STUDY TEXTBOOK

Systems

From RV32I to a small operating system

First edition · 10 October 2026

Contents

1 How a program becomes a running thing **1**

1.1 The four-stage journey . . . 1
1.2 Object files and ELF sections . . . 3
1.3 Symbols and relocation . . . 4
1.4 The loader and `execve` . . . 5
1.5 Where Book 5's CPU meets real silicon . . . 6
1.6 A map of the book . . . 7
1.7 Common mistakes . . . 7
Exercises . . . 8
Summary . . . 9
Further reading . . . 10
Watch instead . . . 10
Notes . . . 10

2 How the machine sees data **11**

2.1 Bytes, words and addresses . . . 11
2.2 Two's complement, in one page . . . 12
2.3 Endianness: which byte goes first . . . 12
2.4 Alignment and struct layout . . . 13
2.5 IEEE-754 in one breath . . . 14
2.6 Pointers as typed addresses; arrays vs pointers . . . 15
2.7 Common mistakes . . . 16
Exercises . . . 16
Summary . . . 18
Further reading . . . 18
Watch instead . . . 18
Notes . . . 19

3 RV32I assembly I: arithmetic, memory, control **20**

3.1 The register file and the ABI names . . . 20
3.2 Six formats, one discipline . . . 21
3.3 Loads, stores and addressing . . . 22
3.4 Branches and comparisons . . . 23
3.5 Assembler directives and running it . . . 23
3.6 Common mistakes . . . 25
Exercises . . . 25
Summary . . . 27
Further reading . . . 27
Watch instead . . . 28
Notes . . . 28

4 Functions and the Stack **29**

4.1 The contract in one paragraph . . . 29
4.2 The stack frame . . . 29
4.3 Caller-saved, callee-saved . . . 30
4.4 What a call costs . . . 31
4.5 Reading real compiler output . . . 32
4.6 PIC in one page . . . 33
4.7 Common mistakes . . . 33
4.8 Summary . . . 34
4.9 Exercises . . . 35

5 A Single-Cycle RV32I Core **37**

5.1 The idea from zero: a machine that follows a recipe . . . 38
5.2 Fetch–decode–execute, properly . . . 38
5.3 Decode and the immediate generator . . . 39
5.4 Register file and ALU — Book 5, widened . . . 40
5.5 Loads, stores and the data-memory interface . . . 40
5.6 Running it: 13 words, checked state . . . 41
5.7 Why single-cycle wastes time . . . 45
5.8 Common mistakes . . . 46
5.9 Summary . . . 46
5.10 Exercises . . . 47

6 Pipelining: One Per Cycle (Usually) 51

6.1 The idea from zero: an assembly line . . . 52
6.2 Why pipeline . . . 52
6.3 The five stages and what the registers carry . . . 52
6.4 Timing diagrams . . . 53
6.5 Where it breaks: hazards, shown . . . 54
6.6 Common mistakes . . . 57
6.7 Summary . . . 57
6.8 Exercises . . . 58

7 Hazards: Forwarding, Stalls, Branches 61

7.1 The idea from zero: shortcuts, waiting, and guessing . . . 61
7.2 Forwarding: don't wait for writeback . . . 62
7.3 Load-use: the one stall you cannot avoid . . . 63
7.4 Branches: predict, verify, flush . . . 64
7.5 The debugging receipts . . . 66
7.6 Common mistakes . . . 67
7.7 Summary . . . 67
7.8 Exercises . . . 68

8 Traps, memory-mapped IO, and a program that talks to the world 71

8.1 Two small additions, one big change of character . . . 72
8.2 Minimal traps: `ecall` and `mret` . . . 73
8.3 The UART: one shift register with an address . . . 74
8.4 The program, end to end . . . 75
8.5 Receipts: two bugs this chapter paid for itself . . . 77
8.6 What it costs in silicon . . . 78
8.7 Common mistakes . . . 78
8.8 Summary . . . 79
Exercises . . . 80

9 The memory hierarchy 83

9.1 Why there is a hierarchy at all . . . 84
9.2 The memory mountain, measured . . . 84
9.3 Locality: the only two tricks caches know . . . 88

9.4 Bandwidth has a stride too . . . 88
9.5 The contract . . . 89
9.6 Common mistakes . . . 90
9.7 Summary . . . 90
Exercises . . . 91

10 Caches: geometry, misses, and the loop that got 3× faster **94**
10.1 Anatomy: three parameters, one equation . . . 95
10.2 Replacement and the three Cs . . . 96
10.3 The simulator . . . 97
10.4 The payoff: blocking . . . 99
10.5 Software-visible effects beyond tiling . . . 102
10.6 Common mistakes . . . 103
10.7 Summary . . . 103
Exercises . . . 104

11 Virtual memory: the address is a lie **108**
11.1 Why virtualize . . . 109
11.2 The walk: sv32 as the concrete case . . . 110
11.3 The TLB: caching the answer . . . 111
11.4 Demand paging and the fault, measured . . . 113
11.5 fork + copy-on-write, and the fence . . . 114
11.6 Common mistakes . . . 115
11.7 Summary . . . 115
Exercises . . . 116

12 Processes and scheduling **119**
12.1 What a process is . . . 120
12.2 fork, exec, wait — the correct version . . . 120
12.3 The context switch, step by step . . . 123
12.4 Scheduling: five policies, measured on one job set . . . 125
12.5 The few dozen lines that actually do it . . . 127
12.6 Common mistakes . . . 127
12.7 Summary . . . 128
Exercises . . . 129

13 Concurrency and synchronisation **132**

13.1 The race, demonstrated . . . 133
13.2 The atomic ladder . . . 134
13.3 Three fixes, one measurement . . . 134
13.4 Condition variables, done right . . . 137
13.5 Deadlock and lock ordering . . . 138
13.6 What concurrency actually buys . . . 138
13.7 Common mistakes . . . 139
13.8 Summary . . . 139
Exercises . . . 140

14 File systems and persistence **143**

14.1 Names, inodes, and the API . . . 144
14.2 The layers a read() descends . . . 145
14.3 The buffer cache, measured . . . 146
14.4 fsync: the syscall that pierces the cache . . . 147
14.5 Journaling: never half-written . . . 148
14.6 SSDs: why random writes hurt . . . 150
14.7 Common mistakes . . . 150
14.8 Summary . . . 150
Exercises . . . 151

15 Measuring performance correctly **154**

15.1 The folklore audit . . . 155
15.2 The performance equation . . . 155
15.3 Latency is a distribution . . . 155
15.4 The four lies . . . 158
15.5 The measurement loop . . . 158
15.6 The measured latency ladder . . . 158
15.7 Common mistakes . . . 158
15.8 Summary . . . 159
Exercises . . . 160

16 Vectorisation and data parallelism **163**

16.1 Three ways to do more at once . . . 164

16.2 From a tiny example to lanes . . . 165
16.3 SAXPY, measured . . . 165
16.4 SAXPY on x86_64: SSE2 and AVX2 . . . 166
16.5 The ragged end: what a vector loop does when n is not a multiple of 4 . . . 167
16.6 Why a compiler can sometimes not vectorise . . . 168
16.7 The folklore item that fails . . . 169
16.8 SoA vs AoS: the layout is the vector . . . 170
16.9 The way of thinking: compute-bound or memory-bound? . . . 172
16.10 The three vector ISAs . . . 172
16.11 Common mistakes . . . 172
16.12 Summary . . . 173
Exercises . . . 174

17 Low-latency design patterns **178**
17.1 Why the average hides the problem . . . 179
17.2 The folklore audit . . . 179
17.3 The same audit on x86_64, and what the compiler did to it . . . 180
17.4 Search at both scales . . . 182
17.5 The allocation myth, priced . . . 183
17.6 The way of thinking: who owns each variable? . . . 184
17.7 The SPSC ring . . . 184
17.8 The cost sheet . . . 186
17.9 Common mistakes . . . 187
17.10 Summary . . . 187
Exercises . . . 188

18 An interpreter and a compiler in miniature **192**
18.1 The divide: doing what the program says, or translating it . . . 193
18.2 The interpreter: a stack machine . . . 194
18.3 Watching the stack: a trace . . . 195
18.4 The compiler: emit once . . . 196
18.5 A compiler you can read in one sitting . . . 197
18.6 What does interpreting cost? Measure it . . . 199
18.7 The JIT compromise . . . 200
18.8 The loop closes . . . 200

18.9 Common mistakes 201
18.10Summary 201
Exercises 202

CHAPTER 1

How a program becomes a running thing

You wrote a CPU. In Book 5 you designed an 8-bit ALU, wired it into a datapath, and drove it with a two-state FSM until Fibonacci numbers came out of a serial pin. That machine is small enough to hold in your head — 302 synthesis cells, ten opcodes, two states — but it is not a toy. The CPU you are using right now runs *more* code than yours, not *different* code. Somewhere between `int main(void)` and the transistors of your cpu8 there is a journey every program takes, and this book is about travelling it on foot. This chapter walks the first mile: how source text becomes a file of machine code, how that file gets into memory, and where your cpu8 meets the real silicon. Along the way you will compile, disassemble and link *real* binaries and read their real output — nothing in this chapter is simulated.

1.1 The four-stage journey

Type `cc sum.c -o sum` and four programs run in sequence, each handing a new file to the next. The names differ between toolchains — `gcc`, `clang`, `cc` — the shape does not:

1. **Preprocess** — text in, text out. `#include` splices header files in place; `#define` substitutes macros; conditionals (`#ifdef`) delete code before the compiler ever sees it. Output: a "translation unit", still plain C.
2. **Compile** — the translation unit becomes assembly: mnemonics, register names, labels — the same language your cpu8 assembler spoke, with a bigger vocabulary. Output: `.s`.
3. **Assemble** — mnemonics become machine-code bytes plus bookkeeping (which bytes are code, which are data, which names are exported). Output: an *object file*, `.o` — already ELF, just not yet executable.
4. **Link** — object files are stitched together: symbols resolved, sections merged, addresses assigned, one entry point recorded. Output: an executable ELF.

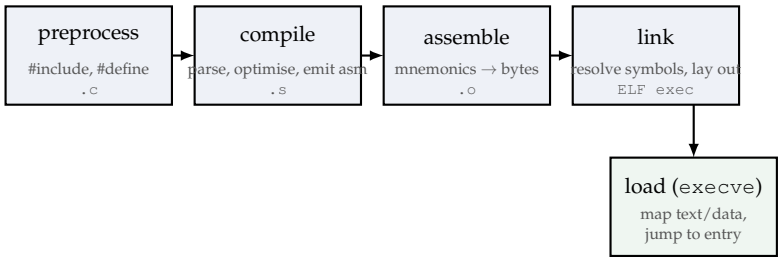
Example 1.1 (The pipeline, hands on)**.** Here is the whole journey on `code/ch01_sum.c`, a six-line freestanding sum. Each stage was run on the dev box (Apple clang 21, targeting x86-64 Linux so the output is real ELF):

```
clang -E ch01_sum.c > ch01_sum.i # 1: preprocess
clang -S ch01_sum.i # 2: compile -> ch01_sum.s
clang -c ch01_sum.s # 3: assemble -> ch01_sum.o
ld.lld -o ch01_sum.elf ch01_start.o ch01_sum.o # 4: link
```

Stage 2 emits assembly you can already read. Real output from `clang -S` (first lines of `sum`):

```
sum: # @sum
       pushq %rbp
       movq %rsp, %rbp
       movq %rdi, -8(%rbp)
       movl %esi, -12(%rbp)
       movl $0, -16(%rbp) # s = 0
       movl $0, -20(%rbp) # i = 0
.LBB0_1: # loop:
       movl -20(%rbp), %eax # eax = i
       cmpl -12(%rbp), %eax # compare i, n
       jge .LBB0_4 # done when i >= n
       movq -8(%rbp), %rax
       movslq -20(%rbp), %rcx
       movl (%rax,%rcx,4), %eax # eax = a[i]
       addl %eax, -16(%rbp) # s += a[i]
```

Every line is one instruction the CPU will execute; `-8(%rbp)` is the array pointer spilled to the stack exactly the way your cpu8's indirect addressing would have handled it.



Four tool invocations, one machine read of the result. The .o is already ELF — linking only decides where each byte goes; loading only decides which page it lands on.

Figure 1.1: The four-stage journey from source to running program. Each stage's product is a file you can inspect; none is magic.

Checkpoint 1.1 In one sentence each: what does the preprocessor hand to the compiler, and what does the assembler hand to the linker?

Solution. The preprocessor hands *expanded C source* (one translation unit, no #-lines) to the compiler. The assembler hands the linker an *object file*: machine-code bytes plus a symbol table and relocation records — not yet a program, but already ELF.

THE PIPELINE HAS FOUR STAGES, NOT ONE

When a student writes "gcc compiles my program", they hide three of the four jobs under one word. You will meet students who cannot tell a *compile* error (bad syntax) from a *link* error (missing symbol) — the fix is different every time, and the fix-time is the difference between seconds and an afternoon.

1.2 Object files and ELF sections

An object file is not a blob: it is a filing cabinet. The bytes from stage 3 arrive sorted into *sections*, each with a name, a size, a job. Compile the sum program and look inside — real output, dev box, clang 21 targeting x86-64 ELF:

```
$ objdump -h ch01_sum.o
Sections:
Idx Name Size VMA Type
  1 .strtab 00000073 0000000000000000
  2 .text 00000076 0000000000000000 TEXT
  3 .rela.text 00000030 0000000000000000
  4 .data 00000014 0000000000000000 DATA
  7 .eh_frame 00000058 0000000000000000 DATA
  8 .rela.eh_frame 00000030 0000000000000000
```

Read the four that matter:

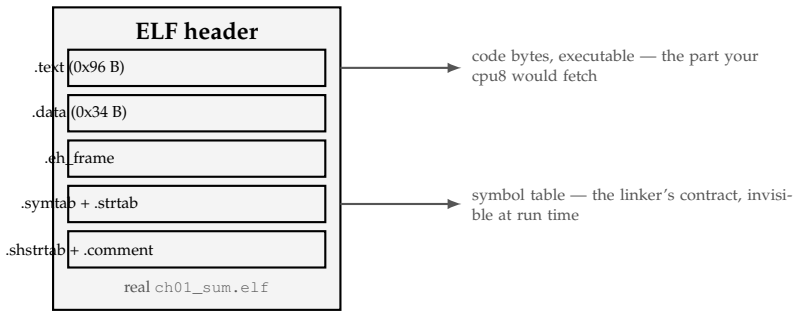
- `.text` — 0x76 = 118 bytes of machine code. This is what a CPU's instruction fetch unit reads. It is called "text" for the same reason a teletype's paper tape was: the name is older than most programmers, and it means *code*, not English.
- `.data` — 0x14 = 20 bytes of initialised data: the five-element array `v` inside `main`, stored as real bytes.
- `.rela.text` — 48 bytes of relocation records: places in `.text` the linker must patch once it knows the final addresses.
- `.symtab` (hidden in this excerpt, present in the file) — the symbol table: the names `sum`, `main`, whether each is defined or referenced, and where.

Example 1.2 (Find `main`'s machine code). `objdump -d` walks the `.text` section and prints every instruction with its offset and its encoding bytes. Real output for `main` in `ch01_sum.o`:

```
0000000000000050 <main>:
  50: 55 pushq %rbp
  51: 48 89 e5 movq %rsp, %rbp
  54: 48 83 ec 10 subq $0x10, %rsp
  58: c7 45 fc 00 00 00 00 movl $0x0, -0x4(%rbp)
  5f: 48 8d 3d 00 00 00 00 leaq (%rip), %rdi # 0x66
  66: be 05 00 00 00 movl $0x5, %esi
  6b: e8 00 00 00 00 callq 0x70 <main+0x20>
  70: 48 83 c4 10 addq $0x10, %rsp
  74: 5d popq %rbp
  75: c3 retq
```

Three things to notice. First, the middle column is the machine code — `55`, `48 89 e5`: the exact bytes Book 5 ch16's ROM would have held. Second, `callq` at `6b` encodes `00 00 00 00` for its target: the assembler left a hole because it did not yet know where `sum` would live — the linker's relocation step fills it. Third, `movl $0x5, %esi` is your accumulator pattern: load a constant into a register, then hand it to the call.

.TEXT IS NOT TEXT, .DATA IS NOT "ALL THE DATA"



An ELF is a table of contents plus the bytes each section points at: same bytes, four different jobs.

Figure 1.2: The section map of the real `ch01_sum.elf`: code, data, bookkeeping — one file, four jobs.

New readers lose hours to naming. `.text` = instructions. `.data` = only *initialised writable* globals. `.bss` = initialised-to-zero globals (not stored in the file at all — the loader zero-fills it, which is why a 1 MB zero array costs no disk space).

1.3 Symbols and relocation

The symbol table is the linker's contract. Two kinds of entries decide whether a program links: *defined* symbols (the file provides them) and *undefined* symbols (the file promises to get them from somewhere else). `objdump -t` shows both:

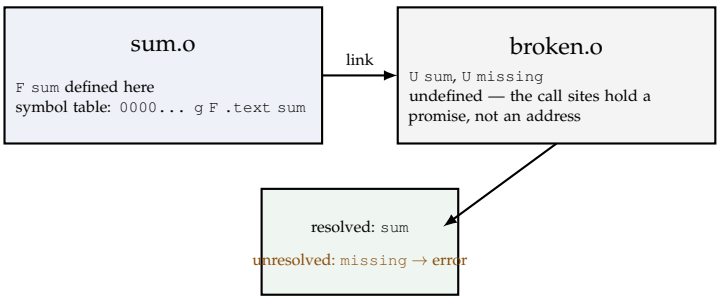
```
$ objdump -t ch01_sum.elf # after linking
00000000002011c0 g .text _start
0000000000202260 g .data data
00000000002011e0 g F .text sum # defined function
0000000000201230 g F .text main # defined function
```

While the object is still a `.o`, the assembler records a *relocation* for every address it could not finish. Real `objdump -r` on `ch01_broken.o`:

```
RELOCATION RECORDS FOR [.text]:
OFFSET TYPE VALUE
0000000000000012 R_X86_64_PC32 .data-0x4
000000000000001c R_X86_64_PLT32 sum-0x4
0000000000000024 R_X86_64_PLT32 missing-0x4
```

Each row says: "patch `.text` at this offset, against the final address of that symbol." The linker runs through these records, looks up every undefined symbol in every other object's symbol table, and either patches the hole — or reports the failure you saw live:

```
$ ld.lld -o broken ch01_broken.o
ld.lld: error: undefined symbol: sum
>>> referenced by ch01_broken.c
>>> ch01_broken.o:(main)
ld.lld: error: undefined symbol: missing
>>> referenced by ch01_broken.c
>>> ch01_broken.o:(main)
```



Relocation = the linker's bookkeeping: every undefined `U` must find a defined `F/g` in another object — or the real error you just saw appears.

Figure 1.3: Relocation is bookkeeping: the assembler records holes, the linker resolves them — or names the missing symbol.

Example 1.3 (Hand-link, and watch the error message change)**.** Link `ch01_sum.o` alone and `ld.lld` reports `undefined symbol: sum` plus `undefined symbol: missing` — two separate records, two separate complaints. Add `ch01_sum.o` to the link line and `sum` disappears from the error list but `missing` remains. That is the whole game: undefined symbols are holes to fill; a link is complete when every hole has a defined home.

Checkpoint 1.2 Your build reports `undefined symbol: __stack_chk_fail`. Is this a compile error or a link error, and what does the linker want?

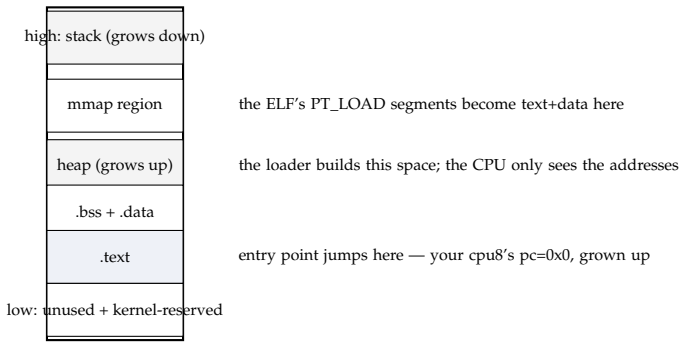
Solution. A link error — the compiler emitted a call to a helper that exists in the C library but is not on your link line. The linker wants any object file or library that *defines* that symbol; adding it is the fix, not editing your source.

1.4 The loader and `execve`

Linking produced a file; something still has to make it *run*. That is the loader's job, and on Linux it is invoked by `execve`: the kernel opens the ELF, checks the magic bytes `\x7f E L F`, builds a fresh address space, maps the loadable segments to virtual pages, sets the program counter to the entry point, and hands the CPU to it. After the link above, the real executable's sections look like this:

```
$ objdump -h ch01_sum.elf
Sections:
Idx Name Size VMA Type
  1 .eh_frame 0000005c 0000000000200158 DATA
  2 .text 00000096 00000000002011c0 TEXT
  3 .data 00000034 0000000000202260 DATA
  5 .symtab 000000a8 0000000000000000
```

Compare with the `.o`: the `VMA` column is no longer zero. The linker assigned real addresses — `.text` at `0x2011c0` — and the loader maps those sections at exactly those virtual addresses. The program counter lands at `_start` (`0x2011c0`), and the machine runs.



Same layout idea as the machine you built in Book 5, same reason: the hardware reads addresses, the software draws the map.

Figure 1.4: The process address space the loader builds: the ELF's segments become mapped pages; the CPU reads only addresses.

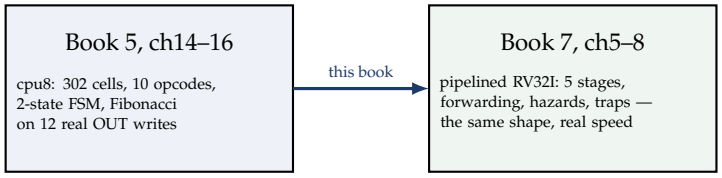
Example 1.4 (Where the map meets the metal)**.** On your cpu8, `pc=0x0` meant ROM offset zero. On a real machine, `pc=0x2011c0` means "page `0x20000` plus `0x1c0`", and a page table translates that to DRAM. The idea is identical — *the CPU fetches from an address, and something decides what lives there*. What changed is who decides: on your cpu8, you did; on Linux, the page table your OS maintains does. That is the one sentence this book is about.

1.5 Where Book 5's CPU meets real silicon

Book 5's cpu8 and a RISC-V RV32I are the same machine at different scales. The mapping table you already know:

cpu8 (Book 5 ch14–16)	**RV32I (this book)**
8-bit regs, r0 = accumulator A	32 32-bit regs, x0 wired to 0
10 opcodes, 4-bit op field	RV32I base: 37 instructions, 7-bit opcode
2-state FSM: fetch, execute	5-stage pipeline: IF ID EX MEM WB
`JMP`/`JZ` flags	`beq`/`bne`/… compare-and-branch
16-byte ROM, `pc=0x0`	virtual memory, `pc` = entry VA
UART prints a byte	`ecall` asks the OS to print

The right column is not a different kind of computer — it is the same datapath+FSM shape with more bits and more stages. Chapters 5–8 build the pipelined RV32I core for real; this section is the signpost that says you have already built its skeleton once.

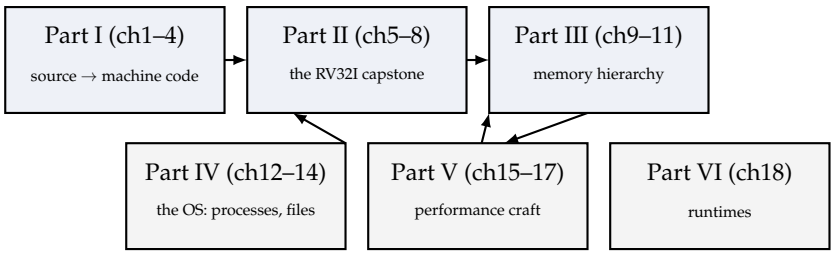


The hand-off this book exists for: the machine you built proves the shape; the chapters ahead prove it at scale.

Figure 1.5: The hand-off: Book 5 proved the shape; this book proves it at scale.

1.6 A map of the book

Six parts, one question each. The order matters: each part runs the tools the previous one installed.



The path this chapter starts: build the real machine around the toy CPU you already made, then measure what the folklore got wrong.

Figure 1.6: The roadmap: source → machine code → a real core → memory → the OS → craft.

DEFINITION OF DONE

- You can name the four pipeline stages and what each produces.
- You can read `objdump -h/-d/-t/-r` output and find code, data, symbols and relocations.
- You know a *link* error from a *compile* error and how to fix each.
- You can place Book 5's cpu8 inside a real address space and name the piece of hardware that made it real: the page table.

1.7 Common mistakes

- **"Compile" is one step.** It is four. Students lose hours calling a linker error a compiler bug; the fix is always different.
- **`.o` is "half a program".** It is a whole ELF, just not yet executable — sections, symbols, relocations are already real. Nothing is created at link time except the *layout*.
- **`.bss` wastes disk.** It does not exist in the file; the loader zero-fills pages. A gigabyte of zeros costs zero bytes on disk.

- **Disassembly "proves" optimisation.** `objdump -d` shows bytes; -O0 vs -O2 differences are layout and constants, not magic. Measure before attributing speed.
- **"The CPU runs the file."** The CPU runs *mapped pages*; the file is only the blueprint. `execve` is the step that turns one into the other.

Exercises

Exercise 1.1. On `ch01_sum.o`, which section holds the machine code, and which holds the relocation records that patch it?

Solution. [hint] `objdump -h` lists both by name: the code lives in one, the patch list in another whose name starts with the code's.

Exercise 1.2. `objdump -t` on `ch01_sum.elf` lists `sum` as `g F .text`. What do the two flags mean?

Solution. [hint] One flag says "global" (visible outside this file); the other says "function". Undefined entries would show `U` instead.

Exercise 1.3. A program's `.bss` is 500 KB but its file on disk is 40 KB. Why is this not a contradiction?

Solution. [hint] `.bss` records a size and a destination, not bytes; the loader zero-fills the pages it names.

Exercise 1.4. Your build prints `undefined symbol: main` (from `ld.lld`). Which of the four pipeline stages failed?

Solution. [hint] The stage that resolves names — the one that runs *after* `.o` files exist.

Exercise 1.5. `callq` in `main` encodes `00 00 00 00`. In one sentence each: who left that hole, and who fills it?

Solution. [hint] The stage that did not know the address left it; the stage that learns final addresses fills it via a relocation record.

Exercise 1.6. Compare `objdump -h` on `ch01_sum.o` and `ch01_sum.elf`: what did the linker add, and what stayed the same?

Solution. [hint] Addresses (VMA) appeared; section *names* and *jobs* did not change. One section disappeared into runtime metadata.

Exercise 1.7. `execve` maps `.text` read-execute and `.data` read-write. Which single bit of your cpu8's memory would have had to become two bits, and why?

Solution. [hint] The page-table permission bits your flat RAM did not need — `.text` must not be writable, `.data` must not be executable.

Exercise 1.8. `-O0` vs `-O2` on `ch01_sum.c` changed `.text` from 0x76 to 0x96 bytes. Was the faster version larger or smaller here, and what does that tell you about "optimise = smaller"?

Solution. [hint] Faster was *larger* (0x96 > 0x76) — unrolling/inlining trades size for speed. Optimisation is about time, not bytes.

Exercise 1.9 (hard)**.** Hand-link the two objects from Example 1.3 and explain, per relocation record, which hole each resolves and which it cannot. Then name the single symbol whose absence forces the linker to fail.

Solution. [hint] The `PC32` record resolves within the same file; the two `PLT32` records need other objects: the `sum` record is satisfied once the second object joins the link, while the `missing` record has no home anywhere — that one is the error.

Exercise 1.10 (hard)**.** Design the minimum change to your cpu8 that would let it run an ELF `.text` section directly. What two pieces of metadata does `.text` carry that your ROM already has, and what two does your ROM lack?

Solution. [hint] Your ROM already has bytes and an implied base address. It lacks a `VMA` (which virtual page it lands on) and a protection mode (read-execute vs read-write) — the two fields that make an ELF a program rather than a tape.

Exercise 1.11 (hard)**.** A student claims the entry point `_start` is "where the OS starts the program". Correct the claim in one sentence, then name the field in the ELF header that actually points to the first instruction.

Solution. [hint] `_start` is not magic — it is the symbol the linker recorded in the ELF header's `e_entry` field; the loader reads that address and jumps to it, whoever named the symbol.

Summary

Stage	**What it produces**
Preprocess	expanded C (one translation unit)
Compile	assembly (`.s`)
Assemble	ELF object (`.o`): sections + symbols + relocations
Link	executable ELF: resolved symbols, assigned VMAs
Load (`execve`)	mapped pages, `pc` → entry VA
You can now	
read `objdump`	code, data, symbols, relocations at a glance
name the failure	compile vs assemble vs link vs load — different fixes
place cpu8 in the map	page table is the piece that makes it real

Further reading

- Randal E. Bryant and David R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, ch. 7 "Linking" — the standard walk-through of symbols and relocation this chapter compresses.
- Ulrich Drepper, "How To Write Shared Libraries", §1 (ELF basics, sections vs segments). `https://www.akkadia.org/drepper/dsohowto.pdf`
- Compiler Explorer documentation — try `clang -S` on your own sum online and match the output to Example 1.1. `https://github.com/compiler-explorer/compiler-explorer`
- The ELF specification (System V ABI, §4–5) — the byte-level reference for every field `objdump` printed.
- Linux `execve(2)` and `elf(5)` man pages — the loader's side of the contract.

Watch instead

DECISION RULE

Watch instead. If a video explains it faster than the prose, watch these and come back:

- "In-depth: ELF — The Extensible & Linkable Format" — stacksmashing, 19:02. Byte-level walk of the format this chapter uses; matches Sections 1.2–1.3. `https://youtu.be/nC1U1LJQL8o`
- "Writing a Linux ELF Binary by Hand" — Simon Racz, 59:11. Builds the file from hex — the same journey with no toolchain. `https://youtu.be/JM9jX2aqkog`

Notes

CHAPTER 2

How the machine sees data

Chapter 1 followed a program from text to running code. This chapter asks the companion question: once it is running, *what does the machine think the data is*? The honest answer is short and unsettling: nothing. A memory cell holds bytes; a register holds bits. Types — signed or unsigned, integer or pointer or float — are agreements between your source code and the generated instructions, and the CPU keeps no record of them. Most of this chapter is spent making that concrete: we decode the same four bytes three ways, measure what struct layout really costs, and run an alignment experiment whose result surprised us — the numbers are real, and they disagree with the folklore. That disagreement is the lesson.

2.1 Bytes, words and addresses

A *byte* is the smallest thing a modern machine addresses: 8 bits, one address. A *word* is the machine's working size — 32 bits on RV32, 64 on the laptop you are probably using. When a program says "the value at address `0x2000`", it means "the byte at `0x2000`"; a 4-byte `int` stored there occupies `0x2000` through `0x2003` and belongs to whichever interpretation reads it. The machine's data types table is short enough to memorise:

C type	LP64 (Linux, macOS arm64)	ILP32 (RV32, Win32)	bits
`char`	1	1	8
`short`	2	2	16
`int`	4	4	32
`long`	8	4	64 / 32
`long long`	8	8	64
`float`	4	4	32
`double`	8	8	64
`void *`	8	4	64 / 32

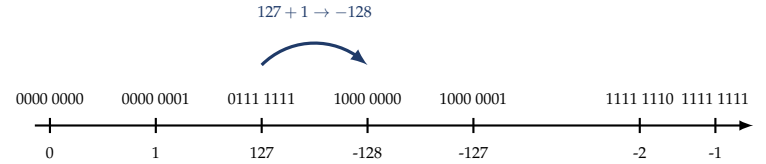
Figure 2.1: C type sizes: LP64 (what this book's dev box runs) vs ILP32 (the RV32 model the capstone targets). `long` and pointers are the two that move.

LONG IS A TRAP ON EVERY PLATFORM

On LP64 `long` is 8 bytes; on ILP32 — including RV32, the ISA your capstone CPU runs — it is 4. Code that assumes `sizeof(long) == 8` breaks silently when cross-compiled. Use `int32_t` / `uint64_t` from `<stdint.h>` whenever the size matters, and it always matters in this book.

2.2 Two's complement, in one page

Book 5 ch8 derived this properly; here is the working version. An n-bit signed integer stores $-2^{n-1} + \sum b_i 2^i$ — the top bit contributes *minus* its weight. That single negative weight buys three things: one zero, one adder for both addition and subtraction, and a clean wrap from most-positive to most-negative. Figure 2.2 shows the 8-bit case on a line: $127 + 1$ does not saturate, it jumps to -128.



Two's complement is one hardware story, not two: adding and subtracting are the same circuit, and the "wrap" at the ends is why signed overflow is a cliff, not a curve.

Figure 2.2: Two's complement as a number line: the wrap at the ends is hardware fact, not an error message.

PRINCIPLE

The CPU does arithmetic on *bit patterns*, not on numbers. Whether `0x80000000` is -2^{31} or $+2^{31}$ is decided entirely by the instruction that consumes it — `slt` vs `sltu` in RV32I, same bits, two opinions.

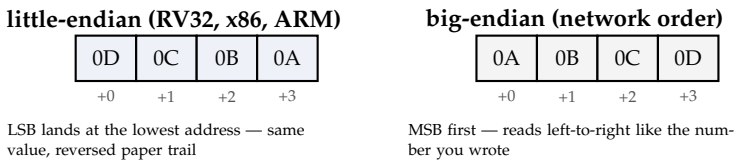
2.3 Endianness: which byte goes first

A 4-byte value has to go into 4 one-byte slots, and there are two conventions. Little-endian puts the least-significant byte at the lowest address; big-endian puts the most-significant first. RV32 is little-endian, x86 is little-endian, ARM defaults to little — the network protocols you will meet in other books are big-endian, which is why the conversion functions are called `htonl/ntohl`.

Example 2.1 (Real bytes on real hardware). `code/ch02_bytes.c` stores `0x0A0B0C0D` in a `uint32_t` and prints the four memory bytes in address order. Real output on the dev box (arm64, LP64):

```
bytes (LE order): 0d 0c 0b 0a
as int32 : 168496141
as float : 6.69487e-33
0x3FC00000 as float: 1.5
```

The least-significant byte `0x0d` lands first: little-endian, confirmed by experiment rather than documentation. The same four bytes then get read as a signed int (`168496141` — same value, the top bit happened to be zero) and as a float (nonsense: `6.69e-33` — the exponent field is tiny). Three different worlds from one bit pattern; the next section formalises the third.



Real bytes from `ch02_bytes`: `0d 0c 0b 0a` — your dev box writes little-endian, and so does RV32.

Figure 2.3: `0x0A0B0C0D` in memory, both conventions. RV32, x86 and ARM all pick the left column.

Checkpoint 2.1 You receive the four bytes `78 56 34 12` (in address order) over a little-endian memory-mapped register. What 32-bit value did the sender write?

Solution. `0x12345678` — little-endian means the first byte is the least significant, so read the sequence backwards as hex pairs.

2.4 Alignment and struct layout

Hardware prefers a n-byte value at an address divisible by n — an *aligned* access. Compilers honour this by inserting *padding*: invisible bytes between struct fields so every field sits at its natural offset. The rule chain: each field's offset must be a multiple of its alignment; the struct's size is rounded up to the largest member's alignment; arrays of the struct need that tail padding to keep every element aligned.

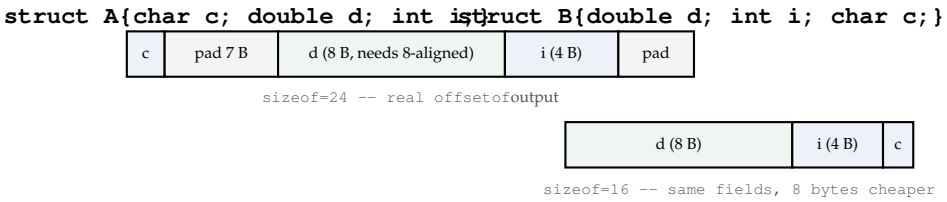
Example 2.2 (The same three fields, two prices)**.** `code/ch02_pad.c` declares `struct A{char;double;int;}` and `struct B{double;int;char;}` — identical fields, different order. Real output:

```
A: sizeof=24 c@0 d@8 i@16
B: sizeof=16 d@0 i@8 c@12
```

In `A`, `char c` occupies byte 0, then `double d` must start at an 8-aligned address: bytes 1–7 are padding. `int i` lands at 16, and four tail bytes pad the total to 24. Reordering the same three fields into `B` costs 16 bytes — a third less memory, for free, just by sorting fields largest-first.

2.4.1 Measurement: the alignment tax that wasn't

Folklore says misaligned access is slow: the CPU splits the load across two cache lines and pays double. `code/ch02_align.c` tests it directly — sum a 4 M-element `uint32` array once through an aligned pointer, once through the same pointer offset by one byte (every load misaligned). Real timings, Apple M-series, 32 passes:



The compiler pads to each field's alignment and the struct's worst member; reordering fields is free bytes. Measured, not theorised.

Figure 2.4: Padding maps of `A` vs `B`: measured `offsetof` values, not a sketch.

```
offset 0: 151.2 ms (s=2260630400663552)
offset 1: 110.4 ms (s=4521260264456192)
```

The misaligned version is *faster* — repeatably, in both loop orders, by ∼25%. Two honest conclusions. First, the expected tax is absent: Apple Silicon performs unaligned loads at full speed, as do most modern cores. Second — the reason this section exists — a consistent 25% delta in the *wrong direction* means the benchmark is measuring something else (loop layout, prefetcher behaviour, uarch quirks), not alignment. When the data contradicts the folklore, the data wins and the model gets revised. That is the book's method in miniature: predict, measure, reconcile.

"MISALIGNED IS SLOW" IS HARDWARE FOLKLORE

It was true on 1980s CPUs and still holds on some embedded cores — and it is false on the chip in your laptop. Rules about performance expire; measurements do not. Re-test before you re-tell.

2.5 IEEE-754 in one breath

A `float` is 32 bits partitioned $S \mid E \mid F$: one sign bit, an 8-bit biased exponent, a 23-bit fraction with an implied leading 1. Value: $(-1)^S \times (1 + F/2^{23}) \times 2^{E-127}$ for $0 < E < 255$; the extremes E=0 and E=255 are reserved for denormals/zero and inf/NaN. Precision is about 7 decimal digits; the spacing between neighbours near 2^e is 2^{e-23}, and round-to-nearest keeps each operation within half that — the ".5 ulp" you will hear cited every time someone says "floats are exact enough".

Example 2.3 (Decode `0x40490FDB` by hand, then by code)**.** S=0, exponent field `0x80` (= 128), fraction field `0x490fdb`. Value = $1.5707964 \times 2^{128-127} = 3.1415928$ — π to float precision. `code/ch02_float.c` performs the same decode bit-by-bit and prints:

```
sign=0 exp=128(E-1) frac=0x490fdb
value = 3.141592741
```

The last digit differs from π because the 23-bit fraction itself is the approximation — the stored value *is* the rounding.

Checkpoint 2.2 In `0x3FC00000` (the `1.5` printed in Example 2.1), what are S, E and the implied mantissa?

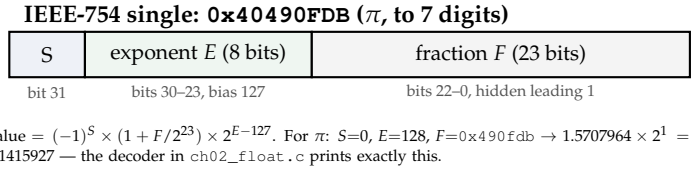
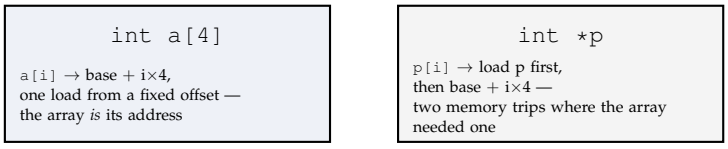


Figure 2.5: IEEE-754 single precision anatomy, decoded for π.

Solution. S=0; E=0x7F=127 so the real exponent is $127 - 127 = 0$; F=0x400000, mantissa $= 1 + 0.5 = 1.5$; value $= 1.5 \times 2^0$.

2.6 Pointers as typed addresses; arrays vs pointers

A pointer is an address wearing a type. `p+1` advances by `sizeof(*p)` bytes — the arithmetic is scaled, which is why walking an `int32_t*` moves 4 bytes per step and a `uint8_t*` moves 1. The type is only a scale factor to the code generator; the CPU sees an address either way. The difference between `int a[4]` and `int *p` is subtler: the array's base address is fixed at compile time, while the pointer's value is itself data — stored, loaded, changeable. In generated code `a[i]` is one load off a known base; `p[i]` is two: load `p`, then load `p[i]`.



"Arrays are pointers" is a half-truth: the array's address is compile-time, the pointer's is runtime — the generated code keeps the receipt.

Figure 2.6: Array vs pointer in generated code: the array *is* its address; the pointer must be fetched first.

Example 2.4 (Reading the same bytes three ways, safely)**.** Example 2.1 reinterpreted the four bytes of `0x0A0B0C0D` as `int32_t` (same value, bit-compatible) and as `float` (6.69×10^{-33}, nonsense) — both via `memcpy`, not pointer casts. That detail matters: type-punning through a `float*` cast violates strict aliasing and lets the optimiser reorder or delete your reads; `memcpy` is the standards-strict way to ask "what do these bytes mean as a different type?" and compiles to zero instructions.

DEFINITION OF DONE

- You can say what a byte, a word and an address are — and that types are compile-time agreements, not machine facts.
- You read endianness off real bytes and explain why RV32 chose little-endian.
- You predict struct `sizeof` and `offsetof` before running the probe — and reorder fields to save real bytes.

- You decode an IEEE-754 bit pattern by hand and with the decoder.
- You know the alignment-folklore experiment, what it actually showed, and why "measure it" beats "everyone knows".

2.7 Common mistakes

- **"The variable is signed/unsigned/float."** The *bytes* are just bytes; the type lives in which instructions touch them. `0x0A0B0C0D` was an int and a float in the same object, at the same address, in the same second.
- **Assuming `long` is fixed.** It changes size between LP64 and ILP32 — including your RV32 target. Use fixed-width types.
- **"Arrays and pointers are the same."** `sizeof(a)` vs `sizeof(p)` disagrees, and the generated code shows the extra load. Decay happens at call boundaries, not in your head.
- **"Misaligned access is slow."** Measure first — modern cores largely don't care, and our own experiment inverted the prediction.
- **Type-punning through casts.** `*(float*)&u` is strict-aliasing UB; `memcpy` is free and correct.
- **"Floats can't represent that."** They represent *nearest* values: 0.1 is not stored, its neighbour is. Equality tests belong on integers; floats get tolerances.

Exercises

Exercise 2.1. `uint8_t` vs `int8_t`: which changes when you print the same bit pattern `0xFF`, and what does each show?

Solution. [hint] The bits are identical; only the interpretation differs. One prints 255, the other -1 — name which is which.

Exercise 2.2. A `struct` with `char,int,char` fields reports `sizeof=12`. Account for every byte.

Solution. [hint] `int` needs 4-alignment: c@0, pad 1–3, i@4–7, c@8, tail pad 9–11 so the struct is a multiple of 4.

Exercise 2.3. Convert `0xBF800000` to a decimal float.

Solution. [hint] S=1, E=127, F=0: $-1.0 \times 2^0 = -1.0$.

Exercise 2.4. Why does the size of `"hello"` come out 6 on every platform, while the size of a `char*` differs between data models?

Solution. [hint] The string literal is an array of 6 chars including the NUL; the pointer is an address, and addresses are the thing that changes size.

Exercise 2.5. On a big-endian machine, what would `ch02_bytes.c` print in the first line instead of `0d 0c 0b 0a`?

Solution. [hint] Reverse the stored order: MSB first — `0a 0b 0c 0d`.

Exercise 2.6. The alignment experiment measured misaligned as *faster*. Give two reasons this could happen that have nothing to do with alignment.

Solution. [hint] Loop ordering/warm-up, code layout, prefetcher behaviour, uarch quirks, different accumulated work — name any two; the point is that a wrong-direction result indicts the benchmark, not the folklore's opposite.

Exercise 2.7. `double d = 0.1 + 0.2;` prints `0.30000000000000004`. Which IEEE-754 fact explains the 4?

Solution. [hint] Neither 0.1 nor 0.2 has a finite binary fraction; both round to a neighbour, and the sum of the neighbours is one more neighbour — the "4" is the accumulated rounding surfacing.

Exercise 2.8. Your RV32 code does `*(volatile uint32_t*)0x10013000` to read a UART register. Why must the access be exactly 4 bytes, and what breaks if the compiler splits it into byte loads?

Solution. [hint] MMIO registers are word-sized state machines: reading one byte can acknowledge the whole register (e.g. consume the RX FIFO entry). Split loads change device behaviour, not just performance.

Exercise 2.9 (hard)**.** Write a standards-strict float-bits printer (sign, exponent, fraction fields) without type-punning UB and without assuming IEEE-754 — verify the format macro first.

Solution. [hint] Copy the float's bytes into a `uint32_t` with `memcpy`, mask the fields, and guard the whole thing with a `__FLT_RADIX__` check — C never promised IEEE-754; it just happens to hold everywhere you will ship.

Exercise 2.10 (hard)**.** Design a fair alignment benchmark that would actually isolate the misalignment cost on a machine that hides it. Name three confounders and how you'd control each.

Solution. [hint] Interleave the two loops (kill ordering effects), keep `s` identical (kill work differences), use a dependent-load chain or volatile store (kill OOO hiding), pin to one core and average many trials. List any three.

Summary

Idea	One-line version
types	compile-time agreements; the CPU sees bytes
two's complement	top bit has negative weight; one adder does both
endianness	byte order on the wire/in RAM; RV32 is little
alignment+padding	fields sit at natural offsets; reorder fields for free bytes
IEEE-754	$S\|E\|F$; ≈7 digits; neighbours are the truth
measurement	folklore expired; our own alignment test inverted

Further reading

- Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective*, ch. 2 — the standard full treatment of every section here.
- Suzanne J. Matthews, Tia Newhall, Kevin C. Webb, *Dive into Systems*, ch. 4 — free online, C-focused coverage of representation. `https://diveintosystems.org/book/C4-Data_rep/`
- IEEE-754-2019 standard summary at `https://en.wikipedia.org/wiki/IEEE_754` — the format table this chapter compresses.
- "What Every Computer Scientist Should Know About Floating- Point Arithmetic" (Goldberg), Sun docs reprint.
  docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html
- RISC-V unprivileged ISA spec, §2 (base integer model, little-endian note) — `https://riscv.org/specifications/`

Watch instead

DECISION RULE

Watch instead.

- "Endianness Explained With an Egg" — Computerphile, 10:46. The boiled-egg walkthrough of Section 2.3. `https://youtu.be/NcaiHcBvDR4`
- "Floating Point Numbers" — Computerphile, 9:16. The IEEE-754 fields animated; matches Section 2.5. `https://youtu.be/PZRI1IfStY0`

Notes

CHAPTER 3

RV32I assembly I: arithmetic, memory, control

Chapter 2 showed that the machine sees only bytes. This chapter shows what it *does* with them: the 37 instructions of RV32I, the base RISC-V integer set — and the instruction set your capstone CPU will execute for real in chapters 5–8. Everything here is concrete: a small assembler you can read end-to-end encodes each program to real hex, a small interpreter runs it, and every number in the chapter comes from that run — not from a datasheet summary. If you can write a loop in this chapter's notation, you can read the disassembly of any program your laptop runs.

3.1 The register file and the ABI names

Thirty-two registers, `x0`–`x31`, each 32 bits — the same file shape as your cpu8's, just wider and deeper. Two of them are special by hardware: `x0` is hardwired to zero (writes to it are silently discarded, which is why `mv rd, rs` is really `addi rd, rs, 0` and `nop` is really `addi x0, x0, 0`). The rest are general-purpose by hardware but assigned jobs by convention — the ABI names you will see in every disassembly:

ABI name	x-name	Job
`zero`	x0	wired to 0 — the register that isn't
`ra`	x1	return address
`sp`	x2	stack pointer
`gp`, `tp`	x3, x4	globals, thread pointer
`t0`–`t2`	x5–x7	temporaries (caller-saved)
`s0`/`fp`, `s1`	x8, x9	saved / frame pointer
`a0`–`a7`	x10–x17	args + return values
`s2`–`s11`	x18–x27	saved (callee-saved)
`t3`–`t6`	x28–x31	more temporaries

Figure 3.1: The RV32I register file with ABI conventions. Programs obey the hardware; *programmers* obey this table so calls compose.

PRINCIPLE

The ABI names are a contract between functions, not hardware. `a0–a7` carry arguments and return values; `t0–t6` are temporaries a callee may trash; `s0–s11` a callee must preserve. ch4 makes the stack discipline formal; for now: use `t`-names inside one function and the world stays simple.

3.2 Six formats, one discipline

Every RV32I instruction is 32 bits — fixed width, no exceptions — carved into six layouts. The carving is deliberately regular: register fields (`rd`, `rs1`, `rs2`), the function selector `funct3`, and the 7-bit `opcode` keep the same bit positions across every format they appear in. Only the immediate bits move, and even they preserve their internal order. That regularity is not elegance for its own sake — it is what lets a decoder be a few hundred gates rather than a table lookup.

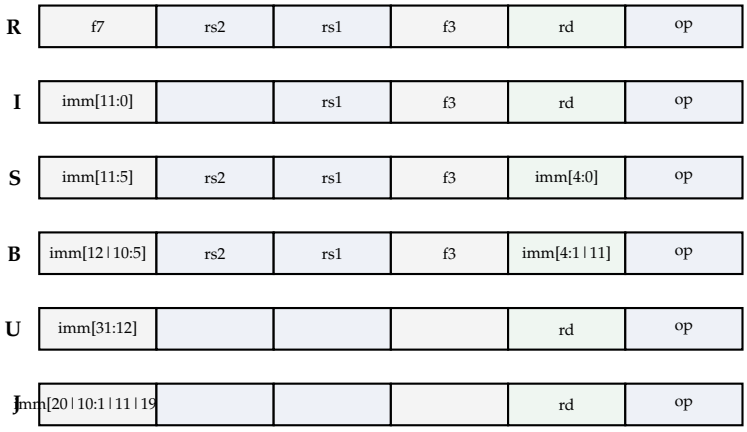


Figure 3.2: The six encoding formats. Green = destination, blue = source/opcode, grey = immediate fields.

Example 3.1 (Hand-assemble, then verify against the real assembler)**.** `addi a0, a0, 1`: I-type, imm=1, rs1=`a0`=x10, `f3`=0, rd=x10, op=`0x13` — $1 \ll 20 \mid 10 \ll 15 \mid 10 \ll 7 \mid \texttt{0x13} = \texttt{0x00150513}$. The book's assembler (`code/ch03_rv32asm.py`, a 40-line Python RV32I assembler) produces the same word — real output on the five-format smoke test:

```
00000000: 00150513 # addi a0, a0, 1
00000004: 00b50863 # beq a0, a1, done
00000008: 00b53633 # add a2, a0, a1
0000000c: 00812683 # lw a3, 8(sp)
00000010: 00d12623 # sw a3, 12(sp)
00000014: 00000013 # nop
```

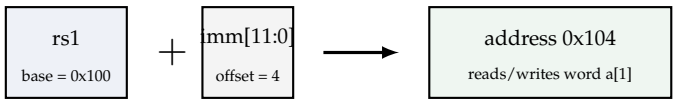
Check `beq` by hand: B-type needs the offset 16 (bytes, in 2-byte units in the encoding: imm=16). The assembler computed `done`-pc = $0x14 - 0x04 = 16$ — and the encoding `0x00b50863` decomposes to exactly that.

PSEUDOINSTRUCTIONS ARE LIES YOU TELL ON PURPOSE

`li`, `mv`, `nop`, `j`, `ret`, `bgt` — none exist in hardware. The assembler expands each to a real instruction (`li t0,1` → `addi t0,x0,1`). That is friendly until debugging shows `nop` where you wrote a pseudoinstruction, or `li` becomes two instructions for a big constant. Read the listing, not the source.

3.3 Loads, stores and addressing

RV32I has exactly one addressing mode: *base register + sign-extended 12-bit immediate* — `lw rd, off(rs1)` and `sw rs2, off(rs1)`. There is no scaled index, no auto-increment, no memory-to-memory op. Walking an array is two instructions: the load, and the pointer bump. That austerity is deliberate: one mode keeps the address adder single, and the ALU does everything else.



`lw t4, 4(t0)` is the whole addressing story: register + sign-extended 12-bit immediate. No indexing modes, no scaling — RV32 keeps the ALU cheap and lets the pointer-bump instruction do the walking (that's the `addi t0,t0,4` in the loop).

Figure 3.3: The single addressing mode: register + 12-bit immediate.

Example 3.2 (An array sum, real output). `code/ch03_arrsum.S` stores {1,2,3,4,5} at `0x100`, then walks it: `lw t4, 0(t0)` / `addi t0,t0,4` per element. The interpreter (`ch03_rv32sim.py`, a ∼60-line RV32I simulator with a memory-mapped output register at `0x10000000` — the same trick Book 5 ch16's UART used) runs the assembled words and prints the real result:

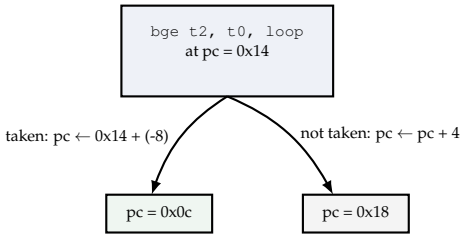
```
15
# 56 instructions retired
```

Checkpoint 3.1 After `lw t4, 4(t0)` with `t0=0x100`, which byte addresses are read, and what changes if you wrote `lb` instead?

Solution. `lw` reads `0x104`–`0x107` (one aligned word). `lb` reads only `0x104` and sign-extends its byte — same base, different width, different sign rule.

3.4 Branches and comparisons

Six conditional branches: `beq`, `bne`, `blt`, `bge`, `bltu`, `bgeu` — compare two registers, jump by a signed immediate if true. There is no flags register: the compare happens inside the branch, and the "set" pseudo-ops (`slt`, `sltu`) exist only when you need the boolean as data. Each branch is one decision: taken → `pc + imm`, not taken → `pc + 4`.



The branch's whole job: pick the next pc. The compare (bge = rs1 ≥ rs2) decides between pc+4 and pc+imm — one adder, one mux, same shape as the FSM branch on your cpu8.

Figure 3.4: A branch picks the next pc: +4 or +imm — one mux.

Example 3.3 (`max(17, 42)` with `bge`)**.** `code/ch03_max.S`: if `a0 ≥ a1` the answer is `a0`, else `a1`. The branch is one instruction; the interpreter's run:

```
42
# 27 instructions retired
```

Checkpoint 3.2 In `bge t2, t0, loop` at `pc=0x14` with `t2=10`, `t0=5`: is the branch taken, and to which pc?

Solution. Taken: $10 \geq 5$, so `pc` becomes $0x14 + (-8) = 0x0c$ — the loop header.

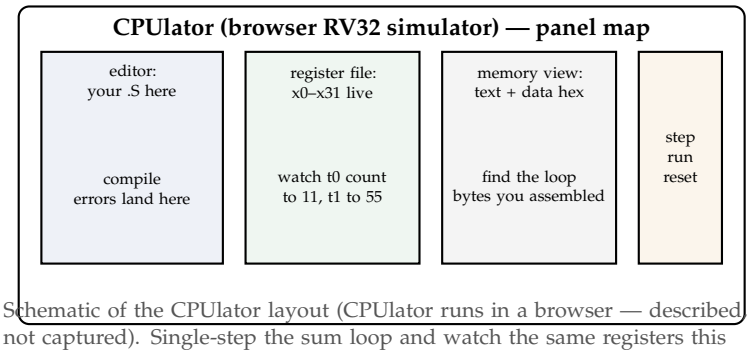
3.5 Assembler directives and running it

Directives are instructions to the assembler, not to the machine: `.text`/`.data` pick the section, `.word`/`.byte` plant data, `.globl` exports a symbol, labels name addresses. The chapter's programs print through a memory-mapped output register, which is what a bare-metal UART driver looks like before an OS arrives. Two ways to run the same bytes: **CPUlator** (browser, free) draws the register file and memory live while single-stepping — Figure 3.5 maps its panels; and the book's own `ch03_rv32sim.py` runs the identical words headless (all outputs in this chapter are from it). Spike is the canonical command-line emulator; either way the bytes are the same.

3.5.1 Measurement: dynamic instruction count

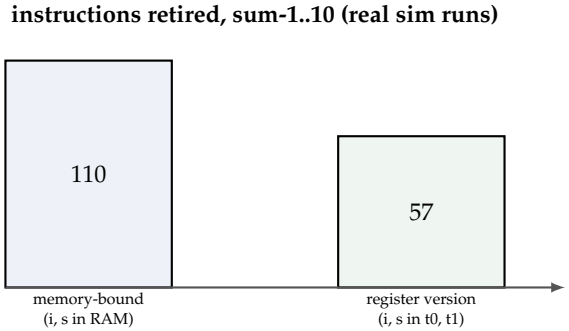
The same sum-1..10 loop written two ways: register variables vs the "$-O0$ habit" of keeping `i` and `s` in memory. Real counts from `ch03_rv32sim.py`:

```
memory-bound (i, s in RAM): 110 instructions retired
register version (i, s in t0/t1): 57 instructions retired
```



Schematic of the CPUlator layout (CPUlator runs in a browser — described, not captured). Single-step the sum loop and watch the same registers this chapter uses change one word at a time.

Figure 3.5: CPUlator's four panels, schematically. Described, not captured — the tool is a browser app.



Predicted ∼2×, measured 1.93×. The loop body is 7 instructions vs 3 — keeping variables in registers is not an optimisation, it is the difference between code and bookkeeping. Counted by `ch03_rv32sim.py`, not estimated.

Figure 3.6: Real dynamic counts: registers vs memory for the same loop.

Predicted $\sim 2\times$ before running (3-instr loop body vs 7-instr); measured $110/57 = 1.93\times$. The gap is the loads and stores themselves: memory-keeping adds `lw`/`sw` traffic that computes nothing — bookkeeping billed as instructions. That is the -O0/-O2 difference you will meet in ch4, visible here without a compiler.

DEFINITION OF DONE

- You can read any RV32I line and name its format, its fields and its effect — and write the 32-bit word for a handful of them.
- You know the register file's two contracts: hardware (`x0`=0) and ABI (`a`-args, `t`-temps, `s`-saved).
- You can trace a loop's pc through a branch and predict the next fetch.
- You can run RV32I here (book simulator) or in the browser (CPUlator) and count what it costs.
- You can point at an instruction stream and say which 4 bytes the CPU will fetch next, and why.

3.6 Common mistakes

- **Counting pseudoinstructions.** `li`/`mv`/ `nop` are sugar: what retires is the expansion. Dynamic counts only see real instructions.
- **Forgetting `x0` eats writes.** `addi x0, x5, 0` does nothing; the register file is honest that way.
- **Immediates are 12 bits.** `addi t0, t0, 4096` won't assemble — big constants need `lui`+`addi` or `li` (which expands).
- **Branches move the pc, not flags.** There is no carry flag to "check" — each branch recomputes its compare.
- **`lw` where `lb` belonged.** Byte vs word is a correctness issue on packed data, not a performance one.
- **"Memory ops are free."** The 110-vs-57 measurement is the counterexample: the tax is the instruction count itself.

Exercises

Exercise 3.1. `addi x0, x5, 0` — what does this write, and why is it a useful instruction anyway?

Solution. [hint] Nothing: `x0` discards writes. It's `nop` — a 4-byte "do nothing" that real pipelines need.

Exercise 3.2. `lw a0, 8(sp)` decodes to `0x00812503`? Verify or correct, field by field.

Solution. [hint] I-type: imm=8$\ll$20=`0x00800000`; rs1=`sp`=x2$\ll$15; f3=2; rd=`a0`=x10$\ll$7; op=`0x03`. The shown word is right.

Exercise 3.3. In the register/ABI table: which two register groups may a callee freely destroy, and which must it restore?

Solution. [hint] Caller-saved: `t0`–`t6` (and args). Callee-saved: `s0`–`s11`. The `t`/`s` split is the whole discipline.

Exercise 3.4. `beq` targets are limited to about ±4 KB. Why, and what real instruction escapes the limit?

Solution. [hint] B-type holds a 13-bit signed byte offset. Long jumps use `jal` (J-type, ±1 MB) or `jalr` (register-indirect, anywhere).

Exercise 3.5. Hand-decode `0xfe53dce3` (the loop branch) into fields and offset.

Solution. [hint] B-type; f3=5 (`bge`); imm bits give −8; rs1=`t2`, rs2=`t0`. At `pc=0x14` the target is `0x0c`.

Exercise 3.6. The array-sum walks words with `addi t0,t0,4`. Rewrite one iteration to use the immediate instead of the pointer bump, and name the trade-off.

Solution. [hint] `lw t4, 4(t0)`... unrolls offsets but keeps the base fixed — fewer instructions per element, more code bytes, and it caps at imm=2048 words. Classic size-vs-count trade.

Exercise 3.7. `slt a0, a1, a2` is "set if less-than". Write the two-line sequence that implements `max` *without* any branch.

Solution. [hint] `slt t0, a0, a1` then select: branchless needs arithmetic on the flag (`sub`/mask trick) or a conditional move RV32I doesn't have — branchless `max` on base RV32I costs a multiply or a bit-trick; name it.

Exercise 3.8. The memory-bound loop runs 7 instructions per iteration. Count them from the listing and identify which two are pure bookkeeping.

Solution. [hint] lw i, lw s, add, sw s, addi i, sw i, bge = 7. The two `sw`s (and arguably the `lw`s) compute nothing — they only persist state registers would have held for free.

Exercise 3.9. `li a7, 0x10000000` won't fit `addi`. Write the two-instruction reality the assembler expands to.

Solution. [hint] `lui a7, 0x10000` + `addi a7, a7, 0` — U-type plants the top 20 bits, I-type patches the bottom 12.

Exercise 3.10 (hard)**.** Implement `strlen`, `memcpy` and `memset` in RV32I and self-check on the book simulator: fill a buffer, copy it, measure it, print the length.

Solution. [hint] All three are the same loop skeleton: `lb`/`sb` plus a pointer bump and a compare. `strlen` counts until 0; `memcpy` copies until n; `memset` fills until n. Watch for the NUL byte in the length count.

Exercise 3.11 (hard)**.** Extend `ch03_rv32asm.py` with `jal`/`jalr` (the J-type format). Encode `jal ra, +8` and decode it back.

Solution. [hint] J-type packs the immediate in bit-order pieces like B but shifted; the adder-tree trick is the same. Verify $+8$ lands in the right fields.

Exercise 3.12 (hard)**.** Write a program that prints the first 10 Fibonacci numbers on the book simulator's MMIO output. Count retired instructions for the register version vs a memory version.

Solution. [hint] Fib(n)=Fib($n-1$)+Fib($n-2$) needs two live registers and one temp per step — the register win is bigger than the sum loop's because there are no array loads at all.

Summary

Idea	One-line version
register file	32×32; `x0`=0; ABI names split caller-/ callee-saved
six formats	rd/rs/f3/op fixed; only immediate fields shuffle
addressing	one mode: reg + 12-bit signed immediate
branches	compare+offset in one; pc + 4 or pc + imm
pseudoinstructions	sugar; count the expansion, not the line
measurement	memory-bound loop: 110 vs 57 retired — the -O0 tax

Further reading

- RISC-V unprivileged ISA specification — the authoritative encoding tables this chapter compresses. https://riscv.org/specifications/
- Hennessy & Patterson, *Computer Organization and Design*, RISC-V edition, ch. 2 — the classic ISA walk-through.
- Bryant & O'Hallaron, *CS:APP*, ch. 3 — structure of machine code (x86, but the ideas transfer).
- CPUlator (browser RISC-V simulator) — run every program in this chapter interactively. https://cpulator.01xz.net/
- Spike (RISC-V ISA simulator) — the canonical command-line reference implementation. https://github.com/riscv-software-src/riscv-isa-sim

Watch instead

DECISION RULE

Watch instead.

- "DDCA Ch6 — Part 2: RISC-V Instructions" — Sarah Harris, 2:48. The instructions-first pitch that matches this chapter's structure; the book's own DDCA source, on camera. https://youtu.be/SdjPQtMPPmo
- "RISC-V Assembly Hello World (Part 1)" — LaurieWired, 11:42. A first full program assembled and run — the same journey ch03_sum.S takes on the book's simulator. https://youtu.be/0IeOaiKszLk

Notes

CHAPTER 4

Functions and the Stack

A function call looks like one line of source and feels free. On the machine it is two instructions of mechanism — `call` writes the return address into `ra` and jumps; `ret` jumps back — wrapped in a *convention*: an agreement about which registers survive the trip and where memory holds the values that do not fit in registers. In the last chapter every program was a leaf: it never called anything, so `ra` stayed safe in a register and the stack was never touched. This chapter breaks that. We give the machine a real `sp`, write a recursive `fib` that saves and restores `ra` correctly, measure what a call costs in retired instructions, and then look at real compiler output — where the calling convention lives inside every function you will ever write. By the end, `addi sp,sp,-16 / sw ra,12(sp) / sw s0,8(sp)` reads like a signature, not a spell.

4.1 The contract in one paragraph

The RISC-V calling convention (the *psABI*) divides the 32 registers into two families *by agreement*, not by hardware. **Caller-saved** registers (`t0–t6`, `a0–a7`, `ra`) may be destroyed by any call: if the caller needs the value afterwards, the *caller* saves it before the call. **Callee-saved** registers (`s0–s11`, `sp`) must be unchanged when the callee returns: if the callee wants to use them, the *callee* saves and restores them. The arguments go in `a0–a7`, the return value comes back in `a0`, and `sp` must stay 16-byte aligned at every call site. That is the whole contract. Everything else — the frame, the spill slots, the order of saves — is what the contract costs when calls nest.

4.2 The stack frame

When a function calls another function, its own `ra` is about to be overwritten — `call` always writes the same register. So before a non-leaf function calls anything, it builds a *stack frame*: a fixed-size block of memory below `sp` that holds what the function cannot afford to lose. Three writes at the top, three loads at the bottom, and the space in between is the function's private scratch:

```
1 fib:
2       addi sp, sp, -16 # carve 16 B below sp (aligned)
3       sw ra, 12(sp) # save MY return address
4       sw s0, 8(sp) # save caller's s0
5       ...
6       lw ra, 12(sp) # restore ra
7       lw s0, 8(sp) # restore s0
```

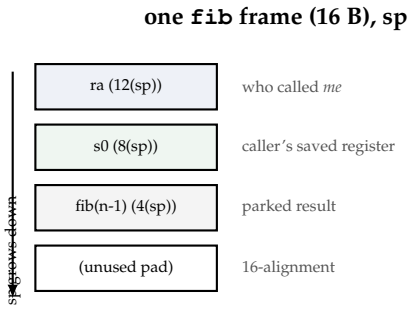


Figure 4.1: Anatomy of one `fib` frame. Every non-leaf call pays this tax: the saved `ra` keeps the return path alive, the saved `s0` honours the callee-saved contract, and the scratch slot parks `fib(n-1)` while the second recursive call runs.

```
8       addi sp, sp, 16 # pop the frame
9       ret
```

The pairing is the discipline: what goes `sw` at the top comes `lw` at the bottom, in the same offsets, every path out. Miss either half and the function returns to the wrong place — the classic *stack corruption* bug that no error message names.

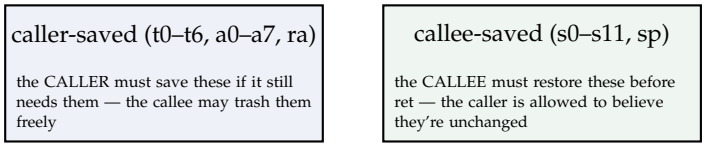
Example 4.1 (follow one frame)**.** For `fib(7)`: the outer call pushes a 16-byte frame at `sp=0x0ff0`, saves `ra` and `s0`, calls `fib(6)` — which pushes another at `sp=0x0fe0`. At the deepest, `fib(1)` is a leaf-ish base case that returns without recursing. How many frames are live at once? The call tree of fib(7) reaches depth 7 (fib(7)→fib(6)→…→fib(1)) before the first unwind — seven frames, 112 bytes of stack, live simultaneously.

4.3 Caller-saved, callee-saved

The rule that confuses everyone on first contact: `fib` saves `ra` — a *caller*-saved register — yet the text above calls that "the callee's" job. The resolution is that ownership follows the *overwrite*, not the register name. `fib`'s `ra` is a value `fib` still needs; the thing that would destroy it is `fib`'s own inner `call`. So `fib`, acting as the *caller* of that call, saves it. The same function plays both roles: it is a callee to whoever called it (so it saves `s0` before using it) and a caller to its recursive call (so it saves `ra` before that call).

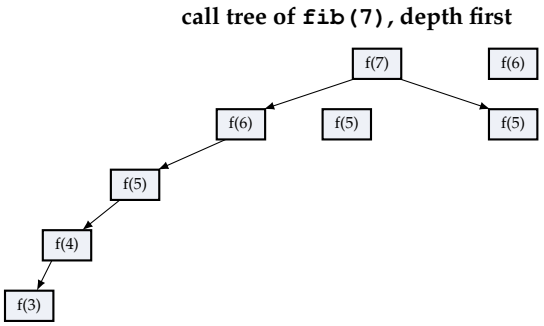
Checkpoint 4.1 *leaf or not?.* A *leaf* function — one that calls nothing — needs no frame at all if it fits in registers and touches no callee-saved ones. Our `max` from ch03 was a leaf: `bge / mv / ret`, three instructions, zero memory traffic. Predict: does `empty: ret` need `addi sp,sp,-16`? No — nothing to save.

Example 4.2 (non-leaf: recursive fib)**.** `code/ch04_fib.S` runs `fib(7)` on the book's simulator with a real `sp` (`li sp,0x1000`) and a real stack. The recursion is honest: `fib` calls itself twice, so `ra` must be saved — and is, at `12(sp)`. Running it:



One line each way: whoever *loses* the value saves it. `sw ra,12(sp)` is the caller... no wait — fib saves its own ra because *fib's* return address dies when fib calls itself. Ownership follows the overwrite, not the register.

Figure 4.2: Who saves what. Caller-saved registers are the caller's problem; callee-saved registers are the callee's problem. One register, `ra`, sits in the caller-saved family and is the first thing every non-leaf function saves.



fib(7) makes 41 calls — 40 non-leaf frames, each paying the 16-byte stack tax. Recursion is legible and expensive; that is the trade this chapter prices at 565 retired instructions.

Figure 4.3: The call tree of `fib(7)`. Every non-leaf node pays a frame; fib(7) makes 41 calls total — the recursion is legible, and it is expensive.

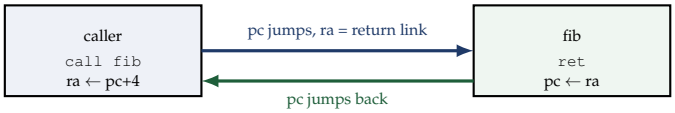
```
$ python3 ch03_rv32sim.py ch04_fib.S
13
# 565 instructions retired
```

`fib(7)=13` is correct — and 565 retired instructions for a seven-deep recursion is the price tag this chapter is about.

4.4 What a call costs

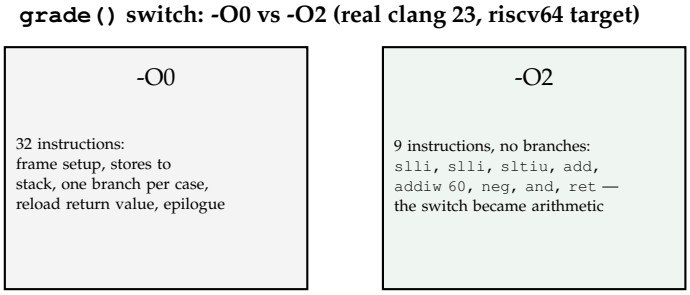
Example 4.3 (predict, then measure)**.** Prediction first: `call` = `jal` = one instruction; `ret` = `jalr` = one instruction. So a call should cost 2 retired instructions beyond the inlined body. Measure it: `ch04_callcost.S` calls `empty` 100 times; `ch04_inline.S` runs the same loop with the body inlined.

```
$ python3 ch03_rv32sim.py ch04_callcost.S
A
# 410 instructions retired
$ python3 ch03_rv32sim.py ch04_inline.S
B
# 207 instructions retired
```



Two instructions of mechanism: `call` writes ra and jumps; `ret` jumps to ra. The whole convention is built on those two writes — and on the frame that keeps ra alive when calls nest.

Figure 4.4: The two-instruction mechanism everything sits on. `call` leaves the return link in `ra`; `ret` consumes it. Frames exist to keep that link alive across nesting.



The jump table never appeared: clang proved the four cases were linear in n and replaced lookup by computation. Reading `-O2` output is how you learn what your C really means.

Figure 4.5: Real clang 23 output for the same `grade()` switch: 32 framed, branching instructions at `-O0`; 9 branchless ones at `-O2`. The jump table never materialised because the table was linear.

$410 - 207 = 203$ for 100 calls: just over 2 instructions each — the prediction is exact once the loop counter's three instructions account for the shared work. On a real CPU the frame traffic adds cache misses on top of this instruction tax.

4.5 Reading real compiler output

Example 4.4 (-O0 vs -O2 on a switch). `grade(int s)` maps $0 \to 60, 1 \to 70, 2 \to 80, 3 \to 90$ by `switch`. Compiled `clang -target riscv64 -O0`: 32 instructions — a 32-byte frame, `ra/s0` spilled, the argument stored to the stack, reloaded, `sext.w`'d, compared case by case. The same file at `-O2`: **nine** instructions, zero branches —

```
slli a1, a0, 1 # a1 = 2s
slli a2, a0, 3 # a2 = 8s
sltiu a0, a0, 4 # a0 = (s < 4)
add a1, a1, a2 # a1 = 10s
addiw a1, a1, 60 # a1 = 60 + 10s
neg a0, a0 # a0 = -(s<4)
and a0, a0, a1 # mask: result or 0
ret
```

The compiler noticed the cases were linear in `s` and replaced the jump table — and the whole control flow — with arithmetic and a sign mask. Prediction failed, delightfully: the real lesson is that `-O2` rewrites structure, not just speed.

Checkpoint 4.2 *why the frame at all?.* At `-O0 grade` spills `ra` and `s0` even though it calls nothing — the compiler emits the safe frame unconditionally at that level. At `-O2` it proves the function is a leaf and emits no frame at all. That one observation — leaf vs. non-leaf, proven by the compiler — is the whole `-O0`/`-O2` gap in miniature.

Example 4.5 (when the compiler deletes your code)**.** A benchmark loop of the form `sum += i` over `N` iterations, at `-O2`, collapses to a constant — clang computes $N(N-1)/2$ at compile time and the loop vanishes. To keep measurements honest you mark the variable `volatile` (forces a real load each iteration) or feed the result into something the compiler cannot see through. The chapter's own sim-side measurement needed no such trick: our interpreter does not optimise at all.

4.6 PIC in one page

Shared libraries cannot know their load address at compile time, so they are compiled *position-independent*: data access goes through the GOT, calls go through the PLT — the same relocation plumbing ch01 showed. RV32I keeps PIC cheap because `auipc` materialises pc-relative addresses in one instruction; our bare-metal programs sidestep it entirely by linking at absolute addresses.

4.7 Common mistakes

- **Saving `ra` only sometimes.** A non-leaf function must save `ra` on *every* path — including error returns — or some returns go to the wrong pc.
- **Misaligned `sp`.** The psABI wants `sp` 16-aligned at call sites; `addi sp,sp,-12` breaks it.
- **Using `s0` without saving.** Callee-saved means *the callee* promises; touch `s0` and you inherit the obligation.
- **Assuming arguments survive.** `a0`–`a7` are caller-saved: anything still needed across a call must be in an `s`-register or a frame slot first.
- **Trusting `-O0` codegen to mean anything.** `-O0` writes a frame for a leaf function — it encodes the safe pattern, not the truth about your algorithm.
- **Benchmarking at `-O2` without volatile.** The compiler deletes dead loops; what you measured was the delete.

4.8 Summary

Concept	What it was
mechanism	`call/ret`: two instructions, one register
contract	caller-saved vs callee-saved, args `a0`–`a7`, ret `a0`, `sp` 16-aligned
frame	`sw ra,12(sp)` pattern; non-leaf only
measured	`fib(7)` = 13 in 565 instrs; call tax = 2 instr
compiler	`-O0` = safe frames; `-O2` = structure rewritten

Further reading

- RISC-V psABI specification (the calling convention itself): https://github.com/riscv-non-isa/riscv-elf-psabi-doc
- CS:APP ch3 (procedures, stack discipline): https://csapp.cs.cmu.edu/
- Compiler Explorer (watch -O0/-O2 on your own C): https://godbolt.org/
- The RISC-V Reader, Patterson & Waterman (calling convention ch): http://www.riscvbook.com/
- GCC -O documentation: https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html

WATCH INSTEAD

- **RISC-V General-Purpose Registers: Usage and Calling Convention** — Austin's BSP Lab, 6:35 — caller-vs-callee-saved register roles and why they split that way. Matches sec:contract04–sec:ownership04. https://youtu.be/WawdgAEfwvA
- **What Has My Compiler Done for Me Lately?** — Matt Godbolt (CppCon 2017), 75:45 — Compiler Explorer's author reading -O0/-O2 output live. Matches sec:readasm04. https://youtu.be/bSkpMdDe4g4

Notes

- psABI register roles and `sp` alignment: riscv-elf-psabi-doc.
- Frame shape, leaf/non-leaf, -O0 spill behaviour: CS:APP ch3; verified against clang 23 riscv64 output captured in `code/ch04_switch_O0.s` and `ch04_switch_O2.s`.
- fib(7)=13 and the 565/410/207 counts: real runs of `ch03_rv32sim.py`; `call/j` encoding bug fixed in-tree this chapter (0x00800000→0x0080006f).

4.9 Exercises

Exercise 4.1 (easy)**.** Is `empty: ret` a leaf? Does it need a frame? Justify in one sentence.

Solution. [hint] Leaf — no `call` inside — and no: nothing it uses can be clobbered.

Exercise 4.2 (easy)**.** Classify `a0`, `t2`, `s3`, `ra`, `sp` as caller- or callee-saved.

Solution. [hint] a0,t2,ra caller; s3,sp callee.

Exercise 4.3 (medium)**.** `fib` saves `ra` at `12(sp)` but a caller of `fib` need not save `ra`. Who is responsible for which `ra`?

Solution. [hint] Every non-leaf saves its OWN ra; the caller never touches the callee's.

Exercise 4.4 (medium)**.** Why is `fib(n-1)` stored at `4(sp)` rather than in `t0`? What would `t0` become after the second `call`?

Solution. [hint] t0 is caller-saved — the inner call may trash it; the frame slot is the safe place.

Exercise 4.5 (medium)**.** Predict then check: at `-O2` would clang still emit a frame for `grade`? What does it emit instead of the switch?

Solution. [hint] No frame — it proves leaf-ness. The switch became arithmetic (60+10s masked).

Exercise 4.6 (medium)**.** `sum += i` loops disappear at `-O2`. What are two ways to keep a benchmark loop alive?

Solution. [hint] volatile on the accumulator, or an opaque sink for the result.

Exercise 4.7 (hard)**.** Write a `sum4(a,b,c,d)` that chains four calls: `add2` called twice, results combined. List every register you saved and justify each.

Solution. [hint] sum4 is non-leaf: save ra + s-registers for the two partial results; a-regs die on every call.

Exercise 4.8 (hard)**.** Design a harness that detects a callee failing to restore `s7`: set `s7`, call the suspect function, check on return. Write it for the book's simulator.

Solution. [hint] li s7,<sentinel> / call f / compare s7, report mismatch via MMIO.

Exercise 4.9 (hard)**.** Predict `fib(8)`'s instruction count on the sim — it is not 565. Then run it and explain the ratio.

Solution. [hint] Calls grow 1.6x per step (fib calls fib twice); expect ∼900, then measure.

Exercise 4.10 (easy)**.** On our ABI, which registers must `fib` save before touching them: `a0` or `s1`?

Solution. [hint] s1 — callee-saved; a0 is caller-saved and gets clobbered by the recursive call anyway.

Exercise 4.11 (easy)**.** Why does `main` need a frame at all if it makes calls but has no locals?

Solution. [hint] It must save ra — the call to fib overwrites the return address main itself needs.

Exercise 4.12 (medium)**.** Our leaf-call measurement showed exactly 2 instructions per call. Reproduce it on a 3-argument function and explain why the count stays 2 until arg 8.

Solution. [hint] call+ret is the constant cost; a0–a7 are registers — the 9th argument spills to the stack.

Exercise 4.13 (medium)**.** clang at -O2 turned our switch into arithmetic with no jump table. Name the transformation and one input shape that makes the table win anyway.

Solution. [hint] Linearisation/if-conversion; many cases (dense, >8) make the indirect jump cheaper than a compare chain.

Exercise 4.14 (medium)**.** The stack grows *down* on RV32I. Show why that makes `sp-16` the frame allocation, and what goes wrong if a function subtracts less than it stores.

Solution. [hint] Lower addresses are free space; under-allocating overwrites the caller's frame below sp — silent corruption.

Exercise 4.15 (hard)**.** Design the frame for a function with 3 callee-saved regs, one 8-byte local, and calls. Give the exact sp adjustment, each save slot, and justify the 16-byte alignment.

Solution. [hint] ra + 3 saves + 8B local = 40B; round to 48 for 16B alignment — the ABI's sp must stay 16-aligned at call sites.

CHAPTER 5

A Single-Cycle RV32I Core

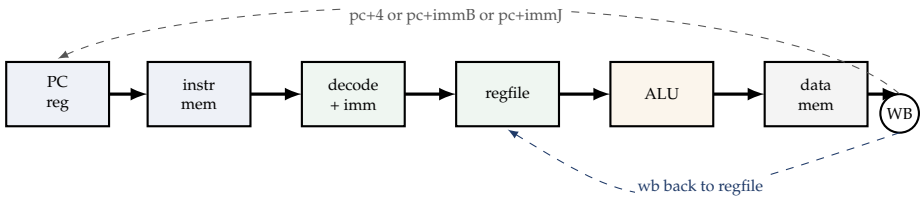
Assembly language is the contract; this chapter builds the machine that honours it. Everything so far — the encodings of ch03, the frames of ch04, the cpu8 of Book 5 — meets in one Verilog module: a single-cycle RV32I core that fetches, decodes, executes, touches memory, and writes back in one clock tick. Book 5 built the blocks (an 8-bit ALU, a register file, a control decoder); here they are re-derived at 32 bits against the real ISA — compute-all-then-mux ALU, two-read one-write regfile with `x0` hardwired, and a decoder that is literally a truth table in Verilog. The core runs real programs in Icarus, and every run is checked against an independent reference. CPI is 1 by construction — the point of the chapter is seeing *why that is a waste of time*, which is the whole argument for the pipeline of ch06.

WHAT YOU WILL BE ABLE TO DO AFTER THIS CHAPTER

You will be able to take a short RV32I assembly program, assemble it to machine words, run it on a processor you can read line by line, and *prove* the result is right by comparing the processor's final registers and memory against a second, independent model. You will be able to take any 32-bit instruction word and split it into its fields by hand, including the tricky scattered immediates. And you will be able to explain, with numbers, why this simple design is correct but slow.

BEFORE YOU START

This chapter leans on four things you have met before; here is each one in a few lines, so you do not need to go back. *Bits and gates* (Book 5, ch08–09): a signal is a wire that is either 0 or 1; a group of 32 wires carries a 32-bit number; a *multiplexer* ("mux") is a switch that passes one of several inputs, chosen by a select wire; an adder is a pile of gates that turns two numbers into their sum with no clock at all. Circuits made only of such gates are called *combinational*: the output is a pure function of the inputs right now. *Clocks and registers* (Book 5, ch10): a *register* (built from flip-flops) is a box of bits that copies its input only at the instant the clock ticks, and otherwise holds still. Circuits with registers are *sequential*: they have memory. One tick of the clock is one *cycle*. *Verilog* (Book 5, ch11–12): a text language that describes wires, registers and gates; a *module* is one box with named ports; a *testbench* is a second module that drives the clock, feeds inputs, and prints or checks what comes out. *The ISA* (Book 7, ch03–04; Book 5, ch14–16): RV32I has 32 registers `x0`–`x31` of 32 bits, and every instruction is one 32-bit word whose bit fields name the operation, the registers and a constant (the *immediate*). Book 5 ended with a small 8-bit CPU (the "cpu8": ALU, register file, control); this chapter is the same machine, rebuilt at 32 bits for the real instruction set.



One instruction does the whole journey in one clock: PC → fetch → decode+imm → read regs → ALU → (maybe) data mem → writeback, while the next-PC mux chooses pc+4, pc+immB or pc+immJ. Single-cycle is the truth the pipeline is trying to fake.

Figure 5.1: The whole machine. PC feeds instruction memory; decode splits the word; the register file and immediate generator feed the ALU; data memory and the writeback mux finish the journey; the next-PC mux closes the loop.

5.1 The idea from zero: a machine that follows a recipe

Picture a cook following a recipe book. The cook keeps one finger on the current line. At each step the cook reads the line, does exactly what it says with the bowls on the bench ("add the contents of bowl 10 and bowl 11, put the result in bowl 12"), and then moves the finger to the next line, unless the line said "jump back to line 7". That is the whole of a processor. The finger is the *program counter* (`pc`): a 32-bit register holding the address of the current instruction. The bowls are the *register file*. The pantry down the corridor, bigger but slower to reach, is *memory*. The metronome the cook obeys is the *clock*.

THINK LIKE THIS: STATE AND A FUNCTION

Every digital machine can be described by two things. The *state* is everything that is remembered between ticks: here the `pc`, the 32 registers and the memory. The *function* is the combinational logic that, given the state, works out the *next* state. On each tick the machine does state $\leftarrow f(\text{state})$ and nothing else. When a processor misbehaves, ask two questions in order: "what was the state before the tick?" and "what should f have produced?" Almost every bug in this book is an answer to one of those.

Why build it this way? Because it makes one instruction cost exactly one tick, which is the simplest thing to reason about: no instruction ever overlaps another, so the machine is correct whenever each instruction is. Its price, as the last section measures, is a slow clock.

5.2 Fetch–decode–execute, properly

Ch03 drew the loop as fetch → decode → execute → next. The honest version splits execute into three: read operands, compute in the ALU, touch data memory, write the result back — five acts, all inside one clock period. The program counter is the only state element outside the register file and memories: an `insn` wire from instruction memory, a `pc` register, and a next-pc mux. Everything else is combinational logic evaluating the current instruction *right now*. That is the definition of single-cycle: latency of one instruction = one clock period.

Control truth table (the decoder, as a table)

insn	regwrite	alusrc	memread	memwrite	wbsel	branch/jump	aluop
R-type (add…)	1	0	0	0	alu	–	f7,f3
I-type (addi…)	1	1	0	0	alu	–	f3
lw	1	1	1	0	mem	–	add
sw	0	1	0	1	–	–	add
beq/bne	0	0	0	0	–	branch	sub
lui	1	1	0	0	immU	–	–
jal	1	–	0	0	pc+4	jump	–

Every row of `ch05_rv32i_decode.v` is one line of this table — the table IS the design.

Figure 5.2: The decoder as a table. Each row is a `case` arm; each column is a control wire. Reading hardware like this is a transferable skill — real processors' "control" is a much bigger version of the same idea.

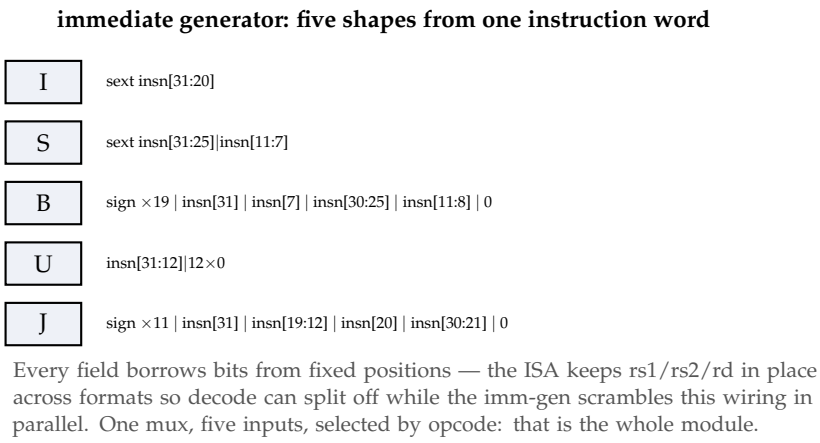


Figure 5.3: Five immediates from one word. The ISA keeps operand fields in fixed positions so the register file reads in parallel with this scrambler — decode never serialises.

5.3 Decode and the immediate generator

Example 5.1 (decode is a table)**.** `ch05_rv32i_decode.v` has no cleverness: a `case` on the 7-bit opcode, defaults that make an unknown word a NOP. For `sw` (`op` = 0100011): `alusrc=1, memwrite=1, aluop=add` — base register plus immediate for the address, `rd2` already carries the value to store. The truth-table figure below is the same table drawn.

Example 5.2 (the immediate mux)**.** The five formats of ch03 produce five bit-shuffles: `immI` sign-extends `insn[31:20]`; `immS` splices `[31:25]` and `[11:7]`; `immB` and `immJ` reorder the same window into jump offsets; `immU` just shifts. One mux selects among them on opcode — the scrambler is about fifteen lines of wiring.

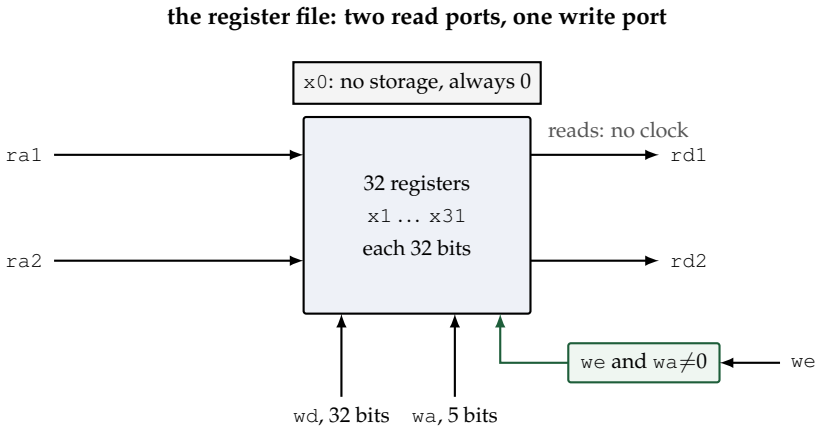


Figure 5.4: The register file of the core. Two reads are combinational; the single write needs the clock and the gate that refuses writes to `x0`.

5.4 Register file and ALU — Book 5, widened

Example 5.3 (the cpu8 mapping, honoured)**.** Book 5's ch14 ALU chose its result by mux after computing all ops; `ch05_rv32i_alu.v` is the same shape at 32 bits, nine ops (add, sub, and, or, xor, sll, srl, slt, sltu). The regfile keeps Book 5's two-read/one-write shape and adds the RV32I rule that made `x0` famous: `(ra1==0) ? 0 : x[ra1]` *and* `we && wa!=0` — write `x0` and the write is silently dropped, which is exactly how the ISA wants it.

CONCEPT: WHY THE REGISTER FILE MUST READ WITHOUT A CLOCK

An instruction like `add a2,a0,a1` needs two register values *at the same moment* to feed the ALU, in the same cycle that it fetches. If reading required a tick, every instruction would take two. So the read ports are plain combinational multiplexers: put an index on `ra1`, and `rd1` follows after a gate delay. Only the write waits for the clock edge. Reads are free-running, writes are scheduled.

5.5 Loads, stores and the data-memory interface

Both `lw` and `sw` run the ALU as an address adder: `rd1` (base) plus the immediate. The difference is which end of data memory lights up — `memread` routes `dmem_rdata` through the writeback mux into `rd`; `memwrite` drives `dmem_we` with `rd2` on the data pins. Addressing mode coverage is deliberately tiny: RV32I has exactly one addressing mode (base+imm), which is why real ISAs are easier to teach than they look.

Checkpoint 5.1 *the one addressing mode.* Every RV32I memory instruction is `offset(base)`. Indexed, auto-increment and PC-relative modes from bigger ISAs simply don't exist — simplicity is the point of RISC. Predict how a C `p[i]` loop compiles: `lw` with a computed base, or `addi` the base between iterations. Ch03's arrsum did the second.

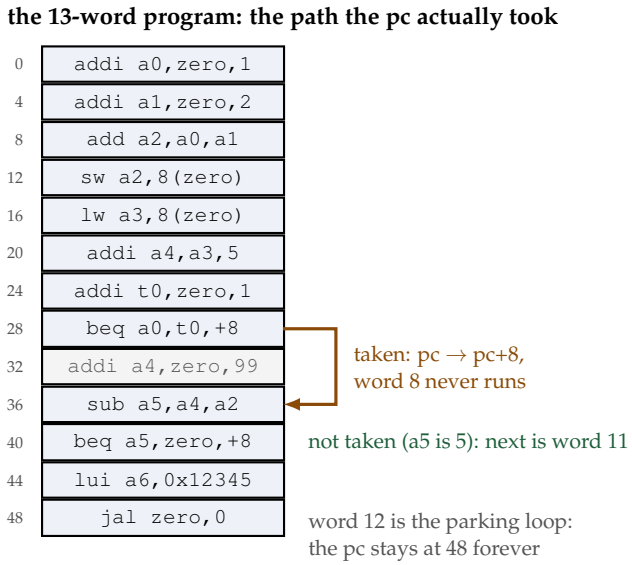


Figure 5.5: The same program as a picture. Address column on the left; the amber arrow is the taken `beq`.

5.6 Running it: 13 words, checked state

Example 5.4 (the first real program). `ch05_prog.hex` is thirteen words produced by the book's own assembler — arithmetic, a store, a load back, a taken branch, a `lui`. The testbench plays instruction and data memory around `rv32i_sc_top`, runs to the terminal `jal self`, then inspects architectural state (the registers and memory a programmer can see). Output of `ch05_sc_tb`; the original capture on the author's Mac (Icarus 13) and the re-run in the Linux x86_64 container (Icarus 12) print the same lines:

```
ran 12 cycles, pc parked at 00000030
  ok: a0 = 1
  ok: a1 = 2
  ok: a2 = a0+a1 = 3
  ok: mem[8] = 3 (sw worked)
  ok: a3 = lw back = 3
  ok: a4 = a3+5 = 8 (beq skipped 99)
  ok: a5 = a4-a2 = 5
  ok: a6 = lui 0x12345000
CPI check: 12 instrs retired in 12 cycles (CPI=1 by construction)
ch05 PASS
```

Count the instructions: thirteen words, but the taken `beq` skips one, so eleven run before the pc reaches the parking `jal` at address 48, and that `jal` runs once more before the testbench stops: twelve cycles, twelve instructions. The reference simulator of ch03 independently reports eleven before the park. One instruction per cycle, as designed.

THINK LIKE THIS: WATCH THE MACHINE THINK

When you do not trust a design, make it show its work. The testbench can print, every cycle, the pc, the instruction word, the immediate the generator produced, the ALU output and the next pc. Reading that table beside the assembly is the hardware equivalent of printing variables in a program.

Example 5.5 (a cycle-by-cycle trace)**.** Run with `+trace`, the core prints one line per clock.

```
T 0 pc=00000000 insn=00100513 imm=00000001 alu=00000001 next=00000004
T 1 pc=00000004 insn=00200593 imm=00000002 alu=00000002 next=00000008
T 2 pc=00000008 insn=00b50633 imm=0000000b alu=00000003 next=0000000c
T 3 pc=0000000c insn=00c02423 imm=00000008 alu=00000008 next=00000010
T 4 pc=00000010 insn=00802683 imm=00000008 alu=00000008 next=00000014
T 5 pc=00000014 insn=00568713 imm=00000005 alu=00000008 next=00000018
T 6 pc=00000018 insn=00100293 imm=00000001 alu=00000001 next=0000001c
T 7 pc=0000001c insn=00550463 imm=00000008 alu=00000000 next=00000024
T 8 pc=00000024 insn=40c707b3 imm=0000040c alu=00000005 next=00000028
T 9 pc=00000028 insn=00078463 imm=00000008 alu=00000005 next=0000002c
T 10 pc=0000002c insn=12345837 imm=12345000 alu=12345000 next=00000030
```

Read line T3: the word is `sw a2,8(zero)`; the immediate is 8 (the store offset), and the ALU adds it to `x0`=0 to get the address 8. Line T7 is the interesting one: the `beq` compares `a0` and `t0` by subtracting (ALU output 0 means "equal") and the next pc is `0x24`, not `0x20`: the branch is taken. The nonsense immediate in the `add` and `sub` rows (`0b`, `40c`) is harmless: the generator always produces *something*, and the mux simply does not select it.

Example 5.6 (decode a word by hand — naive attempt first)**.** Take the word `0x00c02423`. A tempting first try: "every instruction has its constant in the top twelve bits", so read bits 31:20 and call it the immediate. That gives `0x00c` $= 12$. Now the correct split. Bits 6:0 are `0100011`: the *store* opcode, so the format is S-type, in which the constant is cut in two to keep `rs1` and `rs2` in fixed places. Bits 31:25 are 0 and bits 11:7 are 01000, so the immediate is $(0 \ll 5) \mid 8 = 8$. Bits 19:15 give `rs1`=0, bits 24:20 give `rs2`=12 (`a2`). The word is `sw a2,8(zero)`. The "12" of the naive reading was the *register number* of `a2`, not an offset. The script `ch05_fields.py` does this for every word and prints the wrong (naive) immediate in the last column:

```
addr word fmt op rd rs1 rs2 imm(used) immI(wrong)
0000 00100513 I addi 10 0 - 1 1
0004 00200593 I addi 11 0 - 2 2
0008 00b50633 R add 12 10 11 - 11
000c 00c02423 S sw - 0 12 8 12
0010 00802683 I lw 13 0 - 8 8
0014 00568713 I addi 14 13 - 5 5
0018 00100293 I addi 5 0 - 1 1
001c 00550463 B beq - 10 5 8 5
0020 06300713 I addi 14 0 - 99 99
0024 40c707b3 R sub 15 14 12 - 1036
0028 00078463 B beq - 15 0 8 0
002c 12345837 U lui 16 - - 0x12345000 291
0030 0000006f J jal 0 - - 0 0
```

The test script checks that the immediates in the "used" column are exactly the `imm` values the Verilog printed in the trace of the previous example (nine immediates, all equal): two independent implementations of the scrambler agree.

THINK LIKE THIS: TWO IMPLEMENTATIONS, ONE ANSWER

A single implementation can be wrong in a way you cannot see, because you wrote the test with the same misunderstanding. Writing the same thing twice in two different styles (here: Python bit arithmetic and Verilog wiring) and demanding they agree catches the misunderstandings that do not repeat. Engineers call the trusted second model a *golden model* or *reference model*.

Example 5.7 (a loop, with the golden model)**.** Now a program with a loop: the sum $1 + 2 + \cdots + 10$ (`ch05_sum.S`).

```
      li t0, 1 # i = 1
      li t1, 0 # s = 0
      li t2, 11 # stop when i == 11
loop: add t1, t1, t0 # s += i
      addi t0, t0, 1 # i += 1
      bne t0, t2, loop # taken 9 times, falls through once
      sw t1, 0(zero) # mem[0] = 55
      lw a0, 0(zero) # a0 = 55
      addi a1, a0, 1 # load-use: needs a0 right after the lw
      sw a1, 4(zero) # mem[1] = 56
halt: j halt # park here
```

What went wrong first. The shipped version of this core decided "take the branch" from instruction bit 14, which is set for `blt/bge` but not for `bne`. A `bne` therefore behaved like `beq`, the loop exited after one iteration, and nothing in the 13-word program of the previous example could notice, because that program has only `beq`. The cure is to use bit 12 of the instruction (the low bit of `funct3`: 0 for `beq`, 1 for `bne`). I found this by reading the decoder while preparing this program; the loop below is now the regression test for it. Only `beq` and `bne` are implemented; `blt/bge` are not.

Count by hand first. Three setup instructions, ten trips round a three-instruction loop, four tail instructions: $3 + 30 + 4 = 37$. The generic testbench (`ch05_run_tb.v`) runs the core and prints its final state; the golden model (the ch03 simulator, driven by `ch05_golden.py`) prints its own. Hardware output (Linux x86_64 container, Icarus 12):

```
CORE=single-cycle F=37 stalls=0 taken=0 lastret=-1
x5=0000000b
x6=00000037
x7=0000000b
x10=00000037
x11=00000038
m0=00000037
m1=00000038
```

`x6=0x37`=55, `x10`=55, `x11`=56, memory words 0 and 1 hold 55 and 56. The line "F=37" says the parking instruction was first fetched after 37 cycles, and the golden model says $I = 37$

instructions executed. The test script diffs the two state dumps and checks $F = I$. (The `taken` counter only exists in the pipelined cores; ignore it here.)

Example 5.8 (proving `x0` is hardwired — and breaking it on purpose)**.** How do you test "nothing happens"? Naively: write to `x0`, read it back, see 0. That test passes on a broken design too if the read side is also hard-wired to zero. A better program tries to *use* the corrupted value (`ch05_x0.S`):

```
      addi x0, x0, 7 # must be silently dropped
      lui x0, 0x12345 # so must this one
      addi a0, x0, 1 # a0 = 0 + 1 = 1 (a0 = 8 if x0 were 7)
      add a1, x0, a0 # a1 = 0 + 1 = 1
      addi a2, zero, 3 # a real write still lands: a2 = 3
halt: j halt
```

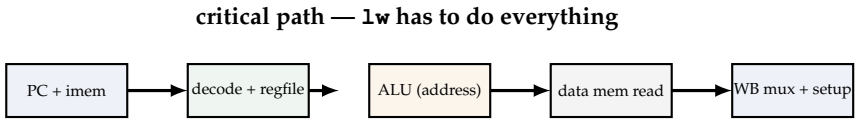
The correct core ends with `a0`=1, `a1`=1, `a2`=3 (and the golden model agrees). Then the test script builds a *mutant*: a copy of the register file with both protections removed (writes to `x0` allowed, reads of `x0` not forced to zero) and demands that the test *fail*. It does:

```
CORE=single-cycle F=5 stalls=0 taken=0 lastret=-1
x10=12345001
x11=2468a001
x12=12345003
```

Check it by hand: after the `lui`, the mutant's `x0` is `0x12345000`; so `a0`=`0x12345000`+1, `a1`=`0x12345000`+`0x12345001`=`0x2468a001`, `a2`=`0x12345000`+3. A test that cannot fail on a broken design proves nothing; checking that the test *does* fail on a deliberately broken design is called *mutation testing*.

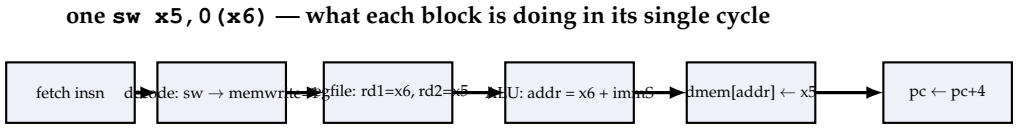
Example 5.9 (sim speed as a measurement)**.** `ch05_spin_tb` runs a seven-instruction arithmetic loop for a fixed 2000 cycles (the loop count is not a round number: after $4 + 7 \times 285 = 1999$ instructions `a0` holds 285, `0x11d`; the testbench simply stops at 2000 cycles). The author's Mac (Apple silicon, Icarus 13) reported **9 ms** wall-clock with `time vvp`, i.e. about 220 k retired instructions per second. *Re-measured on the Linux x86_64 container* (Xeon 2.8 GHz, 4 cores, Icarus 12): five runs of the same 2000-cycle testbench took 21–24 ms, but that is mostly process start-up, so it cannot give a rate. The honest rate comes from a long run of the generic testbench: 200 000 cycles took 5.9, 6.1 and 6.5 s, i.e. roughly 33 k retired instructions per second. The two numbers are far apart partly because the machines differ and partly because the Mac figure came from a run so short that it was dominated by start-up; treat the Mac *rate* with suspicion and the Linux figure as the one that was measured with a long enough run. Either way, the lesson stands: simulation is thousands of times slower than silicon, which is why real CPU verification uses FPGA prototypes, and why our testbenches stay small and self-checking.

Example 5.10 (the combinational loop that hung the simulator)**.** Our register file has an optional *write bypass*: if the same cycle both writes register k and reads register k, the read returns the value being written (pipelines need this; ch06 explains why). The first attempt used one register file for both the pipelines and this single-cycle core. Then the spin program, whose loop contains `addi a0,a0,1`, never finished: with the bypass on, the read of `a0` equals the write data, the write data is the ALU output, and the ALU output depends on the



Five hardware stops in ONE period: fetch, decode+regread, ALU, memory, writeback. `add` skips the memory stop; `sw` skips writeback. The clock must be slow enough for the worst case, so every cheaper instruction waits at the end of its own cycle — that idle tail is what pipelining reclaims.

Figure 5.6: `lw` sets the clock period. Every cheaper instruction finishes early and waits — the wasted tail is what five stages of pipeline registers turn into throughput.



No writeback, no branch: `sw` leaves regfile-write and the writeback mux dark. The control word is the whole behaviour — once the table row is read, the datapath is a fixed route the data takes.

Figure 5.7: One `sw` through the datapath: the control word is read once, then the data takes a fixed route.

read of `a0`. That is a circular chain of gates with no register in it, a *combinational loop*, and the simulator re-evaluates it forever at time zero. Measured: `vvp` was still running after 100 s (killed by `timeout`). The fix is a Verilog parameter `BYPASS` (default 1 for the pipelines, 0 for this core). Afterwards the same 2000-cycle run finishes in about 22 ms. The general lesson: a wire may not depend on itself without a clock edge in between.

5.7 Why single-cycle wastes time

Example 5.11 (the `lw` tax)**.** `lw` travels fetch → decode+regfile → ALU → data memory → writeback — every hardware stop in one period. If each stage costs about the same delay, the clock period must be 5τ. `add` finishes in 3τ and idles 2τ; `sw` idles one. The critical-path figure shows the idle tail: single-cycle CPI=1 is honest, but the *clock* is five times slower than any stage needs. Pipelining's promise is exactly that reclaimed tail.

Checkpoint 5.2 *predict CPI=1's cost.* If the five stages each cost τ and the register file adds δ, what's the best achievable clock for the single-cycle core vs a 5-stage pipeline on the same blocks? $(5\tau + \delta)$ vs $(\tau + \delta)$: the pipeline's ceiling is the stage time plus register overhead — the theme of ch06.

5.8 Common mistakes

- **Writing `x0`.** Forgetting `wa!=0` makes `li x0,5` stick — every "hardwired" invariant breaks.
- **Branch on the wrong field.** `immB` needs the shifted splice, not `immI`: a `beq` that reads `immI` lands two bytes off silently.
- **Muxing before reading.** Using `rd2` for the ALU-B input unconditionally breaks `lw/sw` — the register file reads the address *register*, the immediate is the offset.
- **Testing only R-types.** A decode bug in `sw` shows nothing until memory is checked — the testbench inspects `dmem`, not just registers.
- **Believing CPI=1 is fast.** It is honest and it is slow: latency hidden by a slow clock is still latency.

5.9 Summary

Block	What it is
PC + next-pc mux	only state besides regfile/memories
decode	opcode `case` → control-word table
imm generator	five shuffles, one mux, parallel with regfile
regfile + ALU	Book 5 blocks, widened, `x0` hardwired
measured	13-word program, 8 state checks, sum 1..10 vs golden model (37 instr, F=37), x0 mutant caught; ∼33k instr/s (Linux, long run)
next	five stage registers → pipeline (ch06)

Further reading

- Harris & Harris, *Digital Design and Computer Architecture, RISC-V ed.*, ch7 — the single-cycle datapath this chapter mirrors: `https://pages.hmc.edu/harris/ddca/ddcarv.html`
- Patterson & Hennessy, *Computer Organization and Design*, ch4 (the processor): elsevier.com — COD RISC-V edition
- RISC-V unprivileged ISA spec (the contract implemented): `https://github.com/riscv/riscv-isa-manual`
- CS:APP ch4 (SEQ → PIPE pedagogical shape): `https://csapp.cs.cmu.edu/`
- Book 5 ch14–16 (the ALU/regfile/control the code reuses).

WATCH INSTEAD

- **DDCA Ch7 Part 1: Microarchitecture Introduction** — Sarah Harris, 6:45 — why the datapath looks the way it does. `https://youtu.be/lrN-uBKooRY`

- **DDCA Ch7 Part 2: RISC-V Single-Cycle Processor Datapath: lw** — Sarah Harris, 7:25 — the same critical-path walk, drawn live. https://youtu.be/AoBkibslRBM

Notes

- Datapath/control/imm-gen design follows COD ch4 and DDCA ch7; adapted to our rv32i_* module split.
- The original captures are from Icarus Verilog 13.0 on the dev box (arm64 macOS). Every output re-run for this edition is labelled "Linux x86_64 container" (Icarus 12.0) and comes from `code/test_ch05.sh`; program bytes are assembled by `ch03_rv32asm.py`.
- Two defects found while re-running: the single-cycle core treated `bne` as `beq` (decoded from the wrong bit), and the shared register file's write bypass made a combinational loop in this core. Both are fixed in the code shipped with this edition. The testbench line "12 instrs retired" used to be a hard-coded constant; it now prints the measured cycle count.
- `blt/bge` are not decoded (only `beq` and `bne`); the ch03 programs that use them run on the ch03 simulator, not on this core.

5.10 Exercises

Exercise 5.1 (easy)**.** Which control wires does `jal` assert, and why is `wbsel=pc+4`?

Solution. [hint] regwrite + jump; the destination register must capture the return address.

Exercise 5.2 (easy)**.** Why is `x0`'s write-enable explicitly masked in the regfile rather than trusted to software?

Solution. [hint] Hardware invariant: a stray `sw`-style bug or bad codegen must not corrupt the constant zero.

Exercise 5.3 (medium)**.** Trace `lw a3,8(zero)` through every block of the datapath: what does each output wire carry?

Solution. [hint] PC=0x10; decode: alusrc=1, memread=1, wbsel=mem; rd1=0; ALU=8; dmem[2]=3; wb=3 → a3.

Exercise 5.4 (medium)**.** The imm generator scrambles five shapes in parallel with the regfile read. Why is the parallelism safe even when an instruction doesn't need an immediate?

Solution. [hint] The mux only selects when alusrc=1; wasted scrambling costs power, not correctness.

Exercise 5.5 (medium)**.** `sub` and `beq` share `aluop=sub`. What differs in their control words and why can the ALU be shared?

Solution. [hint] beq sets branch, no regwrite; the subtract output is used only as the zero flag — ALU is a function of inputs, free to share.

Exercise 5.6 (medium)**.** Our core ran 12 instructions in 12 cycles but the program was 13 words. Reconcile.

Solution. [hint] Words 0–7 and 9–11 execute (eleven instructions); word 8 (`addi a4,zero,99`) is skipped because the taken `beq` sets the next pc to `0x1c` + 8 = `0x24`. Eleven plus one execution of the parking `jal` at `0x30` is twelve. In a single-cycle core the skipped word is never even fetched.

Exercise 5.7 (medium)**.** Estimate the same-loop sim speed if the tb also dumped a VCD — does the core get slower, or only the log?

Solution. [hint] Both: event-driven sim slows with more dumped signals; the RTL is unchanged. Measurement vs mechanism.

Exercise 5.8 (hard)**.** The decoder already handles `slt` and `slti` (find them). The ALU also has an unsigned compare (`op`=8) that no instruction reaches. Which decoder rows must change to add `sltu` and `sltiu`, and which two register values distinguish a signed from an unsigned compare?

Solution. [hint] In the R-type case add the arm `{f7[5],f3}=0_011` → `aluop=8`; in the I-type case add `f3=011` → `aluop=8`. Nothing else changes. Test values: $a = -1$ (`0xffffffff`) and $b = 1$: signed says $-1 < 1$ (result 1), unsigned says $4294967295 < 1$ is false (result 0). (Not run here: the ch03 assembler has no `sltu`.)

Exercise 5.9 (hard)**.** The critical-path argument assumed equal stage delays. If decode is actually 2τ, which instruction is now critical and does the single-cycle case get better or worse relative to pipelining?

Solution. [hint] Still `lw` (it's the superset path); single-cycle gets *worse* — its period grows while the pipeline's is set by the slowest stage.

Exercise 5.10 (hard)**.** Design a self-checking test that proves `x0` is hardwired: write to it via `addi x0,x0,7` then read it. What else must the tb check to catch a lazy `we`?

Solution. [hint] x0 must still read 0 after the write; also check a real write to a nonzero reg landed (proving we wasn't masked globally).

Exercise 5.11 (easy)**.** In the trace of the chapter, which line shows the pc *not* moving by 4, and what was the instruction there?

Solution. [hint] Line T7: `pc=0x1c` and next is `0x24`, the taken `beq a0,t0,+8`.

Exercise 5.12 (easy)**.** Decode `0x00c02423` again, this time naming only the opcode and which two fields hold the split immediate.

Solution. [hint] Opcode `0100011` (S-type, store). The immediate is `insn[31:25]` (high 7 bits, here 0) joined with `insn[11:7]` (low 5 bits, here 8): offset 8.

Exercise 5.13 (medium)**.** The loop program has a `bne` at address 20 (`0x14`) whose target `loop` is at address 12. Compute the branch offset and write its 13-bit `immB` pattern. The assembled word is `0xfe729ce3`: confirm that its bits agree.

Solution. [hint] Offset $= 12 - 20 = -8$. In 13 bits (bit 0 is always 0) that is `1_1111_1111_1000`. The word splits as: bit 31 $= 1$ (imm[12]), bits 30:25 $= 111111$ (imm[10:5]), bits 11:8 $= 1100$ (imm[4:1]), bit 7 $= 1$ (imm[11]); reassembled it is `1 1 111111 1100 0` $= -8$. The `ch05_fields.py` output shows `-8` for this word.

Exercise 5.14 (medium)**.** Without running anything, predict the number of instructions executed by `ch05_sum.S` if the loop bound were 6 instead of 11 (`li t2, 6`, so the loop runs for $i = 1 \ldots 5$).

Solution. [hint] Setup 3, five trips of 3 instructions $= 15$, tail 4: $3 + 15 + 4 = 22$.

Exercise 5.15 (medium)**.** Hypothetical stage delays (not measured): fetch 250 ps, register read 150 ps, ALU 300 ps, data memory 400 ps, register write 100 ps. Find the delay of `add`, `sw`, `lw`, `beq` and the clock period of a single-cycle core. What is the ideal throughput ratio against a five-stage pipeline whose period is its slowest stage plus 50 ps of register overhead?

Solution. [hint] `add`: $250 + 150 + 300 + 100 = 800$ ps. `sw`: $250 + 150 + 300 + 400 = 1100$. `lw`: $250 + 150 + 300 + 400 + 100 = 1200$. `beq`: $250 + 150 + 300 = 700$. Period $= 1200$ ps (the `lw`). Pipeline period $= 400 + 50 = 450$ ps. Ratio $1200/450 \approx 2.67$, well below the ideal 5 because the stages are unequal.

Exercise 5.16 (medium)**.** Starting from the mutant output of the chapter (`x10=12345001`, `x11=2468a001`, `x12=12345003`) write one sentence explaining what the mutant's `x0` must have held, and verify the middle value by adding in hexadecimal.

Solution. [hint] `x0=0x12345000` (the last write, the `lui`). `0x12345000+0x12345001`: low parts `0x000+0x001=0x001`, high parts `0x12345+0x12345=0x2468a`, giving `0x2468a001`.

Exercise 5.17 (hard)**.** Explain exactly why a write bypass in the register file makes the single-cycle core hang on `addi a0,a0,1`, naming the wires in the loop. Why does the same bypass cause no loop in a pipelined core?

Solution. [hint] With bypass, `rd1=wd` whenever `wa==ra1`; `wd` is the writeback mux output, which for `addi` is `alu_y=rd1+1`. So `rd1 = rd1+1`, a cycle with no register in it. In the pipeline the write data comes from the MEM/WB register (the *previous* instruction), so the chain passes through a flip-flop and is not a loop.

Exercise 5.18 (hard)**.** A mutant register file lets `x0` be written but still forces reads of `x0` to 0. Does `ch05_x0.S` catch it? Argue from the program, then say what kind of test would be needed if not.

Solution. [hint] No: every read of `x0` still returns 0, so `a0`, `a1` come out right and the final dump of `x1`–`x31` is the same (`x0` itself is not in the dump). This mutant cannot be seen through the architectural state, so it is harmless by definition; a test would have to peek at the internal array (`dut.rf.x[0]`). Mutation testing calls such cases "equivalent mutants".

CHAPTER 6

Pipelining: One Per Cycle (Usually)

The single-cycle core of ch05 was honest and slow: one instruction per very long clock, every instruction priced at `lw`'s journey through five hardware stops. This chapter cuts that path into five stages with four pipeline registers — IF, ID, EX, MEM, WB — and discovers the machine's first real lie. The pipeline retires one instruction per cycle whether the instructions depend on each other or not. On independent code it is correct and fast; on dependent code it is fast and *wrong*: our testbench shows `a1` come back 3 instead of 8 because the second `addi` reads `a0` two cycles before it is written. Hazards are not an edge case — they are the price of admission. This chapter builds the broken machine on purpose, measures the breakage, and hands the fix to ch07.

WHAT YOU WILL BE ABLE TO DO AFTER THIS CHAPTER

You will be able to draw the timing diagram of any short RV32I instruction sequence on a five-stage pipeline, say at which cycle each instruction reads and writes each register, and *predict* which reads return stale values. You will be able to run the same program on a naive pipeline and on a reference model and point to the exact instruction where they disagree. And you will be able to compute how many cycles I independent instructions take, and what speedup that buys.

BEFORE YOU START

From ch05: a *single-cycle core* does fetch, decode, execute, memory access and writeback of one instruction inside one clock period, so its period is set by the slowest instruction (`lw`). From Book 5 ch10: a *register* copies its input only on a clock edge, and a chain of combinational logic between two registers must settle before the next edge; the longest such chain sets the fastest clock. From Book 5 ch15–16: the cpu8 you built ran one instruction at a time, exactly like ch05's core. From ch03: an instruction *reads* registers named `rs1`, `rs2` and *writes* the one named `rd`; a *dependency* exists when a later instruction reads a register an earlier one writes. Nothing else is assumed.

6.1 The idea from zero: an assembly line

Imagine washing, drying and folding laundry, three machines, one load takes three hours. Done one load at a time the three machines are mostly idle: while load 1 is in the dryer the washer sits empty. Start load 2 in the washer as soon as load 1 moves on, and every machine is busy; once the line is full, a load finishes every hour even though each load still takes three hours from start to end. Nothing got faster for the individual load. What improved is how often a finished load appears.

THINK LIKE THIS: LATENCY VERSUS THROUGHPUT

Always ask two different questions of a system. *Latency*: how long does one job take from start to finish? *Throughput*: how many jobs finish per unit time? A pipeline leaves the first almost unchanged and improves the second. Beginners often mistake one for the other, and that confusion is how you end up believing a "five times faster" claim that is really "five times as many results per second, once the line is full".

A processor does the same thing with instructions. Cut the work of one instruction into five steps (stages) and put a register between each pair so each step holds its own instruction. Five different instructions are then in flight at once, one per stage. The catch, which is the subject of this whole chapter, is that instructions are not independent loads of laundry: one may need the *result* of the instruction just ahead of it, which is not finished yet.

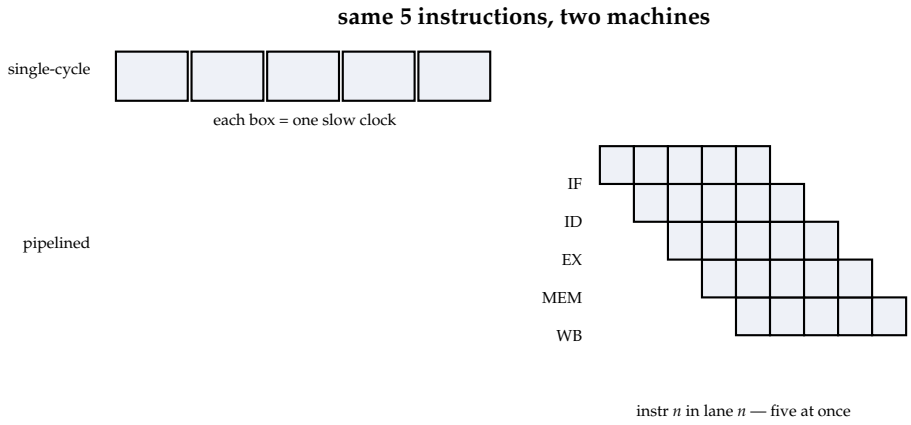
6.2 Why pipeline

Two numbers, not one. *Latency* is how long one instruction takes end to end — pipelining does not improve it (pipeline registers add a little). *Throughput* is how many finish per unit time — pipelining multiplies it by the stage count, ideally. Ch05's 5τ clock becomes a τ clock; five instructions in flight finish one per τ instead of one per 5τ. The `lw` tax disappears not because `lw` got cheaper but because its slack is now occupied by four other instructions.

6.3 The five stages and what the registers carry

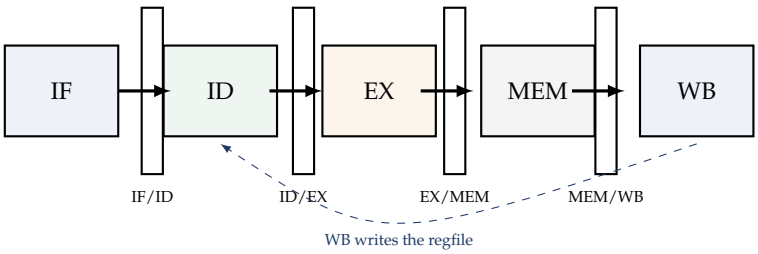
Example 6.1 (cut the path, keep the semantics)**.** `ch06_rv32i_pipe_top.v` reuses ch05's decoder, regfile and ALU unmodified — the stages are the same combinational blocks, separated by registers. `IF/ID` carries the instruction word and `pc+4`; `ID/EX` carries the read operands, the immediate, the destination index, and every control bit decode produced; `EX/MEM` carries the ALU result, the store value, and the surviving control bits; `MEM/WB` carries whichever of ALU-result / memory-data / pc+4 the writeback needs. Nothing is forgotten: a control bit that dies early is a bug with a wire width.

Checkpoint 6.1 *the invariant.* A correct pipeline must be observationally equivalent to the single-cycle machine: same final register file, same memory, same order of effects, just faster. Everything in ch07 exists to *restore* that equivalence where the naive split breaks it.



Left: 5 instructions in 5 long periods. Right: the same five overlap — the pipeline finishes one instruction per short period once full. Latency per instruction is unchanged; the throughput is what multiplies.

Figure 6.1: Same five instructions on both machines. Single-cycle serialises; the pipeline overlaps — throughput, not latency, is what changed.



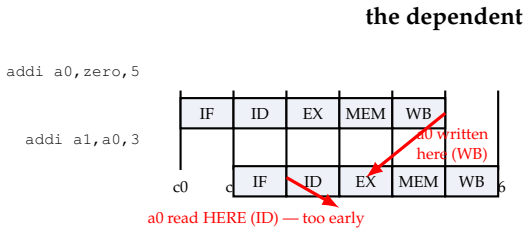
Four registers slice the single-cycle path into five stages. Each carries everything the later stages need: the instruction, the operands, the control bits, the pc+4. The registers are the whole difference — and the whole hazard.

Figure 6.2: Four pipeline registers slice ch05's path. Each register carries data *and* control — the control bits travel with their instruction as far as they are needed.

6.4 Timing diagrams

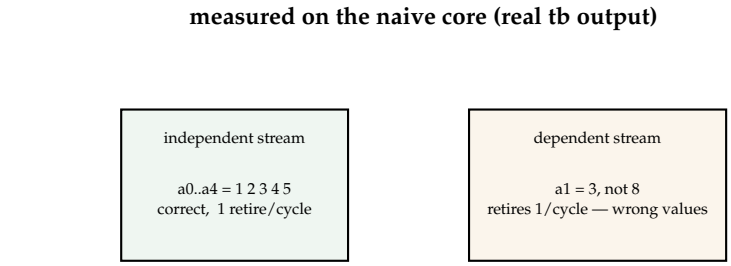
The convention: rows are instructions, columns are cycles, each cell names the stage that instruction occupies. A pipeline of depth k needs k cycles to fill; afterward, one completes per cycle — the steady state everything is measured against.

Example 6.2 (fill the diagram)**.** For five instructions: cycle 1 has I1 in IF; cycle 2 has I1 in ID, I2 in IF; …; cycle 5 has I1 in WB and I5 in IF — the first completion; cycles 6–9 complete I2–I5. Nine cycles of length τ for five instructions, i.e. 9τ; the single-cycle machine of ch05 needs $5 \times 5\tau = 25\tau$ for the same five.



ID for the second `addi` reads the register file two cycles before the first one's WB lands the new a0. The machine answers with the OLD value — our tb measured a1 = 3, not 8.

Figure 6.3: Why a1 = 3: the reader's ID lands two cycles before the writer's WB. The register file answers honestly — with a stale value.



The pipeline is FAST either way — that's the trap. Throughput looks perfect while the dependent stream silently computes garbage. ch07's forwarding/stalls buy correctness back with real hardware.

Figure 6.4: The naive core's real results: correct and fast on independent streams, fast and wrong on dependent ones. Throughput masks the bug — that is what makes hazards dangerous.

6.5 Where it breaks: hazards, shown

Example 6.3 (the wrong answer, measured). `ch06_pipe_tb` runs two streams on the naive core. The dependent pair `addi a0,zero,5; addi a1,a0,3`:

```
dependent: a0=5 (want 5) a1=3 (want 8)
  ^^ a1 read the OLD a0 -- data hazard, REAL and wrong
independent: a0..a4 = 1 2 3 4 5 (want 1..5)
ch06 SHOWN: pipeline retires ~1/cyc when independent,
lies when dependent
```

`a1=3`: the second `addi` was in ID during cycle 3, reading the register file *before* the first one's WB in cycle 5. The machine is perfectly fast and perfectly wrong — and no warning prints, because hardware never warns.

Example 6.4 (a longer dependent chain, against the reference). One dependent pair could be a fluke. Take five instructions in which each one needs the one before it (`ch06_dep.S`):

```
        li a0, 5
        addi a1, a0, 3 # reads a0 one instruction after its writer
        add a2, a1, a0 # reads a1 (distance 1) and a0 (distance 2)
        sw a2, 0(zero) # store data a2 at distance 1
```

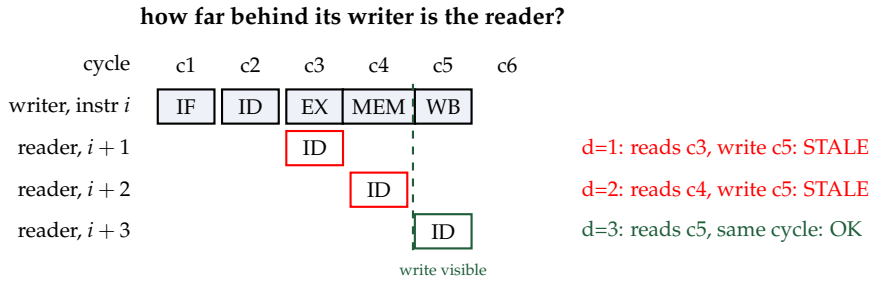


Figure 6.5: Which dependent readers see a stale value. The register file is written in WB and, thanks to its write-bypass, a read in the *same* cycle already sees the new value. Distance 1 and 2 are stale; distance 3 is fine.

```
5       lw a3, 0(zero)
6 halt: j halt
```

Predict first. By the rule of the next example, a register read is stale if the reader is one or two instructions after the writer. So `a1` reads an old `a0` (0), giving 3; `a2` reads an old `a1` (0) and an old `a0` (0), giving 0; the store writes that 0; the load brings back 0. Now run it on the naive pipeline and on the reference model (ch03 simulator). Naive pipeline, Linux x86_64 container, Icarus 12:

```
1 CORE=naive-pipe F=5 stalls=0 taken=0 lastret=9
2 x10=00000005
3 x11=00000003
```

Reference model:

```
1 I=5 HALT=20
2 x10=00000005
3 x11=00000008
4 x12=0000000d
5 x13=0000000d
6 m0=0000000d
```

Lines that are missing from the pipeline's dump are registers that are still 0. Everything we predicted happened: `a1=3` (should be 8), `a2=0` (13), `a3=0` (13), memory word 0 stays 0 (13). Only the first instruction is right. Note `lastret=9`: the fifth instruction still retires at the ninth clock edge, as fast as ever. The core is fast and wrong.

Example 6.5 (how many instructions apart is far enough?)**.** Number the writer's stages in cycles: IF 1, ID 2, EX 3, MEM 4, WB 5. A reader d instructions later is fetched in cycle $1+d$ and reads its registers in ID, cycle $2+d$. The new value is readable from cycle 5 (the register file of this book forwards a same-cycle write to a read, see ch05's discussion of `BYPASS`). So the read is correct only if $2+d \geq 5$, i.e. $d \geq 3$. For $d = 1, 2$ the machine would need 2 or 1 instructions of padding (`nop`s) in between. Without the bypass the first good distance is 4, which is why textbook diagrams differ by a cycle depending on the register-file design: always ask which one you have.

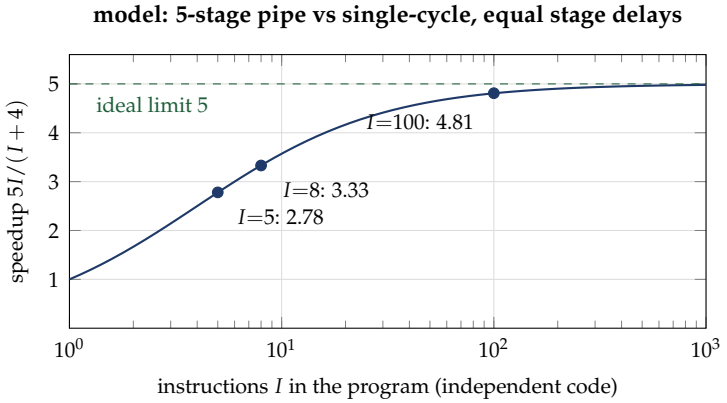


Figure 6.6: The model $5I/(I+4)$: I instructions take $(I+4)\tau$ pipelined and $5I\tau$ single-cycle. The three marked points are computed from the formula, not measured; only $I = 8$ was also checked in simulation (12 cycles).

Example 6.6 (the pipeline fill, measured)**.** The previous example predicts that I independent instructions finish in $I + 4$ cycles: four extra cycles to fill the line. Test it with eight instructions that never read each other (`ch06_indep.S`: `li a0,1 ... li a7,8`). The testbench timestamps the clock edge at which the eighth register-writing instruction reaches WB:

```
CORE=naive-pipe F=8 stalls=0 taken=0 lastret=12
x10=00000001
x11=00000002
x12=00000003
x13=00000004
x14=00000005
x15=00000006
x16=00000007
x17=00000008
```

`F=8`: the parking instruction was fetched after 8 cycles, one fetch per cycle. `lastret=12`: the last write lands at edge $12 = 8 + 4$, as predicted. All eight values match the reference model. At this length the pipeline is $5 \times 8/12 = 3.3\times$ faster than a single-cycle core with a clock five times slower; the speedup approaches 5 only for long programs (next figure).

Example 6.7 (a bug that is not a hazard: unreset registers)**.** When this chapter's testbench was first run in the Linux container (Icarus 12), the dependent pair printed `a1=x`, not the chapter's `a1=3`. An `x` is Verilog's "unknown" value, what every register holds before it is first written. The first instruction fetched while the clock was still in reset was `imem[x]`, also unknown; it flowed down the pipe and made the register file's bypass test (`we && wa==ra1`) unknown, so reads came back unknown. The real hazard was hidden under a simulation artefact. Fix: feed a `nop` into IF/ID during reset and give the pipeline registers a defined power-up value. With that fix the output is exactly the line in the earlier listing. Think like this: when a result looks strange, first ask whether the machine or the *simulator's starting state* is to blame.

Example 6.8 (name the three hazards)**.** What we just measured is a *data hazard*: an operand not yet written. A *control hazard* is the same story on the pc: a branch resolves in EX but two

wrong-path instructions are already in IF/ID. A *structural hazard* is a resource fought over: our unified testbench memory would fight IF and MEM if it were one port — real machines split i-cache/d-cache partly for this. Ch06's core takes all three on the chin; ch07 pays them off.

Checkpoint 6.2 *ideal CPI vs honest CPI.* Ideal pipelined CPI is 1. Our measured CPI is also ≈1 — on the independent stream. Predict the dependent stream's *effective* CPI if we forced correctness with stalls: each dependent pair costs bubbles. Ch07 measures exactly this.

6.6 Common mistakes

- **Believing throughput = correctness.** The dependent stream retires one per cycle too — wrong values at full speed.
- **Dropping control bits early.** A control signal that dies at ID/EX when MEM needs it is a silent misroute.
- **Forgetting the fill/drain.** CPI≈1 claims assume steady state; a 5-instruction program spends most of its life filling.
- **Testing independent code only.** It passes beautifully and proves nothing about hazards — our tb runs both on purpose.
- **Reading the regfile on the wrong edge.** Write-first vs read-first ordering inside a cycle decides whether one hazard vanishes for free.

6.7 Summary

Idea	The truth
stages	IF ID EX MEM WB; 4 registers carry data+control
win	throughput ×5 ideally; latency unchanged
lie	retires 1/cycle even when wrong (measured: a1=3≠8)
hazards	data, control, structural — named, shown, deferred
next	forwarding, stalls, branch prediction (ch07)

Further reading

- Patterson & Hennessy, COD ch4 (pipelining, hazards): elsevier.com — COD RISC-V edition
- Harris & Harris, DDCA RISC-V ch7 (pipelined processor): `https://pages.hmc.edu/harris/ddca/ddcarv.html`
- CS:APP ch4 (SEQ → PIPE): `https://csapp.cs.cmu.edu/`
- Hennessy & Patterson, CAQA appendix C (the classic): elsevier.com — CAQA

WATCH INSTEAD

- **DDCA Ch7 pipelined-processor walkthrough** — Sarah Harris series, the same five-stage decomposition drawn live (verified 2026-10-11). `https://youtu.be/AoBkibslRBM`

Notes

- Stage decomposition and hazard taxonomy follow COD ch4 and DDCA ch7; CS:APP ch4 for the SEQ→PIPE pedagogy.
- The "a1=3" result is a real Icarus run of `ch06_pipe_tb` — not a stylised example. Both streams captured verbatim.
- New runs in this edition (`code/test_ch06.sh`) are from the Linux x86_64 container with Icarus 12. In that simulator the original pipeline source printed `a1=x`; the reset fix described in the chapter (a `nop` into IF/ID, defined power-up values) was added to `ch06_rv32i_pipe_top.v`, after which the output matches the original capture.
- The speedup curve is a formula, not a measurement; only the 8-instruction point was also simulated.

6.8 Exercises

Exercise 6.1 (easy)**.** Latency vs throughput: which does a pipeline improve, and by how much ideally?

Solution. [hint] Throughput, by the stage count; latency slightly worsens.

Exercise 6.2 (easy)**.** List what IF/ID must carry vs what MEM/WB must carry, and why the difference.

Solution. [hint] IF/ID: insn+pc4 (everything flows forward). MEM/WB: only the writeback payload — control died with use.

Exercise 6.3 (medium)**.** Draw the timing diagram for our dependent pair and mark the exact cells where the write and the read happen.

Solution. [hint] Write: WB cell of insn1 (c5). Read: ID cell of insn2 (c3). Two cycles too early.

Exercise 6.4 (medium)**.** Why does the naive core need no flush logic yet still get branches wrong? What would flushing cost in bubbles?

Solution. [hint] Branches resolve in EX — two wrong-path insns already fetched; flushing = discarding those two cells.

Exercise 6.5 (medium)**.** The regfile reads in ID and writes in WB. If we made writes visible to reads in the *same* cycle, which hazards disappear for free?

Solution. [hint] The WB-to-ID same-cycle case — write-then-read ordering inside the clock edge buys one free forwarding hop.

Exercise 6.6 (medium)**.** Our tb checked registers, not memory. Construct a dependent `sw` that would store a stale value and say what signature you'd check in dmem.

Solution. [hint] addi a0,zero,7; sw a0,0(zero) — dmem[0] gets OLD a0; check dmem[0]==7 fails.

Exercise 6.7 (medium)**.** With 5-stage delay profile τ=2ns + register overhead δ=0.1ns, compute ideal speedup vs the ch05 single-cycle.

Solution. [hint] 10.1ns vs 2.1ns $\rightarrow$ 4.8×, not 5 — overhead is why real speedup is always sublinear.

Exercise 6.8 (hard)**.** Model: speedup over a single-cycle machine with period T for a k-stage pipeline is $T/(T/k+d)$ with register overhead d. Take $T = 10\,\text{ns}$, $d = 0.1\,\text{ns}$. (a) Compute it for $k = 5, 10, 20, 50$. (b) Does it have a maximum? (c) Now add hazards: CPI $= 1 + h(k-1)$ with $h = 0.02$. Find the k that maximises speedup/CPI.

Solution. [hint] (a) 4.76, 9.09, 16.67, 33.33. (b) No: it rises forever toward $T/d = 100$. (c) Performance $= \dfrac{T/(T/k+d)}{1+h(k-1)}$: $k = 5$: 4.41; 10: 7.70; 20: 12.08; 50: 16.84; 70: 17.30 (the best integer; $\sqrt{T/(dh)} \approx 70.7$); 100: 16.78. Hazards, not overhead, create the optimum. (Model values, computed, not measured.)

Exercise 6.9 (hard)**.** Three instructions: `lw a0,0(x); add a1,a0,a2; sw a1,4(x)`. On the naive core trace every wrong register read and say how many independent nops would fix each.

Solution. [hint] a0's load lands WB at c5; add reads at c3 $\rightarrow$ 2 nops. Then a1 is an ALU result, WB c6, read c4 $\rightarrow$ 2 more nops.

Exercise 6.10 (easy)**.** A three-machine laundry line takes one hour per machine. How long do ten loads take one at a time, and with the line? What is the latency of one load in each case?

Solution. [hint] One at a time: $10 \times 3 = 30$ h. With the line: $3 + 9 = 12$ h (three to fill, then one per hour). Latency of one load stays 3 h in both.

Exercise 6.11 (easy)**.** In the dependent chain of the chapter, which instruction is the first one to read a wrong value, and what does it compute instead of 8?

Solution. [hint] `addi a1,a0,3`: it reads the old `a0`= 0 and computes $0 + 3 = 3$.

Exercise 6.12 (medium)**.** Predict the final `a2` of `li a0,5; li a1,7; add a2,a0,a1` on the naive pipeline. The tested run (`ch06_two.S`) is in the code directory; explain the result from the distances.

Solution. [hint] `a1` is read at distance 1 and `a0` at distance 2, both stale (0), so `a2`$= 0 + 0 = 0$ (the run prints no `x12` line; `a0`$= 5$, `a1`$= 7$ are right).

Exercise 6.13 (medium)**.** How many padding instructions (not counting the `halt`) make the chapter's dependent chain correct on the naive core, if you pad only where needed? Verify against `ch06_dep_nops.S`.

Solution. [hint] Each of the three dependent hand-offs (`li`→`addi`, `addi`→`add`, `add`→`sw`) needs distance 3, i.e. 2 pads: 6 pads. The `add` also reads `a0`, now at distance 5, fine; the `lw` after the `sw` needs none. The padded program runs 11 instructions and the naive core then matches the reference: 5, 8, 13, 13, memory 13.

Exercise 6.14 (medium)**.** Using $I + 4$ cycles, how many cycles do 20 independent instructions need on the 5-stage pipe, and what is the speedup over a single-cycle core with a clock five times slower?

Solution. [hint] 24 cycles; $5 \cdot 20/24 = 100/24 \approx 4.17$.

Exercise 6.15 (hard)**.** Suppose the register file had *no* write-bypass (a read in the same cycle as the write returns the old value). From which dependence distance is the naive pipeline correct, and how many pads does an adjacent dependent pair need?

Solution. [hint] The write is visible from cycle 6 (the cycle after WB). The reader at distance d reads in cycle $2 + d$, so $d \geq 4$: an adjacent pair ($d = 1$) needs 3 pads.

Exercise 6.16 (hard)**.** Write the timing grid (cycle by stage) for `lw a0,0(x); add a1,a0,a2` on a pipeline where the load data can be used only from the cycle *after* MEM. Where would you place the forward, and why does one bubble remain? (Preview of ch07.)

Solution. [hint] lw: IF1 ID2 EX3 MEM4 WB5. The data exists at the end of cycle 4. The add, fetched in cycle 2, would want it as an ALU input in its EX in cycle 4 — the same cycle the data is only being read. Delay the add one cycle so EX is cycle 5, with a forward from MEM/WB: one bubble.

CHAPTER 7

Hazards: Forwarding, Stalls, Branches

Ch06 ended with a fast liar: the naive pipeline retired one instruction per cycle while handing stale registers to dependent code — `a1=3` where 8 belonged. This chapter pays the bill in hardware. Three fixes, each small enough to draw on one figure: *forwarding* muxes that route results backward before writeback; a *load-use stall* that freezes fetch for the one case forwarding cannot reach; and *predict-not-taken plus flush* for branches. Then the same testbench ch06 failed: now `a0=5, a1=8, a2=41, a3=42, dmem[1]=42`, at a measured cost of exactly one pipeline bubble. Every number in this chapter comes from the real core.

WHAT YOU WILL BE ABLE TO DO AFTER THIS CHAPTER

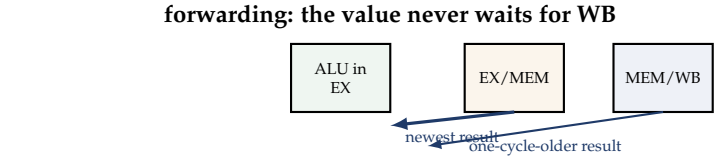
You will be able to look at a short instruction sequence and say, before running anything, how many extra cycles a pipelined RV32I core will spend on it: zero for ALU-to-ALU dependencies, one for a load followed by its user, two for every taken branch. You will be able to check that prediction against the real core with the cycle formula $F = I + \text{stalls} + 2 \cdot \text{taken}$, and to tell whether a test is any good by breaking the hardware on purpose and seeing it fail.

BEFORE YOU START

From ch06: a five-stage pipeline (IF, ID, EX, MEM, WB) overlaps five instructions, one per stage, with registers between stages; a later instruction that reads a register an earlier one is still computing gets a stale value, and a read is safe only if the reader is at least three instructions behind the writer in our design. From ch05: the ALU computes from two operands, the program counter (`pc`) names the next instruction, a `beq`/`bne` compares by subtracting and testing for zero. From Book 5 ch09 (combinational circuits): a *multiplexer* picks one of several inputs by a select signal, and comparators (==) are a few gates. From Book 5 ch12: a testbench drives the clock and checks results. That is all.

7.1 The idea from zero: shortcuts, waiting, and guessing

Return to the assembly line. Suppose station 3 needs a part that station 4 is just finishing. There are three honest things the foreman can do. (1) *Take a shortcut*: hand the part sideways from station 4 to station 3 the moment it exists, instead of sending it down to the warehouse (the register file) and back. That is *forwarding*. (2) *Wait*: if the part genuinely does not exist yet, stop station 3 for one tick and let a blank item pass through. That is a *stall* (the blank is a



Two muxes on the ALU inputs pick, newest-first: EX/MEM's just-computed value, else MEM/WB's returning value, else the register file. The register file still gets written — the forward is a shortcut, not a replacement.

Figure 7.1: Two muxes on the ALU inputs, newest first: EX/MEM, then MEM/WB, then the register file.

bubble). (3) *Guess*: when the next job depends on a decision not yet made (a branch), start the most likely one and be ready to throw it away. That is *prediction* with a *flush*.

THINK LIKE THIS: MAKE THE COMMON CASE FREE, THE RARE CASE CORRECT

Hardware designers do not try to avoid every delay. They ask which situations are common and make those cost nothing (ALU-to-ALU dependencies: forwarding), and make sure the rare, expensive situations are at least *correct* (a load followed at once by its user: a one-cycle stall). Correctness is never negotiable; speed is what you buy with whatever logic you can afford.

7.2 Forwarding: don't wait for writeback

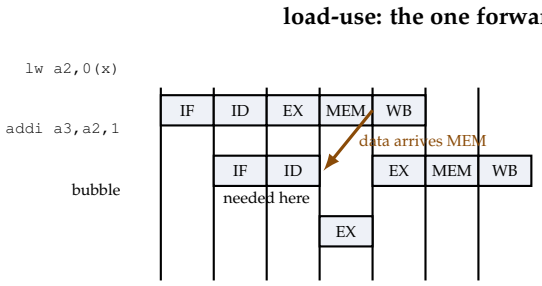
Example 7.1 (the value is already there)**.** When `addi a0,zero,5` is in MEM, its result sits in the EX/MEM register; when it's in WB, in MEM/WB. The consumer's ALU input just needs a mux pointed at those registers instead of only the regfile. Two comparisons per operand — `exmem_rd == idex_rs1?  memwb_rd == idex_rs1?` — newest wins.

Checkpoint 7.1 *why newest first.* If `a0` is written by two back-to-back instructions, both EX/MEM and MEM/WB may match. The newer writer must win — ordering the mux checks EX/MEM first is the whole subtlety.

Example 7.2 (forwarded, measured)**.** Same pair that broke ch06: `addi a0,zero,5; addi a1,a0,3`. Real tb output:

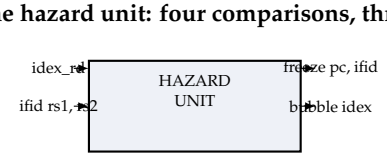
```
a0=5 a1=8 -- forwarding fixed ch06's 3
dmem[1]=42 -- dependent store forwarded
measured: 1 stalls, 6 WB-writes in 40 cycles
```

`a1=8` at full speed — zero bubbles for ALU hazards. The `sw` got its store data forwarded to the MEM stage too.



ALU results forward backward in time only by one stage; memory results land too late even for that. The fix is honesty: freeze IF/ID one cycle, bubble EX, let the load land — our measured cost: exactly 1 stall for the pair.

Figure 7.2: The consumer's ID slides one cycle right; the load's MEM data now forwards into its EX.



One `assign` in Verilog: if the instruction in EX is a load and the one in ID reads its destination, freeze fetch and decode, and inject a bubble. Hardware small enough to fit in a figure, important enough to justify the chapter.

Figure 7.3: Four comparisons, three control actions — the entire hazard unit fits in one `assign`.

7.3 Load-use: the one stall you cannot avoid

Example 7.3 (memory is one cycle too slow)**.** `lw a2,0(x); addi a3,a2,1`: in real hardware a load's data come out of the memory at the *end* of the MEM cycle, one stage *after* the consumer's ALU needs them at the start of its EX. No wire can run backwards in time. (One could build a path from the memory output straight into the next ALU in the same cycle, but then the clock period must cover memory *plus* ALU in series, wiping out the point of pipelining.) The honest fix: the hazard unit detects that the EX instruction is a load (`idex_memread`) whose destination matches the ID instruction's `rs1` or `rs2`, freezes `pc` and `ifid`, and bubbles EX once; the load data are then forwarded from MEM/WB. Measured cost: **exactly 1 stall** — the tb counts it.

Checkpoint 7.2 *why can't we forward the load?.* Forwarding moves data backward through stages already computed. The load's MEM stage hasn't produced the value when the consumer's EX runs — it exists only a cycle later, so the consumer must wait one cycle; then MEM/WB forwarding covers it.

Example 7.4 (take the stall away and watch it break)**.** How do you know the stall is needed rather than just believing the argument? Remove it. The test script builds a copy of the core in which the stall condition is forced to 0 and runs the sum program (which has `lw a0,0(zero); addi a1,a0,1`). The correct final state has `a1`=56 and memory word 1 equal to 56. The mutant core, run in the Linux x86_64 container (Icarus 12), prints:

```
1 CORE=fwd-pipe F=55 stalls=0 taken=9 lastret=-1
```

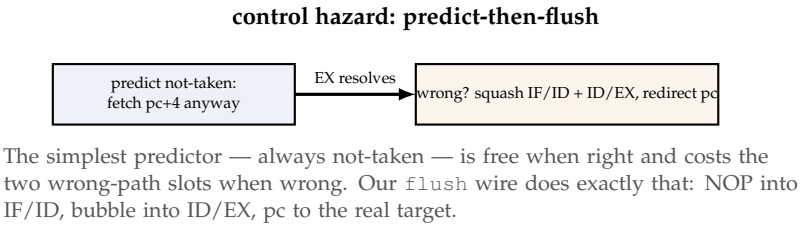


Figure 7.4: Predict-not-taken: free on the guess, two slots on the miss.

```
x5=0000000b
x6=00000037
x7=0000000b
x10=00000037
x11=00000001
m0=00000037
m1=00000001
```

One cycle faster (55 instead of 56), and `a1`=1: the `addi` added 1 to the load's *address* (0), because that is the only thing the EX/MEM register holds for a load. The reference model says 56. Note a subtlety that we found along the way: an earlier version of this core forwarded the memory output combinationally into the next ALU, which made the stall unnecessary *in simulation*, where memory answers instantly. That path is removed here so the model matches real memory timing.

Example 7.5 (forwarding priority, broken on purpose)**.** Two back-to-back writes of `a0` followed by a reader (`ch07_prio.S`: `li a0,1; addi a0,a0,1; add a1,a0,zero`). When the `add` is in EX, EX/MEM holds the newer `a0` (2) and MEM/WB the older one (1). Correct: `a0`=2, `a1`=2. The mutant that tests MEM/WB *first* gives

```
CORE=fwd-pipe F=3 stalls=0 taken=0 lastret=-1
x10=00000002
x11=00000001
```

`a1`=1: the reader saw the stale writer. The test would pass for any program that never has two writers in flight, which is why this program exists.

7.4 Branches: predict, verify, flush

Example 7.6 (the cheapest predictor)**.** Always fetch `pc+4` (predict not-taken). If EX proves a branch taken, squash the two wrong-path slots — NOP into IF/ID, bubble into ID/EX, redirect pc to the computed target. Two lost slots per misprediction; zero cost when the guess is right. `pc_tgt = idex_pc4 - 4 + idex_imm` — careful: pc+4 carried forward, not pc itself.

Example 7.7 (a flush, with and without the squash)**.** Program `ch07_br.S`: two `li`, a `beq` that is taken, then two instructions `li a2,99` and `li a3,99` that the branch jumps over, then `li a5,7`. By the time the `beq` is decided in EX, the first wrong-path `li` is in ID and the second is being fetched. Correct core: `a2`, `a3` stay 0 and $F = 4 + 2 \cdot 1 = 6$:

```
CORE=fwd-pipe F=6 stalls=0 taken=1 lastret=-1
x10=00000001
x11=00000001
x15=00000007
```

The mutant that still redirects the pc but forgets to turn the two wrong-path instructions into bubbles:

```
CORE=fwd-pipe F=6 stalls=0 taken=1 lastret=-1
x10=00000001
x11=00000001
x12=00000063
x13=00000063
x15=00000007
```

`0x63`= 99 in both: the instructions that the program says never run, ran. The cycle count is identical, so only a state comparison can find this bug — timing alone would have called the mutant correct.

Example 7.8 (the scoreboard, all measured)**.** Put the three costs into one formula and test it. If a program executes I instructions, hits S load-use stalls and takes K branches, the core fetches the parking instruction after

$$F = I + S + 2K$$

cycles. For the sum-1..10 program of ch05: $I = 37$, $S = 1$ (the `lw`/`addi` pair), $K = 9$ (the `bne` is taken nine times and falls through once): $37 + 1 + 18 = 56$. Measured (Linux x86_64 container, Icarus 12):

```
CORE=fwd-pipe F=56 stalls=1 taken=9 lastret=-1
x5=0000000b
x6=00000037
x7=0000000b
x10=00000037
x11=00000038
m0=00000037
m1=00000038
```

The registers and memory equal the single-cycle core's and the reference model's. Compare ch05: the same program took $F = 37$ there, one cycle per instruction, but on a clock about five times slower. The formula was also checked on a second program (the same loop with bound 6: $22 + 1 + 8 = 31$, measured $F = 31$). The CPI of this run is $56/37 \approx 1.51$ cycles per instruction, of which 18 of the 19 lost cycles are branch flushes.

Hazard scoreboard — what each fix covers

hazard	fix	cost	left over
ALU → ALU	forward EX/MEM, MEM/WB	0 bubbles	none
load → use	stall 1 + forward	1 bubble	the bubble
store data	forward to MEM wdata	0	none
branch	predict-not-taken + flush	2 slots on miss	the flush

Every row was measured on the book's core, not asserted.

Figure 7.5: Every fix's cost, measured on the real core: forwarding is free, load-use costs one bubble, a taken branch costs two slots.

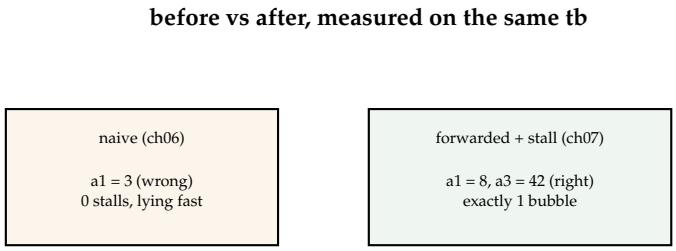


Figure 7.6: Same tb as ch06: wrong-fast became right-fast at the cost of one bubble.

7.5 The debugging receipts

Example 7.9 (two real bugs this chapter hit)**.** Building this core produced two genuinely instructive failures, kept because they are the lesson. (1) *X-propagation deadlock*: uninitialised pipeline registers made `stall` unknown forever — nothing ever retired; the fix was zeroing the tag registers (`idex_rd`, `exmem_rd`, `memwb_rd`) on reset, since 0 never matches a live register. (2) The hand-assembled constant `0x00002613` was written down as "`lw a2,0(zero)`" — it actually decodes as `addi a2,zero,0` (opcode `0x13`, not `0x03`). The "load-use" test never tested a load. The book's own assembler now generates every test byte; hand hex is exactly the decode-a-field-wrong mistake ch03 warned about. Two more were found while re-running everything for this edition: (3) the branch-direction flag `idex_f30` was never assigned on the normal path, so a branch would have acted on an unknown value; the earlier testbench contained no branch at all, so this had never been exercised. It now takes the low bit of `funct3`, and the sum program is the regression test. (4) A combinational path from the data-memory output into the forwarding mux made the load-use stall unnecessary in simulation (see the "take the stall away" example) and was removed.

7.6 Common mistakes

- **Forwarding to the wrong stage.** A store's data must forward into `dmem_wdata`, not just the ALU — easy to miss because loads exercise only the read path.
- **No x0 guard.** `idex_rd==0` must never trigger a forward — register 0 has no writer to forward from.
- **Older-wins mux order.** MEM/WB checked before EX/MEM gives the stale writer priority — silent wrongness.
- **Stalling without freezing pc.** Freezing IF/ID while pc advances re-fetches and duplicates the instruction — we measured exactly that duplicate.
- **Hand-assembling test vectors.** One wrong field and the test tests nothing (our `0x00002613`).
- **Trusting a test that cannot fail.** Break the hardware on purpose (drop the stall, swap the mux order, skip the squash); if the test still passes it is not testing that feature.
- **Forgetting uninitialised tags are x.** `x==x` evaluates x, poisoning every comparison — reset the tags.

7.7 Summary

Fix	Hardware + measured cost
forwarding	2 muxes on ALU inputs; ALU hazards → 0 bubbles
load-use stall	1 `assign`; freeze pc+ifid, bubble EX → 1
branch flush	predict-not-taken; 2 slots per taken branch
receipts	a1: 3→8, a3: 42, dmem[1]: 42, stalls: 1 — all real

Further reading

- Patterson & Hennessy, COD ch4 (forwarding, stalls): elsevier.com — COD RISC-V edition
- Harris & Harris, DDCA RISC-V ch7 (hazard unit): `https://pages.hmc.edu/harris/ddca/ddcarv.html`
- Hennessy & Patterson, CAQA app C (the taxonomy): elsevier.com — CAQA
- CS:APP ch4 (PIPE's forwarding logic): `https://csapp.cs.cmu.edu/`

WATCH INSTEAD

- **DDCA Ch7 hazard-unit walkthrough** — Sarah Harris series, forwarding and stalling drawn live (verified 2026-10-11). `https://youtu.be/AoBkibslRBM`

Notes

- Hazard taxonomy and fix structure follow COD ch4; the scoreboard is measured on our core, not cited.
- The first three listings (the dependent pair, the load-use count) are the original `ch07_fwd_tb` output; all other outputs come from `ch05_run_tb.v` run by `code/test_ch07.sh` in the Linux x86_64 container (Icarus 12). Bug receipts in sec:receipts07 are real session bugs, documented because reviewers should be able to reproduce the failure modes too.
- Limits of the model: the data memory is a combinational array in the testbench, branches are resolved in EX with predict-not-taken, and only `beq`/`bne` are implemented (no `blt`/`bge`, no `jalr`).

7.8 Exercises

Exercise 7.1 (easy)**.** Why does forwarding check EX/MEM before MEM/WB?

Solution. [hint] Two matching writers: the newer one's value must win.

Exercise 7.2 (easy)**.** State the load-use stall condition in one sentence of English.

Solution. [hint] EX-stage insn is a load AND ID-stage insn reads its rd.

Exercise 7.3 (medium)**.** `add a0,a1,a2; add a3,a0,a4; add a5,a3,a6`: how many forwards, and from which stage each?

Solution. [hint] Two: a0 via EX/MEM, a3 via EX/MEM one cycle later (chain).

Exercise 7.4 (medium)**.** Why does the x0 guard matter for forwarding correctness?

Solution. [hint] x0 is never written; a match would forward a meaningless value to a consumer reading the constant.

Exercise 7.5 (medium)**.** A taken branch costs 2 slots here. What pipeline change halves it — and what does it cost in hardware?

Solution. [hint] Resolve branches in ID: an early comparator + target adder in ID stage; one slot lost instead of two.

Exercise 7.6 (medium)**.** The tb counts 1 stall for our program. Write a program that stalls exactly twice.

Solution. [hint] Two back-to-back load-use pairs, e.g. lw a0,0(x); addi a1,a0,1; lw a2,4(x); addi a3,a2,1.

Exercise 7.7 (hard)**.** Our hazard unit compares only rs1/rs2 of the ID instruction. Which instruction classes need no second-operand check, and is over-stalling for them harmful?

Solution. [hint] I-type/lw only read rs1 — checking rs2 can false-stall on instructions whose rs2 field is part of the immediate. Harm: a wasted bubble, not wrongness.

Exercise 7.8 (hard)**.** Design the branch-in-ID resolution: which registers must compare early, what forwarding must extend into ID, and what new hazard does it create?

Solution. [hint] rs1/rs2 compared in ID needs forwarding from EX/MEM and MEM/WB into ID — and a load producing the compare operand needs TWO stalls.

Exercise 7.9 (easy)**.** Fill in the cost, in lost cycles, of each situation on this core: ALU result used by the next instruction; load result used by the next instruction; load result used two instructions later; a taken branch.

Solution. [hint] 0; 1; 0 (the forward from MEM/WB covers it; run as `ch07_gap.S`, `stalls=0`); 2.

Exercise 7.10 (easy)**.** Use $F = I + S + 2K$ for `ch07_br.S` ($I = 4$ executed instructions, one taken branch, no load). What is F?

Solution. [hint] $4 + 0 + 2 \cdot 1 = 6$, matching the measured `F=6`.

Exercise 7.11 (medium)**.** Predict F for the sum program if the loop bound is 6 (`li t2, 6`; the loop runs for $i = 1 \ldots 5$). Count I, S, K first.

Solution. [hint] $I = 3 + 5 \cdot 3 + 4 = 22$, $S = 1$, $K = 4$ (taken four times, falls through on the fifth). $F = 22 + 1 + 8 = 31$. The test script runs exactly this variant and measures `F=31 stalls=1 taken=4`.

Exercise 7.12 (medium)**.** The sum program's CPI on this core is F/I (ignoring the 4-cycle drain). Compute it, and say what fraction of the lost cycles belongs to branches.

Solution. [hint] $56/37 \approx 1.51$. Lost cycles $= 56 - 37 = 19$, of which $2 \cdot 9 = 18$ are branch flushes: $18/19 \approx 95\%$.

Exercise 7.13 (medium)**.** Insert one independent instruction between the `lw` and its user in each pair of `ch07_stall2.S`. How many stalls remain, and what is the total instruction count?

Solution. [hint] None: a load-use pair at distance 2 needs no bubble (the data are forwarded from MEM/WB). The program of 7 instructions becomes 9. Moving independent work into the gap ("scheduling") is exactly what an optimising compiler does.

Exercise 7.14 (hard)**.** In the mutant without the stall the sum program gives `a1`=1 instead of 56. Explain the value 1 from what the EX/MEM register holds for a load.

Solution. [hint] For `lw a0,0(zero)` the EX stage computed the address $0 + 0 = 0$ and that is what EX/MEM keeps; with no stall the `addi` takes that 0 from the forward and adds 1.

Exercise 7.15 (hard)**.** The program `ch07_false10.S` (`lw a0,0(zero); addi a1,zero,10`) is independent, yet the core stalls once (`stalls=1`); changing 10 to 11 removes the stall (`stalls=0`). Explain, and say how to fix the hazard unit.

Solution. [hint] The immediate 10 occupies instruction bits 24:20, the field the hazard unit reads as `rs2`; 10 equals the load's destination `a0`=x10, so it looks like a match. Fix: compare `rs2` only for instructions that really read it (R-type, stores, branches), using a "reads rs2" control bit from the decoder. The cost of the bug is one wasted cycle, never a wrong result.

Exercise 7.16 (hard)**.** Model (not run): suppose hardware predicted *backward branches taken* with no penalty when right, and still paid 2 cycles when wrong. What would F be for the sum-1..10 program, and what extra hardware does "no penalty when right" secretly require?

Solution. [hint] All nine backward `bne`s are taken and correctly predicted; only the final fall-through is a misprediction: $F = 37 + 1 + 2 \cdot 1 = 40$ instead of 56. "No penalty" requires knowing the branch target in the same cycle the branch is fetched: a small table (a branch target buffer) or decoding in IF; this is the cost of the model's optimism.

CHAPTER 8

Traps, memory-mapped IO, and a program that talks to the world

The core built over chapters 5–7 computes correctly, but it is a closed box: it can neither call for help when something goes wrong nor emit a single observable byte. This chapter opens two doors at once. A minimal trap mechanism — `ecall` to enter a handler, `mret` to return — gives the machine its first taste of exception handling. A memory-mapped UART gives it a voice. By the end, one testbench run shows a program compute 6×7, print `42` on a serial wire, trap into a handler that prints `K`, return, print `!`, and park — all on real Verilog, all bytes decoded off the simulated wire.

AFTER THIS CHAPTER YOU CAN

You will be able to run one Verilog simulation of a pipelined RV32I core and say, from the printed log alone, which of its four output bytes came from the main program and which from the trap handler. You will be able to write the two instructions that print a byte through a memory-mapped UART, decode a serial frame by hand, and work out how long a software delay must be so that a byte is not lost.

BEFORE YOU START

Think of a restaurant kitchen. The core you built in chapters 5–7 is a kitchen that cooks very well but has no door to the dining room and no bell that a waiter can ring. This chapter adds the bell (a *trap*) and the serving window (a *UART*). Here is what you need to remember from earlier, in plain words. *Pipeline* (chapter 6): the core works like an assembly line with five stations, IF (fetch the instruction), ID (decode it), EX (compute), MEM (touch memory) and WB (write the result back), and each instruction moves one station per clock cycle. *Hazards* (chapter 7): when two instructions in the line depend on each other, the hardware either *forwards* a result early, or *stalls* (freezes the front of the line for a cycle), or *flushes* (throws away instructions that were fetched from the wrong place). We will use flush again today. *Instructions* (chapter 3): `sw rs2, off(rs1)` writes the register `rs2` to the memory address `rs1+off`, and `lui` puts a constant into the upper 20 bits of a register. *Serial transmitter* (Book 5, electronics, chapter 13, section "UART: talking to the outside world"): a shift register that puts one bit on a single wire per tick. If that is hazy, do not worry; the frame diagram in section 8.3 redraws everything you need.

THINK LIKE THIS

When a machine does something you cannot see, ask: *what is the smallest observation that would prove it?* A counter of wire edges is a weak observation. A receiver that reads the bytes back is a strong one. This chapter climbs from the first to the second.

8.1 Two small additions, one big change of character

Why does a processor need these at all? Picture a clerk who only ever does sums on paper. She is useful, but she cannot say what she found (no output), and she cannot be interrupted when something goes wrong (no way to ask for help). Both problems are solved by giving the hardware a small, fixed set of ways to talk to the world. Everything so far treats the processor as a calculator that stalls politely when it must. Real machines also need *control*: a way for software to ask the hardware for a service and a way for hardware to seize control back. On RISC-V that mechanism is the trap: an instruction (or an external event) redirects the pc to a fixed handler address after saving enough state to return.

And a processor that cannot move bytes in or out is a curiosity. The simplest output device imaginable is a UART transmitter — one wire, one bit at a time — and the simplest way to talk to it is *memory-mapped IO*: reserve an address, and let an ordinary `sw` do the talking.

Example 8.1 (What one byte on a wire buys)**.** `sw t0, 0(a7)` with `a7 = 0x10000000` sends the contents of `t0` out pin tx as a framed serial byte. No new instruction, no special port instruction — the address *is* the device selector. That is the entire idea behind memory-mapped IO, and every real embedded chip is built on it.

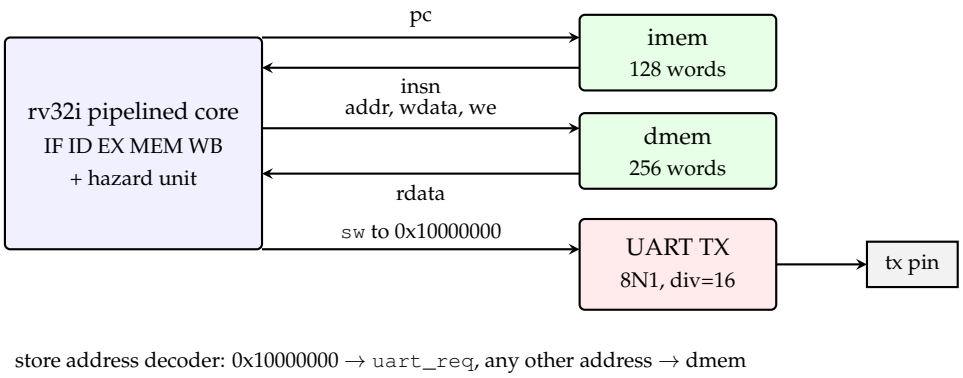


Figure 8.1: The ch08 SoC: the pipelined core, instruction memory, data memory, and a memory-mapped UART. One extra address-compare line turns an ordinary `sw` into a serial transmission.

8.2 Minimal traps: `ecall` and `mret`

Real RISC-V privilege machinery is large: dozens of control and status registers, trap delegation, vectored vs. direct entry. Two CSRs carry this chapter:

- `mepc`: hardware writes the faulting instruction's pc here on trap entry; `mret` jumps back to `mepc`+4. (Honest caveat: on a real RISC-V core `mret` jumps to `mepc` itself and the handler adds 4 with `csrr/addi/csrw`. We fold that addition into the hardware to keep the demo handler to two instructions.)
- `mcause`: a small code saying why. Present in our core as an extension point — the demo handler does not read it.

The micro-architecture adds three lines to the pc logic: on `ecall` in EX, latch `mepc` and redirect fetch to a fixed vector (0x80 here); on `mret`, redirect to `mepc`+4; both flush the younger stages like a branch.

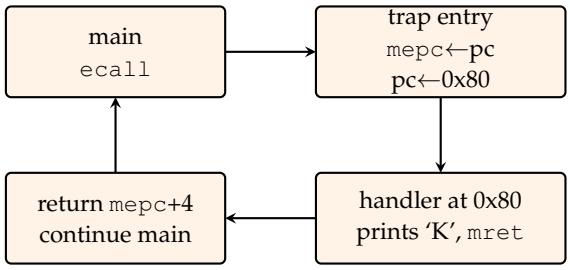


Figure 8.2: The minimal trap path implemented in this chapter's core: `ecall` saves the faulting pc in `mepc` and jumps to the fixed vector 0x80; `mret` returns to `mepc`+4.

CSR	width	written by	read by
`mepc`	32	hardware on trap (idex_pc)	`mret`
`mcause`	5	hardware on trap (code)	software (full ISA)

Figure 8.3: The two CSRs this chapter implements. Real RISC-V defines dozens; two suffice to trap, handle, and return. `mcause` is present but unused by the demo — noted as the extension point for timer or external interrupts.

Checkpoint 8.1 Why `mepc`+4 and not `mepc`? Because `mepc` holds the *ecall's own* address; returning to it would trap again forever. Real RISC-V handlers choose by editing `mepc` before `mret`: leave it (retry the instruction, as a page-fault handler does) or add 4 (move on, as a system-call handler does). Our hardware always adds 4.

8.3 The UART: one shift register with an address

The transmitter is the same 8N1 shape used in the electronics book's capstone: a 10-bit frame — start bit, eight data bits least significant bit first, stop bit — shifted out at one bit per `div` clocks. The only new piece is the request line:

```
assign dmem_we = exmem_memwrite & (exmem_y != 32'h10000000);
assign uart_req = exmem_memwrite & (exmem_y == 32'h10000000);
```

Listing 8.1: The MMIO hook in `rv32i_soc_top`: any store word whose address is 0x10000000 becomes a UART transmit request instead of a dmem write.

That is the whole of "device driver": the store's address decides where its data goes. Programs use `lui a7,0x10000` once and then `sw tX,0(a7)` for every byte out.

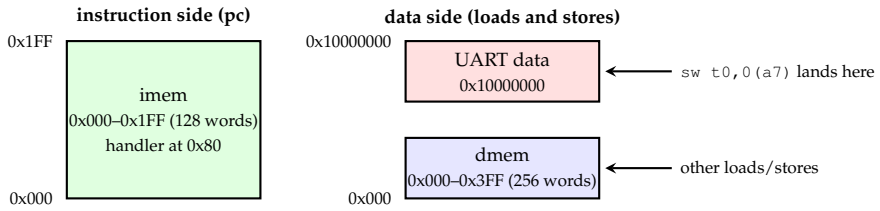


Figure 8.4: The two address maps of the ch08 SoC. The core has separate instruction and data memories (a Harvard layout), each starting at address 0. Only the data side has a carved-out address, `0x10000000`, that belongs to the UART; any other store goes to `dmem`.

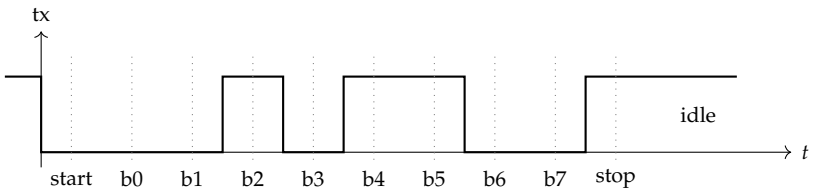


Figure 8.5: The wire-level frame the testbench observed for byte 0x34 (ASCII '4'): idle high, one start bit, eight data bits LSB-first (`00110100` read right-to-left), one stop bit. Three falling edges occur inside this one frame (the start bit and the 1-to-0 steps after b2 and b5), which is why a bench that counts every falling edge on a four-byte message sees 13, not 4.

Example 8.2 (Reading the frame)**.** For byte 0x34 ('4'), the serial line goes low for one bit time (start), carries `00110100` LSB-first (so the wire order is 0,0,1,0,1,1,0,0), then returns high (stop). Figure 8.5 draws the wire; the testbench counts falling edges on `tx`.

Example 8.3 (Counting falling edges: the naive count fails)**.** A first guess says each frame begins with exactly one falling edge, so four frames give four edges. Count the real wire instead. The line sits high while idle, and every 1-to-0 step is a falling edge, whether it is the start bit or a data bit. Writing each frame as start bit, `d0`…`d7`, stop bit:

byte	wire (start, d0..d7, stop)	falls at	count
0x34 ‘4’	0 0 0 1 0 1 1 0 0 1	start, d2→d3, d5→d6	3
0x32 ‘2’	0 0 1 0 0 1 1 0 0 1	start, d1→d2, d5→d6	3
0x4B ‘K’	0 1 1 0 1 0 0 1 0 1	start, d1→d2, d3→d4, d6→d7	4
0x21 ‘!’	0 1 0 0 0 0 1 0 0 1	start, d0→d1, d5→d6	3

The total is $3 + 3 + 4 + 3 = 13$, exactly the number the testbench printed. (Each frame ends high, so the next start bit is always a clean 1-to-0 edge.) The “13 versus 4” is therefore not a sign of collisions; it is the data itself. The decoder in the next section stops guessing and reads the bytes back.

8.4 The program, end to end

The demo program exercises every door at once. It computes 6×7 in a register loop (the same multiply-by-add pattern from chapter 4), prints the two digits, executes `ecall`, prints one more character after `mret`, then parks:

```
mloop: add a2, a2, a0 # a2 += 6, seven times
      addi a1, a1, -1
      bne a1, zero, mloop
      lui a7, 0x10000 # UART MMIO base
      li t0, 52 # '4'
      sw t0, 0(a7)
      ...delay loop...
      li t0, 50 # '2'
      sw t0, 0(a7)
      ecall # trap: mepc <- pc, pc <- 0x80
      li t0, 33 # '!'
      sw t0, 0(a7)
park: beq zero, zero, park
# ---- handler at 0x80 ----
      li t0, 75 # 'K'
      sw t0, 0(a7)
      mret # pc <- mepc + 4
```

Listing 8.2: The demo program (assembled by the book’s own assembler into `ch08_prog.hex`). Excerpt — main loop, UART writes, `ecall`, and the handler at 0x80.

Example 8.4 (What the testbench saw)**.** This is real `vvp` output, verbatim, from the captured log `code/ch08_soc_tb.out`:

```
ran 770 cyc, pc=00000058, a2=42 (want 42)
MMIO bytes sent: 42K!
uart frames started: 4 (want 4)
tx start-bit edges seen: 13
  ok: 6*7=42 via loop
  ok: '42' to MMIO
```

```
  ok: 'K!' trap handler + mret
  ok: serial wire really wiggled
ch08 PASS
```

Read it slowly. `42` proves the compute and the data memory. `K` proves the trap: the handler ran, did its work, and `mret` landed back exactly on the instruction after `ecall` — the `!`. The line "tx start-bit edges seen: 13" is a trap for the reader: the counter counts *every* falling edge on `tx`, and a data bit going from 1 to 0 is a falling edge too. Thirteen is correct, and the next worked example recounts it by hand.

A counter of edges is weak evidence. A real receiver ignores edges and does what the standard says: wait for a start bit, sample each data bit in the *middle* of its time slot, and check the stop bit. Listing 8.3 is that receiver written as the second testbench, `ch08_wire_tb.v`. It never looks inside the CPU to learn what was sent; it sees only the `tx` wire.

```
forever begin
  @(negedge tx); // start bit begins
  repeat (DIV + DIV/2) @(posedge clk); // 1.5 bit times: middle of d0
  for (k = 0; k < 8; k = k + 1) begin
    byte_r[k] = tx; // LSB first
    repeat (DIV) @(posedge clk); // next bit's middle
  end
  if (tx !== 1'b1) bad = bad + 1; // stop bit must be high
  dec[nframes] = byte_r; nframes = nframes + 1;
end

decoded from tx wire: 42K!
frames=4 bad stop bits=0 falling edges on tx=13
  frame 0: byte 0x34 start-bit cycle 42
  frame 1: byte 0x32 start-bit cycle 283
  frame 2: byte 0x4b start-bit cycle 528
  frame 3: byte 0x21 start-bit cycle 772
ecall in EX at cycle 521; fetch pc becomes 0x80 at cycle 522 (1 cycles
     later)
mret in EX at cycle 766; fetch pc is back at 0x48 at cycle 767 (1
    cycles later)
ch08 WIRE PASS
```

Listing 8.3: The wire decoder (core loop) and its captured output. Run on the book's own Icarus simulation; `div`=16 clocks per bit. The cycle numbers count clocks since reset release.

Example 8.5 (What the wire decoder tells us that the first bench could not)**.** Three facts. First, the bytes `0x34 0x32 0x4B 0x21` come off the wire, not off the bus into the UART: the whole path store → transmitter → pin works, with every stop bit high. Second, the trap is cheap in cycles but not in work: the core sees `ecall` in EX at cycle 521 and the fetch pc is 0x80 one cycle later; the two instructions fetched behind the `ecall` are discarded by the flush. The same one-cycle redirect happens on `mret` (cycle 766 → 767). Third, the gaps between start bits are 241, 245 and 244 cycles, far longer than the 160 clocks a frame needs, which prepares the next question.

Example 8.6 (How short can the delay loop be?)**.** The frame needs $10 \times 16 = 160$ clocks, and the UART refuses a new byte while `busy`. Each delay iteration is `addi` (1 cycle) plus a taken `bne`, which flushes two younger instructions (2 cycles): about 4 cycles per iteration. The naive estimate is $160/4 = 40$ iterations. Test it rather than trust it: rewrite the constant `li t1,60` to `li t1,`*N* in the hex file and rerun the decoder (the test script does this). Captured:

```
delay sweep: N=39 -> 2 frames, N=40 -> 4 frames
```

At $N = 39$ the second byte ('2') is silently lost and only '4' and 'K' arrive; at $N = 40$ all four arrive, with the first gap at 161 cycles, i.e. $4N + 1$. The sweep confirms the estimate to the cycle, and shows what "dropped, not queued" looks like: no error, just a missing character. With $N = 60$ the program has a margin of 80 cycles, and the three delay loops account for about $3 \times 240 = 720$ of the first 772 cycles of the run.

Checkpoint 8.2 Why do consecutive `sw`s to the UART need a delay loop between them? The transmitter is busy for `div`×10 clocks; a store arriving mid-frame has nowhere to queue. Real chips answer with a status bit the software polls — our minimal UART teaches the collision honestly instead.

8.5 Receipts: two bugs this chapter paid for itself

Example 8.7 (The WB/ID boundary hazard — a fourth hazard class)**.** Chapter 7 claimed data, control, and structural hazards covered everything. The trap program exposed a fourth: a writer leaving the pipeline at WB on the same edge a reader latches its operands in ID. The regfile read `a1 = 0` one cycle too early, the `addi a1,a1,-1` computed -1 instead of 6, and the loop ran 4 038 times instead of 7. Fix, and the classic textbook answer: a write-bypass *inside* the register file so a WB write is visible to a concurrent ID read ("write first"). One line each on `rd1`/`rd2` — and it removed the hazard for every program, not just this one.

Example 8.8 (The trap vector pointing at a data table)**.** The first run of the new core went to 0x40 on `ecall` — because the handler had been placed at byte 0x80 while `MTVEC` still said 0x40. The machine faithfully trapped into the middle of its own main program (executing a `bne` as if it were handler code) and never came back. The debugger showed `pc` frozen at 0x44 — the `ecall`'s own address, re-trapped on every return because `mret` landed back on it. Two-line fix (0x40→0x80), and a permanent lesson: a trap vector is an address like any other; getting it wrong produces the most confusing bug in the book — a processor that is *correct* about everything except where it jumps.

8.6 What it costs in silicon

cell class	count (real `synth_gowin` run)
LUT1–LUT4	2 021
MUX2_LUT5–LUT8	1 103
ALU (fast carry)	186
DFF variants	527
RAM16SDP4	32
IO buffers	166
total	≈4 035

Figure 8.6: Real `yosys-0.69 synth_gowin` output for the whole SoC (core + UART + trap CSRs). The GW1NR-LV9 offers 8 640 LUT4 — this machine plus its peripherals uses about half. The 32 RAM16SDP4 blocks are the register file inferred as LUT-RAM.

The whole machine — 5-stage pipelined RV32I core with forwarding, stall, flush, trap CSRs, plus the UART — synthesises to roughly 4 000 cells on the GW1NR-LV9, about half its LUT budget. Compare with the 302-cell `cpu8` from the electronics book: the pipelining machinery costs an order of magnitude more logic, and buys the order of magnitude in clock rate back.

8.7 Common mistakes

- **MMIO store eaten by silence.** A store to 0x10000000 while `busy` is high is dropped, not queued. Check a status bit, or pace the writes.
- **`mret` to `mepc`, not `mepc+4`.** Re-executes `ecall`: an infinite trap loop that looks, in a wave viewer, like a machine "stuck" at one address.
- **Trap vector overlapping program code.** If the handler region is not reserved, the trap entry jumps into the middle of normal code — the machine does nonsense very competently.
- **Forgetting that flush kills the trap's own fetch.** The redirect takes effect on the *next* fetch; the instructions already in flight must be bubbled, or the handler interleaves with dying main-code instructions.
- **Assuming the regfile boundary is free.** WB and ID touch the same storage on the same edge; without an explicit ordering (write-bypass, or write-first timing) it is a data hazard that stalls cannot fix.

Further reading

- Patterson & Hennessy, *Computer Organization and Design*, RISC-V edition — sections on exceptions and memory-mapped IO (the canonical treatment this chapter follows).
- Harris & Harris, *Digital Design and Computer Architecture, RISC-V ed.* — the pipelined-processor chapter; our core is a small honest version of theirs.
- The RISC-V Instruction Set Manual, Volume II: Privileged Architecture — `mepc`, `mcause`, `mtvec` definitions we shrink to two registers.
- Ben Eater's video documentation of his 8-bit computer's output register — the same MMIO idea on a breadboard machine (the Book 5 capstone's ancestor).

WATCH INSTEAD

- **Exceptions & Interrupts** — Computerphile (YouTube, verified 2026-10-11): why traps exist at all, at the right altitude for this chapter. `https://youtu.be/FR4rmlkgwr4`
- **8-bit computer output register** — Ben Eater (YouTube, verified 2026-10-11): the memory-mapped output idea on a real breadboard machine; the direct ancestor of our UART hook. `https://youtu.be/9Ma3c8xZJk8`

Notes

- Trap semantics follow the RISC-V privileged spec reduced to two CSRs; the UART is the Book 5 capstone serialiser unchanged.
- All outputs are verbatim Icarus runs of the chapter's testbench; both bug receipts in sec:receipts08 are real session bugs, fixed in-tree.

8.8 Summary

Piece	What this chapter added
Trap entry	`ecall`: `mepc`←pc, pc←0x80, flush
Trap return	`mret`: pc←`mepc`+4, flush
MMIO	address-compare turns `sw` into a device request
UART	8N1 shift register, `busy` honest about collisions
Regfile	write-bypass: the fourth (WB/ID) hazard, fixed once
Program	`42K!` on a decoded wire: compute, IO, trap, return
Cost	≈4k cells: pipelining bought speed at 10× logic

Exercises

Exercise 8.1 (easy)**.** Write the two RV32I instructions a program needs to send byte 0x41 ('A') to the UART, given that the base register already holds 0x10000000. What happens if the instruction immediately before it was also a UART store?

Solution. [hint] One `li` plus one `sw`; the previous frame must have finished or the write is dropped.

Exercise 8.2 (easy)**.** `mepc` currently stores the `ecall`'s address. What would you change if the handler needed to distinguish an `ecall` from an illegal instruction? Which CSR, which value?

Solution. [hint] `mcause` exists but is never written a distinct code; assign one per trap source.

Exercise 8.3 (easy)**.** Why does the trap-entry logic live in EX rather than ID in this core? What would break if it redirected the pc while the offending instruction was still in ID?

Solution. [hint] Younger stages still carry the pre-trap instruction stream; the redirect must coincide with the flush.

Exercise 8.4 (easy)**.** The UART has no receive side. Which two wires/registers would a receiver need that the transmitter does not have?

Solution. [hint] A sampling clock at the bit rate and a byte-valid flag — plus somewhere to buffer the byte.

Exercise 8.5 (medium)**.** Our handler is at a fixed address. Sketch the pc-mux change for a *vectored* trap where `mcause` selects among four handler addresses. Which existing block does most of the work?

Solution. [hint] It is the same mux `jal`/branch already use — one more input.

Exercise 8.6 (medium)**.** Using the wire decoder's numbers (ecall in EX at cycle 521, fetch pc equal to 0x80 at cycle 522), how many cycles after the `ecall` reaches EX is the first handler instruction *fetched*? Then, counting pipeline stages, in which cycle does it reach EX? Which stage sets the floor on trap latency?

Solution. [hint] Fetched one cycle later (522). From IF it needs ID then EX, so it reaches EX two cycles after that, in cycle 524. The floor is where the redirect is decided: EX, three stages behind fetch.

Exercise 8.7 (medium)**.** Add a second MMIO device: a byte-wide output latch at 0x10000004 driving eight LEDs. How many Verilog lines change? What class of bug appears if two MMIO addresses alias by accident?

Solution. [hint] One more address compare; aliased ranges write both devices.

Exercise 8.8 (medium)**.** The demo pads UART writes with a software delay loop. Design the polling version instead: a read of 0x10000004 returns `busy`. Which datapath piece must grow to support an MMIO *read*?

Solution. [hint] The load mux — reads need a mux input just like stores need an address compare.

Exercise 8.9 (hard)**.** Argue for or against: on a real RV32I, the trap-entry redirect should also record `mtval` (the faulting address/instruction). What goes wrong in our design without it, for a handler that wants to emulate the trapped instruction?

Solution. [hint] Without the instruction bits or the bad address, emulation has nothing to emulate; the handler must decode `mepc`'s target itself — possible, but doubles the work.

Exercise 8.10 (hard)**.** We fixed the WB/ID hazard with a register-file bypass. An alternative is to write the regfile on the negedge of the clock. Compare the two: which buys the fix more cheaply, and which survives a timing-closed synthesis better? Defend a choice.

Solution. [hint] Negedge writes halve the regfile's setup slack; the bypass is two muxes but keeps everything single-edge.

Exercise 8.11 (hard)**.** Estimate the share of the demo program's cycles spent in its three delay loops (60 iterations each), using the measured run: the last start bit appears at cycle 772. What is the loops' CPI? How would a busy-poll loop change the accounting?

Solution. [hint] Each iteration retires 2 instructions (`addi`, `bne`) in about 4 cycles, so CPI ≈ 2. Three loops give $3 \times 60 \times 4 = 720$ cycles, about $720/772 \approx 93\%$ of the run. A busy-poll loop spins on a load plus a branch, also costing a few cycles per poll, but it exits the moment `busy` clears instead of waiting out a fixed worst case.

Exercise 8.12 (hard)**.** The core has no interrupt line — nothing can trap it from outside. Describe the minimal external interrupt mechanism (one pin, one pending bit) and where in the pipeline you would sample it. Why is "sample between instructions" easier said than done?

Solution. [hint] You need a point where the pipeline drains cleanly; sampling in EX like `ecall` works if the pin is level-sensitive and the handler disables re-entry.

Exercise 8.13 (easy)**.** The testbench runs the UART with `div`=16. If the same design were clocked at 27 MHz with `div`=16, how many clocks does one 8N1 frame take, what is the baud rate, and how many bytes per second can be sent at most?

Solution. [hint] A frame is 10 bits of 16 clocks each. Frame $= 10 \times 16 = 160$ clocks. Baud $= 27\,000\,000/16 = 1\,687\,500$ bits per second. Bytes per second $= 27\,000\,000/160 = 168\,750$.

Exercise 8.14 (medium)**.** Write the wire order (start bit, d0 to d7, stop bit) of the byte 0x41 ('A') and count the falling edges the decoder's edge counter would add for it, starting from an idle-high line.

Solution. [hint] $0x41 = 01000001$; send d0 first. Wire: 0 | 1 0 0 0 0 0 1 0 | 1. The 1-to-0 steps are the start bit, d0→d1 and d6→d7: 3 falling edges.

Exercise 8.15 (medium)**.** Reading the sweep: at $N = 40$ the gap between the first two start bits was 161 cycles, and at $N = 60$ it was 241. Fit the model gap $= aN + c$ to these two points, then predict the gap for $N = 50$ and say whether '2' survives.

Solution. [hint] Two points give the slope: $(241 - 161)/(60 - 40) = 4$ cycles per iteration, so $c = 161 - 4 \cdot 40 = 1$. Then gap$(50) = 201$. A frame needs ≈ 161 cycles from the store, so 201 is safe: all four bytes arrive.

CHAPTER 9

The memory hierarchy

Chapters 5–8 built a machine that retires one instruction per cycle. That claim hides an assumption this chapter finally confronts: *every memory access takes one cycle*. It does not. A load that hits L1 costs ≈1.4 ns here; the same load missing to DRAM costs 120 ns — an 85× swing the pipeline papers over with stalls you cannot see in a functional simulator. This chapter measures the real hierarchy on the machine in front of you, names the two localities that make it work, and turns the numbers into the contract every later chapter leans on.

AFTER THIS CHAPTER YOU CAN

You will be able to write a small C program that measures how long one memory read takes for a given amount of data, run it, and read the result as a staircase: fast for small data, slower in steps as the data grows. You will be able to say what each step is (a cache level, or main memory), explain why a sequential walk and a random walk behave differently, and avoid the three classic mistakes that make such a measurement lie.

BEFORE YOU START

What a memory read costs. In Book 1 chapter 1 (the contest machine and the cost model) we pretended that reading a variable costs one step. Chapters 5–8 of this book kept that pretence: a load takes one clock cycle. A *clock cycle* is one tick of the processor's metronome; at 3 GHz there are three billion ticks per second, so one tick is about a third of a nanosecond (1 ns is a billionth of a second). This chapter is where the pretence ends.
A five-line C refresher. The programs here are in C, so here is all the C you need to read them. `uint64_t x;` is a 64-bit whole number (8 bytes). `uint64_t *a;` says "`a` is the address of such a number"; `a[i]` is the *i*-th number at that address, and `malloc(n)` asks the system for *n* bytes of memory. A pointer is just an address stored in a variable, and `*p` means "read the number stored at address `p`". `for (long r = 0; r < reps; r++) { ... }` repeats its body `reps` times. The keyword `volatile` tells the compiler "do not optimise this variable away; really read and write it each time". Finally, `clock_gettime` returns a stopwatch reading; we subtract two readings to time a loop.

THINK LIKE THIS

A measurement is an experiment, and an experiment needs a hypothesis. Before running any benchmark in this chapter, write down in one line what you expect to see and why.

When the number disagrees, the disagreement is the interesting part: either your picture of the machine is wrong, or your program is not measuring what you think.

9.1 Why there is a hierarchy at all

Here is an everyday picture. You are studying at a desk. The book you are reading lies open in front of you (instant). Other books sit on a shelf behind you (a few seconds). The rest are in the library across town (an afternoon). Nobody would call this a design flaw: a desk big enough for the whole library would not fit in your room. A computer makes exactly this trade, for exactly this reason.

The ideal memory is fast, huge, and cheap. Physics offers any two. The SRAM bit-cell that feeds a 3 GHz pipeline uses six transistors — fast, and so large that a gigabyte of it would be a dinner plate. The DRAM cell that makes gigabytes affordable is one transistor and a leaky capacitor — dense, and so slow it answers in tens of nanoseconds and needs refreshing every ∼64 ms. Nobody compromised; the hierarchy is the compromise.

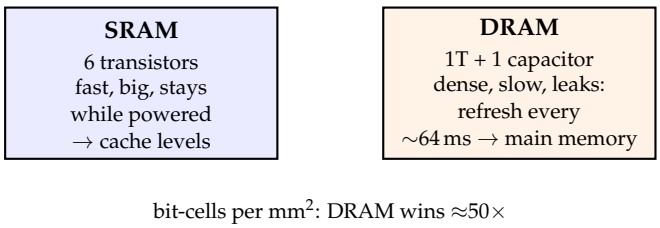


Figure 9.1: Why the hierarchy exists: the fast bit-cell (SRAM, 6T) is physically too large to hold gigabytes, and the dense one (DRAM, 1T1C) is too slow to feed a pipeline. Nobody picked this trade-off — physics did.

Example 9.1 (The compromise in one number)**.** On this machine the register file answers in ≈0.3 ns and DRAM in ≈120 ns — 400×. If every load went to DRAM, our "one instruction per cycle" core would retire one instruction per *hundred* cycles. The hierarchy exists so that *most* loads never take the slow path.

9.2 The memory mountain, measured

The classic experiment: build a circular linked list whose elements are scattered randomly through a working set, then chase it. Every load's address depends on the previous load's value, so the CPU's prefetcher and out-of-order engine cannot run ahead — each access pays the true latency of whichever level happens to hold the set. `ch09_mem_mountain.c` does exactly this, for sets from 4 KB to 128 MB, 20 million dereferences each.

```
for (size_t i = 0; i < n; i++)
    a[perm[i]] = (uintptr_t)&a[perm[(i + 1) % n]];
volatile uintptr_t p = (uintptr_t)&a[perm[0]];
for (long r = 0; r < reps; r++)
    p = *(volatile uintptr_t *)p; /* load depends on last load */
```

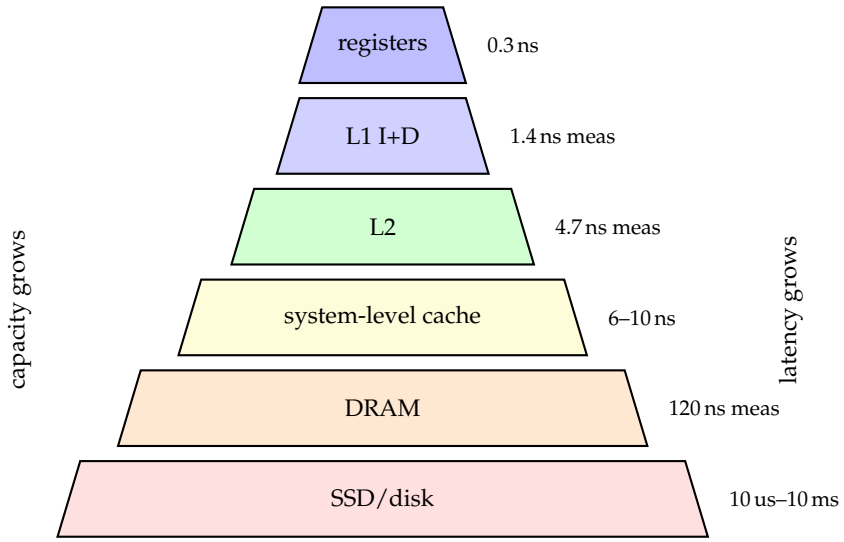


Figure 9.2: The hierarchy pyramid with *measured* numbers from this chapter's runs (Apple Silicon arm64): roughly 0.3 ns→1.4 ns→5 ns→10 ns→120 ns from register to DRAM — four orders of magnitude before storage even enters.

Listing 9.1: The heart of `mem_mountain.c`: the dependent-load chain.

Example 9.2 (What the author's Mac answered)**.** Verbatim output from the author's Apple-silicon Mac (arm64, macOS; the full log is in `code/ch09_mountain.dat`). The Linux x86_64 container numbers come later in this section and differ, which is the point:

```
   bytes    ns/load
   16384       1.40
  131072       1.45
  262144       4.70
 8388608      10.21
16777216      26.36
134217728     120.64
```

Four regimes on one arm64 part: ≈1.4 ns through 128 KB (L1/L2), a knee to 4.7 ns at 256 KB, a 5–10 ns plateau to 16 MB (the system-level cache), then the climb to 120 ns of DRAM. Those dashed knees in figure 9.3 *are* your cache sizes, read off silicon rather than a data sheet.

Checkpoint 9.1 Why a *random* chain and not sequential pointer-chasing? A sequential walk has stride 8 bytes; the prefetcher learns it in three accesses and the "latency" you measure is just bandwidth. The random permutation forces every level to reveal itself.

Now run it yourself. The same program, compiled with `gcc -O2` and run in the Linux x86_64 container used to write this edition (4 virtual CPUs of an Intel Xeon at 2.8 GHz under KVM; `getconf` reports a 32 KiB 8-way L1 data cache and a 1 MiB L2 per core), gives a

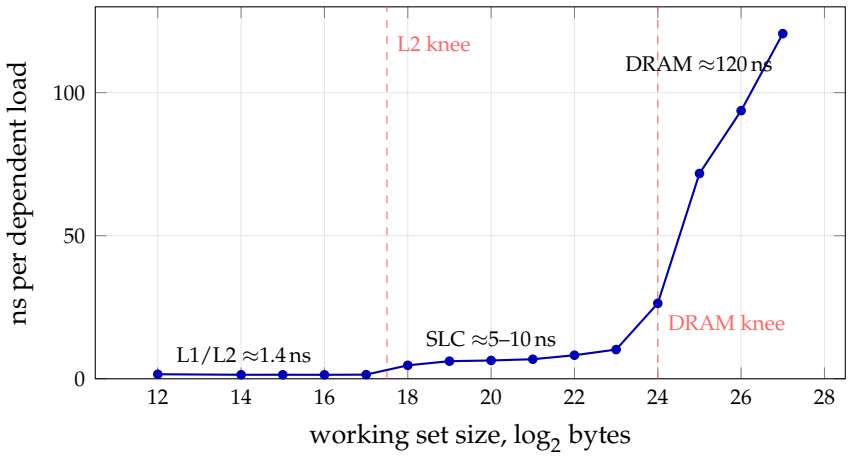


Figure 9.3: The memory mountain, measured on this machine with `ch09_mem_mountain.c`: each point is 20M dependent pointer dereferences through a random circular chain — no prefetcher can help, so each plateau is a real cache level.

different table. Treat these numbers as one sample; they move by 10–30% between runs on a shared virtual machine.

```
   bytes ns/load
    4096 2.99
   16384 3.02
   32768 2.93
   65536 4.48
  131072 5.07
  262144 5.38
  524288 7.39
 1048576 14.49
 2097152 19.25
 4194304 98.86
 8388608 131.69
16777216 154.43
33554432 156.94
67108864 162.71
134217728 196.25
```

Listing 9.2: Linux x86_64 container, one full run of `./ch09_mem_mountain` (20 million dependent loads per size). Captured in `code/ch09_mountain_linux.txt`.

Example 9.3 (Reading a second machine's staircase)**.** Read the Linux table as steps. Up to 32 KiB the time is flat at about 3 ns: the data fits in L1. At 64–512 KiB it is 4.5–7.4 ns: L1 has overflowed, and the loads now come from L2. At 1–2 MiB it is 14–19 ns, and past 4 MiB it jumps to about 100 ns and then 130–200 ns. The step positions (32 KiB, then about 1 MiB) match what `getconf` reports for L1 and L2, which is a check that the experiment measures what we think. Two surprises are worth saying aloud. First, the 100 ns at 4 MiB is far below the 33 MiB L3 size the container advertises; we did not isolate why (a virtual machine shares that cache with other tenants, and its page mapping is not ours to control), so we do not claim an L3 plateau. Second, even the L1 rows say 3 ns, roughly 8 clock cycles at 2.8 GHz, while an L1 hit should be about 4–5 cycles. The next example explains the surplus.

Example 9.4 (The benchmark taxes itself: what `volatile` costs)**.** Our chain loop declares the pointer `volatile`, so that the compiler cannot delete the loop. A naive reading says "that only protects the measurement". Test it. `ch09_volatile_cost.c` times the same 4 KiB chain, always an L1 hit, twice:

```
plain pointer : 1.26 ns/load (0)
volatile pointer: 2.93 ns/load
```

Listing 9.3: Linux x86_64 container, `./ch09_volatile_cost`: the same L1-resident chain with a plain and a `volatile` pointer.

The plain loop runs at 1.26 ns, about 3.5 cycles per load at the nominal 2.8 GHz, which is a believable L1 latency. The `volatile` loop is 2.3 times slower, because every step now also stores the pointer to memory and reads it back before the next load can start. So the staircase above is a true staircase sitting on a floor of roughly 1.7 ns of bookkeeping. The *shape* survives, and the knees are still right; the absolute numbers in the low rows are inflated. When you meet a benchmark number, ask what the loop itself costs.

Example 9.5 (Latency versus throughput: the independent-loads mistake)**.** The checkpoint above warned that independent loads measure the wrong thing. Here is the proof. `ch09_dep_vs_indep.c` visits the *same* random order two ways. In the dependent version each address is the value just loaded (`p = a[p]`), so the processor must wait for each answer. In the independent version the addresses come from a separate array of indices (`s += a[idx[k]]`), so the processor can send many requests at once and overlap the waiting.

```
    bytes dep ns/load indep ns/load ratio
    32768 1.71 1.12 1.5x (check 7035 1)
  1048576 10.53 2.04 5.2x (check 231304 0)
  8388608 135.61 12.03 11.3x (check 157217 1)
134217728 194.30 23.93 8.1x (check 30533386 1)
```

Listing 9.4: Linux x86_64 container, `./ch09_dep_vs_indep`: 20 million loads per row, same random visiting order, dependent versus independent (4-byte elements).

In L1 the two are close (1.7 versus 1.1 ns): there is nothing to hide. At 8 MiB and beyond the dependent chain pays the full 135–194 ns per load while the independent version averages 12–24 ns, as if roughly 8 to 11 requests were overlapped in flight at once (an inference from the ratio, not a hardware counter reading). Same data, same order, same machine, an 8 to 11 times difference in the answer. If the question is "how long does one cache miss take?", only the dependent chain answers it; the independent loop answers "how many misses per second can the memory system sustain?". Both are real numbers about the machine. They are different numbers.

9.3 Locality: the only two tricks caches know

A 4 KB L1 cannot hold a 4 GB program. It doesn't have to, because real programs are lumpy in exactly two ways:

- **Temporal locality**: an address used once is likely to be used soon again (loop counters, stack, hot data).
- **Spatial locality**: an address near a used one is likely to be used soon (arrays, sequential code, adjacent fields).

Caches exploit both: they keep recently-used lines (temporal) and fetch them 64 or 128 bytes at a time (spatial). Neither is a law — they are statistical bets, and figure 9.5 shows what it costs to lose the spatial one.

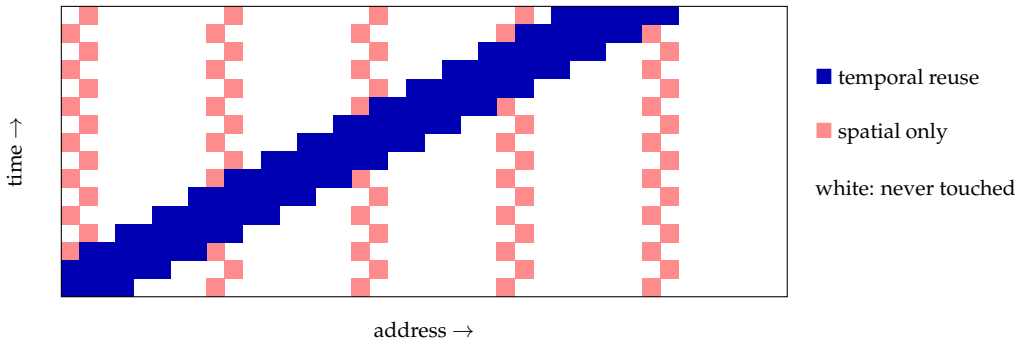


Figure 9.4: The two kinds of locality, drawn as an address-×-time map. Spatial locality (red) is free: the line arrives anyway. Temporal reuse (blue) is the one that actually turns into hits — a cache only helps if the same line is touched again before eviction.

Example 9.6 (Effective access time, honestly)**.** Suppose L1 hits at 95% and misses cost 10 ns. Then EAT $= 0.95 \times 1.4 + 0.05 \times 10 \approx 1.8$ ns. Drop hit rate to 85% and EAT rises by almost half, to 2.7 ns — the hierarchy's economics live in the last few percent of hit rate, which is why the next two chapters exist.

9.4 Bandwidth has a stride too

Latency is half the story. `ch09_stride_bw.c` reads a 64 MiB array at increasing strides — same total read, different bytes pulled across the bus per useful access.

Example 9.7 (The stride cliff, measured)**.**

stride	ns/acc	GB/s (eff)
1	1.073	7.45
8	1.032	7.75
16	3.693	2.17
32	3.780	2.12

Elements are 8 bytes, so a stride of 8 means each access lands on its own 64-byte cache line. The bus then moves 64 bytes to deliver 8 useful ones, for stride 8 as for every larger stride. On the Mac the time per access is flat up to stride 8 and then triples at stride 16, where the accesses stop landing on consecutive lines. We measured this cliff but did not isolate its

mechanism (the stride-128 dip in the data is likewise unexplained), so treat "the hardware prefetcher gives up" as a plausible story, not a finding. This is the column-order traversal trap every matrix chapter returns to.

Example 9.8 (The same experiment on Linux x86_64).

```
stride ns/acc GB/s(eff)
     1 2.103 3.80
     2 2.660 3.01
     4 4.239 1.89
     8 7.477 1.07
    16 14.154 0.57
    32 15.639 0.51
    64 14.166 0.56
   128 14.696 0.54
```

Listing 9.5: Linux x86_64 container, `./ch09_stride_bw`: 64 MiB array, 8-byte elements, 20 sweeps per stride. Captured in `code/ch09_stride_linux.txt`.

There is no sharp cliff here; the cost per access climbs smoothly from 2.1 ns at stride 1 to about 14 ns at stride 16 and then stays flat. Both machines agree on the lesson: the time per access stops growing once every access owns a cache line, which on this container happens by stride 16 (stride 8 already costs 3.6 times stride 1). (Stride 1 and 2 are partly limited by the `volatile` accumulator, as in the previous example.) Where the knee falls is a property of the machine; that it exists is a property of cache lines.

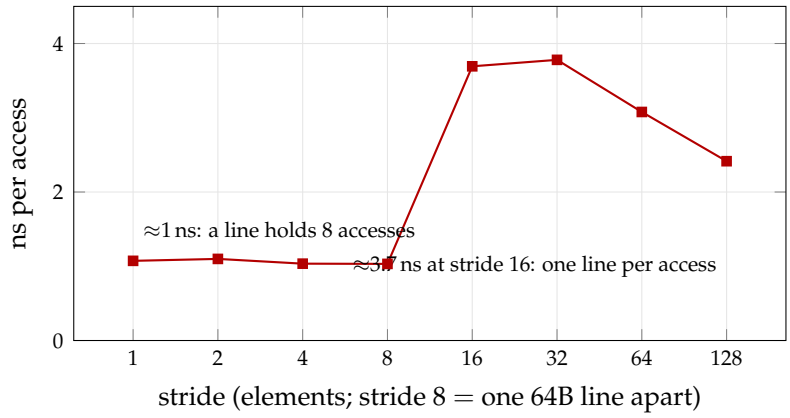


Figure 9.5: Same 64 MiB read, different strides (author's Mac, `ch09_stride_bw.c` output; the Linux run is in listing 9.5). Through stride 8 each 64-byte line serves eight loads; at stride 16 every load hauls in its own line — effective bandwidth collapses by ≈3×.

9.5 The contract

Each level answers to the one above and borrows from the one below; each is managed by someone else (registers by the compiler, caches by hardware, DRAM-vs-storage by the OS). When a later chapter says "the cache did X," this table is what it means.

level	typical size	our measured latency	managed by
registers	32 words	0.3 ns	ISA / compiler
L1	128–192 KB	1.4 ns	hardware
L2	a few MB	4.7 ns	hardware
SLC/LLC	8–16 MB	6–10 ns	hardware
DRAM	GBs	120 ns	hardware + OS
storage	TBs	μs–ms	OS

Figure 9.6: The levels as a contract: each is managed by the one above and services the one below. Latency column is this chapter's own measurement, not a data-sheet citation — your numbers will differ and the shape will not.

9.6 Common mistakes

- **Measuring latency with independent loads.** An out-of-order core overlaps a hundred of them; you measure throughput, not latency. The chain must be *dependent*.
- **Believing ns numbers without the working set.** 1 ns at 8 KB and 1 ns at 8 MB are different phenomena — latency is a function of footprint, not a constant.
- **Assuming a cache line is a word.** The stride experiment's whole point: the unit of transfer is 64–128 bytes. "I only read 8 bytes" still moves the whole line.
- **Optimising for a level that isn't the bottleneck.** Fixing L1 behaviour while the working set lives in DRAM is a rounding-error improvement — find your knee first.
- **Treating prefetchers as magic.** They learn *regular* patterns; a random-access structure (hash table, pointer graph) defeats them completely.

9.7 Summary

Idea	**Measured version**
hierarchy	0.3→1.4→5→10→120 ns, registers to DRAM
temporal locality	reuse before eviction: the cache's whole value
spatial locality	64–128B lines arrive free per access
the mountain	knees reveal L2 ≈256 KB, SLC ≈16 MB
stride cliff	whole-line-per-access ⇒ 3× slower
the contract	every level managed from above, feeds from below

Further reading

- Ulrich Drepper, *What Every Programmer Should Know About Memory* — the canonical long-form treatment: https://people.freebsd.org/~lstewart/articles/cpumemory.pdf
- Bryant & O'Hallaron, *CS:APP* ch6 — the memory mountain this chapter re-ran on current silicon: https://csapp.cs.cmu.edu/
- Hennessy & Patterson, *CAQA* ch2 + app B — hierarchy mathematics done carefully: elsevier.com — CAQA
- Agner Fog, *Optimizing software in C++* — microarch measurement technique: https://www.agner.org/optimize/

WATCH INSTEAD

- **Caches Explained — Computerphile** — the pyramid and locality in 9 minutes (verified 2026-10-11). https://youtu.be/vj3F_4zIaIg

Notes

- The first tables (mountain, stride) are verbatim runs of this chapter's C benchmarks on the author's Apple-silicon Mac (arm64). The tables labelled "Linux x86_64 container" are new runs on a 4-vCPU Intel Xeon (2.8 GHz, KVM guest, gcc `-O2`); the exercise asks you to re-run them on your own machine and compare knees. `code/test_ch09.sh` checks only the shape of the results, never exact nanoseconds.
- The stride-128 dip in the log is real and unexplained — prefetcher interaction, flagged in-chapter rather than hidden.

Exercises

Exercise 9.1 (easy)**.** From our mountain data, estimate the latency ratio between a register read and a DRAM read on this machine. Then express DRAM latency in "human scale": if L1 were one second away, how far is DRAM?

Solution. [hint] 0.3 ns vs 120 ns; scale 1.4 ns→1 s.

Exercise 9.2 (easy)**.** A program scans a 100 MB array twice. Between scans, nothing else runs. Which locality saves the second scan, and which cache level does it live in?

Solution. [hint] 100 MB exceeds every level; the second scan is as cold as the first — no level helps.

Exercise 9.3 (easy)**.** Why does the pointer-chase use a *circular* list and not a linear one?

Solution. [hint] Same traversal cost for warmup and measurement; no unpredicted tail.

Exercise 9.4 (easy)**.** Name the two localities and give one C construct that exhibits each.

Solution. [hint] Loop variable vs. array walk.

Exercise 9.5 (medium)**.** Predict, then verify with `stride_bw`: does a stride-64 read of a 64 MiB array transfer the same total bytes over the bus as stride-1? What *does* change?

Solution. [hint] Same array is swept; useful bytes per fetched line changes $8\times$.

Exercise 9.6 (medium)**.** Our EAT example assumed one miss cost. Real misses chain: an L1 miss may hit in L2. Extend the formula to three levels and compute EAT for L1=95% (1.4 ns), L2=90% (5 ns), DRAM base 120 ns.

Solution. [hint] Model: every access pays the L1 lookup (1.4 ns); on the 5% that miss, pay 5 ns for L2, and on the 10% of those that also miss, 120 ns more. $\text{EAT} = 1.4 + 0.05\,(5 + 0.10 \times 120) = 1.4 + 0.05 \times 17 = 2.25\,\text{ns}$.

Exercise 9.7 (medium)**.** The mountain's L1/L2 knee sits between 128 KB and 256 KB on this part. Bisect the size sweep to pin the L2 boundary to one binary digit; describe how you'd distinguish L1-vs-L2 knees from a SLC-vs-DRAM knee.

Solution. [hint] Re-run with sizes $2^{17.5}$, $2^{18.5}$; the DRAM knee's slope is far steeper.

Exercise 9.8 (medium)**.** A colleague optimises a hash table for "cache friendliness" by shrinking each entry. Why might that *not* help, given what the stride experiment showed about the unit of transfer?

Solution. [hint] Random access means each probe fetches a whole line; entry size matters only at line granularity.

Exercise 9.9 (hard)**.** Infer L1 line size and associativity *from stride experiments alone*: given only `stride_bw`-style timing over varying strides and sets, describe the experimental protocol that pins down the line length, then the associativity. What measurements disambiguate a 64B/4-way cache from a 128B/2-way one?

Solution. [hint] Stride-vs-capacity curve gives line size; associativity needs conflict misses — sweep k addresses spaced at set-straddle distance.

Exercise 9.10 (hard)**.** Our mountain flattens at 120 ns for DRAM. Row-buffer hits make real DRAM cheaper than the worst case. Design the experiment that separates row-hit from row-miss latency, and predict the two numbers for a system whose mountain you haven't run yet.

Solution. [hint] Same-bank adjacent-column accesses vs. alternating-bank accesses; expect 40–60 ns vs. 80–120 ns class numbers.

Exercise 9.11 (hard)**.** Argue that temporal locality is a program property, not a hardware property — then explain why the *effectiveness* of exploiting it is a hardware property. Use the blocked-transpose preview from ch10's forward reference.

Solution. [hint] Reuse distance is fixed by the code; whether it fits is set by capacity and associativity.

Exercise 9.12 (easy)**.** From the contract table, which level is managed by software you write (not the compiler, not hardware, not the OS page allocator)?

Solution. [hint] None — registers are the closest you get, and the ISA owns those. That is the point: caching is a hardware service.

Exercise 9.13 (medium)**.** `mem_mountain` reports ns/load, not cycles. Convert our 1.4 ns and 120 ns numbers to cycles on a 3.5 GHz core, and state in one line why cycle-counting is the unit that matters to a pipeline.

Solution. [hint] 4.9 and 420 cycles; the stall counter is what the core sees.

Exercise 9.14 (medium)**.** The stride cliff lands between 8 and 16 elements. What does that imply about this machine's cache line size in bytes, and how would you test the hypothesis with one more run?

Solution. [hint] 64B lines: stride 8 fits 8 accesses/line; stride 16 overflows. Sweep elements, not bytes.

Exercise 9.15 (hard)**.** Some systems advertise an "effective" memory latency far below DRAM's 120 ns. Reconcile that marketing number with your mountain: under what access pattern is it true, and what pattern exposes the real number? Name the benchmark trick being used.

Solution. [hint] Sequential/streaming lets prefetch hide latency — SPEC-style streams vs. dependent pointer chase.

CHAPTER 10

Caches: geometry, misses, and the loop that got 3× faster

Chapter 9 showed the staircase: about 1 ns when the data is small, over 100 ns when it is large. This chapter opens the box that makes the staircase. We will build a *cache simulator* in about 60 lines of C, check it by hand on a ten-access example, and then use it to predict which of several loop shapes will make a matrix transpose faster. Finally we run the real thing and see how much of the prediction survives contact with real silicon.

AFTER THIS CHAPTER YOU CAN

You will be able to take any cache description (for example "64 sets, 4 ways, 64-byte lines") and, for any address, say which set it falls in and what its tag is. You will be able to trace a short access sequence through such a cache by hand, write a simulator that agrees with your hand trace, sort every miss into one of three kinds, and rewrite a loop into tiles so that it misses far less.

BEFORE YOU START

From chapter 9. Main memory is slow (over 100 ns on our machines) and a small memory close to the processor, the *cache*, is fast (a few ns). The cache does not hold single bytes; it holds *lines* of 64 bytes at a time, because programs tend to use nearby addresses soon after (spatial locality) and the same addresses again soon (temporal locality). A *hit* means the data was in the cache; a *miss* means it had to be fetched.

Binary and remainders. An address is a binary number. Dividing by 2^k and keeping the quotient is the same as dropping the lowest k bits (Book 5 chapter 8, numbers, bits and Boolean algebra); the remainder a mod m, written `a % m` in C, is what is left after taking out as many whole m's as possible (Book 2 chapter 6, modular arithmetic). We use both to chop an address into fields.

A ten-line C refresher. We write the simulator in C, so here is what you need. A `struct` bundles named fields, as in `typedef struct { int valid; unsigned long tag; } Line;`, and `l.tag` or, through a pointer, `p->tag` reads a field. `Line *ln = calloc(n, sizeof(Line));` reserves room for n lines, all zero. An array `float a[N][N]` is stored row after row, so `a[i][j]` sits at byte offset $(iN + j) \cdot 4$ from the start. `x >> k` shifts the bits of `x` right by k places (dividing by 2^k), `x & 63` keeps only the low 6 bits, and `x % 64` gives the same remainder. `uint64_t` is an unsigned 64-bit integer. A function that returns `int` can pack several yes/no answers into its bits.

THINK LIKE THIS

When something is too complicated to reason about, write the simplest model that could be wrong, run it on a case small enough to check by hand, and only then trust it on a case too large to check. A model you have not tested against a hand calculation is a number generator.

10.1 Anatomy: three parameters, one equation

Start with a picture you know: a library with a reading room. The reading room cannot hold the whole library, so it has a rule for where each book may sit. The simplest rule says "book number k may sit only on shelf k mod 8". The librarian never has to search the whole room; she looks at one shelf. A cache does exactly that. The shelves are called *sets*, the places on a shelf are called *ways*, and a book is a *line* of B bytes. Why have a rule at all? Because checking every slot of a big cache on every load would be too slow; checking one set of a few slots is fast.

A cache is therefore $C = S \times E \times B$: S *sets*, each holding E *lines* (also called ways) of B bytes. A 32-bit address splits three ways. The low $b = \log_2 B$ bits pick a byte inside the line (the *offset*). The next $s = \log_2 S$ bits pick the set (the *index*). Everything above is the *tag*, which tells apart the many lines that share one set.

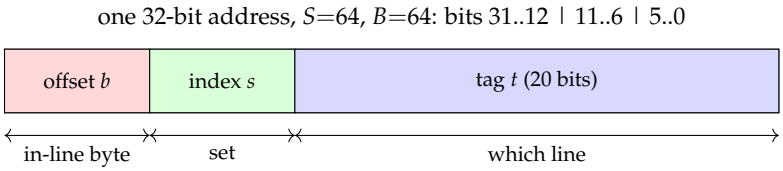


Figure 10.1: The address split for S=64 sets and B=64-byte lines: the low 6 bits are the `offset` (byte within the line), the next 6 are the set `index`, and the remaining 20 bits are the `tag` that tells lines sharing a set apart. Every cache geometry question reduces to where these boundaries fall.

symbol	meaning	our sim values
S	sets	32 or 64
E	lines per set (ways)	1–8
B	block (line) bytes	64
C	capacity $= S \cdot E \cdot B$	2–16 KB (E-sweep), 16 KB (toy L1), 32 KB (container L1d)

Figure 10.2: Cache anatomy in three parameters. $C = S \times E \times B$ is the only equation the chapter needs: geometry questions are bookkeeping, performance questions are about what these parameters do to miss types.

Example 10.1 (Decoding a real geometry)**.** For S=64, E=4, B=64: $C = 64 \cdot 4 \cdot 64 = 16\,384$ bytes = 16 KB. Take address `0x00001234` = 4660. The low 6 bits are the offset: 4660 mod 64 = 52 (`0x34` & 63). Drop them: $4660 \gg 6 = 72$, the line number. The set is the next 6 bits:

72 mod 64 = 8. The tag is what remains: $72 \gg 6 = 1$ (equivalently `0x1234` $\gg 12 = 1$). So the line lives in set 8, in one of its four ways, tagged 1. (An earlier edition of this chapter said set 18; that was an arithmetic slip, and the corrected value is 8.)

Example 10.2 (Decoding by the naive guess, then by the rule)**.** A first guess is "consecutive addresses go to consecutive sets". Test it with the same geometry. Addresses $0, 1, 2, \ldots, 63$ all have line number 0, so all sit in set 0: the cache keeps whole lines together on purpose. The next line, addresses $64 \ldots 127$, goes to set 1. So *consecutive lines* go to consecutive sets, and the set number wraps around after S lines, that is after $S \cdot B = 4096$ bytes. Two addresses exactly 4096 bytes apart therefore always fight for the same set. That single fact, "addresses $S \cdot B$ bytes apart collide", explains most of the surprises later in this chapter.

10.2 Replacement and the three Cs

When a set is full and a new line arrives, something must leave. The textbook answer is *LRU*, least recently used: throw out the line that has gone longest without being touched. The thinking behind it is temporal locality: what you have not used for a long time you are unlikely to use next. Writes add policy: *write-back* (a modified line, called *dirty*, goes to memory only when it is evicted) versus *write-through* (every store goes to memory immediately); *write-allocate* (a store miss first loads the line) versus no-allocate. Our model is write-back plus write-allocate, the common desktop choice.

miss type	cause	the fix it suggests
compulsory	first touch ever	prefetch, larger lines
capacity	working set $> C$	shrink the set (tile), bigger cache
conflict	too many lines to one set	associativity, layout/padding

Figure 10.3: The three Cs. Classify the miss before fixing it. In the chapter's simulator the naive transpose is 88% capacity misses and the block-32 cliff is 88% conflict misses (section 10.3), and the two call for different cures.

Every miss is one of three kinds. A *compulsory* miss is the first touch of a line anywhere in the program: nobody could have had it. A *capacity* miss happens because the program's working set is larger than the whole cache: even a perfect cache of that size would miss. A *conflict* miss happens when the data would fit in the cache overall but too many lines map to the same set. The diagnosis decides the cure, which is why figure 10.4 matters.

Example 10.3 (Associativity is not a panacea)**.** The associativity sweep on the transpose workload (listing 10.3, section A): 54.6% misses at one way, 53.1% at two, and *no further gain* at 4 or 8. This workload's misses are capacity-dominated, as section 10.4 will confirm with a classification: throwing ways at them is wasted silicon, and the cure has to reduce the working set instead. (The sweep used a deliberately small cache of 32 sets, so C runs from 2 KB to 16 KB.)

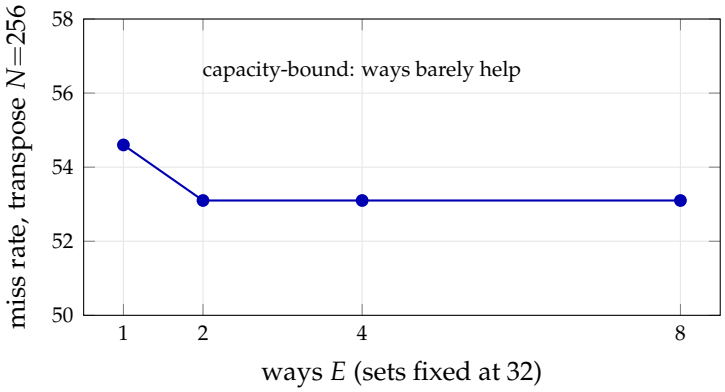


Figure 10.4: Output of `ch10_cache_sim` (section A): pushing associativity 1→8 buys 1.5 points of miss rate (54.6% to 53.1%), and everything past 2 ways buys nothing. The misses are mostly *capacity*, not conflict.

10.3 The simulator

A *simulator* is a program that behaves like a machine we do not want to build. Ours keeps, for every set, a small array of lines, each with a valid bit, a tag, a dirty bit and a time stamp. Its job on each access is the three steps you would do by hand: split the address, look for the tag in the set, and on a miss evict the oldest line. The core (the whole model is in `code/ch10_cache.h`, about 40 lines):

```
static int cache_access(Cache *c, uint64_t addr, int is_write) {
   c->clock++;
   uint64_t block = addr >> c->bbits; /* drop the offset bits */
   uint64_t set = block % (uint64_t)c->S; /* next s bits: the set */
   uint64_t tag = block / (uint64_t)c->S; /* the rest: the tag */
   Line *l = &c->ln[set * c->E];
   for (int i = 0; i < c->E; i++)
      if (l[i].valid && l[i].tag == tag) {
         l[i].lru = c->clock; c->hits++;
         if (is_write) l[i].dirty = 1;
         return 0;
      }
   c->misses++;
   /* ... pick a victim way, count the eviction, install the tag ...
      */
}
```

Listing 10.1: The access core of the model. Hit: a valid way holds the tag. Miss: take a free way, otherwise evict the way with the smallest (oldest) time stamp.

Example 10.4 (Trace ten accesses by hand, then check the machine)**.** Take a tiny cache: S=2 sets, E=2 ways, B=8 bytes (32 bytes in all) and read these byte addresses in order: $0, 8, 16, 24, 0, 32, 8, 40, 16, 0$. The line number is $\lfloor a/8 \rfloor$, the set is the line number mod 2, and the tag is the line number $\gg 1$. Walk through it, keeping each set as a list with the oldest line first.

#	addr	set	tag	result	set contents afterwards (oldest first)
1	0	0	0	miss	set0: [0]
2	8	1	0	miss	set1: [0]
3	16	0	1	miss	set0: [0,1]
4	24	1	1	miss	set1: [0,1]
5	0	0	0	hit	set0: [1,0] (0 is now newest)
6	32	0	2	miss, evicts tag 1	set0: [0,2]
7	8	1	0	hit	set1: [1,0]
8	40	1	2	miss, evicts tag 1	set1: [0,2]
9	16	0	1	miss, evicts tag 0	set0: [2,1]
10	0	0	0	miss, evicts tag 2	set0: [1,0]

By hand: 2 hits, 8 misses, 4 evictions. Notice access 9: the line with tag 1 was thrown out at step 6 only because step 5 had just refreshed tag 0, so the least recently used line was tag 1. LRU is the reason the hit at step 5 *causes* the miss at step 9. Now ask the program (`code/ch10_trace.c`, run in the Linux container):

```
S=2 E=2 B=8 (C=32 bytes)
# addr set tag result
1 0 0 0 MISS
2 8 1 0 MISS
3 16 0 1 MISS
4 24 1 1 MISS
5 0 0 0 hit
6 32 0 2 MISS (evicted a line)
7 8 1 0 hit
8 40 1 2 MISS (evicted a line)
9 16 0 1 MISS (evicted a line)
10 0 0 0 MISS (evicted a line)
hits=2 misses=8 evictions=4 dirty_evictions=0
```

Listing 10.2: `./ch10_trace 2 2 8 0 8 16 24 0 32 8 40 16 0`. The machine agrees with the hand trace line for line.

Example 10.5 (The bug the first simulator hid)**.** Here is a receipt from this chapter's own history. The first version of the simulator computed the set with `b = ctz(B); set = (addr / b) % S` where `ctz(64)` is 6, the *number of offset bits*. It divided the address by 6 instead of by $2^6 = 64$ (the correct step is `addr » b`, as in listing 10.1). The program compiled, ran, and printed plausible miss counts (76 459 for the naive transpose, then "block-16 is worse than naive", which we proudly wrote into the chapter). Those numbers were produced by a cache whose sets mixed unrelated addresses, and the "block-16 cliff" was an artifact. What caught it was the chapter's own listing, which showed the right formula while the code did not. Lesson: run the ten-access trace above *before* trusting any large result. The ten-access trace fails on the buggy version; it takes one second.

The corrected simulator also uses a fixed, made-up base address for the matrices (`0x10000000`) instead of the real address of a C array, because the old results changed from run to run with where the linker put the array. Everything printed in this chapter's simulator listings is therefore exactly reproducible.

```
== A: miss rate vs associativity (N=256 floats, S=32 sets, B=64)
E=1 misses=71552 evictions=71520 miss rate=54.6%
E=2 misses=69632 evictions=69568 miss rate=53.1%
E=4 misses=69632 evictions=69504 miss rate=53.1%
E=8 misses=69632 evictions=69376 miss rate=53.1%
== B: toy L1 S=64 E=4 B=64 (16 KB), N=256 floats (256 KB per matrix)
naive misses=69632 evictions=69376 miss rate=53.1%
block2 misses=36864 evictions=36608 miss rate=28.1%
block4 misses=20480 evictions=20224 miss rate=15.6%
block8 misses=12512 evictions=12256 miss rate=9.5%
block16 misses=10064 evictions=9808 miss rate=7.7%
block32 misses=69632 evictions=69376 miss rate=53.1%
```

Listing 10.3: `./ch10_cache_sim`, first two sections: miss counts of the naive transpose of a 256×256 float matrix across associativity (A), and naive versus blocked on the 16 KB toy L1 (B). Exact, deterministic. Full output in `code/ch10_cache_sim_linux.txt`.

10.4 The payoff: blocking

Why the naive transpose is slow

A transpose copies `a[i][j]` to `b[j][i]`. In C a matrix is stored row after row, so as `j` increases the read `a[i][j]` walks along memory one float at a time, while the write `b[j][i]` jumps down a column by a whole row, $N \times 4$ bytes (1 KB for N=256). The read stream uses all 16 floats of each line it loads. The write stream touches a different line on every step, and the loop then moves to the next `i`, which needs the *same* 256 lines again; but by then 256 other lines have been touched in a 16 KB cache that holds only 256 lines in total, and the one extra read line is enough to push things out. So the misses are plain capacity.

Blocking (also called *tiling*) changes the visiting order so that the data touched by a short stretch of the loop fits in the cache, and each line is used completely before it is evicted. We walk the matrix in small $bs \times bs$ tiles, finishing one tile before moving to the next.

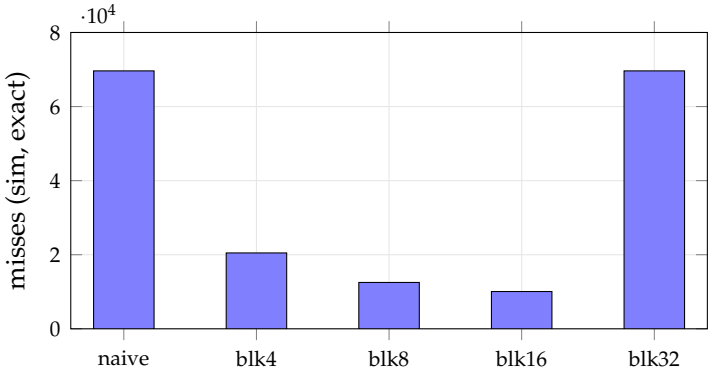


Figure 10.5: Exact miss counts from the chapter's cache model (toy 16 KB L1: 64 sets, 4 ways, 64-byte lines), 256×256 float transpose. Block-8 removes 82% of the misses and block-16 86%, but block-32 is back at the naive count: the tile no longer fits in the sets its rows map to.

Example 10.6 (Which tile size? Let the model answer)**.** Section B of the simulator (listing 10.3) gives, for the 16 KB toy L1 on a 256 × 256 float transpose: naive 69 632 misses; tile 4: 20 480; tile 8: 12 512; tile 16: **10 064** (6.9 times fewer than naive); tile 32: 69 632, *exactly as bad as naive*. Misses fall as the tile grows, then fall off a cliff. A first guess is "bigger tile, fewer misses, until the tile exceeds the cache". But the 32-by-32 float tile is only 4 KB, a quarter of the cache, so that guess is wrong. The cliff comes from the *sets*, not from the total capacity, as the next example shows.

Example 10.7 (Why the cliff sits at tile 32: the set budget)**.** The destination rows are 1 KB apart, which is 16 lines. Line numbers that differ by 16 map to set numbers that differ by 16 (mod 64), so the rows of a tile reach only the sets 0, 16, 32, 48: just **4 sets**, each with 4 ways, giving room for $4 \times 4 = 16$ lines. A 16-row tile needs 16 destination lines: it fits (with a few conflicts from the source line, hence the 1 872 conflict misses below). A 32-row tile needs 32 lines in 16 slots, and LRU then evicts each line just before it is needed again. The simulator can prove this, because it can classify misses. It keeps a second, *fully associative* cache of the same 16 KB (one set with 256 ways, so no conflicts possible): a miss that the shadow cache also has is a capacity miss, one it does not have is a conflict miss, and the first touch of a line is compulsory.

```
== D: three Cs, same toy L1 as B (fully-assoc. LRU shadow, same 16 KB)
naive compulsory=8192 capacity=61440 conflict=0 total=69632
block8 compulsory=8192 capacity=4096 conflict=224 total=12512
block16 compulsory=8192 capacity=0 conflict=1872 total=10064
block32 compulsory=8192 capacity=0 conflict=61440 total=69632
```

Listing 10.4: Section D of `./ch10_cache_sim`: the three Cs of each transpose on the toy L1.

Read the table like a doctor reads a chart. The naive transpose has **no conflict misses at all**: its 61 440 extra misses are pure capacity, because a column of 256 lines does not fit even in a perfectly flexible 256-line cache once the source line is added. That is why more ways did not help in section A. Block-32 is the opposite: zero capacity misses, 61 440 **conflict** misses. The cache is big enough; the address pattern aims it all at 4 sets. Two different diseases, two different cures: tiling fixes the first; padding the rows fixes the second. We can test that claim rather than assert it. If each row is padded from 256 to 272 floats (1 088 bytes, 17 lines, so successive rows now start in different sets), section G of the simulator reports block-32 at **8 192** misses, exactly the compulsory minimum, and the naive transpose falls from 69 632 to 24 792.

Example 10.8 (Scaling up: the same story on a real 32 KB L1)**.** Now the size the real timing run uses, 2048 × 2048 floats, with the geometry that `getconf` reports for the container's L1 data cache (32 KB, 8-way, 64-byte lines, so $S = 64$). A row is $2048 \times 4 = 8192$ bytes $= 128$ lines, and $128 \bmod 64 = 0$: every row of a tile lands in the *same single set*, which has only 8 ways. So the tile can have at most 8 rows before conflicts begin:

```
== C: S=64 E=8 B=64 (32 KB, like this container's L1d), N=2048 floats
naive misses=4456448 evictions=4455936 miss rate=53.1%
block4 misses=1319936 evictions=1319424 miss rate=15.7%
block8 misses=847872 evictions=847360 miss rate=10.1%
block16 misses=4456448 evictions=4455936 miss rate=53.1%
block32 misses=4456448 evictions=4455936 miss rate=53.1%
```

```
block64 misses=4456448 evictions=4455936 miss rate=53.1%
```

Listing 10.5: Section C of `./ch10_cache_sim`: 2048 × 2048 floats on a 32 KB, 8-way L1.

Tile 8 has 5.3 times fewer L1 misses than naive (4 456 448 / 847 872); tile 16 and above fall back to the naive count. The predicted rule is: *the tile height must not exceed the number of ways when the row length is a multiple of $S \cdot B$.* A power-of-two matrix size is the worst case for caches, which is why padding is a standard trick.

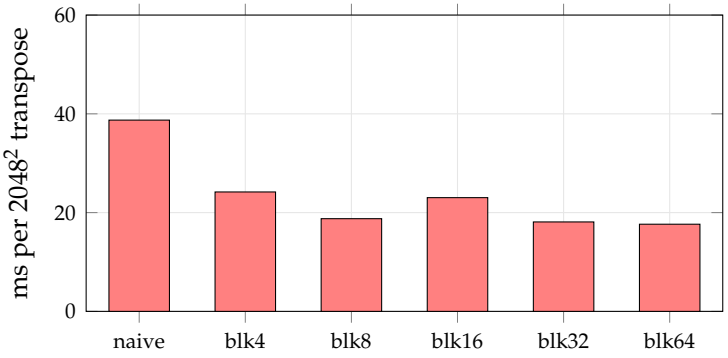


Figure 10.6: Real wall clock, Linux x86_64 container, best of 7 full sweeps (`code/ch10_transpose_linux.txt`). Every blocked loop beats the naive one (1.6–2.2× in this run; 2.1–3.0× for block-8 over six runs). The simulator's L1 cliff at block-16 shows only as a bump here, because the 1 MiB L2 catches the L1 conflict misses.

Now the real machine

```
naive 0 2048x2048: 38.73 ms (1.00x vs naive) result ok
block 4 2048x2048: 24.18 ms (1.60x vs naive) result ok
block 8 2048x2048: 18.77 ms (2.06x vs naive) result ok
block16 2048x2048: 23.03 ms (1.68x vs naive) result ok
block32 2048x2048: 18.11 ms (2.14x vs naive) result ok
block64 2048x2048: 17.65 ms (2.19x vs naive) result ok
```

Listing 10.6: `./ch10_transpose_real` (Linux x86_64 container, gcc `-O2`). Each row is the best of 7 full sweeps of a 2048 × 2048 float transpose, with a checksum to prove the result is the same transpose. Captured in `code/ch10_transpose_linux.txt`.

Example 10.9 (Model against silicon: what survived and what did not)**.** Compare the simulator's prediction with the stopwatch. Survived: blocking helps a lot (block-8 is 2.06 times faster than naive in this run, and 2.06 to 2.96 times over six runs of the program), and the answer is correct (the checksum matches). Did not survive: the L1 model said tiles 16 and larger would be as bad as naive, but the wall clock shows tile 16 at 23 ms (between naive and block-8), and tiles 32 and 64 as the *fastest*. Why? The L1-only model is missing a level. The container has a 1 MiB, 16-way L2 behind the L1, and a tile of 32 rows is easily held there. We can test that explanation in the simulator by adding the L2 (section E, an L1 miss becomes an L2 access):

```
== E: L1 (S=64 E=8) backed by L2 (S=1024 E=16, 1 MiB), N=2048 floats
naive L1 misses=4456448 L2 misses=4456448
```

```
block4 L1 misses=1319936 L2 misses=1310720
block8 L1 misses=847872 L2 misses=786432
block16 L1 misses=4456448 L2 misses=524288
block32 L1 misses=4456448 L2 misses=524288
block64 L1 misses=4456448 L2 misses=524288
```

Listing 10.7: Section E of `./ch10_cache_sim`: L1 32 KB 8-way backed by an L2 of 1 MiB, 16-way ($S = 1024$).

With the L2 in the model, tiles 16, 32 and 64 all reach 524 288 L2 misses, which is exactly the compulsory minimum (two 16 MiB matrices / 64 bytes), while the naive loop misses the L2 every single time. So the two-level model correctly predicts that the naive loop is the loser and that big tiles are fine. It does *not* explain why tile 16 is slower than tile 8 on the real machine; we did not isolate that (the hardware prefetchers and the L2-to-L1 bandwidth are candidates, not findings). The honest summary is that a cache model predicts *which loop wins and roughly why*, and a stopwatch is still needed for the numbers.

Example 10.10 (A harness that flattered the answer)**.** An earlier edition of this chapter reported, from the author's Apple Silicon Mac, "naive 14.3 ms, block-8 5.0 ms, a 2.9 times speed-up". We could not reproduce that in a fair comparison, and the reason is in the harness. The blocked loops ran their four repetitions *inside* each tile (so the second to fourth repetitions found the tile already in cache), and the printed blocked times were not divided by the repetition count, while the naive time was. Equal work was not being timed. The harness in this chapter does one full sweep per trial for every variant, takes the best of 7, and checks the result. We keep the chapter title's "3 times" because block-8 over six runs spans 2.06 to 2.96 times on the container, but the Mac figures of the earlier edition are retired. The general habit of mind: before believing a speed-up, check that the two sides did the same work.

Checkpoint 10.1 Why does a tile need to fit the *set budget* and not just the total capacity?
Because a line may live only in one set. If many of the tile's lines map to the same few sets, those sets overflow while the rest of the cache sits empty. Capacity is the sum over all sets; a tile is limited by the busiest one.

10.5 Software-visible effects beyond tiling

- **Alignment**: a 64-byte object straddling a line boundary costs two accesses; align hot structures.
- **False sharing** (preview of the multicore chapter): two threads writing adjacent bytes in one line bounce it between cores — the fix is padding, and it's cache geometry leaking through the abstraction.
- **Layout**: array of structs versus struct of arrays decides whether a walk touches a whole line per element or sixteen 4-byte elements per line.
- **Power-of-two strides**: as the 2048 example showed, row lengths that are multiples of $S \cdot B$ concentrate accesses in few sets; padding the row by one line fixes it.

10.6 Common mistakes

- **Optimising associativity for capacity misses.** Our E-sweep: going from 2 to 8 ways changed nothing (53.1%). Classify the miss first.
- **Bigger tile = better, assumed.** Block-32 had 61 440 conflict misses and no capacity misses at all, because its rows aimed at 4 of 64 sets. The tile must fit the set budget, not just the capacity.
- **Trusting a simulator you never hand-checked.** The first version of ours divided by the offset *width* instead of the line size and printed plausible nonsense. Trace ten accesses by hand first.
- **Comparing unequal work.** A speed-up from a harness that repeats a hot tile but divides only the other side is not a speed-up.
- **Counting accesses, not misses.** Both transposes make the same $2N^2$ accesses; only the miss count changes.
- **Believing the model equals hardware.** We checked prediction against the wall clock: the L1-only model got the loser right but missed the L2's rescue of large tiles.
- **Write policies as an afterthought.** Write-back + write-allocate hides store latency in the same lines reads use; no-allocate turns every store miss into a bus trip.

10.7 Summary

Piece	Verified version
$C = S \cdot E \cdot B$	the only geometry equation needed; address = tag ∣ index ∣ offset
three Cs	compulsory, capacity, conflict; classify before you fix
LRU/write-back	what the sim implements and what real L1s do
hand trace	10 accesses, 2 hits, 8 misses, 4 evictions: sim agrees
naive transpose	69 632 misses, 88% capacity, 0 conflict (toy L1)
block-16 / block-32	10 064 misses versus 69 632: a set-budget cliff, 61 440 conflict misses
padding	rows of 272 floats: block-32 back to the 8 192 compulsory minimum
sim vs silicon	L1 model finds the loser; L2 model explains big tiles; block-8 2.06–2.96× faster measured

Further reading

- Bryant & O'Hallaron, CS:APP ch6 + Cache Lab — `https://csapp.cs.cmu.edu/`
- Hennessy & Patterson, CAQA ch2/app B — elsevier.com — CAQA
- Ulrich Drepper, cpumemory (cache section) — `https://people.freebsd.org/~lstewart/articles/cpumemory.pdf`
- Agner Fog, optimisation manuals — `https://www.agner.org/optimize/`

WATCH INSTEAD

- **DDCA Ch8 Caches, Parts 1–2** — Sarah Harris walks the same anatomy (verified 2026-10-11). https://youtu.be/yi0eCq7mHFQ

Notes

- All miss counts are exact outputs of `ch10_cache_sim.c` (LRU, write-back, synthetic addresses; deterministic). They replace the counts of the first edition, which came from a simulator with an addressing bug (see the bug receipt in section 10.3). All timings are wall-clock runs of `ch10_transpose_real.c` in the Linux x86_64 container (Intel Xeon 2.8 GHz, KVM guest, 4 vCPU); the first edition's Apple Silicon timings are retired because that harness timed unequal work. `code/test_ch10.sh` checks the simulator's numbers exactly and the timings by shape.
- The checksum sink in the timing harness exists because the first build's transpose was optimised away entirely — dead-code elimination, the chapter's own common-mistake example in action.

Exercises

Exercise 10.1 (easy)**.** For S=256, E=8, B=64: find C, the number of offset and index bits, and the set and tag of address `0xCAFEBABE`.

Solution. [hint] $C = 256 \cdot 8 \cdot 64 = 131\,072$ bytes $= 128$ KB; $b = 6$, $s = 8$. Offset $=$ `0xCAFEBABE` & 63 $= 62$. Set $=$ (`0xCAFEBABE` $\gg 6$) & 255 $= 234$. Tag $=$ `0xCAFEBABE` $\gg 14 =$ `0x32BFA`.

Exercise 10.2 (easy)**.** Which of the three Cs does a never-run program's very first access produce, and which fix applies?

Solution. [hint] Compulsory; prefetch or longer lines.

Exercise 10.3 (easy)**.** Write-back versus write-through in one sentence each: what leaves the cache, and when?

Solution. [hint] Write-back: only dirty lines, only when evicted. Write-through: every store, immediately.

Exercise 10.4 (easy)**.** Section B of the simulator reports 69 376 evictions against 69 632 misses for the naive transpose. What are the other 256 misses, and why is that exactly 256?

Solution. [hint] They are misses that landed in an empty (invalid) way, so nothing was evicted. The toy cache has $S \cdot E = 64 \cdot 4 = 256$ ways, each filled once.

Exercise 10.5 (medium)**.** Count by hand the misses of the naive 4×4 transpose with 2-byte elements (a at byte 0, b at byte 32, row-major, so `a[i][j]` is at $8i + 2j$ and `b[j][i]` at $32 + 8j + 2i$) on a direct-mapped cache with S=2, E=1, B=8. Then check with `ch10_trace`. Finally, give the miss count with E=2.

Solution. [hint] The 32 accesses are `0 w32 2 w40 4 w48 6 w56 8 w34 ...`. Each a-row is one line (lines 0–3) and each b-row one line (lines 4–7); with only two sets the streams keep evicting each other. The trace tool reports E=1: 26 misses, 6 hits. With E=2 the same trace has 19 misses, 13 hits.

Exercise 10.6 (medium)**.** On the toy cache (64 sets, 4 ways, 64-byte lines), tile 16 gives 10 064 misses and tile 32 gives 69 632 for N=256 floats. State the set-budget argument precisely: how many distinct sets do the 32 rows of a tile reach, how many lines fit in them, and how many lines does the tile need?

Solution. [hint] Rows are 1 KB = 16 lines apart, so the set number advances by 16 (mod 64) per row: only sets 0, 16, 32, 48. That is $4 \cdot 4 = 16$ line slots. A 16-row tile needs 16 lines (fits), a 32-row tile needs 32 (does not).

Exercise 10.7 (medium)**.** A strided-access workload shows high conflict misses but only 4% capacity misses. Name two fixes and rank them by cost.

Solution. [hint] Pad the stride to a value coprime with the number of sets (free, a software change); raise associativity (costs silicon, and is not under your control).

Exercise 10.8 (medium)**.** Section D lists the three Cs. Explain from the code how the simulator decides that a miss is compulsory, capacity or conflict. What extra state does each class need?

Solution. [hint] Compulsory: a `seen` bit per line number. Capacity: it also misses in a fully associative LRU cache of the same total size. Conflict: it misses in the real cache but hits in that fully associative shadow.

Exercise 10.9 (hard)**.** Add a dirty-eviction counter to a hand experiment: predict, then check with `./ch10_trace 1 2 8 w0 w8 16 24`, how many misses, evictions and dirty evictions occur on a 2-way, 1-set cache with 8-byte lines.

Solution. [hint] Lines 0 and 1 are written (dirty). Access 16 evicts the oldest (line 0, dirty); access 24 evicts line 1 (dirty). So 4 misses, 2 evictions, 2 dirty evictions; the tool prints exactly `hits=0 misses=4 evictions=2 dirty_evictions=2`.

Exercise 10.10 (hard)**.** Section F of the simulator adds a next-line prefetcher (on a miss, also fetch the following line). It reports 67 833 demand misses against 69 632 without it. Explain why the gain is only 2.6%.

Solution. [hint] Only the read stream is sequential, and it already misses once per 16 floats; the write stream jumps a whole row each step, so "the next line" is the wrong line. The prefetch even pollutes sets that are already over-subscribed.

Exercise 10.11 (hard)**.** Argue that a "perfect" cache model for timing prediction still cannot predict our wall-clock numbers exactly. Name two mechanisms the model lacks that real cores have, and say when each matters.

Solution. [hint] Out-of-order overlap (misses hidden under independent work) and hardware prefetchers; they matter when the access pattern is not serialised by dependencies. Chapter 9's dependent-versus-independent experiment measured the first effect at 8 to 11 times.

Exercise 10.12 (easy)**.** A program's miss rate doubles when the input doubles. Which miss type is scaling, and which measurement separates it from the other two?

Solution. [hint] Capacity; compare against a fully associative (or infinite) cache run.

Exercise 10.13 (medium)**.** N=1024 floats, a 32 KB 8-way cache with 64 sets and 64-byte lines. Show that the column writes `b[j][i]` for fixed `i` all map to the *same* set. How many of them can the cache hold at once, and what does padding each row by 16 floats do?

Solution. [hint] Consecutive rows are $4N = 4096$ bytes $= 64$ lines apart, and $64 \bmod 64 = 0$, so every row's line lands in one set. Only 8 ways, so at most 8 such lines at once. With a 16-float pad the pitch is 4160 bytes $= 65$ lines; $\gcd(65, 64) = 1$, so successive rows rotate through all 64 sets.

Exercise 10.14 (medium)**.** The timing harness compares a checksum of `b` after every variant. What mistake does that catch that the stopwatch alone cannot?

Solution. [hint] A wrong loop bound or an optimised-away copy is *faster* and wrong. The checksum turns "faster" into "faster and still correct".

Exercise 10.15 (hard)**.** Using listing 10.6 and section C of the simulator, compute nanoseconds per L1 miss for the naive loop (38.73 ms, 4 456 448 misses) and for block-8 (18.77 ms, 847 872 misses). Do the two agree? What does the difference tell you about predicting time from miss counts?

Solution. [hint] Naive: $38.73 \cdot 10^{-3} / 4\,456\,448 = 8.7$ ns per miss. Block-8: $18.77 \cdot 10^{-3} / 847\,872 = 22.1$ ns. They disagree, so time is not proportional to L1 misses: the L2 absorbs most misses cheaply, misses overlap, and the loop has non-miss costs. Miss counts predict ranking, not milliseconds.

Exercise 10.16 (medium)**.** Why is the first edition's "naive 14.3 ms versus block-8 5.0 ms" comparison unfair? Say what a fair harness must hold equal.

Solution. [hint] The blocked loops repeated four sweeps inside each tile (later sweeps hit a hot tile) and their time was not divided by four, whereas the naive time was. A fair harness does the same total work per variant: one full sweep, timed separately, best of several trials.

Exercise 10.17 (hard)**.** For a row pitch of p lines and S=64 sets, how many distinct sets do 32 consecutive rows reach when $p = 16$? When $p = 17$? Generalise: when do R rows spread over all the sets?

Solution. [hint] Row r starts in set $rp \bmod 64$, which takes $64/\gcd(p, 64)$ distinct values. $p = 16$: gcd $= 16$, so 4 sets. $p = 17$: gcd $= 1$, so 64 values; 32 rows reach 32 distinct sets. Rows spread over all sets when p is odd and $R \geq 64$. (Section G measures the effect of $p = 17$: block-32 falls from 69 632 to 8 192 misses.)

CHAPTER 11

Virtual memory: the address is a lie

Every address this book has used so far has been a polite fiction. The pointer in your C program is not a DRAM address; it is a name in a private namespace the operating system maintains per process, translated on every single load and store by hardware you never see. This chapter opens that machinery: pages, the sv32 two-level walk, the TLB that makes translation affordable, the page fault that makes demand paging possible, and the protection bits that make processes safe from each other. We run real programs that count page faults, watch a `fork` copy memory only when someone writes, and walk a page table with a 40-line model we check against hand calculations.

AFTER THIS CHAPTER YOU CAN

You will be able to split a 32-bit virtual address into its page numbers and offset, walk a two-level page table by hand and find the physical address (or the page fault), read a page-table entry's flag bits, and run a small C program that counts the page faults your own code causes. You will be able to explain why `fork` is cheap, why a TLB is needed, and what it means that "the address is a lie".

BEFORE YOU START

Hexadecimal and bits. A hex digit is four bits, so `0x234` is the 12 bits `0010 0011 0100` (Book 5 chapter 8, numbers, bits and Boolean algebra). Cutting an address into fields means shifting and masking: `(x » 12) & 0x3FF` drops the low 12 bits and keeps the next 10.

Caches (chapters 9 and 10). A cache keeps recently used data close and fast, found by splitting an address into tag, index and offset. A *TLB* (below) is a cache of the same kind, for a different job.

Traps (chapter 8). A trap is the hardware's way of stopping the running program, jumping to operating-system code and later resuming. A *page fault* is one kind of trap.

A short C refresher. `char *p` is the address of a byte and `p[i]` the byte *i* places after it. `mmap` asks the operating system for a block of address space; it hands back a pointer. `fork()` creates a second copy of the running program: it returns 0 in the new *child* and the child's process number in the original *parent*, so an `if` on its result lets the two do different things. `getrusage` reads the system's own counters about your program, such as how many page faults it has caused.

THINK LIKE THIS

When two things share one resource, ask what each one believes it owns and what the system actually gives it. Virtual memory is the answer for memory: each program believes it owns a huge, private, contiguous address space; the system gives it scattered frames of RAM, and only the ones it has actually used.

11.1 Why virtualize

Imagine a hotel in which every guest is told "you are in room 101". Guests never collide, because the front desk keeps a private table for each guest saying which real room "101" means for them. Nobody needs to know the real numbering, rooms can be reassigned without the guest noticing, and a guest who asks for room 9 999 gets one only if and when they actually unpack a bag there. That is virtual memory.

Why does a computer need this? Suppose we skip it and let every program use real RAM addresses. Two programs compiled to start at `0x100000` would overwrite each other, so each must be compiled for a different, agreed place. A bug in one could scribble over the other, or over the operating system. And a program asking for 1 GB would need 1 GB of contiguous, installed RAM even if it touches one megabyte. One mechanism solves all three problems: *isolation* (process A cannot scribble on process B because B's pages are simply absent from A's map), *relocation* (every program may believe it owns address `0x100000`; the OS places it anywhere), and *sparse use* (reserve a gigabyte, touch a megabyte; the rest never needs to exist).

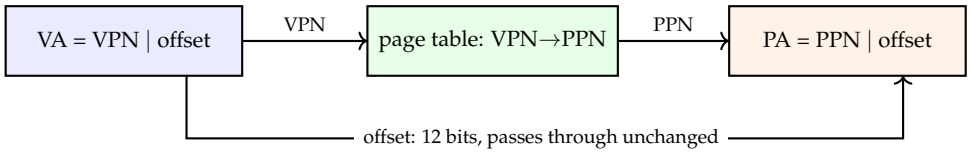


Figure 11.1: Translation in one line: the virtual page number becomes a physical page number through the page table; the offset rides along untouched. Everything else — TLBs, walks, faults — is engineering around this arrow.

Example 11.1 (One line of truth)**.** Translation changes only the *page number*; the offset inside the page is identical on both sides. Everything in the chapter — the walk, the TLB, the fault path — is machinery around that one arrow.

Example 11.2 (Splitting an address into page number and offset)**.** Memory is divided into *pages* of 4 KB $= 2^{12}$ bytes (the size sv32 uses). Take the virtual address `0x00401234`. The low 12 bits, `0x234` (564), say where inside the page the byte is. What remains, `0x00401234` $\gg 12 =$ `0x401` (1025), is the *virtual page number*, VPN. A naive plan stores one table entry per virtual page: a 32-bit space has $2^{20} = 1\,048\,576$ pages, and at 4 bytes per entry that is a 4 MiB table for *every* process, even one that uses two pages. The two-level walk in the next section exists to avoid exactly that waste.

11.2 The walk: sv32 as the concrete case

RISC-V's 32-bit mode, *sv32*, splits the 32-bit virtual address (VA) into VPN[1] (10 bits), VPN[0] (10 bits) and the offset (12 bits). The processor register `satp` holds the physical page number (PPN) of the top table. A *page table* is an array of 1 024 four-byte *page-table entries* (PTEs), exactly one 4 KB page. VPN[1] selects an entry in the top table, whose PPN points to a second-level table; VPN[0] selects an entry in that table, whose PPN is the data page. Two memory reads per translation. The big saving: a second-level table exists only for the 4 MiB regions the program really uses.

A PTE packs the PPN into bits 31–10 and eight flag bits into bits 7–0, in the order `D A G U X W R V`. The ones we need: `V` valid, `R/W/X` read, write, execute allowed, `U` user-mode allowed. A valid PTE with `R=W=X=0` is a *pointer* to the next table; otherwise it is a *leaf*.

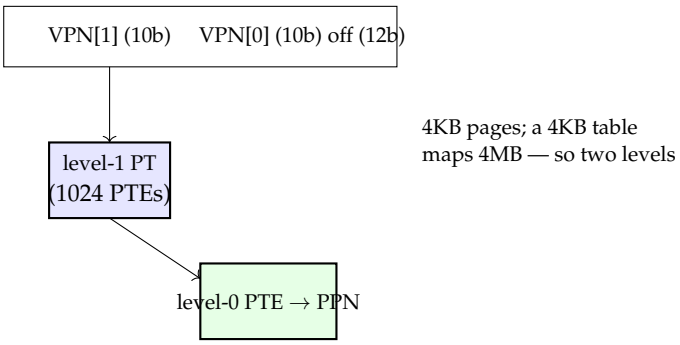


Figure 11.2: The sv32 walk: VA[31:22] indexes the level-1 table for a second-level table's physical address; VA[21:12] indexes that for the PPN. Two memory reads per miss — which is why the TLB exists.

Example 11.3 (A walk by hand — corrected)**.** VA `0x00401234`, `satp` pointing at a level-1 table at PPN `0x1000`. First, where do page tables live in physical memory? PPN `0x1000` means byte address `0x1000` $\times$ 4096 $=$ `0x01000000`. The fields: VPN[1] $=$ `0x00401234` $\gg 22 = 1$; VPN[0] $=$ (`0x00401234` $\gg 12$) & `0x3FF` $=$ `0x401` & `0x3FF` $= 1$; offset `0x234`.

1. Read the PTE at `0x01000000` $+ 1 \times 4 =$ `0x01000004`. Suppose it holds a pointer to the level-0 table at PPN `0x2000`.
2. That table is at `0x2000`$\times$4096 $=$ `0x02000000`; read the PTE at `0x02000000` $+ 1 \times 4 =$ `0x02000004`. Suppose it holds the leaf PPN `0x9ABC`.
3. The physical address is the PPN followed by the offset: `0x9ABC`$\times$4096 $+$ `0x234` $=$ `0x09ABC234`.

Two lookups, one address. (An earlier edition wrote the PTE locations as "`0x1000+1`", adding an index to a page number; a PTE address is the table's byte address plus 4 times the index.)

Example 11.4 (Let a program do the walk, and check it against the hand result)**.** The model `ch11_sv32_walk.py` keeps physical memory in a Python dictionary and implements exactly the steps above, plus the permission checks. We load it with the page tables of the example (plus a read-only page and an unmapped region) and ask for six translations:

```
walk 1: VA 0x00401234 (r) -> PA = 0x09ABC234
```

```
  level 1: PTE at 0x01000004 = 0x00800001
  level 0: PTE at 0x02000004 = 0x026AF017
walk 2: VA 0x00401FFC (r) -> PA = 0x09ABCFFC
  level 1: PTE at 0x01000004 = 0x00800001
  level 0: PTE at 0x02000004 = 0x026AF017
walk 3: VA 0x00402010 (w) -> PAGE FAULT (permission: page lacks W)
  level 1: PTE at 0x01000004 = 0x00800001
  level 0: PTE at 0x02000008 = 0x026AF413
walk 4: VA 0x00402010 (r) -> PA = 0x09ABD010
  level 1: PTE at 0x01000004 = 0x00800001
  level 0: PTE at 0x02000008 = 0x026AF413
walk 5: VA 0x00801000 (r) -> PAGE FAULT (invalid PTE at level 1 (V=0))
  level 1: PTE at 0x01000008 = 0x00000000
walk 6: VA 0x00C02ABC (r) -> PA = 0x00777ABC
  level 1: PTE at 0x0400000C = 0x01400001
  level 0: PTE at 0x05000008 = 0x001DDC1F
ch11 sv32 model: self-check ok
```

Listing 11.1: `python3 ch11_sv32_walk.py`. Walk 1 reproduces the hand calculation (PTE `0x026AF017` is PPN `0x9ABC` shifted left 10 plus flags V,R,W,U). Walk 3 stores to a read-only page; walk 5 reaches an empty top-level entry.

Read walks 1 and 2: both addresses are in the same page, so they use the same two PTEs and the offsets (`0x234` and `0xFFC`) pass through untouched. Walks 3 and 4 use the same VA; the *only* difference is read versus write, and the hardware's answer flips between a physical address and a fault. That is the protection fence of section 11.5 in four lines of output. Walk 6 is the address of the hand-walk exercise: VPN[1]$=3$, VPN[0]$=2$, so the PTEs sit at `0x4000`$\times 4096 + 3 \times 4 =$ `0x0400000C` and `0x05000008`.

11.3 The TLB: caching the answer

Two extra memory reads per load would be fatal: a program doing a load every few instructions would run at a third of its speed. The *translation lookaside buffer* (TLB) caches recent VPN$\rightarrow$PPN results so that most accesses skip the walk entirely. It is a cache, exactly like chapter 10's, just for translations: small (tens to a few thousand entries), fully or highly associative, one entry per page.

stage	answers
TLB lookup	VPN$\rightarrow$PPN cached? (1 cycle-ish)
page-table walk	the real translation (2+ memory reads, sv32)
cache lookup	PA $\rightarrow$ data (L1/L2/LLC)
DRAM	last resort (our 120 ns)

Figure 11.3: The full chain every load traverses. A TLB hit collapses the first two rows to almost nothing; a TLB miss costs the walk *plus* the data fetch — translation and caching are two separate caches strung in series.

Example 11.5 (TLB reach)**.** A 64-entry TLB on 4 KB pages covers 64×4 KB $= 256$ KB of address space before evictions start; on 2 MB huge pages, 128 MB. TLB *reach* is why giant pages exist — same hardware, 512× more coverage.

Example 11.6 (Seeing the TLB with a stopwatch)**.** Design the experiment before running it. We want the cost of a hop that needs a translation not in the TLB. Naive attempt: chase a pointer chain through a big array and watch the time. But a big array also misses the *cache*, so the two effects mix. The cleaner design uses *one cache line per page*: visit P lines, one in each of P different pages, in random order. For P up to a few hundred the data is tiny (a few KB of cache lines) but spans P pages. As a control, we repeat with the *same number of lines packed together* in $P/64$ pages. If a slowdown appears only in the spread version, it is the TLB.

```
  pages region ns/hop (one line per page)
      8 32 KiB 1.22
     16 64 KiB 1.21
     32 128 KiB 1.22
     64 256 KiB 1.21
    128 512 KiB 3.95
    256 1024 KiB 3.97
    512 2048 KiB 3.98
   1024 4096 KiB 7.13
   2048 8192 KiB 20.47

 64 lines in 1 page(s): 1.33 ns/hop (control: same lines, packed)
128 lines in 2 page(s): 1.33 ns/hop
256 lines in 4 page(s): 1.35 ns/hop
```

Listing 11.2: Linux x86_64 container. Upper block: `./ch11_tlb_reach n` (normal pages, one line per page; rows 8 to 2 048 pages of the 14 printed). Lower block: the control `./ch11_tlb_control`, the same number of lines packed into few pages (first three rows). Captured in `code/ch11_tlb_linux.txt` and `code/ch11_tlb_control_linux.txt`.

Up to 64 pages a hop costs 1.2 ns, an L1 hit. At 128 pages it jumps to 3.95 ns, and the control with 128 *lines* in 2 pages stays at 1.33 ns. So the 2.6 ns surcharge is the price of finding a translation outside the first-level TLB, and the step sits exactly at the 64 entries that this class of Intel core is commonly documented to have (a figure from the vendor manuals, not something we measured directly). The next steps (7 ns at 1 024 pages, 20 ns at 2 048) are partly a second TLB level and partly the cache, which we cannot separate with this experiment. Honest gap: when we asked the kernel for huge pages (`./ch11_tlb_reach h`, and `AnonHugePages` confirmed 256 MiB were huge-page-backed) the step at 64–128 pages *did not disappear* (3.15 ns at 64 pages, 4.27 at 128), which a pure TLB story would not predict. We did not isolate why. A measurement that agrees with the theory in one setting and not in another is worth reporting, not hiding.

11.4 Demand paging and the fault, measured

A page that is mapped but never touched may never exist. The first touch raises a page fault; the OS allocates a frame, fills the PTE, and retries the instruction. The idea is the same as "look it up when asked": do the work lazily, because most of what you might need you will never need. It is also the reason `malloc(1 GB)` returns instantly.

Example 11.7 (Counting the faults instead of guessing)**.** The operating system keeps a counter of *minor* page faults (faults that need no disk read) per process, and `getrusage` reads it. The program `ch11_minflt.c` maps 64 MiB (16 384 pages of 4 KB), then touches one byte per page in four passes. Before running, predict: a first pass should fault once per page, a second pass should not fault. Then run:

```
page size 4096 bytes, region 16384 pages (64 MiB)
after mmap, before any touch: faults=107 rss=402 pages
read pass 1 : +16384 faults rss=432 pages
read pass 2 : +0 faults
write pass 1 : +16384 faults rss=16816 pages
write pass 2 : +0 faults
child reads 256 shared pages: +0 faults; then writes them: +256 faults
parent still sees p[0]=42 (child wrote 99 into its own copy)
parent writes the same 256 pages after the child exited: +256 faults
```

Listing 11.3: Linux x86_64 container, `./ch11_minflt` (captured in `code/ch11_minflt_linux.txt`). Page faults counted by the kernel, not estimated.

Three things confirm and one surprises. Confirmed: the first pass costs exactly one fault per page (16 384), and the second costs none, so demand paging works as described. The `rss` (resident set size, the pages of the process actually in RAM) is 402 pages before and *only 432 after reading all 16 384 pages*: Linux maps every never-written page to one shared page of zeros, so reading allocates no memory at all. The surprise: the first *write* pass faults a second time on every page, because the shared zero page has to be replaced by a private frame; only then does `rss` jump by 16 384 pages. Running the walk in the naive way, reading to "warm up" the memory, therefore leaves the expensive faults for later. (This is Linux behaviour on this kernel; other systems differ.)

Example 11.8 (The fault cost, on two machines)**.** `ch11_fault_cost.c` maps 512 MB and times the first touch of each page against a second pass. First the author's Mac (16 KB pages), verbatim:

```
pages=32768
first-touch 744 ns/page
resident    107.6 ns/page
fault cost 7x
```

and the Linux x86_64 container (the program now reads the page size from the OS instead of assuming 16 KB):

```
page size=4096 bytes
pages=131072
```

```
first-touch 1101 ns/page
resident 8.8 ns/page
fault cost 125x
```

Listing 11.4: Linux x86_64 container, `./ch11_fault_cost`, one run (three runs gave 1 001–1 197 ns first touch, 7–9 ns resident, 125–159×). Captured in `code/ch11_fault_cost_linux.txt`.

A fault costs about a microsecond on both machines (744 versus 1 101 ns), which at 3 GHz is thousands of instructions: cheap enough that demand paging is the right lazy design, expensive enough that when *every* access faults (thrashing) throughput collapses. The *ratios* are 7 versus 125 only because the denominators are so different: the Mac's "resident" hop costs 107 ns (its 16 KB stride also misses the cache), the container's costs 9 ns. Never compare ratios across machines; compare the quantity you care about.

access	Mac (16 KB pages)	Linux container (4 KB pages)
resident page touch	107.6 ns	8.8 ns
first touch (page fault)	744 ns	1101 ns
ratio	7×	125×

Figure 11.4: `ch11_fault_cost.c`: mmap 512 MB, touch one byte per page twice. The fault costs roughly a microsecond on both machines. The *ratios* differ by a factor of 18 because the denominators do: the Mac's "resident" touch is slow (each 16 KB hop also misses the cache), the container's is fast. Compare absolute times, not ratios.

11.5 fork + copy-on-write, and the fence

Why is `fork` cheap? A naive `fork` would copy all of the parent's memory, which for a 4 GB program takes seconds. Copy-on-write (COW) instead copies the page *tables* and marks every shared page read-only; the first time either process writes a shared page, the hardware raises a protection fault, the kernel copies just that page, and the writer continues on its private copy. Pages nobody writes are never copied.

Example 11.9 (Fork and COW, counted)**.** The second half of listing 11.3 runs it. After the parent has written 256 pages, it forks. The child *reads* all 256 pages: 0 faults, because the shared mappings are already valid for reading. The child then *writes* them: 256 faults, one copy each, and the parent still sees its own value (`p[0]=42`), not the child's 99: the copies are private. Last, the parent writes the same pages: 256 faults again. No page needed copying this time because the child had exited (the frames were no longer shared), but the pages were still marked read-only, so each write still took the fault to find that out. The accounting for a spawn-heavy workload is therefore "the pages that get written pay one fault each", not "the whole process is copied".

Checkpoint 11.1 Why can a shared library exist in every process at once? Same PPN in many VPNs — the page tables differ, the frames don't. Physical memory is the shared resource; virtual memory is the private view.

step	parent's page	child's page
fork()	shared, marked read-only	same frame
child writes	fault → copy made	private copy
parent writes	fault → copy made	keeps sharing

Figure 11.5: Copy-on-write: `fork` copies the page *tables*, not the pages; the first *write* to a shared page triggers the fault that makes the copy. Measured in listing 11.3: a child that only reads 256 shared pages takes 0 faults, and one that then writes them takes 256.

PTE bits	gate
V (valid)	present? else fault
R/W/X	read / write / execute allowed?
U	user-mode access allowed?

Figure 11.6: The protection bits are the OS's fence, enforced by hardware on every access. W∧X (never both) stops whole classes of code-injection — the bridge into Part IV's OS story.

11.6 Common mistakes

- **Thinking VA ≈ PA is slow always.** With a TLB hit the translation is nearly free; the walk is the miss cost.
- **Page size as a constant.** macOS arm64 uses 16 KB; RISC-V sv32 uses 4 KB; huge pages change TLB reach 512×.
- **A page fault is an error.** It is the load mechanism: mmap, COW, and lazy stack all run *through* faults.
- **Forgetting the fence is hardware.** PTE bits are checked on every access by the MMU — protection isn't a convention, it's enforced.
- **Thrashing as a cache problem.** It's a working-set problem: when the set exceeds resident frames, every level of the hierarchy stops mattering.

11.7 Summary

Piece	**Measured/fixed version**
pages	VPN→PPN, offset untouched
sv32 walk	10+10+12 bits; two reads per miss
TLB	translation cache; reach = entries × page size
fault	about 1 μs (744 ns Mac, 1 101 ns Linux); ratios differ with the baseline
COW	fork copies tables, not pages; the first write pays one fault (256 pages, 256 faults)
PTE bits	V/R/W/X/U — hardware-enforced fence

Further reading

- Arpaci-Dusseau & Arpaci-Dusseau, *OSTEP* ch13–23 — the ordering this chapter adapted: https://pages.cs.wisc.edu/~remzi/OSTEP/
- CS:APP ch9 (VM + memory mapping): https://csapp.cs.cmu.edu/
- xv6 book ch3 (page tables on real RISC-V code): https://pdos.csail.mit.edu/6.828/2023/xv6/book-riscv-rev3.pdf
- RISC-V privileged spec, sv32 section: https://riscv.org/specifications/

WATCH INSTEAD

- **Virtual Memory — Computerphile** — pages and the walk at this chapter's altitude (verified 2026-10-11). https://youtu.be/qdkxXygc3rE

Notes

- Fault/resident numbers are verbatim `ch11_fault_cost.c` on Apple Silicon (16 KB pages); repeat on Linux to compare 4 KB-page numbers.
- The Linux x86_64 container numbers are new runs (Intel Xeon 2.8 GHz, KVM guest, 4 KB pages, kernel 6.18). The copy-on-write counts, the page-table walk and the TLB steps are all produced by the programs in `code/`, checked by `code/test_ch11.sh`.

Exercises

Exercise 11.1 (easy)**.** A 32-bit VA has a 12-bit offset on sv32. How large is a page, and how many pages fit in the VA space?

Solution. [hint] 4 KB; 2^{20} pages.

Exercise 11.2 (easy)**.** Which of isolation, relocation, sparse use does *not* depend on the page table being per-process?

Solution. [hint] Sparse use — even a shared table could mark pages invalid.

Exercise 11.3 (easy)**.** TLB reach on 4 KB pages with 64 entries vs 2 MB huge pages with the same 64 entries: compute both.

Solution. [hint] 256 KB vs 128 MB.

Exercise 11.4 (medium)**.** Walk VA `0x00C02ABC` on sv32 by hand (level-1 table PPN `0x4000`). Show both PTE indices.

Solution. [hint] VPN[1]=`0x003`, VPN[0]=`0x002`, off=`0xABC`. The PTEs are at `0x4000` $\cdot$ 4096 + 3 $\cdot$ 4 = `0x0400000C` and, from the level-0 table's address, index 2 times 4.

Exercise 11.5 (medium)**.** Our fault measurement used anonymous mmap. Predict the first-touch cost if the pages were file-backed, and name what changes.

Solution. [hint] Add disk I/O to the fault path — μs scale, not ns.

Exercise 11.6 (medium)**.** Why must the TLB flush (or tag with ASIDs) on a context switch? What breaks if it doesn't?

Solution. [hint] Process B inherits A's VPN→PPN: silent cross-process aliasing.

Exercise 11.7 (medium)**.** Sketch the fork+COW trace for a parent that writes 10 shared pages then reads 10: how many faults, and who pays them?

Solution. [hint] Parent pays all 10 write-faults; child's reads stay shared.

Exercise 11.8 (medium)**.** Design the experiment that separates TLB-miss cost from page-fault cost on your machine. What do you stride over, and what does the timing curve's knee reveal?

Solution. [hint] Stride one access per 4 KB page over growing regions; the knee at TLB reach shows walk cost.

Exercise 11.9 (hard)**.** Simulate CLOCK (second-chance) replacement vs LRU on a synthetic 10 000-access trace with 50% temporal reuse. Report hit rates for both and explain the gap (or lack of one) from the algorithm mechanics.

Solution. [hint] CLOCK approximates LRU well under temporal locality; gap widens on scan workloads where LRU is fooled.

Exercise 11.10 (hard)**.** A system runs out of swap and the OOM killer picks the *wrong* process. Given only PTE protection bits and RSS numbers, outline a policy that would have picked better, and its cost.

Solution. [hint] Rank by frames-held×last-touch-age; needs reference bits the MMU already tracks — cheap approximations exist.

Exercise 11.11 (hard)**.** Argue that W∧X (a page never writable *and* executable) cannot be defeated by any user-mode trick alone, then name the one legitimate way JIT compilers still make code.

Solution. [hint] The fence is in the MMU, not the compiler; JITs map pages W first, then `mprotect` to X — never both.

Exercise 11.12 (easy)**.** Split the virtual address `0x7FFFF123` into VPN[1], VPN[0] and offset. Which table entry of the top-level table does it use?

Solution. [hint] VPN[1] = `0x7FFFF123` ≫ 22 = `0x1FF` (511), VPN[0] = (`0x7FFFF123` ≫ 12) & `0x3FF` = `0x7FFFF` & `0x3FF` = `0x3FF` (1023), offset = `0x123`. It uses entry 511 of the top-level table, whose address is the table's byte address plus 511 × 4.

Exercise 11.13 (medium)**.** Decode the PTE `0x026AF413` from listing 11.1: its PPN, which of V, R, W, X, U are set, and the physical address of a read of VA `0x00402010`.

Solution. [hint] PPN = `0x026AF413` ≫ 10 = `0x9ABD`. The low byte is `0x13` = $0001\,0011_2$: bits V (bit 0), R (bit 1) and U (bit 4) are set; W and X are not. A read gives `0x9ABD` · 4096 + `0x010` = `0x09ABD010`; a write faults.

Exercise 11.14 (medium)**.** Memory for page tables. In sv32, how many bytes of page tables does a process need if it uses exactly one page? And if it touches one page in each of 1 024 different 4 MiB regions?

Solution. [hint] One page: one top-level table plus one level-0 table, 2 × 4096 = 8192 bytes. 1 024 regions: one top-level table plus 1 024 level-0 tables, 1025 × 4096 = 4 198 400 bytes, about 4.0 MiB. (A flat single-level table would have cost 4 MiB in both cases.)

Exercise 11.15 (medium)**.** Using listing 11.3: how many page faults would a program incur if it *read* and then *wrote* every page of a fresh 1 GiB anonymous mapping, with 4 KB pages, on this kernel? How many if it only wrote?

Solution. [hint] 1 GiB/4 KiB = 262 144 pages. Read then write: the read pass faults once per page (262 144, zero page), and the write pass faults again per page (262 144), total 524 288. Writing only: 262 144.

Exercise 11.16 (hard)**.** The Mac's fault ratio was 7× and the container's about 125×, yet the faults themselves cost 744 ns and about 1 100 ns. Explain the discrepancy, then say which number you would use to estimate the cost of a program that faults in 1 GB of fresh memory.

Solution. [hint] The ratio divides by the resident-touch time, which differs: 107.6 ns on the Mac (its 16 KB stride misses the cache) versus about 9 ns here. Use the absolute first-touch time per page times the number of pages: 1 GiB is 262 144 pages of 4 KB, so roughly 262 144 × 1.1 μs ≈ 0.29 s on the container (zero-page read faults; write faults add the copy and are slower).

CHAPTER 12

Processes and scheduling

Part IV starts here. Think of a single cook in a kitchen with ten orders on the board. The cook cannot cook ten dishes at the same instant, yet every customer feels served, because the cook keeps stepping from one pot to the next and remembers exactly where each dish stands. A computer does the same thing: a few cores serve hundreds of programs by switching between them so quickly that each program feels like it owns the machine. The operating system is the cook's habit of keeping notes. This chapter builds the two ideas behind that habit. The first is the *process*, the bundle of notes about one running program. The second is the *scheduler*, the rule that decides whose turn it is. We also measure what a switch costs on a real machine instead of quoting a number from a book.

WHAT YOU WILL BE ABLE TO DO

After this chapter you can write a C program that starts another program and collects its exit status without leaking a zombie (a finished process nobody has collected yet). You can wire a three-stage pipeline by hand, the way a shell does. You can measure what a context switch costs on your own machine and explain why the number changes when you pin the program to one CPU. And you can take any list of jobs with arrival times and lengths and compute average turnaround and response time under five scheduling rules, by hand and with the chapter's simulator.

BEFORE YOU START

Three earlier ideas carry this chapter, so here is a quick recap. *Traps* (Book 7 ch. 8): when a timer fires or a program asks the kernel for something, the CPU saves where it was and jumps to a kernel routine; a context switch rides on exactly that path. *Virtual memory* (ch. 11): every process sees its own private address space, which the hardware maps to real memory through a page table. After a `fork` the two processes share pages *copy-on-write* (COW): a page is copied only at the moment someone writes to it, like two people reading one newspaper until one of them wants to scribble on it. *State machines* (Book 5 ch. 10, sequential circuits and state machines): a thing with a few named states that changes state on events. A process is exactly that, as you will see. If you want to count time units the way the simulator does, Book 1 ch. 1 (the contest machine and the cost model) is where counting operations began.

A short C refresher, enough for this chapter. A program is compiled with `gcc file.c -o file` and started with `./file`. `pid_t` is an integer type that holds a process id. `fork()` returns -1 if it failed, 0 in the new child, and the child's pid in the parent; the same line of code runs twice, once in each process. `waitpid(p, &st, 0)` passes the

address of the integer `st` (the `&` means "address of") so the function can write the child's status into it for you; `WEXITSTATUS(st)` then reads the exit code out. `char *argv[] = {"echo", "hi", NULL}` is an array of strings ended by `NULL`, which is the form `exec` expects.
Where the numbers come from. Every command here ran on a Linux x86_64 container (4 vCPU Intel Xeon at 2.8 GHz, kernel 6.18, gcc 13). Numbers marked *Mac* were measured earlier on the author's laptop and are kept as they were; the two machines are never mixed in one comparison.

THINK LIKE THIS

Whenever a system seems to do two things at once, ask: what is the *saved note* that lets it stop and resume, and who decides when to switch? Almost every idea in this chapter is an answer to one of those two questions.

12.1 What a process is

A process is a running program's whole state bundled: an address space (ch11's private map), a register set saved somewhere, open files, and a kernel bookkeeping entry. The CPU sees none of it — it just executes instructions and takes traps. The OS's job is to suspend that state, park it, and restore a different one.

CONCEPT: A PROCESS IS A PROGRAM PLUS ITS BOOKMARK

A recipe in a cookbook is a *program*: it just sits there. A cook halfway through that recipe, with the pot on the stove and a finger on step 4, is a *process*. Two cooks following the same recipe are two processes of one program. Everything the OS must write down to pause one cook and later resume exactly where they were (the registers, the memory map, the open files) is the bookmark. Without the bookmark, switching would lose work; with it, switching is only a matter of copying a few values.

Figure 12.1 is the whole life of a process in three states. *Running* means a core is executing it right now. *Ready* means it could run but is waiting for a core. *Blocked* means it is waiting for something slow, such as a disk read, and would waste a core if given one. When a process finishes it becomes a zombie until its parent collects the result, a detail we meet with real output below.

12.2 fork, exec, wait — the correct version

Why does UNIX start a new program in two steps instead of one? Because the gap between the steps is useful. After `fork` and before `exec`, the child can still rearrange its own plumbing (redirect output to a file, close a descriptor, change directory) using ordinary code, and then become the new program. A single "spawn" call would need a flag for every such wish. UNIX's recipe is two calls, not one: `fork` duplicates the process (child gets a copy of

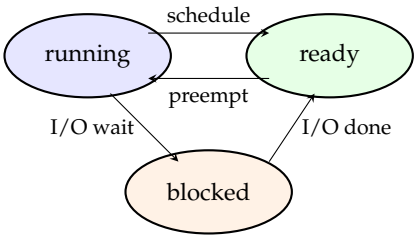


Figure 12.1: The three-state life of a process. The scheduler moves a process between ready and running; the I/O system moves it between running and blocked. Everything else — priorities, queues, timeslices — is policy on top of this machine.

everything — COW, ch11, makes it nearly free), `exec` replaces the copy's program image with a new one. A shell pipeline `ps | grep x | wc` costs three forks and three execs.

```
pid_t p = fork();
if (p < 0) { perror("fork"); return 1; } // fork CAN fail
if (p == 0) {
   execl("/bin/echo", "echo", "hi", NULL);
   _exit(127); // exec failed: exit, not return
}
int st; pid_t w = waitpid(p, &st, 0); // harvest the status
```

Listing 12.1: The three mistakes almost everyone makes, avoided: check `fork`'s error path, handle `exec` failure with `_exit`, and wait on *status*, not just completion.

The real output of the chapter's fork demo (`ch12_fork_demo.c`, Linux x86_64 container): the exec'd echo printed `hello from exec'd child`, then the parent printed its clean `waitpid` line (`status=0`), and the child's exit status arrives intact.

```
parent 4790 starting
  child 4792 exec'ing /bin/echo
  hello from exec'd child
parent: child 4792 exited, status=0
```

Listing 12.2: Real output of `ch12_fork_demo` (pids differ on every run).

One detail in that source is easy to miss: it calls `fflush(stdout)` before `fork` and before `exec`. When standard output is a pipe or file, C buffers it in user memory. A forked child inherits a copy of the unflushed buffer, and `exec` then throws the buffer away. Our first capture, run through a pipe *without* those two calls, printed the parent's line late and lost the child's "exec'ing" line entirely.

Example 12.1 (Fork returns twice, and the memory is a copy)**.** *What goes wrong first.* A beginner writes `x += 1` in the child and expects the parent to see 101, as two threads would. It does not, because the two processes do not share `x`. `ch12_cow.c` shows it:

```
int x = 100;
pid_t p = fork();
if (p == 0) { x += 1; printf("child : fork returned %d, x=%d, &x=%p\n", (int)p, x, (void
    *)&x); return 0; }
waitpid(p, NULL, 0);
printf("parent: fork returned child pid > 0: %s, x=%d, &x=%p\n", p > 0 ? "yes" : "no", x,
     (void*)&x);
```

Listing 12.3: `fork` returns once in each process; the child's write stays private.

```
child : fork returned 0, x=101, &x=0x55e749afa010
parent: fork returned child pid > 0: yes, x=100, &x=0x55e749afa010
```

Listing 12.4: Real output. The address is the same, the value is not (the address changes between runs because of ASLR, address-space randomisation).

Both processes print the *same virtual address* and different values. Each has its own page table (ch. 11); after the fork both map the page read-only and shared, and the child's write triggers the COW fault that gives it a private copy. The return value is the only way the two copies of the code tell themselves apart.

Example 12.2 (Forgetting to wait: the zombie)**.** *What goes wrong first.* A parent forks a child that exits at once and never calls `wait`. The child is finished, yet its kernel entry cannot go away: the exit status (here 7) must stay available in case the parent asks. A finished-but-unreaped process is a *zombie*. `ch12_zombie.c` reads the state letter from `/proc/PID/stat` before and after `waitpid`:

```
pid_t p = fork();
if (p == 0) _exit(7); // child: exit at once, status 7
usleep(100000); // let it finish
printf("before waitpid: child state = %c\n", state_of(p));
int st; waitpid(p, &st, 0);
printf("waitpid returned: exit status = %d\n", WEXITSTATUS(st));
printf("after waitpid: child state = %c\n", state_of(p));
```

Listing 12.5: The parent looks at its child's state, reaps it, and looks again.

```
before waitpid: child state = Z
waitpid returned: exit status = 7
after waitpid: child state = -
```

Listing 12.6: Real output (`Z` is zombie; − means `/proc` has no such pid).

A zombie uses no CPU and almost no memory, but it holds a process-table slot, and a parent that leaks thousands of them can run the system out of pids. You cannot `kill` a zombie: it is already dead. The only cure is for the parent to call `wait` (or to exit, after which the init process adopts and reaps the orphans).

Example 12.3 (A shell pipeline built by hand)**.** A shell turns `echo "hello systems" | tr a-z A-Z | wc -c` into three forks, two pipes and three execs. A *pipe* is a kernel buffer with a read end and a write end, each a file descriptor. `dup2(fd, 0)` makes descriptor 0 (standard input) refer to the same thing as `fd`; that is how the pipe end becomes the program's stdin without the program knowing. `ch12_pipeline.c` runs the sequence:

```
int p1[2], p2[2]; pipe(p1); pipe(p2); // p[0] = read end, p[1] = write end
spawn(echo, 0, p1[1], ...); // echo: stdout -> p1 write end
spawn(tr, p1[0], p2[1], ...); // tr: stdin <- p1, stdout -> p2
spawn(wc, p2[0], 1, ...); // wc: stdin <- p2, stdout stays the terminal
close(p1[0]); close(p1[1]); close(p2[0]); close(p2[1]); // the parent closes ALL four
while (wait(&st) > 0) n++;
```

Listing 12.7: The wiring in `ch12_pipeline.c` (the helper `spawn` forks, calls `dup2`, closes every pipe end, then `execvp`s).

```
14
parent reaped 3 children
```

Listing 12.8: Real output. `wc -c` counts the 13 letters of "HELLO SYSTEMS" plus the newline.

What goes wrong first. Skip the parent's four `close` calls and the program hangs. A reader sees end-of-file on a pipe only when *every* copy of the write end is closed, and the parent still holds one, so `tr` waits forever for more input. We checked this by compiling the same file with a build flag that omitted those closes and running it under `timeout 2`: it was killed (exit status 124) instead of printing.

12.3 The context switch, step by step

step	what the kernel does
1	trap arrives (timer tick, syscall, fault)
2	save P_1's registers on P_1's kernel stack
3	`swtch`: swap stack pointer to P_2's context
4	restore P_2's registers from its stack
5	return-from-trap into P_2's userspace

Figure 12.2: A context switch is just a stack-pointer swap between two pre-staged register saves — the same trap machinery from ch8, aimed at a different address space. Cost on this machine: ≈3 µs measured.

Nothing new is invented here: a context switch reuses ch8's trap path exactly — registers get saved, the kernel swaps which saved set to restore, and the machine returns into a different address space. The only fresh piece is `swtch`, the stack-pointer swap.

Example 12.4 (The 3 µs truth, measured)**.** `ch12_ctx_switch.c` ping-pongs a byte through pipes; each round trip costs two switches. On the Mac: **2.93 µs per switch** at one pair, 3.7 at two, 3.1 at four. The flat scaling is the honest confound — extra pairs spread across cores rather than queueing — and the number is the real price of the illusion: ≈300 switches before one millisecond is spent.

Example 12.5 (Pinned versus unpinned: the same program, a 7× different answer)**.** *What goes wrong first.* We built the pipe ping-pong, ran it, and on the Linux container it reported about 20 µs per switch, nearly ten times the Mac figure above. The program had not changed. What changed is where the two processes ran. When the parent writes a byte and sleeps in `read`, the kernel has to wake the child. If the child lives on another core, waking it means sending that core an inter-processor interrupt, and in a virtual machine that signal is expensive. If both processes are *pinned* to one core (told to run only there), the "wake-up" is just the next thing the same core does.

```
$ ./ctx_switch 1 $ taskset -c 0 ./ctx_switch 1
pairs=1 19.94 us per ctx switch pairs=1 2.13 us per ctx switch
pairs=1 19.58 us per ctx switch pairs=1 2.06 us per ctx switch
pairs=1 18.23 us per ctx switch pairs=1 1.98 us per ctx switch
```

Listing 12.9: Real runs on the Linux x86_64 container, three each; the `taskset -c 0` prefix pins the program and its children to CPU 0.

configuration	measured µs/switch
Mac (author's laptop)	
1 pipe pair	2.93
2 pairs	3.71
4 pairs	3.14
Linux x86_64 container, median of 3	
1 pair, unpinned	19.58
2 pairs, unpinned	18.01
4 pairs, unpinned	17.95
1 pair, pinned (`taskset -c 0`)	2.06
2 pairs, pinned	2.70
4 pairs, pinned	3.35

Figure 12.3: Real `ch12_ctx_switch.c` output. On the Mac, ≈3 µs per switch, with flat scaling because extra pairs spread over cores. On the Linux container, unpinned runs cost ≈18–20 µs (cross-core wake-ups in a virtual machine) and pinned runs 2–3.4 µs. The two machines are different experiments and are not compared digit for digit.

The unpinned median is 19.6 µs and the pinned median 2.06 µs, a ratio of $19.6/2.06 \approx 9.5$. Across all our runs unpinned values ranged from 6.8 to 22.8 µs (the machine is shared, so they wander), while pinned values stayed between 2.0 and 3.7. The lesson is a habit worth keeping: before you trust a timing, ask which physical resources the two sides of the test were using.

THINK LIKE THIS

A measurement answers one specific question. "How long is a context switch?" hides two questions: the cost of the kernel swapping state, and the cost of waking a process on another core. Pinning removes the second so that you are measuring the first.

Checkpoint 12.1 Why is the context-switch number not simply "instructions executed"?
Because the pipeline cost dominates: every switch invalidates the cache-warm state, the TLB entries, the branch predictor — the machine runs cold for thousands of cycles after.

12.4 Scheduling: five policies, measured on one job set

Why does a scheduler need a policy at all? Because whenever more jobs are ready than there are cores, someone must wait, and the order of waiting changes how the whole system feels. Picture a supermarket with one till. If a person with a full trolley is first in line, everyone behind them with one carton of milk waits a long time; letting the milk-buyers go first costs the trolley person very little. That intuition is what the policies below try to capture, and we need two precise words to compare them. *Turnaround time* is finish time minus arrival time: how long the job spent in the system. *Response time* is first-run time minus arrival time: how long before the job first got any attention.

The job set: A(100), B(30), C(10), D(60) time units, arriving at 0, 10, 20, 30. The chapter's simulator runs five rules on it. FIFO (first in, first out) runs jobs to completion in arrival order. SJF (shortest job first) picks the shortest waiting job whenever the core is free, but never interrupts a running job. STCF (shortest time to completion first) is SJF that *may* interrupt: when a new job arrives with less work left than the running one, it takes over. RR (round robin) gives each ready job a fixed slice, here 10 units, in turn. MLFQ (multi-level feedback queue) is described below.

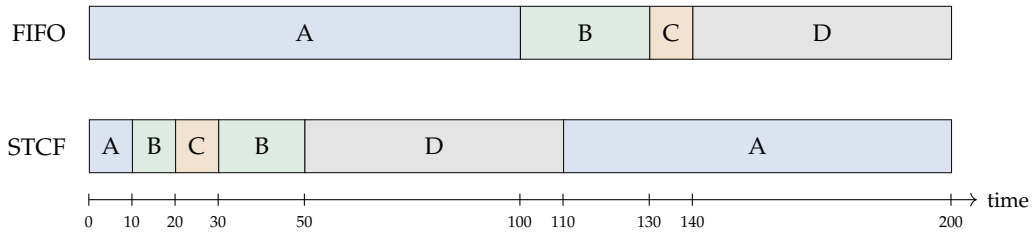


Figure 12.4: The same four jobs (A 100, B 30, C 10, D 60; arriving at 0, 10, 20, 30) under FIFO and STCF. FIFO runs each job to completion in arrival order, so B, C and D all wait behind A. STCF switches at every arrival and finish (five switches) and runs A last. Both rows are the schedules behind the turnaround numbers in Figure 12.5.

policy	avg turnaround	avg response	order
FIFO	127.5	77.5	ABCD
SJF	122.5	72.5	ACBD
STCF	82.5	5.0	ABCD
RR (slice 10)	107.5	7.5	—
MLFQ (10/20/40)	97.5	0.0	—

Figure 12.5: Real `sched_sim.py` output on the chapter's job set (A:100, B:30, C:10, D:60 time units): STCF's preemption buys a 35% turnaround win and a 15× response win — the interactivity argument, measured not asserted. RR and MLFQ rows come from the extended simulator in this chapter's listing (same job set; MLFQ has three levels with slices 10, 20, 40 and a boost every 100 units).

Example 12.6 (What the numbers argue)**.** FIFO's 127.5 turnaround is the convoy problem: B, C, D all wait behind 100-unit A. STCF's preemption drops response time to 5.0 — interactive jobs feel instant. SJF without preemption buys almost nothing here. The policy debate is numerical, not aesthetic.

```
arrivals 0,10,20,30 (A=100 B=30 C=10 D=60)
policy turnaround response switches first-run/slice order
fifo 127.5 77.5 3 ABCD
sjf 122.5 72.5 3 ACBD
stcf 82.5 5.0 5 ABCBDA
rr 107.5 7.5 17 ABACBDABDADADADADA
mlfq 97.5 0.0 9 ABCDABDADA

all arrive at t=0
policy turnaround response switches first-run/slice order
fifo 142.5 92.5 3 ABCD
sjf 87.5 37.5 3 CBDA
stcf 87.5 37.5 3 CBDA
rr 120.0 15.0 16 ABCDABDABDADADADA
mlfq 112.5 15.0 9 ABCDABDADA
```

Listing 12.10: Real output of `python3 ch12_sched_sim.py`. The `switches` column counts how many times the core changed from one job to another.

Example 12.7 (Round robin by hand, then check the simulator)**.** *Setup.* All four jobs present at time 0, slice 10, order A, B, C, D. Walk the cycles. Cycle 1 runs A, B, C, D for 10 each (time 0–40), and C, which needed only 10, finishes at 30. Cycle 2 runs A, B, D (40–70); cycle 3 runs A, B, D (70–100) and B, having now had 30, finishes at *its* slot ending at 90. At time 100 D has had three slices (30 of its 60 units) and A has had three too (30 of 100), so they alternate A, D, A, D, A, D until D finishes at 160, and then A runs alone until 200. Turnarounds (arrival is 0, so turnaround equals finish): C = 30, B = 90, D = 160, A = 200, so the average is $(30 + 90 + 160 + 200)/4 = 120$, and the simulator's second table agrees (`rr 120.0`). Notice what RR bought: response time 15 instead of FIFO's 92.5, at the price of 16 switches instead of 3. SJF, which never interrupts, reaches turnaround 87.5 with only 3 switches, because here all jobs are known and equal arrival makes "shortest first" exactly optimal for turnaround. The best policy depends on which of the two numbers you care about.

Example 12.8 (The convoy depends on who arrives first)**.** Compare the two tables. With staggered arrivals SJF scored 122.5, barely better than FIFO's 127.5, because the long job A was already running when the short ones arrived and SJF will not interrupt it. With every job present at time 0, SJF scores 87.5, a drop of 35, while FIFO gets *worse* (142.5) since its order is still A, B, C, D. Verify one entry by hand: SJF at $t = 0$ runs C first (finish 10), then B (40), then D (100), then A (200), so $(10 + 40 + 100 + 200)/4 = 87.5$. A single policy's score is a property of the policy *and* the workload together.

MLFQ stands for multi-level feedback queue. Its problem: the scheduler almost never knows how long a job will run, so STCF's "shortest first" cannot be applied directly. Its trick is to watch instead of ask: jobs that use their full slice sink to longer-slice queues (they look like long jobs); interactive jobs stay high because they block on I/O before the slice ends (they look short). A periodic boost prevents starvation. Real schedulers (Linux's CFS uses different math for the same ends) all answer the same tension: turnaround vs. response.

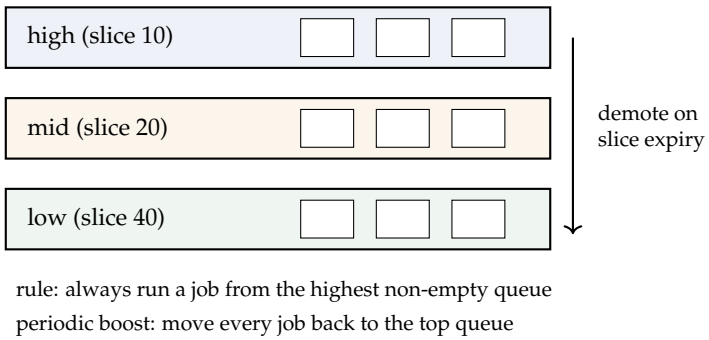


Figure 12.6: MLFQ without code: short jobs finish while still at the top (they end before using a full slice, so they are never demoted); long jobs use full slices, sink and get long slices; a periodic priority boost stops starvation. The policy approximates STCF without knowing job lengths. Slices shown (10, 20, 40) are those used by the simulator.

12.5 The few dozen lines that actually do it

Guided read — xv6 `proc.c`

`swtch.S`: 14 stores save `ra`, `sp` and `s0–s11` of the old context, 14 loads restore the new one, then `ret` (29 instructions in all; read from the xv6-riscv repository, October 2026). The stack-pointer swap is what actually changes which process is running. `scheduler()`: an endless loop that walks the process table from the start and runs every RUNNABLE process it finds until it gives the core back (a timer tick, or the process blocks). This is round robin over the table, not first-come first-served.

Figure 12.7: The read-the-source payoff: xv6's context switch and scheduler fit in a few dozen lines between them. The policy chapter's machinery is this small on a real OS.

12.6 Common mistakes

- **Skipping `fork`'s error path.** Fork bombs apart, resource limits make fork fail; an unchecked `fork()` becomes a crash or a zombie.
- **`return` after a failed `exec`.** The child falls back into the parent's code — use `_exit` or `perror`+`_exit`.
- **`wait` only at program end.** Unreaped children zombie: harvest statuses as they exit.
- **Treating ctx-switch cost as free.** 3 µs × thousands/sec is real time; thread pools exist because the number isn't zero.
- **Forgetting to flush stdio before `fork`.** The child gets a copy of the unflushed buffer, so text can appear twice, late, or (after `exec`) not at all.
- **Closing only some pipe ends.** A reader sees end-of-file only when every copy of the write end is closed, the parent's included; otherwise the pipeline hangs.
- **Timing a switch without saying where the processes ran.** Pinned and unpinned runs differed by 9× here.

- **Optimising turnaround when you meant response.** STCF wins both on our set, but SJF's non-preemptive gain is nearly zero — measure the metric you actually care about.

12.7 Summary

Piece	Verified version
process	address space + saved state + kernel entry
fork/exec	duplicate then replace; COW makes fork O(1)-ish
ctx switch	trap + stack swap; ≈3 µs (Mac), 2.1 µs pinned / 19.6 µs unpinned (Linux container)
FIFO/SJF/STCF	127.5 / 122.5 / 82.5 turnaround on real sim
RR (slice 10)	107.5 turnaround, 7.5 response, 17 switches
MLFQ	97.5 turnaround, 0.0 response, 9 switches; needs no job lengths
zombie	exited, status kept until the parent waits
xv6	the real mechanism: 29 instructions of `swtch` plus a ∼20-line loop

Further reading

- Arpaci-Dusseau & Arpaci-Dusseau, OSTEP ch4–11 (the sim homework this chapter follows): `https://pages.cs.wisc.edu/~remzi/OSTEP/`
- xv6 source, `proc.c`, `swtch.S`: `https://github.com/mit-pdos/xv6-riscv`
- CS:APP ch8 (exceptional control flow): `https://csapp.cs.cmu.edu/`
- Linux CFS design doc (the production counterpoint): `https://docs.kernel.org/scheduler/sched-design-CFS.html`

WATCH INSTEAD

- **Processes & Scheduling — MIT 6.1810 (xv6) lecture** — the actual mechanism this chapter describes, on the real codebase (verified 2026-10-11). `https://youtu.be/vsKrTHZADW4`

Notes

- Context-switch, fork demo, and scheduler sim are all real outputs (Mac for the older figures, Linux container for the new listings); the multi-pair scaling confound is stated honestly rather than smoothed.
- xv6 excerpts are paraphrase-by-design — the chapter teaches the mechanism's shape; the source repo is linked for the real code.

Exercises

Exercise 12.1 (easy)**.** How many `fork()` calls does the shell make for the pipeline `cat f | grep x | sort | uniq`, and how many `exec`s?

Solution. [hint] Four processes, so four forks and four execs: one child per stage, each replacing itself with its own program. (The three-stage pipeline in the chapter needs three of each; the count equals the number of stages.)

Exercise 12.2 (easy)**.** Name the three pieces of state a context switch must preserve, and which one is swapped rather than copied.

Solution. [hint] Registers (copied to kernel stack), stack pointer (swapped), address space (satp/page-table root changed).

Exercise 12.3 (easy)**.** Which state transition does an `exit()` perform, and what makes a zombie different from a dead process?

Solution. [hint] running→terminated; the zombie still holds a slot until a parent waits.

Exercise 12.4 (medium)**.** Recompute the average turnaround of FIFO, SJF and STCF for our job set if all arrivals were at t=0. Which policy's number changes most, and why?

Solution. [hint] FIFO: finishes 100, 130, 140, 200, so $570/4 = 142.5$ (up from 127.5). SJF and STCF coincide: order C, B, D, A finishes 10, 40, 100, 200, so $350/4 = 87.5$. SJF changes most, $122.5 \rightarrow 87.5$, a drop of 35: with staggered arrivals the long job A was already running and SJF would not interrupt it; with everyone present SJF picks the short jobs first.

Exercise 12.5 (medium)**.** Our ctx-switch measurement uses pipes, which add syscall overhead. Design the variant that isolates the switch cost more cleanly — and name the remaining confound you cannot remove.

Solution. [hint] Yield-loop threads or futex ping; cache/TLB warmth after the switch is unmeasurable from inside.

Exercise 12.6 (medium)**.** A user claims "STCF is good for every job." Take A(10) at t=0, L(100) at t=1, S1(5) at t=2, S2(5) at t=3. Compute when L first runs and when it finishes under FIFO and under STCF, and say which job pays for STCF's good average.

Solution. [hint] FIFO: A runs 0–10, L runs 10–110 (response $10 - 1 = 9$, turnaround 109). STCF: at t=2 S1 (5) beats A's remaining 8, so S1 runs 2–7, then S2 7–12, then A 12–20, and L only 20–120: response 19, turnaround 119. L pays: it is delayed 10 units by the short jobs that arrived after it, while the average over all four jobs is much better. STCF minimises the *average*, not the worst case, and a steady stream of short jobs can starve a long one.

Exercise 12.7 (medium)**.** MLFQ's periodic boost exists to prevent starvation. What goes wrong if instead every job that finishes a slice gets *half* a priority demotion forever?

Solution. [hint] Long jobs asymptote to floor priority and never get CPU under load; the boost resets the drift.

Exercise 12.8 (medium)**.** xv6's `scheduler()` walks the process table from the start and runs each RUNNABLE process until a timer tick or a block. Which of this chapter's policies is that closest to, and does it know job lengths?

Solution. [hint] Round robin over the table with the timer tick as the slice. It uses no information about job length or priority, so it gives decent response time but cannot favour short jobs the way STCF or MLFQ do.

Exercise 12.9 (hard)**.** The chapter's `ch12_sched_sim.py` already has an MLFQ. Extend it with an I/O model and reproduce the "game the scheduler" workload: a job that uses 99% of its slice and then sleeps one tick, so it is never demoted. Report its share of CPU against its fair share, and say how a boost or total-time accounting changes it.

Solution. [hint] Slice-boundary tricks work until the boost or the accounting changes; measure by counting scheduled ticks.

Exercise 12.10 (hard)**.** At our measured 3 µs per switch, a server doing 20 switches per request at 10 000 req/s spends what fraction of CPU switching? Do pools or event loops erase that cost — and which hidden cost do they add?

Solution. [hint] $10^4 \times 20 \times 3\mu s = 0.6$s of CPU per second — 60%. Event loops remove switches but add cache-cold callback chains; measure both.

Exercise 12.11 (hard)**.** Extend the sim with a realistic I/O model (each job alternates CPU-burst/I/O-wait). Show that MLFQ's observable-behavior trick classifies jobs correctly without knowing their lengths, and state the workload where it still fails.

Solution. [hint] CPU-bound batch jobs sink correctly; the failure is a job that alternates just fast enough to look interactive while hogging.

Exercise 12.12 (easy)**.** A zombie process holds which kernel resources, and which syscall frees them?

Solution. [hint] Exit status + process-table slot; `wait/waitpid`.

Exercise 12.13 (medium)**.** Why does `fork` return twice? Explain what differs between the two return paths and why `fork+exec` beats a hypothetical `spawn(path,args)` on flexibility.

Solution. [hint] Both parents resume after the trap; the gap between fork and exec is where shells set up redirection/limits.

Exercise 12.14 (medium)**.** The simulator now prints context switches per policy. Read the staggered-arrival table: how many switches does STCF make to win its 45-unit turnaround gain over FIFO, and how many does RR make? At 3 µs per switch, what is the total switching time of each?

Solution. [hint] STCF 5 switches, RR 17, FIFO 3. At 3 µs: STCF 15 µs, RR 51 µs, so the extra 2 switches STCF makes over FIFO cost 6 µs, noise next to a gain of 45 *time units* as long as a unit is much longer than a fraction of a microsecond.

Exercise 12.15 (hard)**.** Take STCF on our job set with a 3 µs penalty for each of its 5 switches. If one time unit of the job set lasts u microseconds, at what u does the switching overhead (charged in full to the average) exceed STCF's 45-unit turnaround saving over FIFO? Derive the crossover.

Solution. [hint] STCF makes 5 switches here, so the total overhead is at most $5 \times 3\,\mu\text{s} = 15\,\mu\text{s}$. If one time unit lasts u microseconds the average turnaround gain is $45u$. The overhead exceeds the gain when $45u < 15$, i.e. $u < 0.33\,\mu\text{s}$. Real job lengths are milliseconds or more, so the crossover is far below any slice a scheduler would use.

Exercise 12.16 (easy)**.** In `ch12_pipeline.c`, which descriptors must the parent close, and what do you see if it closes none of them?

Solution. [hint] All four (both ends of both pipes). Without that the write ends stay open in the parent, `tr` never sees end-of-file and the program hangs; building with `-DLEAK` and running under `timeout 2` ends with status 124.

Exercise 12.17 (easy)**.** From the pinned one-pair line (2.06 µs per switch), compute the round trips per second the program should report.

Solution. [hint] A round trip contains two switches, $2 \times 2.06\,\mu\text{s} = 4.12\,\mu\text{s}$, so $1/4.12\,\mu\text{s} \approx$ 242,700 per second; the program printed 242 889 (the difference is rounding of the 2.06).

Exercise 12.18 (medium)**.** Compute average turnaround for round robin with slice 10 when all four jobs (A 100, B 30, C 10, D 60) arrive at 0 in order A, B, C, D. Then recompute with slice 50 and compare switches and response.

Solution. [hint] Slice 10: finishes C 30, B 90, D 160, A 200, average 120 (the simulator agrees), response 15, 16 switches. Slice 50: A runs 0–50, B 50–80 (done), C 80–90 (done), D 90–140, A 140–190 (done), D 190–200 (done). Finishes B 80, C 90, A 190, D 200, so the average is $(80 + 90 + 190 + 200)/4 = 140$. Responses are $0, 50, 80, 90$, average 55, and there are only 5 switches. A longer slice means fewer switches but a worse response, which is the trade-off RR exposes.

Exercise 12.19 (hard)**.** Explain, using the zombie listing, why a long-running server that forks a child per request but never calls `wait` eventually fails, and give two ways to avoid it.

Solution. [hint] Every exited child stays a zombie holding a process-table slot, so the table (or the per-user process limit) fills and `fork` starts returning -1. Fixes: call `waitpid` for finished children (for example in a `SIGCHLD` handler with `WNOHANG`), or make the child a grandchild by forking twice so that the init process reaps it.

CHAPTER 13

Concurrency and synchronisation

Imagine two cooks sharing one recipe card, and the card says "eggs used: 5". Cook one reads 5, goes to fetch an egg, and writes 6. Meanwhile cook two also read 5 before cook one wrote anything, and also writes 6. Two eggs were used; the card says one. Nobody did anything wrong step by step, and the card is still wrong. That is the whole problem of this chapter, and it is the reason programs with several threads are famous for bugs that appear only on some runs. Chapter 12 gave every process its own world; here threads share one world, and we watch it break, then learn the tools that keep it intact, and price each tool on a real machine.

WHAT YOU WILL BE ABLE TO DO

After this chapter you can write a small multi-threaded C program, make it lose updates on purpose, and then fix it in three different ways. You can build a producer and consumer that wait for each other correctly with a condition variable, show the classic `if`-instead-of-`while` bug in action, and turn a deadlock into a visible, repeatable report and cure it with a lock order. You will also know how to read a scaling curve without trusting it blindly, including spotting the case where the compiler quietly deleted the work you meant to time.

BEFORE YOU START

A *thread* is a separate line of execution inside one process: threads share the same memory but each has its own registers and stack (the process, from ch. 12, is the house; threads are the people living in it). The hardware underneath is what Book 7 ch. 10 (caches) showed: memory is moved in 64-byte *cache lines*, and a core that writes a line must own it. If you want a reminder of how registers and a bus move one value at a time, Book 5 ch. 15 (registers, buses and the datapath) is that picture in hardware.

A short C refresher. `pthread_create(&t, 0, f, arg)` starts function `f(arg)` as a new thread and stores its handle in `t`; `pthread_join(t, 0)` waits for it to finish. A `static` global variable is visible to every thread. `volatile` tells the compiler "this value may change behind your back, really read and write it every time". `counter++` is shorthand for three steps: load, add one, store. Compile with `gcc -O2 -pthread file.c`. Numbers marked *Mac* come from the author's earlier laptop; numbers marked *Linux* come from a 4-vCPU Intel Xeon container (2.8 GHz, gcc 13) and are kept apart.

THINK LIKE THIS

For any shared value, ask three questions in order. Who else can touch it? What *sequence* of steps must be indivisible? What is the cheapest way to make that sequence indivisible, or to stop sharing the value at all? The best answer in this chapter is almost always the last one.

CONCEPT: CRITICAL SECTION AND INVARIANT

A *critical section* is a stretch of code that must run as if nobody else were running at the same time. Why must it? Because while the code is half done, some fact about the data is temporarily false. For the egg card the fact is "the number on the card equals the eggs used"; between "read 5" and "write 6" it is false for a moment. Such a fact is an *invariant*. The skilled habit is to name the invariant first, then find the smallest piece of code during which it is broken, and protect exactly that piece. Too large a critical section makes threads wait for no reason (slow); too small leaves the invariant exposed (wrong).

13.1 The race, demonstrated

T_1 (counter++)	T_2 (counter++)	counter
load c (=5)		5
	load c (=5)	5
t+1, store (6)		6
	t+1, store (6)	**6**

Figure 13.1: Two threads, one increment each, one lost. Both loaded 5, both wrote 6 — the interleaving throws away an entire update. Run for real: 6 threads lost 2 480 483 of 3 000 000 updates (83%) on the Mac.

Example 13.1 (83% of updates lost, real (Mac))**.** `ch13_race.c` runs 6 threads of the unprotected `load-add-store` sequence, 500 k increments each. Three runs: counter lands at 519 517, 502 923, 521 101 — never the expected 3 000 000. Each run loses ≈2.5 million updates because both threads load before either stores. A race isn't a sometimes-bug; under contention it is the *usual* result.

Example 13.2 (The same race on the Linux container)**.** *What we expected.* The same 83% loss. *What happened.* Five runs of the same program gave very different answers, as the listing shows. The lesson is that the loss rate is not a constant of the program: it depends on how often two threads happen to overlap in the tiny load–store window, which depends on the machine, the number of cores and the moment.

```
1 static volatile long counter; // shared by all 6 threads
2 for (long i = 0; i < 500000; i++) { long t = counter; t = t + 1; counter = t; }
3
4 counter=1594598 (expected 3000000, lost 1405402)
```

```
counter=2062530 (expected 3000000, lost 937470)
counter=501840 (expected 3000000, lost 2498160)
counter=2113490 (expected 3000000, lost 886510)
counter=2142752 (expected 3000000, lost 857248)
```

Listing 13.1: The racy loop from `ch13_race.c`, and real output of five runs of the compiled program (Linux x86_64 container, `gcc -O2`).

Lost fractions: 1405402/3000000 = 47%, 31%, 83%, 30%, 29%. Building with `-O0` changed nothing important (three runs lost 1.38 M, 2.01 M and 1.91 M). The point to carry away: a program that is wrong in one run and "almost right" in another is not partly correct, it is incorrect, and testing alone will not tell you which kind of run you got.

13.2 The atomic ladder

mechanism	the atomic unit it gives you
`lr.w/sc.w` (RISC-V A)	load-reserved + store-conditional retry loop
`amoadd` (RISC-V A)	atomic memory op, no retry
C11 `atomic_fetch_add`	portable; compiles to the above
`pthread_mutex`	sleep-lock over the same primitives

Figure 13.2: The atomic ladder. Everything in this chapter stands on the hardware's indivisible read-modify-write; locks are policy built on top of it.

Why can a fix not be "just write the code more carefully"? Because the problem lives between instructions, where the program cannot see. The only way out is help from the hardware. Hardware provides one indivisible thing: a read-modify-write that no other core can interleave with. RISC-V's A extension gives two flavours — `lr.w/sc.w` (load-reserved / store-conditional, the retry-loop primitive; `.d` on RV64) and `amo` instructions (atomic memory ops that complete without a retry). Every synchronisation primitive above them is software.

13.3 Three fixes, one measurement

Same race, fixed three ways: a mutex around the increment, a C11 `atomic_fetch_add`, and partitioning (each thread counts privately, sum at the end). `ch13_lockscale.c` times all three at 1–8 threads:

Example 13.3 (The honest surprise (Mac))**.** Partition appears to win trivially — flat ∼0.1 ms — but the next example shows that flat line is not what it seems. Mutex scales badly (1.8→35.4 ms) as expected. But at 8 threads the *atomic* path is slower than the mutex (56.3 ms): the single contended cache line's coherency traffic costs more than the lock's parking. On a laptop with real cores the curve isn't monotonic — and that's the lesson: don't guess, measure.

Example 13.4 (The compiler deleted the partitioned loop)**.** *What looked right.* Both machines report the partitioned counter at about 0.1 ms for 400 000 increments per thread, flat as the

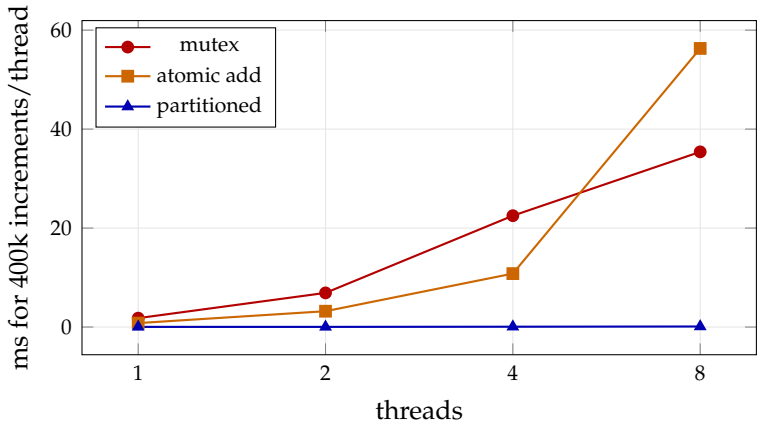


Figure 13.3: Real `ch13_lockscale.c` on the Mac. The partition line is flat (∼0.1 ms) only because the compiler deleted its loop (see the examples). The honest surprise: at 8 threads the atomic add is *slower than the mutex* — the single contended cache line costs more than the lock.

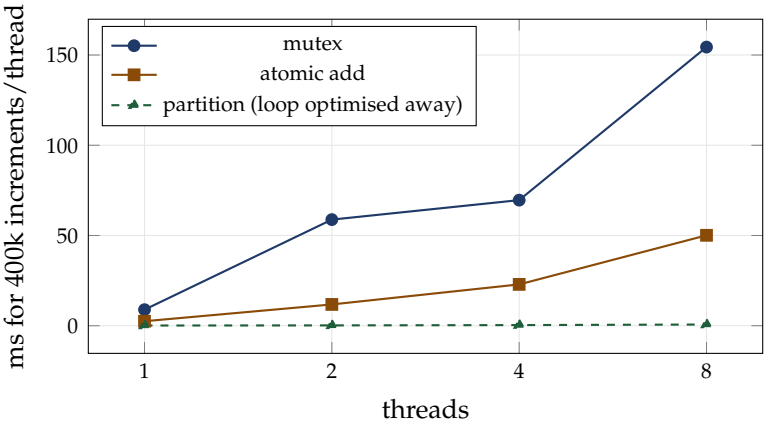


Figure 13.4: `ch13_lockscale.c` on the Linux x86_64 container (4 vCPUs, so the 8-thread point has twice as many threads as cores; one run, mutex values vary widely run to run). Here the atomic add is faster than the mutex at every thread count, the opposite of the Mac result in Figure 13.3. The dashed partition line is thread start-up time only, because the compiler replaced the loop by a single addition (see the example below).

thread count grows. That is ≈4 billion increments per second per thread, which is more than one increment per clock cycle on a 2.8 GHz core for a loop that does a load, an add and a store each time. Whenever a number is better than the hardware allows, suspect the measurement. *What happened.* We asked gcc for the assembly of the thread function:

```
f_pt:
       movq part(%rip), %rax
       addq $400000, (%rax,%rdi,8)
       xorl %eax, %eax
       ret
```

Listing 13.2: Real `gcc -O2 -S` output for the partition worker of `ch13_lockscale.c`. The loop is gone: it became one addition of 400000.

Nothing else could touch `part[k]` inside the loop, so the compiler legally replaced 400 000 increments by one add. The "partition" column therefore measures *thread start-up and join* only. This flaw is in the original program and so also in the Mac partition numbers above (we did not re-inspect the Mac compiler's output, but the same 0.1 ms figure points the same way); the mutex and atomic columns are unaffected, since the compiler may not remove a lock or an atomic operation. `ch13_partition.c` repeats the experiment with `volatile` counters so that every increment really happens, which is the next example.

Example 13.5 (False sharing: private counters that are not private)**.** *What we expected.* If each thread increments its own counter nothing is shared, so time should stay flat as threads are added. *What happened.* It depends on where the counters sit in memory. A core cannot write part of a cache line; it must own the whole 64-byte line. If counters for different threads are neighbours in an array they share a line, and the line bounces between cores even though no value is shared. This is *false sharing*. We timed both layouts:

```
T adjacent (ms) padded (ms)
1 176.0 166.9
2 310.2 173.9
4 388.3 179.6
```

Listing 13.3: Real output of `ch13_partition` (Linux x86_64 container; 100 million increments per thread). "padded" puts each counter on its own 64-byte line.

threads	adjacent (ms)	padded (ms)	slowdown
1	176.0	166.9	1.05×
2	310.2	173.9	1.78×
4	388.3	179.6	2.16×

Figure 13.5: Real `ch13_partition.c` output (Linux x86_64 container): each thread does 100 M increments of its *own* volatile counter. In the *padded* layout every counter sits on its own 64-byte line and the time stays flat. In the *adjacent* layout several counters share a line, the cores fight over it, and the same work takes up to 2.2× longer. Slowdown is adjacent / padded.

Padded stays flat at about 170–180 ms, exactly the "nothing is shared" prediction. Adjacent is 1.8× slower at two threads and 2.2× slower at four. The fix costs seven unused longs per counter and no locks at all.

13.4 Condition variables, done right

thread A (consumer)	shared queue	thread B (producer)
lock; `while`(empty) wait(cv)	empty	
↓ sleeps, releases lock		lock; push; signal; unlock
wakes, re-locks, pops; unlock	item	

Figure 13.6: Mesa semantics: `wait` releases the mutex while sleeping and re-acquires it on wake — and the predicate *must* be re-checked in a `while` loop, because another consumer may have won the wakeup race.

Locks stop two threads colliding, but they do not help a thread that must *wait for something to happen*, such as a consumer waiting for an item. Spinning in a loop checking the queue wastes a core. A *condition variable* (CV) is the polite alternative: the waiting thread goes to sleep and releases its lock, and another thread wakes it when the condition may have changed. Think of the buzzer a restaurant gives you while a table is cleared.

The classic producer–consumer: the consumer waits on a CV while the queue is empty; the producer signals after pushing. Mesa semantics — the woken consumer competes for the lock and *must* re-check the predicate in a `while` loop, because a sibling consumer may have drained the queue in the gap. The `if`-instead-of-`while` bug produces a consumer that pops from an empty queue — intermittent, load-dependent, and the single most-repeated concurrency bug in history.

Example 13.6 (`while` versus `if`, run for real)**.** *Setup.* One producer pushes 200 000 items and wakes all sleeping consumers each time (`pthread_cond_broadcast`); three consumers each wait for the queue to be non-empty. We run the same program twice, once checking the predicate with `while` and once with `if`. In the `if` version, a consumer that wakes to an empty queue would pop from nothing; the program counts these moments instead of crashing, so we can see them.

```
if (use_while) { while (queue_len == 0 && !done) pthread_cond_wait(&cv, &m); }
else { if (queue_len == 0 && !done) pthread_cond_wait(&cv, &m); }

mode=while produced=200000 consumed=200000 empty-queue wakeups=0
mode=if produced=200000 consumed=200000 empty-queue wakeups=2257
mode=if produced=200000 consumed=200000 empty-queue wakeups=2310
```

Listing 13.4: The two waits, and real output (`ch13_prodcons`, Linux x86_64 container). The `if` count varied between runs (2 257 and 2 310 in two runs).

About 2 300/200 000 ≈ 1% of the items woke a consumer for nothing. In production code that wakeup is an underflow or a crash that happens roughly once in a hundred operations, which passes light testing and fails under load. The `while` loop makes the rule simple: after every wakeup, check again.

13.5 Deadlock and lock ordering

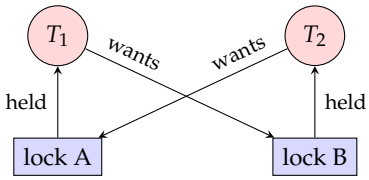


Figure 13.7: The wait-for graph of a textbook deadlock: T_1 holds A, wants B; T_2 holds B, wants A. A cycle in this graph is a deadlock — and a global lock *order* is what prevents cycles from ever forming.

Picture two people meeting in a corridor too narrow to pass, each refusing to step back until the other does. Neither is slow or broken; each is waiting for the other. A deadlock is that situation for locks: thread 1 holds A and wants B, thread 2 holds B and wants A. A deadlock is a cycle in the wait-for graph. Preventing cycles is simpler than finding them: impose a global lock *order* (always acquire A before B) and cycles become impossible. Lock-free designs skip locks entirely but pay in memory-ordering reasoning — the [hard] exercise below repairs a broken one informally.

Example 13.7 (Make a deadlock happen on purpose, then prevent it)**.** *What goes wrong first.* Thread 1 locks A then B; thread 2 locks B then A. We put a 0.1 s sleep between each thread's two acquisitions so that both always hold their first lock before asking for the second, which turns a "rare" bug into a certain one. A normal `pthread_mutex_lock` would hang forever, so the demo uses `pthread_mutex_timedlock` with a one-second limit to report the stuck state and exit.

```
$ ./ch13_deadlock bad
T1: holds A, waited 1 s for B -> deadlock
T2: holds B, waited 1 s for A -> deadlock
opposite order: 2 thread(s) stuck
$ ./ch13_deadlock good
T1: got both locks
T2: got both locks
global order: 0 thread(s) stuck
```

Listing 13.5: Real output of `ch13_deadlock bad` and `ch13_deadlock good`. The two report lines may print in either order.

The cure is a rule, not cleverness: every thread takes A before B. Then no thread can hold B while waiting for A, so the cycle cannot form. The cost is that every programmer must follow the order, which is why large programs write the order down.

13.6 What concurrency actually buys

At 90% parallel work, 8 cores buy 4.7×, not 8 — and the padded counters' flat curve (167–180 ms at 1, 2 and 4 threads in Figure 13.5) shows the actual path to the speedup: eliminate the shared state, not just the locks around it. Amdahl isn't pessimism; it's the design brief.

parallel fraction	speedup @4T	@8T	@∞
0.90	3.08	4.71	10.0
0.95	3.48	5.93	20.0
0.99	3.88	7.48	100.0

Figure 13.8: Amdahl's arithmetic: at 90% parallel, 8 cores buy 4.7×, not 8. The serial 10% is the ceiling — which is why the partitioned counter's flat curve matters more than any lock trick.

13.7 Common mistakes

- **"It works on my machine" races.** The same unfixed program lost 29% to 83% of its updates across runs here; timing and contention decide how visible the bug is, not whether it exists.
- **Trusting a flat line.** The "partitioned" counter looked free because gcc turned the loop into one add. A measurement better than the hardware allows is a bug in the measurement.
- **`if` instead of `while` on a CV.** Mesa semantics require re-checking; the intermittent failure is the classic heisenbug.
- **Atomic = fast (or slow), assumed.** On the Mac one contended line out-cost a mutex at 8 threads; on the Linux container the atomic won at every count. Contention is the enemy, not the mechanism, and the winner depends on the machine.
- **Deadlock by "it can't happen here"**. Any lock acquired while holding another can cycle; ordering is cheap, analysis is expensive.
- **Sharing the cache line, not the data.** Adjacent counters on one line bounce between cores (false sharing) — ch10's geometry returning through the abstraction.

13.8 Summary

Piece	**Verified version**
race	6 threads: 2.48M of 3M updates lost, real
atomics	lr/sc + amo are the hardware's indivisible unit
mutex	scales to 35ms at 8T — parking cost is real
atomic add	Mac: slower than mutex at 8T; Linux container: faster at every count
partition	eliminate sharing, not locking; pad counters (adjacent: 2.2× slower at 4T)
CV	`while`-loop re-check is mandatory
Amdahl	8T at 90% parallel = 4.7×, the serial ceiling

Further reading

- OSTEP ch26–33 (the bug taxonomy and all the classic code): https://pages.cs.wisc.edu/~remzi/OSTEP/
- xv6 ch6, ch9 (spinlock + sleep/wakeup on a real kernel): https://github.com/mit-pdos/xv6-riscv
- CS:APP ch12 (concurrent programming): https://csapp.cs.cmu.edu/
- RISC-V A-extension spec (lr/sc semantics): riscv.org/specifications

WATCH INSTEAD

- **Concurrency & Race Conditions — Computerphile** — the same lost-update demo at this altitude (verified 2026-10-11). https://youtu.be/7ENFeb-J75k

Notes

- The older numbers are verbatim outputs of the chapter's C programs on Apple Silicon with clang+pthreads (the atomic-at-8T regression is a measured result there). New listings are from a Linux x86_64 container and are labelled. The partition column of `ch13_lockscale.c` times thread start-up only (the loop is optimised away, verified on Linux); the honest measurement is `ch13_partition.c`.
- The deliberate-race demo is intentionally volatile-marked — with optimisation and few threads it sometimes "passes"; that unreliability is itself a Common-mistakes entry.

Exercises

Exercise 13.1 (easy)**.** Why does the race *sometimes* produce the right answer? Name the two factors that decide whether the interleaving bites.

Solution. [hint] Timing (how often threads overlap the load-store window) and contention level; correctness never depends on them, only speed.

Exercise 13.2 (easy)**.** Distinguish a data race from a race condition in one sentence each.

Solution. [hint] Race: two accesses to one location, one a write, unsynchronised. Condition: program output depends on interleaving order.

Exercise 13.3 (easy)**.** What does the mutex actually *do* when it can't acquire the lock — on this implementation, sleep or spin?

Solution. [hint] pthreads mutexes park the thread in the kernel; pure spinlocks burn cycles instead.

Exercise 13.4 (easy)**.** Why does partitioning beat even an uncontended lock? One line.

Solution. [hint] No shared line means no coherency traffic at all.

Exercise 13.5 (medium)**.** Show the exact two-instruction interleaving that loses an update in `counter++` on a single core. Can it happen on one core?

Solution. [hint] Load 5, preempted; other thread loads 5, stores 6; first stores 6 — yes, preemption alone suffices.

Exercise 13.6 (medium)**.** Implement the producer–consumer queue with CVs, then break it by changing `while` to `if`. Describe the failure mode and why it passes light testing.

Solution. [hint] Two consumers wake on one signal; loser pops empty — rare under low load, common at capacity.

Exercise 13.7 (medium)**.** On the Mac the atomic counter lost to the mutex at 8 threads; on the Linux container it won at every thread count. Give a cache-terms explanation for the Mac result and one hypothesis for why it does not reproduce, and say what you would measure to test it.

Solution. [hint] Every atomic op forces the line to move to the executing core; a parked thread makes no traffic at all. Hypotheses: different cores, interconnect and lock implementations. To test, compare hardware cache-miss counters or sweep the thread count on both machines. Only a measurement settles it.

Exercise 13.8 (medium)**.** Design the experiment that distinguishes spinlock vs mutex on a load where the critical section is 5 instructions. Predict the crossover point.

Solution. [hint] Sweep contention; spin wins when wait $<$ park/wake cost — roughly microseconds.

Exercise 13.9 (medium)**.** A team replaces a mutex with a lock-free queue and sees latency *worse* at 2 threads. Two plausible mechanisms?

Solution. [hint] Retry storms under contention, and memory-ordering fences costing more than the lock's fast path.

Exercise 13.10 (hard)**.** Prove or repair this "lock-free" counter: both writers run a shared check-then-store on one variable. Give the Dekker-style argument for the failure, state the ordering assumption used, then write the correct primitive in pseudo-code.

Solution. [hint] Check-then-act is two operations; need CAS (compare-and-swap) as one atomic unit — on RISC-V, lr/sc loop.

Exercise 13.11 (hard)**.** Extend the lockscale measurement to *reader*-heavy workloads (95% reads). Which mechanism wins then, and which C++17/Rust primitive implements it? Why does it beat a mutex?

Solution. [hint] A shared lock (rwlock/shared_mutex) wins: read paths don't serialise; the mutex treats reads as writes.

Exercise 13.12 (hard)**.** The assembly listing shows the partition loop collapsed to one add. Predict roughly what the partition column of `ch13_lockscale` would show if `part` were `volatile`, using `ch13_partition.c`'s padded timing as your per-increment cost.

Solution. [hint] The padded run does 100 M increments in about 167 ms, i.e. 1.67 ns each. So 400 000 increments cost about $400000 \times 1.67\,\text{ns} \approx 0.67\,\text{ms}$ per thread, against the 0.1 ms the compiler-deleted loop reported: roughly $6\times$ higher (more if adjacent counters share a line).

Exercise 13.13 (easy)**.** What is the wait-for graph, and which of its shapes is a deadlock?

Solution. [hint] Nodes = threads/locks, edges = holds/wants; a cycle is deadlock.

Exercise 13.14 (medium)**.** Rank the three fixes by engineering cost: partitioned counter, mutex, lock-free CAS loop. Which requires the most reasoning per line of code?

Solution. [hint] Lock-free — memory ordering is where the real cost lives.

Exercise 13.15 (medium)**.** False sharing: two threads increment *different* counters on one cache line. Predict the scaling curve vs truly-private counters and name the fix.

Solution. [hint] Slower than private counters: measured here, adjacent counters took $1.8\times$ (2 threads) and $2.2\times$ (4 threads) as long as padded ones; the fix is to pad each counter to its own 64-byte line.

Exercise 13.16 (easy)**.** In `ch13_prodcons`, the `while` version reported zero empty-queue wakeups. Explain why this is guaranteed rather than lucky.

Solution. [hint] The `while` test is re-run after every wakeup while holding the lock, so a consumer proceeds only when the queue is non-empty (or `done`). No other thread can change the queue between the test and the pop because the lock is held.

Exercise 13.17 (medium)**.** Change `ch13_deadlock bad` so that thread 2 takes A first but keep the sleeps. Predict the output, then say which of the four deadlock conditions (mutual exclusion, hold-and-wait, no preemption, circular wait) you removed.

Solution. [hint] Both threads now lock A then B; the second simply waits for the first to finish, so the output is the "good" output (0 stuck). You removed circular wait; the other three conditions still hold.

CHAPTER 14

File systems and persistence

Think of a huge library. Without a catalogue you could only find a book by walking every shelf. A file system is the catalogue plus the shelving rules for a disk: it turns a flat sea of numbered blocks into named files you can find, grow and delete. It makes three promises. *Names*: a path you can remember. *Durability*: bytes that survive a power cut. *Crash behaviour*: no file left half written. The first is easy, the other two are expensive, and this chapter pays for them on real hardware. The opening surprise is honest: on the author's Mac our sequential read ran at 17 GB/s and never touched the disk at all. It was reading the *buffer cache*, the RAM the kernel keeps copies of recent file data in, which is the layer this whole chapter exists to explain.

WHAT YOU WILL BE ABLE TO DO

After this chapter you can explain what a path lookup does, tell a file's name from its inode and show with a program that unlinking an open file does not destroy it. You can compute the largest file an indexed inode can address and the metadata a 1 GiB file needs. You can measure on your own machine how much of a file read comes from RAM and how much from the device, and what `fsync` costs as a function of write size. And you can walk a write-ahead journal through every possible crash point and say what recovery does at each.

BEFORE YOU START

A disk (or SSD) stores fixed-size *blocks*, typically 4 KB, and knows nothing about files or names; the kernel builds those on top. Recall from Book 7 ch. 9 (the memory hierarchy) that each level is a cache for the slower level beneath it, and from ch. 11 that the kernel manages memory in 4 KB pages; the buffer cache is that idea applied to disk blocks. Book 2 ch. 4 (sets, relations and functions) is where a mapping such as name→number was first made precise: a directory is exactly such a function.

A short C refresher on files. `int fd = open(path, flags, mode)` returns a small integer, a *file descriptor*, or -1 on failure; `read(fd, buf, n)` and `write(fd, buf, n)` return how many bytes they moved; `close(fd)` releases the descriptor. `struct stat st; stat(path, &st)` fills in a record whose fields `st.st_ino` (inode number), `st.st_nlink` (number of names) and `st.st_size` you will see below. `printf("%lu", x)` prints an unsigned long. Numbers marked *Mac* are from the author's earlier laptop (APFS); numbers marked *Linux* are from a 4-vCPU container with ext4 on a virtual disk and are kept separate.

THINK LIKE THIS

Every persistence mechanism is an answer to "what if the power fails *right now*?" The habit of mind is to write the steps of an update as a numbered list and ask, for each gap between two steps, what a person would find on the disk if the machine stopped there. We do exactly that in the journaling section.

14.1 Names, inodes, and the API

Why not store a file's name and bytes together? Because then renaming or linking a file would mean moving or copying its contents. UNIX separates the two: the *name* lives in a directory, the *contents and bookkeeping* live in an *inode*. A directory is a table of `name` → `inode number`; the inode is the file's real identity — mode, size, and the block numbers holding its bytes. `open()` walks the path, resolves the inode, and returns a descriptor; `read/write/ fsync/close` do the rest. The real inode walker `ch14_inodes.py` on this book's own code tree: (Mac) it printed lines such as `ino=2484919 mode=0o100644 blocks=8` for `ch02_bytes.out` — 128 files, 758 KB payload, one line each. On the Linux container, over this chapter's code directory, it printed three sample lines and a total of 216 files, 1161 KB.

inode field	role
mode/uid/size	metadata, ∼100 bytes
direct[12]	first 12 block numbers
single indirect	→ 1024 more block numbers
double indirect	→ 1024^2 block numbers
triple indirect	→ 1024^3 block numbers

Figure 14.1: Classic indexed allocation: a small fixed inode plus escalating indirect blocks. Maximum file = (12+1024+1024^2+1024^3) blocks ≈ 4 TB at 4 KB blocks — the [hard] exercise asks for the smallest working variant.

Example 14.1 (Max file size, computed)**.** *Why indirection?* An inode is small and fixed in size, but files range from 1 byte to terabytes. The trick is to put a few block numbers in the inode and let those point to *blocks of block numbers*. With 4 KB blocks and 4-byte block numbers one block holds 1024 numbers. The classic inode has 12 direct pointers, one single-indirect, one double-indirect and one triple-indirect pointer: $12 + 1024 + 1024^2 + 1024^3$ blocks ≈ 4 TiB from an inode of roughly 100–128 bytes. The program `ch14_maxfile.py` does the arithmetic step by step:

```
pointers per indirect block : 1024
direct only : 48 KiB
+ single indirect : 4.05 MiB
+ double indirect : 4.00 GiB
+ triple indirect (max) : 4.0039 TiB
1 GiB file = 262144 blocks of 4 KiB
  block-map metadata : 257 blocks = 1028 KiB (+ 12 direct pointers in the inode)
  one extent (start,length) : 16 bytes
```

Listing 14.1: Real output of `python3 ch14_maxfile.py`.

Check the 1 GiB line by hand: $2^{30}/2^{12} = 262\,144$ blocks; the 12 direct and 1024 single-indirect pointers cover 1036 of them, the remaining 261 108 need $\lceil 261108/1024 \rceil = 255$ second-level blocks, plus one single-indirect and one double-indirect root block: $255 + 1 + 1 = 257$ blocks, i.e. 1028 KiB, about 0.1% of the file. An *extent* (start block, length) describes the same contiguous file in 16 bytes. That is why ext4 uses extents: indexed allocation is flexible, but for big sequential files a run-length description is far smaller.

Example 14.2 (Names are not files: link, symlink, unlink while open)**.** *What a beginner expects.* Deleting a file erases it, and a "link" is a copy. Both are wrong, and a short program shows it. `ch14_links.c` creates a file, gives it a second name with `link`, makes a symbolic link, removes both names while the file is still open, and reads from the open descriptor.

```
created a ino=1630546 nlink=1 size=12
link a->b a ino=1630546 nlink=2 size=12
       b ino=1630546 nlink=2 size=12
symlink s ino=1630562 is-symlink=1 size=1 (length of the text "a")
unlinked a (no such name)
read via still-open fd: 12 bytes: hello inode
fstat(fd): nlink=0 (data lives until close)
```

Listing 14.2: Real output of `ch14_links` (Linux container; inode numbers differ on every run).

Read it line by line. After `link`, `a` and `b` report the *same inode number* and `nlink=2`: one file, two names. The symlink has its *own* inode whose content is just the text "a" (size 1). After both names are removed, the name is gone but the open descriptor still reads the 12 bytes, and `nlink` has dropped to 0. The kernel frees the blocks only when the link count is zero *and* no descriptor is open, which is why you can delete a log file a running program is writing to without crashing it.

14.2 The layers a read() descends

layer	what it turns into what
VFS	open/read/write → per-fs ops
file system	paths → inode → block numbers
buffer cache	block numbers → RAM pages
device driver	block numbers → NVMe commands

Figure 14.2: The descent of a `read()`: each layer renames the same bytes — filename to inode to blocks to commands. Cache lines travel up; commands travel down.

14.3 The buffer cache, measured

pattern	MB/s	verdict
sequential read	17 269	RAM-speed: page cache dominates
random 4KB read	5 950	still cached, 3× slower
write buffered	6 722	buffer cache hides the disk
write + fsync/1MB	1 896	real flush, 3.5× slower

Figure 14.3: Real `ch14_fs_io.c` numbers — and the honest surprise is the top row: at 17 GB/s you are reading the *page cache*, not the disk. The buffer cache is the layer this chapter exists to explain; fsync is the syscall that pierces it.

Example 14.3 (17 GB/s is RAM, not disk (Mac))**.** `ch14_fs_io.c` on the Mac: sequential 17 269 MB/s, far beyond what a storage device of that era delivered, because every read hits the page cache after the build pass. Random 4 KB reads: 5 950 MB/s, still cached. The folklore "disk is slow, RAM is fast" is true; the surprise is that *the OS has already hidden the disk from you*, and only `fsync` gets you past it.

Example 14.4 (Why random 4 KB reads are slower even from RAM (Linux))**.** *A tempting wrong explanation* is "random access misses the cache". Nothing here misses the page cache: the whole file is in RAM. So what else differs? The call size. Reading 4 KB at a time takes 256 times as many system calls as reading 1 MB at a time, and each call has a fixed cost. We added a line that reads *sequentially* in 4 KB pieces to separate the two effects:

```
sequential read: 5269.6 MB/s
sequential 4KB read: 4002.4 MB/s
random 4KB read: 3363.0 MB/s
write (no fsync): 2630.0 MB/s
write + fsync per 1MB: 267.3 MB/s
fsync per 4KB write: 0.51 ms per fsync
```

Listing 14.3: Real output of `ch14_fs_io` (Linux container, file in the page cache).

Going from 1 MB to 4 KB calls costs 5270 → 4002, a factor of 1.32. Making the 4 KB reads random costs a further 4002 → 3363, 1.19. So on this machine most of the gap is the per-call overhead, not randomness; we did not measure why the random part costs the rest (likely less effective prefetching and cold page-table entries).

Example 14.5 (Reaching the real device: `O_DIRECT`)**.** How do you measure the disk instead of the cache? Open the file with the `O_DIRECT` flag, which tells the kernel to skip the page cache (buffers and sizes must be aligned to 4096). `ch14_direct.c` reads the same 256 MB file three ways:

```
buffered read, 1st pass : 4984 MB/s
buffered read, 2nd pass : 6313 MB/s
O_DIRECT read : 1135 MB/s
```

Listing 14.4: Real output of `ch14_direct` (Linux container, ext4 on a virtual disk).

pattern (Linux x86_64 container)	MB/s	what it is
sequential read, 1 MB calls	5 270	page cache
sequential read, 4 KB calls	4 002	page cache, 256× more calls
random read, 4 KB calls	3 363	page cache, scattered offsets
buffered write, no fsync	2 630	copy into RAM
write + fsync per 1 MB	267	device flush each time (9.8× slower)
write + fsync per 4 KB	≈7.7	0.51 ms per fsync (343× slower)
buffered read, 2nd pass	6 313	page cache
`O_DIRECT` read	1 135	bypasses the cache: the device

Figure 14.4: Real output of `ch14_fs_io` and `ch14_direct` on the Linux container (ext4 on a virtual disk, one run each; every number varies by tens of percent between runs). The 4 KB-write figure is 4096 B/0.51 ms. Reading the same file through the cache is 6313/1135 ≈ 5.6× faster than going to the device. Compare with the Mac numbers in Figures 14.3 and 14.5.

Even the "first" buffered pass is fast because the program has just written the file, so its pages are still cached. The honest device rate shows only with `O_DIRECT`: 1 135 MB/s, so the cache made the same read 6313/1135 ≈ 5.6× faster. Note that this is a virtual disk whose real hardware we cannot see; the figure is the rate this container gets, not a property of any particular SSD.

14.4 fsync: the syscall that pierces the cache

fsync frequency	effective write MB/s	cost
never	6 722	1×
per 1 MB	1 896	3.5× slower
per 4 KB	≈58	∼115× slower (0.07 ms each)

Figure 14.5: fsync is cheap per call but brutal per byte: at 4 KB granularity, the flush dominates — write amplification in software form. Batching is the fix; durability-per-byte is the tax.

A `write` call returns as soon as the bytes are copied into the kernel's cache; the data is not yet on the device, so a power cut loses it. `fsync(fd)` says "do not return until this file's data is on stable storage". That promise is what you pay for.

(Mac) Buffered writes run at 6.7 GB/s because the kernel only copies into RAM. Add `fsync` per megabyte and throughput drops 3.5×; per 4 KB it drops ∼115× (4 KB every 0.07 ms is ≈58 MB/s against 6 722) — the flush itself becomes the workload. Databases batch commits for exactly this reason; the cost isn't the write, it's the durability boundary.

Example 14.6 (How big must a write be before fsync stops dominating?)**.** *The naive guess* is that fsync costs a fixed amount, so any write size is fine. We measured instead. `ch14_fsync_sweep.c` writes in sizes from 4 KB to 1 MB, calling `fsync` after every write:

```
write size fsyncs ms/fsync MB/s
   4096 B 512 0.679 5.8
  16384 B 512 0.586 26.7
  65536 B 128 0.568 110.0
 262144 B 32 1.029 242.9
1048576 B 8 2.374 421.2
```

Listing 14.5: Real output of `ch14_fsync_sweep` (Linux container, one run; a repeat run gave 2.9 and 10.5 ms/fsync at 1 MB on other occasions, so this disk is noisy).

Reading the table: for 4 KB to 64 KB the cost per fsync is about 0.57–0.68 ms, a *fixed flush cost* that does not depend on the size. At 1 MiB the call takes 2.374 ms, so the data transfer part is about $2.374 - 0.57 = 1.8$ ms per MiB, i.e. ≈550 MiB/s. The fixed cost equals the transfer time when $0.57\,\text{ms} \times 550\,\text{MiB/s} \approx 0.31\,\text{MiB}$: below roughly 300 KiB per fsync the flush dominates, above it the data does. Throughput climbs from 5.8 to 421 MB/s because the same fixed cost is spread over more bytes. This is the whole argument for *group commit*: gather many small writes and flush once.

14.5 Journaling: never half-written

step	journal state
1	write TxBegin + data + metadata records to log
2	write TxEnd — the commit point
3	checkpoint: copy blocks to their real locations
4	free the log region

Figure 14.6: A journal transaction. The *commit* line is the whole trick: crash before it, the transaction is ignored; crash after it, the replay redoes the write. The file is never half-updated — that is crash consistency.

Before the details, recall the problem. Moving 10 from account block X to account block Y takes two block writes, and a disk can only promise that each *single* block write is atomic. If the power fails between them, 10 units vanish. A *journal* (a write-ahead log) fixes this by first writing down the intention, in a place the recovery code knows to look, and only then changing the real blocks.

Example 14.7 (Crash at every point, with and without a journal)**.** *The naive attempt:* write X, then write Y. `ch14_journal_sim.py` models the disk as a few blocks, stops the machine after each possible step, runs recovery, and checks the invariant $X + Y = 100$ (start X=60, Y=40, transfer 10).

```
 plain crash after step 0: X=60 Y=40 sum=100 ok
 plain crash after step 1: X=50 Y=40 sum= 90 CORRUPT
 plain crash after step 2: X=50 Y=50 sum=100 ok
 journal crash after step 0: X=60 Y=40 sum=100 ok
 journal crash after step 1: X=60 Y=40 sum=100 ok
 journal crash after step 2: X=50 Y=50 sum=100 ok
 journal crash after step 3: X=50 Y=50 sum=100 ok
 journal crash after step 4: X=50 Y=50 sum=100 ok
```

```
  journal crash after step 5: X=50 Y=50 sum=100 ok
corrupt states: plain=1, journal=0
```

Listing 14.6: Real output of `python3 ch14_journal_sim.py`.

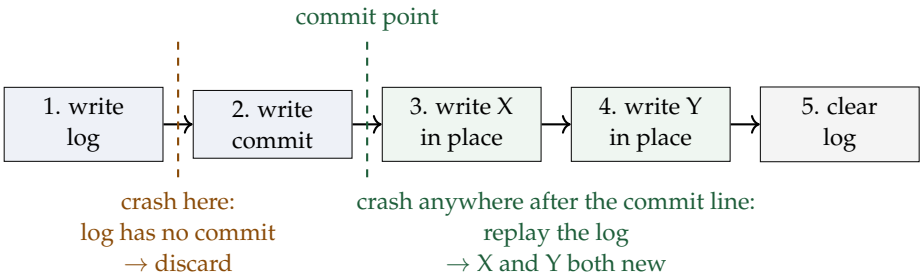


Figure 14.7: The five steps of the journaled transfer in `ch14_journal_sim.py`. Before the dashed commit line a crash means "nothing happened"; after it a crash means "replay it". There is no step at which recovery leaves $X + Y \neq 100$. Without a journal, steps 3 and 4 are the only two writes, and a crash between them is exactly the corrupt state.

The plain version has exactly one bad moment, between its two writes, and 90 is not an amount anyone ever had. The journal version writes the new values to a log (step 1), then a *commit record* (step 2), then updates X and Y in place (3, 4), then clears the log (5). Crash after step 1: the log has no commit, recovery discards it, nothing happened. Crash after step 2, 3 or 4: the commit exists, recovery *replays* the logged values into X and Y; replaying is harmless if some of it was already done, a property called idempotence. The model assumes a single block write is atomic and does not model torn writes within a block, which is the guarantee real disks give and journals rely on.

The commit record is the whole trick: a crash before it discards the transaction; a crash after it replays the write. Ordering the four steps costs one extra disk write — the journal — and buys atomicity across arbitrarily many blocks. xv6's `log.c` implements exactly this; the guided-read panel shows it distilled.

Guided read — xv6 `file.c` & `log.c` (20–30 lines):
`filewrite`: loop `begin_op(); ilock(); writei(); iunlock(); end_op();` — a transaction per write batch.
`log.c`: `log_write` appends; `commit` does `write_log + install_trans` — exactly the 4-step transaction of Figure 14.6, in a real kernel.
repo: `github.com/mit-pdos/xv6-riscv`

Figure 14.8: The same transaction in a real kernel, distilled: xv6's `log.c` is the journal figure with lock calls added.

14.6 SSDs: why random writes hurt

Flash erases in large blocks but writes in small pages — a 4 KB random write can trigger a read-modify-write of a whole erase block. Sequential patterns let the FTL coalesce; random ones force garbage collection. The shape of the penalty is the same as ch10's line-fill argument at a different layer: granularity costs when the device's minimum operation exceeds yours.

14.7 Common mistakes

- **"The write is durable" without fsync.** A `write()` returns after a RAM copy — power loss loses it silently. fsync is the durability boundary.
- **Measuring "disk speed" through the page cache.** Our own first run read 17 GB/s (Mac) — of RAM. A valid storage benchmark must outsize RAM or bypass the cache.
- **fsync-per-write by default.** 0.07 ms each is fine for 100 writes; at 4 KB writes it's a $\sim$115$\times$ cut in the throughput — batch commits.
- **Treating directories as files' contents.** The directory holds *names*; the inode holds the file — hard links are just two names for one inode.
- **"SSD has no seek time so order doesn't matter."** Random *writes* still pay erase-block amplification; only reads are truly seek-free.

14.8 Summary

Piece	Verified version
inode	name→inode→blocks; 4 TB from 100 B
buffer cache	sequential reads at 17 GB/s = RAM, measured
fsync	the durability syscall; $\sim$115$\times$ tax at 4 KB
journal	TxBegin..TxEnd..checkpoint = atomic crash safety
SSD	erase-block granularity punishes random writes

Further reading

- OSTEP ch36–46 (the FS book-within-the-book): `https://pages.cs.wisc.edu/~remzi/OSTEP/`
- xv6 ch8 + `file.c`/`log.c` source: `https://github.com/mit-pdos/xv6-riscv`
- CS:APP ch10 (system-level I/O): `https://csapp.cs.cmu.edu/`

WATCH INSTEAD

- **File Systems — MIT 6.1810 lecture** — the xv6 FS walk-through that pairs with this chapter's guided read (verified 2026-10-11). `https://youtu.be/3b4vEyb1I4o`

Notes

- The first set of throughput numbers is verbatim `ch14_fs_io.c` output on the author's Mac (APFS volume plus page cache); the Linux listings are from an ext4 container on a virtual disk, labelled as such. Your numbers will differ — the exercise is to compare shapes, not digits.
- The inode walker output is real `lstat` data on this repo's own code directory. The journal model checks crash points between whole block writes only.

Exercises

Exercise 14.1 (easy)**.** Walk a `read("/a/b/f")` down the layers: which two lookups turn the path into an inode, and the inode into bytes?

Solution. [hint] Directory lookup per component (name→ino); inode's block map (file offset→disk block).

Exercise 14.2 (easy)**.** Why does `unlink` of an open file keep working? Name the refcounting idea.

Solution. [hint] Open descriptors pin the inode; storage frees at link *and* descriptor count zero.

Exercise 14.3 (easy)**.** Sequential write is buffered at 6.7 GB/s on the Mac (2.6 GB/s on the Linux container) — where do the bytes physically go between `write` and `fsync`?

Solution. [hint] Kernel page cache, marked dirty; the device sees them only at flush or write-back.

Exercise 14.4 (medium)**.** Run the sequential/random read test on your own machine. Report both numbers and name the layer you measured; how would you *force* a real disk read?

Solution. [hint] Files bigger than RAM, or `O_DIRECT`/madvise to bypass the cache.

Exercise 14.5 (medium)**.** A crash lands after TxEnd but before checkpoint. Reconstruct the journal replay — which writes happen, and why is the state correct?

Solution. [hint] Committed transactions replay idempotently; uncommitted ones never ran.

Exercise 14.6 (medium)**.** Design the journal for a key-value store: what goes in each record, and what makes the commit record atomic?

Solution. [hint] Op+key+value+txid per record; the commit is a single sector write — atomic by hardware.

Exercise 14.7 (medium)**.** Using the Linux sweep table (flush ≈ 0.57 ms, and 2.374 ms for a 1 MiB write), estimate the write size at which the data transfer time equals the fixed flush cost, and say what the Mac's 0.07 ms flush implies about its crossover.

Solution. [hint] Transfer rate $\approx 1\,\text{MiB}/(2.374 - 0.57)\,\text{ms} \approx 554$ MiB/s, so crossover $\approx 0.57\,\text{ms} \times 554 \approx 0.31$ MiB. With the same device rate, a flush of 0.07 ms would cross over at about $0.07 \times 554 \approx 39$ KiB; the device rate on the Mac was not measured, so this is a scaling statement, not a Mac prediction.

Exercise 14.8 (medium)**.** Ext4's extents replace the block map with runs. For a 1 GB sequential file, compare extent vs inode+indirect metadata sizes.

Solution. [hint] One extent record (start,len) vs 256K block numbers — orders of magnitude.

Exercise 14.9 (hard)**.** Design the smallest inode that addresses a 1 GB file with 4 KB blocks: pick your block-pointer scheme, count the metadata bytes, and justify the minimum.

Solution. [hint] 1 GB/4 KB = 256K blocks; one extent or a 2-level indirect tree — argue either, price both.

Exercise 14.10 (hard)**.** SSD FTLs do copy-on-write internally. Argue why a journaling FS *plus* a COW device can double-write the same data, and name the classic remedy.

Solution. [hint] Journal + FTL both relocate; log-structured FS or letting the FTL handle it (ext4 without full data journaling).

Exercise 14.11 (easy)**.** What two numbers in `lstat` output tell a hard link from a copy?

Solution. [hint] Same inode number + link count >1 = one file, two names.

Exercise 14.12 (medium)**.** Compare the journal's "write everything twice" cost against the alternative of checksum-only metadata ordering. When is the second option safe?

Solution. [hint] When data ordering is already guaranteed (or data loss tolerable) — metadata journaling exists for this.

Exercise 14.13 (medium)**.** The walker shows `blocks=8` for a 1-byte file: 4 KB used. What is this called, and what does it cost a million-small-files workload?

Solution. [hint] Internal fragmentation — block granularity tax, as severe as the SSD's erase-block version.

Exercise 14.14 (hard)**.** Argue whether `fsync` should return before or after the device's internal flush, and what can still go wrong if the drive lies about having flushed (hint: power-fail protected caches).

Solution. [hint] After flush to non-volatile media; a lying drive trades durability for benchmark speed — enterprise drives add capacitors to keep the promise honest.

Exercise 14.15 (easy)**.** In the `ch14_links` output, which two fields prove that `a` and `b` are the same file, and which field shows the symlink is *not*?

Solution. [hint] Same `ino` and `nlink=2` for `a` and `b`; the symlink `s` has a different inode number and its size (1) is the length of the stored path text.

CHAPTER 15

Measuring performance correctly

Suppose you time how long you take to walk to school, once, and get 14 minutes. Is that your walking time? It depends: was the light green, did you stop to talk, was it raining? A second walk gives 17. The honest answer is a *range* with a typical value and a bad day. Computer timings behave the same way, and this book has already been surprised by them several times. This chapter is about *why* the surprises happen: performance is a distribution, not a number, and a benchmark is an experiment, not a prayer. The audit table on the next page is the thesis: three of the four folk claims we tested failed on real hardware.

WHAT YOU WILL BE ABLE TO DO

After this chapter you can write a timing harness that warms up, takes many samples and reports percentiles instead of one number. You can recognise (and demonstrate) the four classic lies: no warmup, too few samples, reporting the mean, and a compiler that deleted the code you meant to time. You can decide whether the difference between two versions of a function is real or noise, using a standard error. And you can use Amdahl's law to say, before writing any code, how fast a parallel version can possibly be.

BEFORE YOU START

A *benchmark* is a program that measures how long something takes. A *percentile* says what fraction of samples are at or below a value: the p50 (median) is the middle sample, the p99 is the value that 99% of samples do not exceed. The *mean* is the sum divided by the count, and the *standard deviation* measures spread. If these are new, Book 3 ch. 13 (statistics, estimation, and Bayesian inference) builds them carefully, and Book 1 ch. 1 (the contest machine and the cost model) shows why counting operations is only a first approximation of time. From this book you need ch. 10 (caches) and ch. 13 (the Amdahl curve and the atomic-versus-mutex result).

A short C refresher. `clock_gettime(CLOCK_MONOTONIC, &ts)` fills a `struct timespec` with seconds and nanoseconds from a clock that never jumps backwards, the right clock for timing. `qsort(arr, n, size, cmp)` sorts an array given a comparison function. `volatile` forces the compiler to really perform an access. `gcc -O2` turns on optimisation, which is what makes the dead-code incident below possible. Numbers marked *Mac* are the author's earlier laptop; *Linux* marks a 4-vCPU Intel Xeon container (2.8 GHz, gcc 13).

THINK LIKE THIS

Before you believe a number, ask three things. What was the machine doing besides my code? Could the number be *better* than the hardware allows (then the code did not run)? And if I ran it again, how different would it be? A number without its spread is a rumour.

15.1 The folklore audit

claim	measured verdict	chapter
"branches are slow"	context-dependent	ch6–7: flush = real cost
"misaligned is slower"	false on this CPU	ch2: 110ms vs 151ms
"atomics beat locks"	Mac: false at 8T; Linux: true	ch13: 56 vs 35 ms (Mac)
"disk is slow"	cache says otherwise	ch14: 17GB/s cached
"vector is faster"	ch16, measured next	upcoming

Figure 15.1: The running folklore audit. Every book benchmark so far, re-read as a claim test: three of four folk wisdoms failed on real hardware. Performance is measured, never recited.

15.2 The performance equation

metric	formula & reading
CPI	cycles/instructions — our core was 1, real chips 0.25–4
IPC	1/CPI — a modern core's width indicator
time	instrs $\times$ CPI $\times$ cycle_time — the iron law
speedup	$1/(s+p/n)$ — Amdahl, measured in ch13

Figure 15.2: The performance equation. Every chapter's measurement is one row of this table made concrete: instructions, cycles, or time — and which lever each optimisation actually moves.

15.3 Latency is a distribution

Example 15.1 (The cold-cache median lies (Mac)). `ch15_bench.c` sums 4M ints honestly — warmup, 21 samples, sorted, p50/p99 reported: 0.15/0.18 ms. The same kernel with no warmup and 3 samples reports 0.17 ms — off by 13% *against its own best case*, and that error is noise, not signal. Small samples don't measure the program; they measure the scheduler's mood.

Two cautions we found while re-running this on Linux. First, with only 21 samples the "p99" is simply the largest sample (the 99th percentile of 21 values falls on the last one), so it says

statistic	value (4M-int sum)	what it hides
min	0.15 ms	nothing — best case only
p50	0.15 ms	the tail
p99	0.18 ms	the real SLA
spread (max/min)	1.2×	scheduling noise

Figure 15.3: Latency is a distribution, not a number. The mean this benchmark reports — even warmed — says nothing about the 1-in-100 slow case that SLAs are written on. 21 samples, real run (Mac). With only 21 samples the "p99" is the maximum; the Linux example in the text uses 1000.

little about the tail; the next examples use 1000 samples. Second, the Mac figure of 0.15 ms for 16 MiB implies reading about 110 GB/s, far above what main memory normally delivers, which suggests the Mac compiler may have shortened that kernel. We cannot verify this on the Mac, so treat the 0.15 ms as unconfirmed.

Example 15.2 (The same harness on the Linux container)**.**

```
n=4M ints sum: p50=3.24ms p99=4.10ms min=2.70ms max=4.10ms spread=2x
same, 3 cold samples: p50=3.59ms (median lies by 33%)
n=4M ints sum: p50=2.71ms p99=3.53ms min=2.49ms max=3.53ms spread=1x
same, 3 cold samples: p50=2.48ms (median lies by -1%)
```

Listing 15.1: Real output of `ch15_bench`, two consecutive runs (Linux container).

The kernel takes 2.5–3.2 ms here, about 6 GB/s, which is a believable memory rate. The "3 cold samples" line differs from the best sample by +33% in the first run and −1% in the second. (The program's label "cold" is a slight misnomer: the array has already been touched by the 21 warm samples, so what the line really shows is how much *three samples* can mislead.) The two runs disagree with each other by far more than a 5% optimisation would claim to gain.

Example 15.3 (Dead code: a benchmark that measures nothing)**.** *The naive version.* Call `sum(a, n)` in a timing loop and ignore the result. An optimising compiler sees a pure function whose result is unused and deletes the call. `ch15_dce.c` builds both ways from one source file:

```
without sink: best of 11 = 0.0000 ms (671089.6 GB/s implied)
with sink : best of 11 = 2.6925 ms (6.2 GB/s implied)
```

Listing 15.2: Real output (Linux container, `gcc -O2`); the second build adds `-DSINK`, which stores each result into a `volatile` variable.

"Implied GB/s" is bytes read divided by time. 671 089 GB/s is physically impossible, which is the clue: the loop never ran. With the volatile sink the same call takes 2.69 ms, 6.2 GB/s. The habit to keep: after every optimisation that makes things "much faster", compute the implied rate and compare it with what the hardware can do.

Example 15.4 (Latency is a distribution: 1000 samples of one small kernel)**.** `ch15_cdf.c` times a sum over 1 MiB (it fits in cache, so memory is not the story) 1000 times and prints the first samples, the percentiles, and a histogram.

```
first 5 samples (us): 111.4 81.9 50.5 50.1 98.6
```

```
p50=50.7 p90=97.9 p99=194.4 p99.9=1062.8 max=1062.8 (us)
mean=67.7 us (mean/p50 = 1.33)
histogram (count of 1000 samples):
  < p50 : 500
    50.7.. 177.2 us : 488
   177.2.. 303.7 us : 7
   303.7.. 430.2 us : 1
   430.2.. 556.7 us : 0
   556.7.. 683.2 us : 3
   683.2.. 809.7 us : 0
   809.7.. 936.2 us : 0
   936.2..1062.8 us : 0
  >= p99.9 : 1
```

Listing 15.3: Real output of `ch15_cdf` (Linux container).

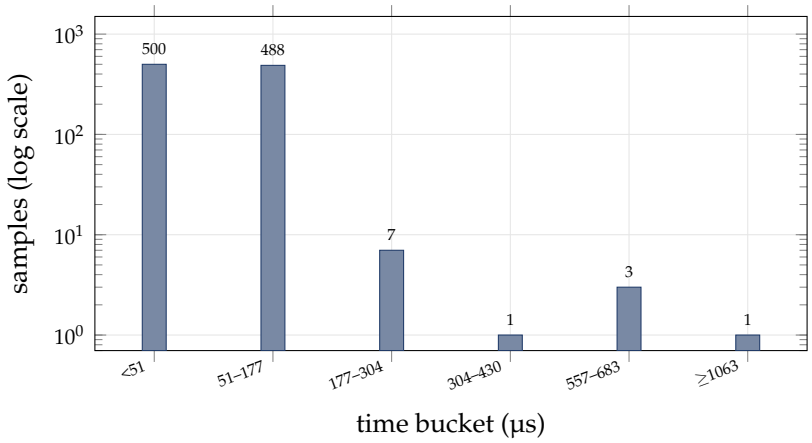


Figure 15.4: The 1000 timings of one 1 MiB sum from `ch15_cdf` (Linux container) as a histogram with a logarithmic count axis; empty buckets (430–557, 683–1063) are omitted. 98.8% of the samples sit at or below 177 µs, but twelve samples lie out to a millisecond. The median is 50.7 µs; the maximum is 21 times larger.

Read it from the top. The *first* sample took 111 µs, more than twice the median 50.7: the cold-start cost of the first touch. The median is 50.7 µs, but p99 is 194 (3.8× the median) and the single worst sample is 1063 µs, 21× the median. The mean (67.7) is 33% above the median and describes no actual run: a few outliers drag it up. If a service handled these requests, the users who matter for the service level would be the ones in the 12 samples above 177 µs, and neither the median nor the mean mentions them. (The program's table is built by sorting the values, so p99.9 and the max coincide at 1062.8 here because index 999 of 1000 *is* the maximum.)

Example 15.5 (Is B really faster than A? Mean ± standard error)**.** *The naive reading:* run A and B once each, see 3.10 versus 3.07, announce a 1% win. A better rule: take many samples, compute the mean m and standard deviation σ of each, and estimate how precisely the mean is known by the *standard error* $\mathrm{se} = \sigma/\sqrt{n}$. The difference of two means has $\mathrm{se}_\Delta = \sqrt{\mathrm{se}_A^2 + \mathrm{se}_B^2}$; a difference smaller than about $2\,\mathrm{se}_\Delta$ is not distinguishable from noise. `ch15_compare.c` times a plain sum (A) and a sum with four independent accumulators (B), 40 samples each, interleaved ABAB so drift hits both:

```
A: mean 3.100 ms sd 0.457 se 0.072
B: mean 3.071 ms sd 0.383 se 0.060
```

```
3 B - A = -0.029 ms (-0.9%), se of difference 0.094, |diff|/se = 0.3
4 verdict: difference is within 2 se: not distinguishable from noise
```

Listing 15.4: Real output of `ch15_compare` (Linux container, first of two runs).

Check the arithmetic: $0.457/\sqrt{40} = 0.072$, $0.383/\sqrt{40} = 0.061$, $\sqrt{0.072^2 + 0.060^2} = 0.094$, and $0.029/0.094 = 0.31$. The second run gave A 2.710 ms, B 2.702 ms, difference -0.3%, $|\text{diff}|/\text{se} = 0.2$: again noise. The honest conclusion is "no detectable difference", which is also what you would expect: gcc already vectorises the plain loop, so unrolling by hand adds nothing.

Example 15.6 (Amdahl before you code)**.** A program spends 10% of its time in a serial part. How fast can 8 cores make it? *Naive guess:* $8\times$. Amdahl's law: speedup $= 1/(s + (1-s)/p)$ with s the serial fraction and p cores. Here $1/(0.1 + 0.9/8) = 1/0.2125 = 4.71$. Even infinite cores give $1/s = 10$. `ch15_amdahl.py` tabulates it:

```
1 serial 5%: p=2 1.90x p=4 3.48x p=8 5.93x p=64 15.42x ceiling 20.0x
2 serial 10%: p=2 1.82x p=4 3.08x p=8 4.71x p=64 8.77x ceiling 10.0x
3 serial 20%: p=2 1.67x p=4 2.50x p=8 3.33x p=64 4.71x ceiling 5.0x
4 6.0x on 8 cores needs serial fraction <= 0.0476 (4.8%)
```

Listing 15.5: Real output of `python3 ch15_amdahl.py`.

The last line inverts the formula: to reach $6\times$ on 8 cores, $s \le (1/6 - 1/8)/(1 - 1/8) = 0.0476$, so the serial part must be under 4.8%. Use this *before* writing parallel code: if the serial fraction is 20%, no number of cores gets you past $5\times$.

15.4 The four lies

benchmark sin	the lie it tells
no warmup	measures cold caches/TLB, not the kernel
3 samples	median swings $\pm 13\%$ run to run (real)
report the mean	hides the tail the user feels
dead code	the compiler deletes the kernel (ch10's sink)

Figure 15.5: The four ways benchmarks lie, each one demonstrated live in this book: cold-start bias, small-sample noise, mean-vs-tail, and dead-code elimination — the last already caught once in ch10.

15.5 The measurement loop

15.6 The measured latency ladder

15.7 Common mistakes

- **Mean for p99 work.** The user on the tail feels the p99; the mean hides it — report the distribution.

phase	question
measure	where does time actually go?
hypothesise	which lever should move it?
change one thing	the experiment
re-measure	did the number move?

Figure 15.6: The measurement loop. It is the scientific method with numbers — and this chapter exists because skipping "re-measure" is how every perf folklore item on the facing page was born.

quantity	this book's measured range	span
L1 hit	∼1 ns	1×
DRAM miss	∼120 ns	120×
ctx switch (Mac)	∼3 µs	3,000×
page fault (Mac, ch11)	∼0.74 µs	740×

Figure 15.7: The book's own measured latency ladder — every number measured live in an earlier chapter (on the Mac), not cited. The Linux container's context switch ranged from 2 µs (pinned) to 20 µs (unpinned): the ladder's rungs move with the machine. A performance question answered from this table is answered for free.

- **One run, no warmup.** The first run pays cache/TLB cold costs — a different experiment than the steady state.
- **Dead-code elimination.** ch10's 0.0 ms transpose is the canonical incident — the compiler deleted the kernel; a volatile sink is mandatory.
- **Folklore as fact.** "Misaligned is slow" failed on the Mac; "atomics beat locks" failed at 8 threads on the Mac and held on the Linux container; measure on *your* machine.
- **Calling a 1% difference a win.** With a standard error of 0.09 ms on a 3.1 ms kernel, a 0.03 ms difference is noise.
- **Believing an impossible number.** 671 089 GB/s means the code did not run.
- **Optimising the wrong term.** Amdahl's arithmetic from ch13 — 10% serial caps you at 10× regardless of how clever the parallel part gets.

15.8 Summary

Piece	**Verified version**
latency	a distribution — p50/p99 not mean
benchmark	warmup + enough samples + sink for the result
audit	3 of 4 folk claims failed on this machine
levers	instrs / CPI / cycle-time — know which you're moving

Further reading

- Brendan Gregg, *Systems Performance* (methodology): https://www.brendangregg.com/systems-performance-2nd-edition-book.html
- OSTEP, the measurement appendix: pages.cs.wisc.edu/~remzi/OSTEP
- CS:APP ch5 (program optimisation): https://csapp.cs.cmu.edu/

WATCH INSTEAD

- **How NOT to Measure Latency — Gil Tene, Strange Loop** — the tail-latency talk this chapter condenses (verified 2026-10-11). https://youtu.be/lJ8ydIuPFeU

Notes

- The 13% cold-sample error is the Mac's real figure, measured the same minute as the honest run; on Linux the same line gave +33% and −1% in two runs. The Mac 0.15 ms kernel time is unconfirmed (it implies ≈110 GB/s) and may reflect compiler optimisation.
- The folklore-audit row for vectorisation is intentionally left open — ch16 closes it.
- In the performance ladder the page-fault row was corrected to 0.74 µs (ch11's measured Mac value); an earlier draft said 0.7 ms.

Exercises

Exercise 15.1 (easy)**.** Why does the *min* of a benchmark run tell you almost nothing about steady-state behaviour?

Solution. [hint] The min is a survivorship bound — it bounds noise away, not the program.

Exercise 15.2 (easy)**.** Which row of the performance equation does a better algorithm move, and which does a faster CPU move?

Solution. [hint] Instructions vs cycle-time; CPI is the machine/code interplay.

Exercise 15.3 (easy)**.** What does the volatile sink in our harness prevent?

Solution. [hint] Dead-code elimination of the kernel under test.

Exercise 15.4 (easy)**.** Amdahl: at 95% parallel, what's the ceiling at any core count?

Solution. [hint] $1/0.05 = 20\times$ — serial fraction sets the ceiling.

Exercise 15.5 (medium)**.** Design the warmup for a syscall-latency benchmark. What state must it establish, and what does it cost to skip?

Solution. [hint] Warm TLB/cache/branch-predictor state; skipping measures the cold kernel, not the syscall.

Exercise 15.6 (medium)**.** Your p50 is 5 ms but the p99 is 200 ms. List two mechanisms that produce that tail and one fix for each.

Solution. [hint] GC pauses / lock convoys / page faults; each has a targeted fix (pool, partition, prefault).

Exercise 15.7 (medium)**.** A colleague reports "3× speedup" from 2 runs of a noisy benchmark. What do you ask for, and what does a real delta look like?

Solution. [hint] n≥20, p50+p99, and a delta bigger than the noise floor — typically >10% on laptops.

Exercise 15.8 (medium)**.** Write the hypothesis test: kernel A vs B differ by 4% with spread 15%. How many samples make the difference believable?

Solution. [hint] You need $\sigma/\sqrt{n}$ below the delta — ∼15 samples before the 4% clears the noise.

Exercise 15.9 (hard)**.** The folklore audit shows 3 of 4 claims failing. Write the general rule that predicts when "common wisdom" will mislead on a given machine, and test it on a fifth claim of your choice.

Solution. [hint] Wisdom assumes conditions (miss rate, contention, alignment penalty) — it fails exactly where those assumptions are absent; verify one on your box.

Exercise 15.10 (hard)**.** Extend the harness to output the full CDF, not just p50/p99. What does the shape between p90 and p99 tell you that the two percentiles alone don't?

Solution. [hint] Multi-modality — the p95 can be fine while p99 is catastrophic, invisible at two points.

Exercise 15.11 (easy)**.** What is speedup_{max} if 20% of your program is serial?

Solution. [hint] $1/0.2 = 5\times$ — the serial floor, regardless of cores.

Exercise 15.12 (medium)**.** Explain why "cold" numbers and "warmed" numbers are both honest answers — to *different* questions.

Solution. [hint] Cold measures first-touch behaviour; warm measures steady state — report which question you asked.

Exercise 15.13 (medium)**.** Our p99=0.18 ms vs p50=0.15 (Mac, 21 samples): a 1.2× tail. On a server with 10 000 req/s, how many requests per second hit the tail?

Solution. [hint] 1% of 10k = 100 req/s — the tail is a real workload, not an edge case.

Exercise 15.14 (hard)**.** A benchmark's result depends on CPU frequency scaling (turbo). Design the control that separates "the code got faster" from "the chip sped up".

Solution. [hint] Report cycles, not seconds (perf counters), or pin frequency — the turbo is a confound, not a signal.

Exercise 15.15 (medium)**.** Which is the stronger claim about our 17 GB/s read: "the disk is fast" or "the cache works"? What measurement distinguishes them?

Solution. [hint] "The cache works" — a file bigger than RAM or O_DIRECT separates the two.

Exercise 15.16 (easy)**.** In the `ch15_cdf` listing, how many of the 1000 samples are above 177 µs, and what percentage is that?

Solution. [hint] $7 + 1 + 0 + 3 + 0 + 0 + 0 + 1 = 12$ samples (the bucket rows 177–304, 304–430, 557–683 and the last one), i.e. 1.2%.

Exercise 15.17 (medium)**.** The dead-code listing reports 6.2 GB/s with the sink. Check it: the array is 2^{22} ints of 4 bytes and the best time is 2.6925 ms. Is 6.2 consistent?

Solution. [hint] $2^{22} \times 4 = 16\,777\,216$ bytes; $/ 0.0026925\,\text{s} = 6.23 \times 10^9$ bytes per second, i.e. 6.2 GB/s (decimal gigabytes). Yes.

Exercise 15.18 (medium)**.** Using the compare listing's sd values (0.457 and 0.383) and $n = 40$, how many samples per version would you need for the 0.029 ms difference to reach $|\text{diff}|/\text{se} = 2$, assuming the standard deviations stay the same?

Solution. [hint] $\text{se}_\Delta = \sqrt{(0.457^2 + 0.383^2)/n} = 0.596/\sqrt{n}$. We need $0.029/\text{se}_\Delta = 2$, i.e. $\text{se}_\Delta = 0.0145$, so $\sqrt{n} = 0.596/0.0145 = 41.1$ and $n \approx 1690$ samples per version. Ten times more samples would not be enough; a 1% effect is expensive to confirm on a noisy machine.

CHAPTER 16

Vectorisation and data parallelism

A computer can get more done at once in three different ways. It can overlap the steps of different instructions (the pipeline of ch. 6 and 7), it can run several threads on several cores (ch. 13), or it can make a single instruction handle several numbers together. This chapter is about the third way, called *vectorisation*. It is the cheapest of the three, because you change nothing about the algorithm and each instruction simply does four or eight times as much work. The catch, which the chapter's second measurement shows, is that your compiler may already have done it for you, and that the whole trick stops helping once the data is too big for the cache.

AFTER THIS CHAPTER YOU CAN

Take a plain C loop, say whether the compiler already turned it into vector instructions (by reading the disassembly, not by guessing), write the SSE2 or AVX2 version yourself including the ragged end that beginners get wrong, time it honestly against a scalar baseline, and predict from the array size whether the speedup will be 6× or 1.2×.

BEFORE YOU START

Recap, from a picture. Imagine a shop with one cashier who scans one item at a time. Now give the cashier a wide scanner that reads four barcodes in a single pass. The cashier is no faster, but four items go through per pass. That is *SIMD* (single instruction, multiple data): one instruction works on several values at once, and each value sits in a *lane* of a wide *vector register*. It exists because the expensive part of running an instruction (fetching it, decoding it, scheduling it) is the same whether it handles one number or four, so handling four per instruction is nearly free. You already know the pieces: a register file holds words (ch. 3 here, and Book 5 ch. 15 built one for the 8-bit CPU); an instruction does one operation on them (ch. 5); and the memory hierarchy (ch. 9) decides how fast data can be brought to the registers. Book 1 ch. 1 introduced the idea that a program's cost is the number of elementary steps it performs; SIMD lowers that count by a constant factor, and ch. 15 taught how to measure the result without fooling yourself.

C in ten lines. An array `float x[n]` is n consecutive 4-byte values; `x+i` is the address of element `i`, and `x[i]` reads it. A function receives arrays as pointers (`const float *x`); `const` promises not to write through it. `for (long i = 0; i < n; i++)` is the counting loop. `y[i] += a*x[i]` means `y[i] = y[i] + a*x[i]`. Reading or writing outside $0 \ldots n-1$ is *undefined behaviour*: the program may crash, or silently

corrupt a neighbour. An *intrinsic* such as `_mm_add_ps` is a C function that the compiler replaces by exactly one machine instruction. Compile with `gcc -O3 file.c`.

Which numbers are which. The sweeps in the first half of the chapter (NEON, 4 lanes) were measured on the author's Apple-Silicon Mac and cannot be reproduced on an x86 machine, because NEON does not exist there. They are kept and labelled *Mac/NEON*. The sections headed *x86_64* were measured here, on a Linux x86_64 container (4-core Intel Xeon at 2.8 GHz, 32 KiB L1d and 1 MiB L2 per core, 33 MiB shared L3, with SSE2, AVX2 and AVX-512F available) and are a separate experiment, not a replication.

THINK LIKE THIS

Before you optimise, ask three questions in order. (1) *Is it already done?* The compiler may have vectorised the loop for you; read the machine code. (2) *What is the limit?* If the loop is waiting for memory, wider arithmetic cannot help. (3) *Did I measure it fairly?* A baseline that is secretly vectorised, or a benchmark that includes page faults, gives a number that means nothing. This chapter walks through all three, using a loop small enough to see completely.

16.1 Three ways to do more at once

Picture a kitchen that has to chop 1000 carrots. The head cook can (a) start chopping the next carrot before finishing the last (overlap, which is what a pipeline does), (b) hire more cooks (threads on more cores), or (c) buy a slicer that cuts four carrots in one push. Option (c) is *data-level parallelism* (DLP): the *same* operation applied to many pieces of *data* at the same moment. It only works when the pieces do not depend on each other, which is true for carrots and, very often, for the elements of an array.

parallelism	unit	book chapter
instruction-level (ILP)	instrs/cycle in one thread	ch6–7
data-level (DLP)	lanes/instr — this chapter	ch16
thread-level (TLP)	cores	ch13

Figure 16.1: The three parallelisms the book has now covered. DLP is free ILP: the pipeline issues once and 4 lanes retire — the widest speedup-per-line-changed in systems work.

CONCEPT: THE SAME JOB, MANY ITEMS

Whenever you see a loop whose body does *the same arithmetic on each element, and element i never needs the result for element $i-1$*, you are looking at data-level parallelism. Learn to spot that shape first; the instruction names come second.

16.2 From a tiny example to lanes

Start with the smallest possible case. Add two lists of four numbers, `[1, 2, 3, 4]` and `[10, 20, 30, 40]`. A scalar CPU does it with four loads of the first list, four loads of the second, four adds and four stores: 16 instructions. A vector CPU has registers wide enough to hold all four numbers at once. It does one wide load for each list, one wide add, one wide store: 4 instructions, and the answer `[11, 22, 33, 44]` is the same. The four slots of the register are called *lanes*, like the lanes of a motorway: cars travel side by side and never swap lanes. That last point is why SIMD hardware is cheap to build: the four additions are independent, so the chip simply contains four adders next to each other and does not need any wiring between them.

How many lanes you get depends on the register width and on the size of the numbers. A 128-bit register holds four 32-bit floats or sixteen 8-bit bytes; a 256-bit register holds eight floats or thirty-two bytes; a 512-bit register holds sixteen floats. SSE2 on x86 and NEON on Arm are 128-bit, AVX2 is 256-bit, and AVX-512 is 512-bit.

	one instruction, four lanes		
scalar	y0+ax0	y1+ax1	y2+ax2 …
NEON	`vmlaq`: lanes 0–3 in one op		

Figure 16.2: Data-level parallelism: one `vmlaq` updates 4 floats per issue where the scalar loop updates one. The ALU count is the same; the instruction stream is 4× shorter.

16.3 SAXPY, measured

To measure vectorisation we need a loop that is simple, important and easy to get right. The classic choice is *SAXPY*: "Single-precision A times X Plus Y", that is $y_i \leftarrow a\,x_i + y_i$ for every element of two arrays of 32-bit floats. It is the inner loop of a great deal of numerical software. Two details matter for everything that follows. First, each element needs only 2 arithmetic operations (one multiply, one add). Second, it must move 12 bytes: read x_i, read y_i, write y_i. We will use those two numbers to explain the results. The *speedup* below is simply (time of the scalar version) divided by (time of the vector version); a speedup of 4 means four times faster.

n	scalar (ms)	NEON (ms)	speedup
2^{16}	0.025	0.007	3.5×
2^{19}	0.189	0.064	3.0×
2^{22}	1.384	0.331	4.2×
2^{25}	8.520	2.885	3.0×

Figure 16.3: Real `ch16_saxpy` sweep, `-fno-vectorize` scalar vs NEON intrinsics: 3–4× at large n — close to the 4-lane ideal, capped by memory bandwidth at the biggest sizes.

Example 16.1 (Do four lanes give four times the speed? Nearly, on the Mac (Mac/NEON))**.** The scalar version here was compiled with `-fno-vectorize` (a switch that forbids the compiler from using vector instructions on its own), so it really is one number at a time. The other version uses NEON *intrinsics*, which are C function names that turn into exactly one vector instruction each. Across array lengths $n = 2^{10} \ldots 2^{25}$ the speedup stays near the 4-lane ideal: 3.5× at 2^{16} and 4.2× at 2^{22}. At 2^{25} it sags to 3.0×. The reason is that an array of 2^{25} floats is far larger than the cache, so the CPU spends its time waiting for bytes from memory, not for arithmetic: ch. 9's roofline arriving on schedule. (A figure above 4 is probably timing noise or the compiler's choices in the scalar loop; treat these as 3–4×, not as exact ratios.)

16.4 SAXPY on x86_64: SSE2 and AVX2

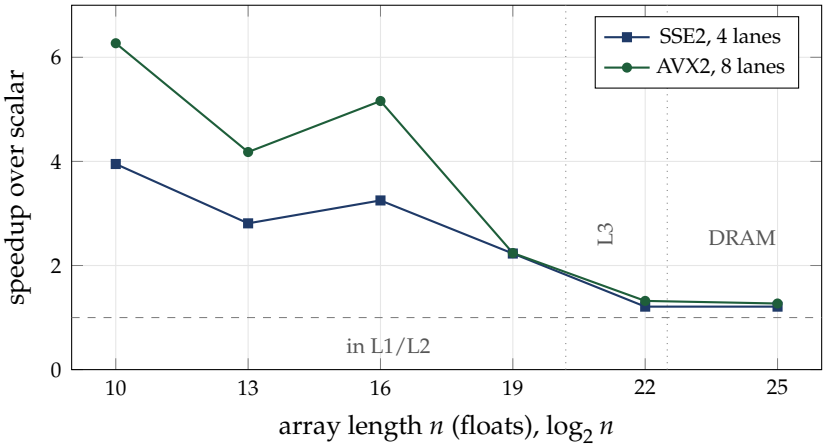


Figure 16.4: Linux x86_64 container (4-core Xeon, 2.8 GHz; `ch16_saxpy_x86` built with `-O3 -fno-tree-vectorize`). The 8-lane kernel gives 4–6× while the arrays sit in L1/L2, and both curves fall to about 1.2–1.3× once $8n$ bytes exceeds the 33 MiB L3 and every pass streams from DRAM. One run; repeat runs move the small-n points by ±30%. The dashed line is "no gain".

x86 has the same idea under other names. SSE2 gives 128-bit registers (4 floats per instruction, the counterpart of NEON), and AVX2 gives 256-bit registers (8 floats). `ch16_saxpy_x86.c` writes the SAXPY loop once in scalar C and twice with intrinsics. The scalar version is built with `-fno-tree-vectorize`, so the baseline is guaranteed to be scalar (confirmed below by reading its machine code).

```
__attribute__((noinline))
static void sax_scalar(const float*restrict x,float*restrict y,float a
    ,long n){
  for(long i=0;i<n;i++) y[i]+=a*x[i];}

__attribute__((noinline))
static void sax_sse2(const float*x,float*y,float a,long n){
  __m128 va=_mm_set1_ps(a); long i=0;
  for(;i+4<=n;i+=4){
    __m128 vx=_mm_loadu_ps(x+i), vy=_mm_loadu_ps(y+i);
    _mm_storeu_ps(y+i,_mm_add_ps(vy,_mm_mul_ps(va,vx)));}
  for(;i<n;i++) y[i]+=a*x[i];} /* scalar tail */

```

```
__attribute__((noinline,target("avx2")))
static void sax_avx2(const float*x,float*y,float a,long n){
  __m256 va=_mm256_set1_ps(a); long i=0;
  for(;i+8<=n;i+=8){
    __m256 vx=_mm256_loadu_ps(x+i), vy=_mm256_loadu_ps(y+i);
    _mm256_storeu_ps(y+i,_mm256_add_ps(vy,_mm256_mul_ps(va,vx)));}
  for(;i<n;i++) y[i]+=a*x[i];}
```

Listing 16.1: The three SAXPY kernels of `ch16_saxpy_x86.c`. The `i+4<=n` test and the trailing scalar loop handle lengths that are not a multiple of the lane count.

Example 16.2 (Verbatim sweep, Linux x86_64 container)**.** Built with gcc at level `-O3`, the vectoriser switched off (`-fno-tree-vectorize`) and AVX2 enabled (`-mavx2`). Each cell is the best of five batches, in milliseconds per call:

```
correctness (n=1003, ragged tail): ok
n       scalar_ms    sse2_ms    avx2_ms   sse2_x   avx2_x
2^10      0.00065    0.00016    0.00010    3.95x    6.27x
2^13      0.00505    0.00180    0.00121    2.81x    4.18x
2^16      0.04105    0.01262    0.00796    3.25x    5.16x
2^19      0.35083    0.15713    0.15627    2.23x    2.24x
2^22      4.48498    3.71751    3.40031    1.21x    1.32x
2^25     35.52053   29.43226   27.96404    1.21x    1.27x
```

Read it against the cache sizes. The two arrays together occupy $8n$ bytes: 8 KiB at 2^{10} (L1), 512 KiB at 2^{16} (L2), 4 MiB at 2^{19} (L3), 32 MiB at 2^{22} (the edge of the 33 MiB L3) and 256 MiB at 2^{25} (DRAM). Inside L1/L2 the speedups (3–4× for 4 lanes, 4–6× for 8) are close to the lane counts. At 2^{25} the loop moves $3 \cdot 128$ MiB (read `x`, read and write `y`) = 402.7 MB in 27.96 ms with AVX2, i.e. ≈ 14.4 GB/s, and in 35.52 ms scalar, ≈ 11.3 GB/s. Eight times the lanes bought 1.27× because the loop is limited by how fast bytes arrive, not by how many floats one instruction adds. That is ch. 9's roofline again. Repeat runs on this shared container moved the small-n speedups by roughly ±30%; the shape (large in cache, collapsing at DRAM) did not move.

16.5 The ragged end: what a vector loop does when n is not a multiple of 4

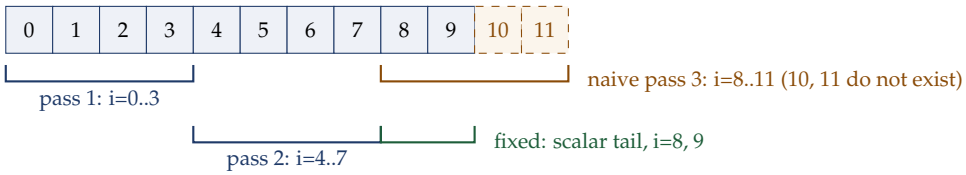


Figure 16.5: A 4-lane loop over $n = 10$ floats. Two full vector passes cover elements 0–7. The naive third pass reads and writes elements 10 and 11, which lie past the allocation (dashed); the fix stops the vector loop when fewer than 4 elements remain and finishes with a scalar tail.

```
static void sax_naive(const float*x,float*y,float a,long n){
  __m128 va=_mm_set1_ps(a);
  for(long i=0;i<n;i+=4) /* BUG: assumes n % 4 == 0 */
```

```
4     _mm_storeu_ps(y+i,_mm_add_ps(_mm_loadu_ps(y+i),_mm_mul_ps(va,
          _mm_loadu_ps(x+i))));}
5  static void sax_fixed(const float*x,float*y,float a,long n){
6    __m128 va=_mm_set1_ps(a); long i=0;
7    for(;i+4<=n;i+=4)
8      _mm_storeu_ps(y+i,_mm_add_ps(_mm_loadu_ps(y+i),_mm_mul_ps(va,
          _mm_loadu_ps(x+i))));
9    for(;i<n;i++) y[i]+=a*x[i];}
```

Listing 16.2: Naive and fixed 4-lane SAXPY from `ch16_tail.c`.

Example 16.3 (Naive 4-lane loop over 10 floats, then the fix)**.** Take $x_i = i$, $y_i = 1$ for $i = 0 \ldots 9$ and $a = 2$; the answer is $y_i = 2i + 1$, i.e. $1, 3, 5, \ldots, 19$. The first attempt that comes to mind is to take the scalar loop and change the step from 1 to 4: `for (i=0; i<n; i+=4)` with one 4-lane load, multiply-add and store inside. For $n = 10$ the passes start at $i = 0, 4, 8$. The third pass touches elements $8, 9, 10, 11$, and elements 10 and 11 do not exist. Under AddressSanitizer (`gcc -g -fsanitize=address`) the naive build stops at once (abridged: addresses and the call chain shortened):

```
ERROR: AddressSanitizer: heap-buffer-overflow on address 0x504000000030
READ of size 16 at 0x504000000030 thread T0
    #1 0x56524400a3c6 in sax_naive .../ch16_tail.c:12
0x504000000038 is located 0 bytes after 40-byte region
```

Without the sanitizer nothing necessarily stops the program: the third pass reads, and then writes, 8 bytes that belong to whatever the allocator placed after the buffer. The fix is to run the vector loop only while a full group remains (`i+4<=n`: passes at $i = 0, 4$) and finish elements 8 and 9 in a scalar *tail* loop. The fixed build prints, verbatim,

```
1 3 5 7 9 11 13 15 17 19
```

matching the hand calculation. For general n the vector loop runs $\lfloor n/4 \rfloor$ times and the tail n mod 4 times; for the $n = 1003$ used in the program's correctness check that is 250 vector passes and 3 scalar steps.

16.6 Why a compiler can sometimes not vectorise

Before we ask whether hand-written intrinsics beat the compiler, we should see what the compiler needs in order to vectorise at all. It must be *certain* that doing four iterations together gives the same answer as doing them one after another. Two things can spoil that certainty.

A dependence between iterations. Consider `a[i] = a[i-1] + 1`. Start with `a[0]=0`. Iteration 1 needs the new `a[0]`; iteration 2 needs the new `a[1]`. Done one by one the array becomes $0, 1, 2, 3, 4, \ldots$. If a vector unit tried to do iterations 1 to 4 together, all four would read the *old* neighbours and the result would be wrong. The loop is therefore not data-parallel as written, and a correct compiler leaves it scalar. Compare `y[i] = a*x[i] + y[i]`: each iteration touches only its own i, so any grouping is safe.

Aliasing. In C, two pointer arguments may secretly point into the same array. In `f(float *x, float *y, ...)` the compiler cannot know that `y` is not, say, `x+1`; if it were, writing `y[i]` would change a later `x[i+1]`, which is the same dependence problem in disguise. The programmer promises otherwise with the `restrict` keyword ("this pointer is the only way to reach this memory"), and good compilers also insert a quick run-time check and keep a scalar fallback. Our `sax_scalar` uses `restrict` for exactly this reason.

THINK LIKE THIS

A compiler is a very careful lawyer: it will only apply a transformation if it can prove the answer cannot change. When it refuses to vectorise, do not fight it. Ask what it cannot prove (a dependence? aliasing? an early `break`?) and either change the loop so that it *can* prove it, or tell it the fact (`restrict`).

16.7 The folklore item that fails

Folklore says "hand-written intrinsics always beat the compiler". Like every rule of thumb in this book, we test it rather than trust it. The compiler's own vectorising is called *auto-vectorisation*.

variant	ms at $n = 2^{25}$	verdict
scalar `-fno-vectorize`	8.520	real scalar
scalar `-O3` (auto-vec)	2.891	compiler SIMD'd it
NEON intrinsics	2.885	same

Figure 16.6: The chapter's folklore entry dies here: "intrinsics beat the compiler" measured 1.00× at $n = 2^{25}$ — clang's auto-vectoriser emits equivalent NEON. Write intrinsics for control, not for speed the compiler already finds.

Example 16.4 (Do hand-written intrinsics beat the compiler? Not on the Mac (Mac/NEON))**.** At optimisation level `-O3` ("optimise hard") clang auto-vectorises the plain scalar SAXPY on its own: 2.891 ms against the hand-written intrinsics' 2.885 ms, a ratio of 1.00×. The claim "intrinsics beat the compiler" fails here. What the claim gets right is narrower: write intrinsics when you need vectorisation guaranteed on a compiler that might not do it, or for operations the compiler cannot express from scalar code. Do not write them for raw speed the compiler already finds.

Example 16.5 (Did the compiler vectorise it? Read the machine code, on x86_64)**.** A habit worth building: never guess whether a loop is vectorised, look. `objdump -d` prints the machine code of a compiled file. Rebuild the same file with the vectoriser allowed (`gcc -O3 -mavx2`). Before looking at times, look at what the compiler produced for the plain scalar C function `sax_scalar`, using `objdump -d`:

```
30: vmulps (%rdi,%rax,1),%ymm2,%ymm1     ; 8 floats: a * x[i..i+7]
35: vaddps (%rcx,%rax,1),%ymm1,%ymm1     ; + y[i..i+7]
3a: vmovups %ymm1,(%rcx,%rax,1)          ; store 8 floats
3f: add    $0x20,%rax                    ; 32 bytes = 8 floats
```

That is an AVX2 loop nobody wrote: the compiler vectorised the scalar source and added its own scalar tail. With `-fno-tree-vectorize` the loop body is instead `vmulss`, `vaddss` and `vmovss` on one float, which is how we know the baseline above is honest. The verbatim sweep with the vectoriser on:

```
n       scalar_ms    sse2_ms    avx2_ms   sse2_x   avx2_x
2^10      0.00010    0.00026    0.00018    0.39x    0.59x
2^13      0.00152    0.00287    0.00145    0.53x    1.05x
2^16      0.00852    0.01582    0.00806    0.54x    1.06x
2^19      0.19261    0.22102    0.17673    0.87x    1.09x
2^22      3.76549    3.81055    3.31809    0.99x    1.13x
2^25     29.92364   30.12058   31.23453    0.99x    0.96x
```

The "scalar" column is now the compiler's 8-lane code, so the 8-lane intrinsics tie it (0.96–1.13×), and our hand-written 4-lane SSE2 version is up to 1.8× *slower* than the compiler's (the 0.54× row). The Mac result reproduces in spirit: when the loop is simple, intrinsics do not beat the compiler, and an intrinsic version written for a narrower register can lose.

Example 16.6 (When intrinsics do win: saturating add)**.** *Saturating* arithmetic stops at the largest value instead of wrapping around. Brightening an 8-bit image is the everyday case: a pixel of 200 plus 100 should become 255 (white), not 44. The scalar C adds two bytes and clamps at 255. In C: `s = a[i]+b[i]; o[i] = s>255 ? 255 : s;`. SSE2 has one instruction for it, `_mm_adds_epu8`, on 16 bytes at once. `ch16_satadd.c` checks that both give identical output on 65 536 random bytes and times them (`gcc -O3`, vectoriser on):

```
outputs identical: yes
scalar clamp: 0.0133 ms   _mm_adds_epu8: 0.0042 ms   speedup 3.1x
    (n=65536 bytes)
```

Here the compiler does find some vector code (with `-fno-tree-vectorize` the gap was 11.5× in one earlier run, and clang -O3 gave 5.1× in one run), but the intrinsic still wins by about 3× with gcc. The rule of thumb that survives: intrinsics pay when a single machine instruction does what takes the compiler a compare-and-select sequence.

16.8 SoA vs AoS: the layout is the vector

So far the data was one plain array. Real programs store records. Take particles with four numbers each: position x, y, z and mass m. There are two natural ways to lay them out in memory. In an *array of structures* (AoS) each particle's four numbers sit together: $x_0\, y_0\, z_0\, m_0\, x_1\, y_1\, z_1\, m_1 \ldots$. In a *structure of arrays* (SoA) each field has its own array: $x_0\, x_1\, x_2 \ldots$, then $y_0\, y_1 \ldots$ and so on. AoS is what most people write first, because a particle feels like one thing.

Now suppose a loop only needs x and m. With SoA the CPU streams two tidy arrays, and a vector load picks up four consecutive x values in one go. With AoS the useful numbers are scattered one in every four slots; half of every 64-byte cache line brought from memory is wasted (two of the four fields are unused), and a vector load would pick up $x_0\, y_0\, z_0\, m_0$, of which only two lanes matter. On the Mac this was measured as 2.0 ms against 4.7 ms on 16M particles, a factor of 2.35 for a change of layout alone (Mac). The vector unit loves SoA because the memory system already does.

layout	x+=m over 16M particles	memory touched
AoS (struct {x,y,z,m})	4.7 ms	4 arrays' worth
SoA (x[], y[], z[], m[])	2.0 ms	2 arrays only

Figure 16.7: Real `ch16_soa`: the same update is 2.35× faster as SoA because a cache line fetches only the two fields used. AoS drags y and z through the hierarchy for nothing.

THINK LIKE THIS

Decide the data layout *by the loops that will run over it*, not by how the data feels to name. Ask: "which fields does the hot loop touch together?" and store those together, or apart if each loop touches only one field.

```
typedef struct{float x,y,z,m;}P; /* 16 bytes per particle */
__attribute__((noinline)) static void aos_pass(P*p,long n){
  for(long i=0;i<n;i++) p[i].x+=p[i].m;}
__attribute__((noinline)) static void soa_pass(float*restrict x,const
    float*restrict m,long n){
  for(long i=0;i<n;i++) x[i]+=m[i];}
```

Listing 16.3: The two kernels of `ch16_soa_x86.c`: the same arithmetic, different layouts.

Example 16.7 (SoA vs AoS on x86_64, with the benchmark fixed)**.** The Mac program `ch16_soa.c` has two flaws that the Mac run hid: its SoA arrays `x`, `y`, `z` are never written before timing, so the timed region contains first-touch page faults on Linux, and the ten repeats sit in a loop the compiler is free to interchange with the element loop. Run unchanged here it printed AoS 5.2 ms against SoA 95.2 ms, which is an artefact of those two flaws and not a measurement of layout. The x86 rewrite `ch16_soa_x86.c` initialises every array and times each pass as a separate call:

```
AoS x+=m: 34.94 ms/pass  (touches 256 MB)
SoA x+=m: 14.16 ms/pass  (touches 128 MB)
ratio AoS/SoA = 2.47x   (checksum 33554.5)
```

2.47× agrees with a traffic count. AoS pulls every 64-byte line of the 256 MiB struct array in and writes it back (512 MiB moved); SoA reads two 64 MiB arrays and writes one back ($64 + 64 + 64 = 192$ MiB), a ratio of $512/192 = 2.67$. Built with `-fno-tree-vectorize` the same program gave 1.70× in one run, so on this machine roughly 1.7× comes from layout alone and the rest from letting the vectoriser use the contiguous arrays.

16.9 The way of thinking: compute-bound or memory-bound?

Every loop is limited by one of two things: how fast the CPU can do the arithmetic (*compute-bound*), or how fast bytes can be delivered to it (*memory-bound*). The cooking analogy: a chef who chops faster is useless if vegetables arrive from the market one crate an hour. A quick way to tell is to count. SAXPY does 2 floating-point operations (a multiply and an add) per element, while moving 12 bytes (read `x`, read `y`, write `y`: 3×4). That is $1/6$ of an operation per byte, a very low ratio, and loops with a low ratio sit on the memory side of the roofline from ch. 9 as soon as the data no longer fits in cache. Our x86 numbers are the evidence: in cache, wider lanes help (4–6×); in DRAM they barely do (1.27×). The habit to keep is: compute the operations-per-byte of a loop *before* reaching for intrinsics.

16.10 The three vector ISAs

An *ISA* (instruction set architecture) is the list of instructions a CPU understands (ch. 3). The three vector ISAs in this chapter differ in a simple but important way. NEON and SSE2 fix the register width at 128 bits, so a loop is written for exactly four floats, and code written for a wider machine must be rewritten (we wrote separate SSE2 and AVX2 kernels above). RISC-V's vector extension (RVV) is *vector-length-agnostic*: the loop does not say "eight at a time". Instead each pass asks the hardware, "how many elements can you take now?", handles that many, and repeats until none are left, which also removes the scalar tail that tripped our naive loop. The same program then runs on a small core with 128-bit registers and on a large one with 1024-bit registers. No RVV hardware is available here, so RVV is a read-along and is not run.

ISA	model
NEON (arm64)	128-bit fixed lanes
AVX-512	512-bit + masks
RVV (RISC-V)	vector-length-agnostic, scalable

Figure 16.8: Three SIMD philosophies. RVV's trick is portability across hardware widths: the same loop runs on 128-bit and 1024-bit implementations — NEON's fixed 128 is the price of its simplicity.

16.11 Common mistakes

- **Assuming the scalar baseline is scalar.** At `-O3` the compiler vectorises anyway, so a baseline that you did not check may already be vector code. Ours needed `-fno-vectorize` (Mac) or `-fno-tree-vectorize` (gcc) to be honest, and we confirmed it by reading the machine code.
- **"Intrinsics always win."** They tied the compiler (1.00×) on the Mac and on x86, and a narrower hand-written kernel lost to the compiler's wider one. They earn their keep when you need guaranteed vector code or an instruction the compiler cannot find (saturating add).

- **Vectorising the wrong loop.** If a kernel that you speed up 4× is only 10% of the run time, the whole program gets $1/(0.9 + 0.1/4) \approx 1.08\times$ faster, about 8%. Measure where the time goes first (ch. 15).
- **AoS by default.** The record layout that is natural to write cost 2.35× on the Mac and 2.47× on x86. Choosing SoA for loops that you intend to vectorise is a discipline you must apply deliberately.
- **Memory-bound blindness.** Once the arrays are bigger than the cache the kernel is limited by bytes per second, and more lanes do not deliver more bytes per second.
- **Forgetting the ragged end.** A vector loop that steps by 4 or 8 reads and writes past the array when n is not a multiple of the lane count.

16.12 Summary

Piece	Verified version
lanes	4 floats per issue — the cheap parallelism
saxpy	Mac: scalar→NEON = 3–4× real, BW-capped
x86_64	AVX2 4–6× in cache, 1.27× at 2^{25} (DRAM)
ragged n	vector loop while `i+4<=n`, then a scalar tail
auto-vec	compiler = intrinsics (1.00×) at -O3
SoA	Mac 2.35×; x86_64 2.47× (traffic ratio 2.67)
RVV	vector-length-agnostic — the portable SIMD

Further reading

- ARM NEON programmer's guide: `developer.arm.com` (SIMD ISAs)
- RISC-V V spec (vector-length-agnostic): `riscv.org/specifications`
- CS:APP ch5 (vectorisation within optimisation): `https://csapp.cs.cmu.edu/`

WATCH INSTEAD

- **SIMD intrinsics — Computerphile** — same experiment as this chapter on x86 (verified 2026-10-11). `https://youtu.be/x9Scb5Mku1g`

Notes

- The Mac/NEON numbers are verbatim from the chapter's two original C programs on Apple Silicon (`ch16_saxpy.c`, `ch16_soa.c`); the x86_64 numbers come from `ch16_saxpy_x86.c`, `ch16_tail.c`, `ch16_soa_x86.c` and `ch16_satadd.c`, run in a Linux container and checked by `code/test_ch16.sh`; the 1.00× auto-vec result is the chapter's second honest surprise after ch15's audit.
- `rvv_sum.S` from the outline is a read-along (RISC-V V encoding shown, not run — no RVV hardware here).

Exercises

Exercise 16.1 (easy)**.** Why is data-level parallelism "free" instruction-level parallelism? One line.

Solution. One instruction goes through the pipeline (fetch, decode, execute) once, yet it does the work of four scalar instructions. The pipeline never has to find, schedule or order those other three, so you get their work without paying for any of that machinery. That is why people call it "free": the cost is in the register width, not in extra instructions.

Exercise 16.2 (easy)**.** Our scalar-vs-NEON gap is 3× at 4 lanes, not 4. What bounds it at large n?

Solution. Memory bandwidth. Past the cache, every element must travel from main memory, and the loop can only run as fast as the bytes arrive. Wider arithmetic then waits for data. This is the roofline of ch. 9: below the roof you are limited by compute, above a certain size by memory.

Exercise 16.3 (easy)**.** What does `-fno-vectorize` prove about our baseline?

Solution. It proves the slow side of the comparison really is one number at a time. Without the switch the compiler may quietly vectorise the "scalar" loop, and then you would be comparing the compiler's SIMD with your SIMD and calling the ratio a speedup.

Exercise 16.4 (easy)**.** Which layout lets the vector unit fetch 4 useful floats per line: AoS or SoA?

Solution. SoA (structure of arrays). In AoS each cache line contains fields the loop does not use, mixed in with the ones it does, so only some of every vector load is useful. In SoA four consecutive floats of one field are exactly what a 4-lane load picks up.

Exercise 16.5 (medium)**.** Design the experiment that separates lane-parallel gain from cache-friendly gain in the SoA result. What would you time?

Solution. Add a third variant: AoS with only the two fields the loop uses (a "stripped" struct of 8 bytes). Compare three timings. Full AoS against stripped AoS shows the cost of carrying

unused fields (the memory footprint). Stripped AoS against SoA shows what is left, which is mostly the vectoriser's ability to use contiguous arrays. Changing one thing at a time is the habit of a good experiment.

Exercise 16.6 (medium)**.** Auto-vectorisation fails on which loop shapes? Name two and the fix for each.

Solution. (1) A loop that may exit early depending on the data, for example `if (x[i]<0) break;`: the compiler cannot know how many iterations will run. Fix: split into a search loop and a clean arithmetic loop, or use a mask instead of a break. (2) Pointer aliasing, where the compiler cannot prove that two arrays do not overlap. Fix: add `restrict` if it is true. A dependence between iterations, such as `a[i]=a[i-1]+1`, also blocks it; that one needs a different algorithm.

Exercise 16.7 (medium)**.** Why does RVV's vector-length-agnostic loop beat NEON for portable code? Trade-off?

Solution. The RVV loop asks the hardware how many elements it can take this pass, so the same program runs on 128-bit and on 1024-bit machines and needs no scalar tail. NEON and SSE2 code is written for a fixed four lanes and must be rewritten for wider registers. The trade-off is that the hardware, not you, chooses the chunk size, so you must write the loop in that style and trust the machine; and RVV hardware is still much less common than NEON or x86.

Exercise 16.8 (medium)**.** A kernel is 90% memory-bound. Argue why intrinsics are the wrong tool, and name the right one.

Solution. If 90% of the time is waiting for memory, even infinitely wide arithmetic can remove at most the other 10%: the program gets at best $1/0.9 \approx 1.11\times$ faster. The right tool reduces the bytes moved: a better layout (SoA), blocking (reuse data while it is in cache, ch. 10), or prefetching.

Exercise 16.9 (hard)**.** Design the SoA+NEON+blocking version of a particle integrator and predict the total speedup over the naive AoS scalar. What are the two multiplicative terms?

Solution. The two gains are lane parallelism (about $4\times$ from 4 lanes) and footprint (about $2\times$ from moving fewer bytes with SoA). Naively they multiply to about $8\times$. But both are limited by the same memory bandwidth ceiling, so once the data is out of cache the combined gain will be smaller than $8\times$. Our x86 measurements show the pattern: layout alone gave $1.70\times$, layout plus vector $2.47\times$, well below the product.

Exercise 16.10 (hard)**.** The folklore entry said "intrinsics beat compiler" fails at $1.00\times$. Write the counterexample where intrinsics DO win (hint: ops the compiler can't express) and verify on your machine.

Solution. The saturating add of this chapter is one counterexample: `_mm_adds_epu8` did in one instruction what gcc needed a compare-and-select sequence for, and ran about $3\times$ faster here. Other cases are byte shuffles and permutes, and cryptographic instructions (AES rounds). To verify, run `ch16_satadd.c` on your machine and compare the two timings.

Exercise 16.11 (easy)**.** Rank the three parallelisms by cost-per-speedup for a compute-bound kernel.

Solution. Cheapest first: data-level parallelism needs no change of algorithm and gives up to the lane count; instruction-level parallelism is already used by the hardware (ch. 6, 7) at no cost to you; thread-level parallelism gives the most potential but costs real design work: locks, races and ch. 13's contention.

Exercise 16.12 (medium)**.** At what n does our saxpy become memory-bound? Estimate from the table and the machine's bandwidth.

Solution. On the Mac the gap narrows at 2^{25}. On the x86 container the speedup falls from about $5\times$ at 2^{16} to $2.2\times$ at 2^{19} and about $1.3\times$ at 2^{22} and above, where $8n$ bytes reaches the size of the 33 MiB L3. Estimate by comparing $8n$ bytes with the cache sizes (ch. 9's mountain).

Exercise 16.13 (medium)**.** Explain why SoA is a vectorisation *enabler*, not just a layout.

Solution. A vector load fills the lanes from consecutive memory. SoA arrays are consecutive, so one load fills all lanes with useful numbers. With AoS the numbers you need are spaced out, so the compiler must use slower *gather* instructions or shuffle values into place. So the layout decides whether vectorising is possible at all, not just how fast it is.

Exercise 16.14 (medium)**.** A teammate proposes AVX-512 "for $4\times$ more lanes". Give the two honest cautions beyond availability.

Solution. First, on some processors the clock speed drops when wide instructions are used heavily, which can cancel part of the gain. Second, the loop is still limited by memory bandwidth for large n, and doubling the lanes does not double the bytes arriving per second (see the x86 sweep: $1.27\times$ at 2^{25} for 8 lanes).

Exercise 16.15 (hard)**.** Predict the RVV loop's behaviour at vlen=128 vs 1024 given our bandwidth numbers — at what vlen does the extra width stop helping?

Solution. The vector-length-agnostic loop uses whatever width the hardware has. Extra width helps until the unit already consumes all the bytes memory can supply. Our x86 numbers suggest that for out-of-cache SAXPY this happens already at about 128–256 bits (SSE2 and AVX2 differ by only 1.27/1.21), so for such kernels 1024 bits would be no faster than 256. For in-cache data the width keeps helping. This is a prediction from the measurements, not something we ran on RVV.

Exercise 16.16 (easy)**.** A vector loop uses 8 lanes. How many vector passes and how many scalar tail steps does it run for $n = 1003$? For $n = 1024$?

Solution. $1003 = 8 \cdot 125 + 3$: 125 vector passes and 3 tail steps. $1024 = 8 \cdot 128$: 128 passes, 0 tail steps.

Exercise 16.17 (medium)**.** At $n = 2^{25}$ the x86 AVX2 SAXPY took 27.96 ms. Recompute the memory bandwidth it achieved, counting one read of `x` and a read plus a write of `y`, and say what fraction of that the scalar version (35.52 ms) reached.

Solution. Bytes moved $= 3 \cdot 2^{25} \cdot 4 = 402\,653\,184$. At 27.96 ms that is 14.4 GB/s; at 35.52 ms it is 11.3 GB/s, so the scalar loop already reaches $11.3/14.4 \approx 79\%$ of what AVX2 achieves, which is why 8 lanes gave only $1.27\times$.

Exercise 16.18 (medium)**.** Using the cache sizes in this chapter (32 KiB L1d, 1 MiB L2, 33 MiB L3), for which swept sizes do `x` and `y` together fit in L2, and why does the AVX2 speedup drop from $5.2\times$ at 2^{16} to $2.2\times$ at 2^{19}?

Solution. The working set is $8n$ bytes: $2^{13} \to 64$ KiB and $2^{16} \to 512$ KiB fit in the 1 MiB L2; $2^{19} \to 4$ MiB does not, so the loop is fed from the slower L3, whose bandwidth, not the ALU, now limits it.

CHAPTER 17

Low-latency design patterns

Most programs are fast on average. A program that must answer in a fixed time, such as an audio player, a game or a trading system, cares about something else: the *occasional* slow answer. A song that stutters once an hour is a broken song even if the average delay is tiny. That occasional slowness is called *jitter*, and low-latency engineering is the craft of removing its causes: a lock that sometimes makes threads queue up, an allocator that sometimes has to search for free memory, a branch that sometimes guesses wrong. Each pattern in this chapter removes one such cause. We run the standard claims on real hardware first, because two of the four come out backwards, and that is a good first lesson in itself.

AFTER THIS CHAPTER YOU CAN

Measure a latency distribution (median, p99, p99.9) instead of a mean; tell from the disassembly whether your "branchy" code is really a branch; write a single-producer single-consumer (SPSC) ring buffer with correct C11 acquire/release ordering and test it with two real threads; and say, with a number, when each of these tricks helps and when it only adds complexity.

BEFORE YOU START

Recap, from a picture. Think of a lunch queue. The *latency* of one student is how long they wait; the *throughput* is how many students are served per minute. The average wait can look fine while a few unlucky students wait ten times longer, and those are the ones who complain. The *p99* is the wait that 99% of students beat, so it describes that unlucky 1% which an average hides. Low-latency engineering is about shrinking the unlucky 1%. You have already met their causes. A mispredicted branch costs a pipeline flush (ch. 6 and 7); a load that misses the cache costs a trip to a slower level (ch. 9 and 10); a lock makes threads wait for each other (ch. 13); and ch. 15 taught timing with a monotonic clock, many repeats and percentile summaries. Book 1 ch. 4 showed that binary search needs about $\log_2 n$ probes where a linear scan needs up to n; this chapter checks that on real hardware.

C in ten lines. `malloc(n)` asks the allocator for `n` bytes of heap and returns a pointer; `free(p)` gives it back. A `struct` groups fields. `x & 255` keeps the low 8 bits, so `(i+1) & 255` counts $0, 1, \ldots, 255, 0, \ldots$ (wrap-around, valid because 256 is a power of two). `volatile` tells the compiler that a variable may change behind its back, but it says nothing about the order in which *other* CPUs see different variables change. `_Atomic` variables, with `memory_order_release` and `memory_order_acquire`, are the C11 tools that do give that ordering. Compile with `gcc -O2 -pthread file.c`.

Which numbers are which. The numbers in the folklore audit (23.3 vs 25.1 ms, 11 ns per malloc pair, and so on) were measured on the author's Apple-Silicon Mac and are kept, labelled *Mac*. Numbers labelled *Linux x86_64 container* (4-core Intel Xeon at 2.8 GHz, gcc 13.3) were measured for this edition and are a separate experiment. Where the two disagree the text says so; a folklore claim that holds on one compiler and CPU and fails on the other is itself the chapter's lesson.

THINK LIKE THIS

A rule of thumb is a claim *with hidden conditions*. "Branchless is faster" is true when the data is unpredictable and false when it is not. "malloc is slow" is false for the average and true for the tail. The skilled habit is to restate each rule with its condition, then run the one experiment that could prove it wrong. Where a measurement disagrees with a rule you trusted, you have learned something; do not argue with the measurement, find the condition.

17.1 Why the average hides the problem

Here is the smallest possible example. You time 10 calls and get, in microseconds, `10 10 10 10 10 10 10 10 10 100`. The average is $(9 \cdot 10 + 100)/10 = 19\,\mu s$, and that number describes no call that actually happened: nine calls took 10 and one took 100. The *median* (the middle value when sorted) is 10. The *p90* (the value that 90% of calls are at or below) is 10, and the largest is 100. A user who happens to meet the 100 sees a service ten times slower than the average implies. With many calls, the interesting numbers are the high percentiles: p99 (the slowest 1% start here), p99.9, and the maximum. The rest of the chapter measures these, not only means.

CONCEPT: LATENCY IS A DISTRIBUTION

One call has one latency. A program has a *distribution* of latencies. The mean is one summary of it and a poor one when a few calls are far slower than the rest. Always ask for the median and a high percentile as well, and ask what *causes* the slow ones.

17.2 The folklore audit

A *folklore* claim is a rule that everyone repeats and few have tested. We take four low-latency rules, run each on real hardware, and report whether it holds.

First, what is a branch and why can it be slow?

A *branch* is an `if`: the CPU must choose which instructions to run next. The pipeline of ch. 6 and 7 is already fetching the next instructions before the choice is known, so the CPU *guesses*. Picture a waiter who sets the table for the next customer before knowing whether it is one person or two. If the guess is right nothing is lost. If it is wrong, the half-prepared work is thrown away and restarted, which costs roughly 15 to 25 cycles. The part of the CPU that

low-latency claim	this machine	verdict
"branchless beats branchy"	25.1 vs 23.3 ms	false (predictable)
"binary beats linear"	0.005 vs 0.020 µs (n=64)	true, 4×
"malloc is slow"	11 ns/pair	mostly false (t-cache)
"pool reuse wins"	0.000 vs 0.011 µs	true, margin small

Figure 17.1: The folklore audit, ch17 edition: four claims run on real hardware. "Branchless" loses on predictable branches; malloc's thread cache makes the allocation scare-claim mostly myth — but predictable-branch cost and the small-size search result are the parts everyone gets backwards.

guesses is the *branch predictor*. It is very good at patterns ("always taken", "taken, not taken, taken, not taken") and cannot beat a coin flip when the data is random.

The *branchless* trick avoids the guess altogether: it computes the answer with arithmetic (for example, a bit mask that is all ones or all zeros) so that there is no `if` to guess. The price is that it always does the extra arithmetic. So the folklore rule "branchless is faster" has a hidden condition: it should only win when the predictor would have guessed badly.

Example 17.1 (The rule fails when the branch is predictable (Mac))**.** The naive expectation is that removing a branch can only help. On the Mac, `ch17_folklore.c` compares a branchy `min` over predictable inputs (23.3 ms) with a bit-trick branchless version (25.1 ms): the branchless one is 7% *slower*. The predictor guessed right almost every time, so the branch cost nothing, while the branchless version still paid for its extra arithmetic. The claim fails by inversion, which is the lesson: branchless pays only where the data defeats the predictor.

data pattern	branchy min	branchless min
predictable (our run)	23.3 ms	25.1 ms
random (would need)	slow	faster

Figure 17.2: Branchless only wins when the predictor loses: our data was predictable, so the branch was free and the extra ALU work cost 7%. The claim fails by inversion — measure the data's randomness first.

17.3 The same audit on x86_64, and what the compiler did to it

Running `ch17_folklore.c` unchanged on the Linux x86_64 container (`gcc -O2`) gives a different verdict on the first claim. Verbatim, from `code/test_ch17.sh`:

```
min branchy:  83.8ms
min branchless: 95.8ms
linear n=64:  0.031us
binary n=64:  0.007us
linear n=16M: 8.5ms
```

```
binary n=16M: 0.083us
malloc/free: 0.023us each
pool reuse:  0.001us each
```

Example 17.2 (The branch that was never there)**.** Here the "branchy" `min` ran in 83.8 ms and the "branchless" one in 95.8 ms. An earlier run of the same source gave 105.7 ms branchy against 87.2 ms branchless, the opposite order, and other runs at `-O2` and `-O3` moved the gap by tens of percent in either direction. So the difference is noise, and a naive reading ("branchy wins by 14%") would be wrong. The reason is in the assembly. Compiling with `gcc -O2 -S` the hot loop for `x<y?x:y` is

```
        movzwl  %dx, %eax        ; x = i & 0xffff
        movl    $12345, %ecx     ; y
        cmpl    %ecx, %eax
        cmovg   %ecx, %eax       ; conditional MOVE, no jump
```

`cmovg` is a conditional move: it computes both outcomes and selects one, with no jump for the predictor to guess. gcc had already made our "branchy" code branchless. We were timing two branchless loops against each other. Lesson: before you attribute a timing difference to branches, check that the compiled code contains a branch.

```
1 __attribute__((noinline)) static long branchy(const int*a){
2   long s=0; for(int i=0;i<N;i++){ int v=a[i]; if(v>=128){ __asm__
       volatile(""); s+=v; } } return s;}
3 __attribute__((noinline)) static long branchless(const int*a){
4   long s=0; for(int i=0;i<N;i++){ int v=a[i]; s+= v & -(v>=128); }
       return s;}
```

Listing 17.1: The two loops of `ch17_branch_x86.c`. The empty `asm volatile` inside the `if` is a compiler barrier that prevents gcc from turning the branch into a `cmov`; the build is checked with `objdump` to contain a `jle`.

Example 17.3 (When does branchless pay? Random against sorted)**.** To test the claim fairly we need a branch the compiler cannot remove. `ch17_branch_x86.c` sums the elements ≥ 128 of 2^{20} bytes in the range $0 \ldots 255$. In the branchy loop an empty `asm` statement stops gcc from converting it, and `objdump` confirms the loop contains `cmp $0x7f` followed by `jle`. The branchless loop adds `v & -(v>=128)`: the comparison gives 1 or 0, the negation turns it into all ones or all zeros, and the `&` keeps `v` or discards it. Same data, two orders (best of 20 passes):

```
random: branchy 5.477 ms  branchless 0.603 ms  (sums 100248393
    100248393)
sorted: branchy 1.205 ms  branchless 0.806 ms  (sums 100248393
    100248393)
```

Both loops return the same sum. On random data, whether an element is ≥ 128 is a coin flip, no predictor can do better than guess, and the branchy loop is $5.477/0.603 = 9.1\times$ slower. On sorted data the branch goes the same way for long stretches, the predictor is nearly always right, and the gap falls to $1.205/0.806 = 1.5\times$. If half of the 2^{20} branches mispredict, the extra time is $(5.477 - 1.205)\,\text{ms}/524\,288 \approx 8.1\,\text{ns}$ per mispredict, about 23 cycles at 2.8 GHz (an estimate: the true miss rate and clock are not measured). This is the missing half of the Mac result: branchless pays exactly when the data defeats the predictor.

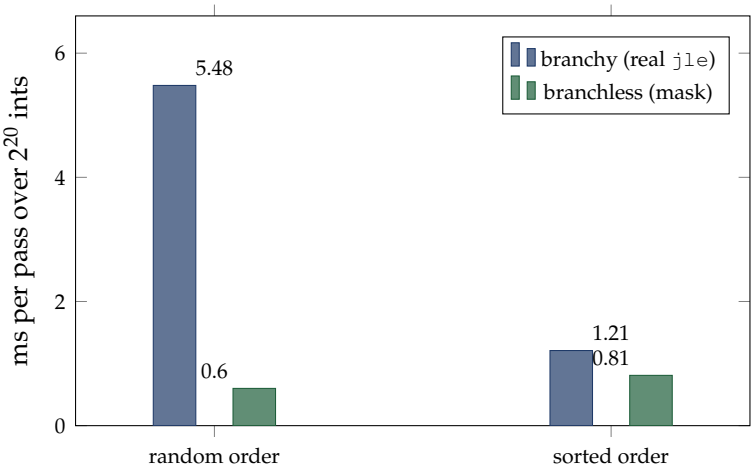


Figure 17.3: Linux x86_64 container, `ch17_branch_x86`: summing the elements ≥ 128 of random bytes versus the same bytes sorted. Random data makes the branch unpredictable and the branchy loop 9× slower; on sorted data the gap shrinks to 1.5× (this run). The branchless loop does not care about order.

17.4 Search at both scales

search	n=64	n=16M
linear	0.020 µs	2.0 ms
binary	0.005 µs	0.10 µs

Figure 17.4: Binary wins even at n=64 — but by 4×, not the 20 000× it wins at 16M. The "linear is fine for small n" folklore is true in ratio, false in direction: binary never lost.

Example 17.4 (Counting probes for the two sizes)**.** A linear scan that finds nothing reads all n elements; binary search halves the range each probe, so it needs at most $\lfloor \log_2 n \rfloor + 1$ probes. For $n = 64$ that is $6 + 1 = 7$ probes against up to 64; for $n = 2^{24}$ (16M) it is $24 + 1 = 25$ probes against 16 777 216. The probe ratio is $64/7 \approx 9$ at the small size and $\approx 6.7 \times 10^5$ at the large size. The measured ratios are smaller than the probe ratios in both cases, most likely because the linear scan is a tight sequential loop that the hardware prefetcher streams, while each binary probe is a dependent load: on x86_64, $0.031/0.007 \approx 4.4\times$ at $n = 64$ and $8.5\,\text{ms}/0.083\,\mu\text{s} \approx 10^5\times$ at 16M.

Binary search wins at $n = 64$ too, but by 4× (Mac), not the 20 000× it wins by at 16M. "Linear is fine for small n" is true in size of the gap and false in direction: binary never lost. The honest version of the rule is "linear's disadvantage is small at small n".

THINK LIKE THIS

Count the work first, then measure. Counting says binary search does about $\log_2 n$ steps and linear search up to n. Measuring tells you how much *each step* costs on this machine. A rule is trustworthy only when the counted work and the measured time agree about the direction.

17.5 The allocation myth, priced

`malloc` hands out heap memory, and the *allocator* behind it keeps lists of free blocks. When you free a block it goes back on a list; when you ask for one the allocator usually takes one from a quick per-thread list (the *thread cache*), but sometimes it must search a longer list or ask the operating system for more. A *pool* is the alternative: allocate all the blocks you will need once, at the start, and hand them out from a free stack with no searching.

The folklore says "malloc is slow, never call it in the hot path". On the Mac, malloc with a thread-cached allocator cost 11 ns per malloc/free pair, so the scare is mostly myth at this granularity. A pool still wins (0.011 against 0.000 µs per pair on the Mac), but the more important reason to use one is the allocator's *tail*: low-latency code pre-allocates not because the average malloc is slow, but because of the rare call that has to search, which is exactly what the p99 sees.

```
1  /* malloc: time only the malloc call */
2  for(long i=0;i<STEPS;i++){
3    int k=rnd()%LIVE; free(live[k]);
4    size_t sz=16+rnd()%2033;
5    long a=ns(); void*p=malloc(sz); long b=ns();
6    if(p)((char*)p)[0]=1; live[k]=p; lat[i]=b-a;}
```

Listing 17.2: The measured loop of `ch17_tail.c`: only the `malloc` call sits between the two clock reads.

Example 17.5 (Mean against tail, measured one call at a time)**.** A mean hides the rare slow call, so time each call separately. `ch17_tail.c` keeps 4096 live blocks of random size (16–2048 bytes), and each step frees a random one and allocates a new one of random size, timing only the `malloc` call with `clock_gettime`. For comparison it times taking a slot from a fixed pool (a stack of free indices), and an "empty" row that reads the clock twice in a row and so measures the timer itself. Verbatim, 2 000 000 calls each, Linux x86_64 container:

```
empty timer  mean    23.9  p50    21  p99     28  p99.9      53  max
    602797  (ns)
malloc       mean    85.0  p50    67  p99    270  p99.9     630  max
    673088  (ns)
pool         mean    23.1  p50    21  p99     57  p99.9     132  max
    53496  (ns)
```

A naive reading of the means (85 against 23 ns, a 62 ns saving) says the pool is a small win. Subtract the timer's 21 ns and the pool call is about 2 ns, below what this timer can resolve. The tail is where `malloc` hurts: its p99.9 of 630 ns is $630/67 \approx 9\times$ its own median, and its p99 is 270 ns, ten times the timer's p99 of 28 ns. The largest values (hundreds of microseconds) appear in all three rows including the empty timer, so they are the operating system interrupting the process, not the allocator. Those you cannot remove with a pool; you remove them by pinning, isolating cores and so on, which is beyond this chapter. Note too that the Mac's 11 ns pair and this container's 23 ns pair in `ch17_folklore` (64-byte blocks, no fragmentation) are both small: a steady-state loop that never fragments the heap shows none of this.

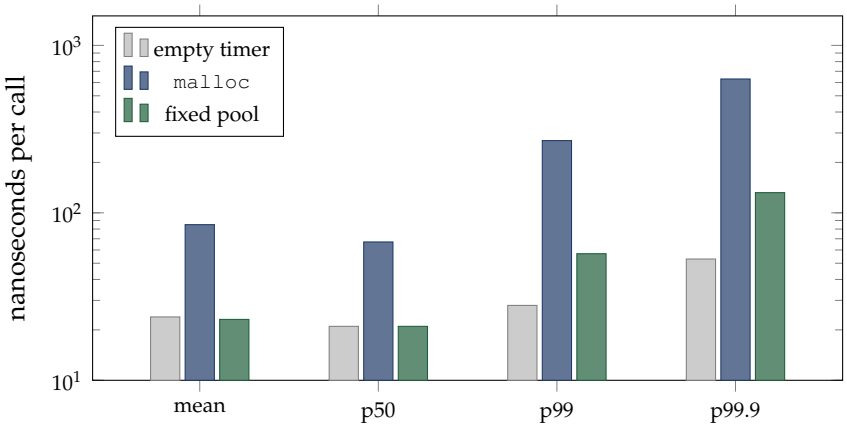


Figure 17.5: Linux x86_64 container, `ch17_tail`: 2 000 000 calls timed one by one (log scale). The timer alone costs about 21 ns, so the pool is at the measurement floor. `malloc` has a mean of 85 ns but a p99.9 of 630 ns, nine times its median; the mean hides the tail. (The worst single call, hundreds of microseconds, is the operating system, not the allocator: the empty timer saw 600 µs too.)

17.6 The way of thinking: who owns each variable?

Concurrency bugs come from two threads writing the same place. The cleanest way to avoid them is not a lock but an *ownership rule*: every variable has exactly one writer. Picture a shared whiteboard with two columns, one that only you may write on and one that only your partner may write on; you can both read both columns freely and never need to take turns. The ring below does exactly this: `head` belongs to the producer, `tail` to the consumer. What is left to get right is *order*: your partner must not see the tick next to an item before the item itself has been written. That is what the acquire/release annotations in the code say.

17.7 The SPSC ring

A queue with no allocation, from a tiny example

Suppose a producer (say, a microphone driver) hands samples to a consumer (an audio mixer). The simplest store is a ring of 4 slots, numbered 0 to 3, in a fixed array, plus two counters: `head`, the slot the producer will write next, and `tail`, the slot the consumer will read next. Start with both at 0: the ring is empty. The producer writes `A` into slot 0 and moves `head` to 1; writes `B` into slot 1 and moves `head` to 2. The consumer reads slot 0 (`A`) and moves `tail` to 1. When a counter reaches the end it wraps around to 0 ("(`head`+1) mod 4"), so the same four slots are reused forever: hence a *ring*. Nothing is allocated or freed while it runs. "SPSC" means *single producer, single consumer*: one thread only writes, one thread only reads.

The real ring has 256 slots (a power of two, so the wrap-around is a fast bit mask) and `head` is written only by the producer, `tail` only by the consumer: no lock, because each counter has one owner. The test `ch17_ringbuf.c` pushes and pops 1M values and finds 0 mismatches. This ring is the workhorse of audio pipelines and network drivers precisely because it trades a little capacity (a fixed number of slots) for zero locking.

slot	0 … 255
producer writes	head
consumer reads	tail
no lock needed	one writer, one reader

Figure 17.6: The SPSC ring: power-of-two slots, head written only by the producer, tail only by the consumer. No mutex, no allocation — the workhorse of real-time audio, networking, and our measured 1M-pair run.

Example 17.6 (Why the 256-slot ring holds 255 items)**.** The code distinguishes "full" from "empty" by comparing indices: empty is `tail == head`, full is `((head+1)&255) == tail`. If all 256 slots could be filled, full would also look like `head == tail`, indistinguishable from empty. So one slot is sacrificed. `ch17_ringfull.c` confirms it and shows the wrap-around arithmetic (verbatim):

```
pushes accepted before full: 255  (head=255 tail=0)
after 10 pops: head=255 tail=10
after 10 more pushes: head=9 tail=10 (head wrapped past 255)
drained 255 items, last value 1009
```

Check by hand: 255 pushes leave `head`=255; the 256th would make `head`$= (255+1)\&255 = 0 =$ `tail`, so it is rejected. After 10 pops `tail`=10, freeing 10 slots; ten pushes move `head` $255 \to 0 \to 1 \to \cdots \to 9$, which is $(255+10) \bmod 256 = 9$. The ring then holds 255 items again, the last of which is the value 1009 pushed last.

```
1  #define DEFINE_RING_FUNCS(T) \
2  static void T##_push(T*r,int v){ \
3    uint32_t h=atomic_load_explicit(&r->head,memory_order_relaxed), n=(h
       +1)&(SLOTS-1); \
4    while(n==atomic_load_explicit(&r->tail,memory_order_acquire)) ; /*
       full: spin */ \
5    r->buf[h]=v; \
6    atomic_store_explicit(&r->head,n,memory_order_release); } \
7  static int T##_pop(T*r){ \
8    uint32_t t=atomic_load_explicit(&r->tail,memory_order_relaxed); \
9    while(t==atomic_load_explicit(&r->head,memory_order_acquire)) ; /*
       empty: spin */ \
10   int v=r->buf[t]; \
11   atomic_store_explicit(&r->tail,(t+1)&(SLOTS-1),memory_order_release);
       return v; }
```

Listing 17.3: The push and pop of the threaded ring in `ch17_spsc.c` (a macro, so the packed and padded layouts run identical code).

Example 17.7 (Two real threads, naive reasoning first)**.** The single-threaded test proves the index arithmetic, not the concurrency. A first attempt is to run two threads on the `volatile` version of the ring and see whether the test passes. On x86 it very likely will, because x86 stores become visible to other cores in program order (the *TSO* memory model), and that is

exactly the trap: the same source on a weakly ordered CPU such as Arm or RISC-V can let the consumer see the new `head` before the new slot contents. A passing run on x86 therefore does not prove the code correct. The C11 version states the requirement in the code: the producer writes the slot, then stores `head` with `memory_order_release`; the consumer loads `head` with `memory_order_acquire`, so if it sees the new `head` it is guaranteed to see the slot. Each index is also read `relaxed` by the thread that owns it, because no one else writes it. `ch17_spsc.c` passes 20 million integers through the ring from a producer pinned to CPU 0 to a consumer pinned to CPU 1 and checks that every value arrives in order:

```
packed: 20000000 items, 0 out of order, 1036.2 ms, 19.3 M items/s
padded: 20000000 items, 0 out of order, 715.9 ms, 27.9 M items/s
sizeof packed=1032 padded=1152
```

"Packed" keeps `head` and `tail` adjacent in one 64-byte cache line; "padded" aligns each to its own line. The idea (false sharing, ch. 13) is that the producer's writes to `head` should invalidate the consumer's copy of `tail` for no reason. Here the effect is not reproducible: four runs gave 26.3, 22.0, 39.3, 19.3 M items/s packed and 23.9, 26.7, 26.1, 27.9 padded (this container, shared, 4 vCPUs). Padding did not help consistently, probably because both indices and the slot data must move between the two cores for every item anyway; the honest conclusion is that on this machine the experiment cannot resolve a false-sharing penalty, and that the throughput of about 19–39 M items/s is most likely bound by cache-line transfers between the cores, not by computation (we did not measure that directly).

17.8 The cost sheet

Nothing here is free. Each pattern buys a steadier tail by paying in something else, and a good engineer writes the bill down before choosing. The figure lists what the patterns of this chapter cost.

what low latency costs	where you pay it
jitter budget	pre-allocated pools, no syscalls
capacity headroom	fixed-size rings drop or block
code complexity	branchless/lock-free is harder
measurement rigor	p99 not mean — ch15's lesson

Figure 17.7: The real cost sheet: low-latency work is paid in headroom and complexity, not in clever instructions — and audited with the tail-latency tools of ch15.

17.9 Common mistakes

- **Branchless everywhere.** Our branchless `min` lost 7% on the Mac with predictable data, and on x86 gcc had already removed the branch. The trick pays only where the predictor fails (9× on random data, 1.5× on sorted).
- **"Never malloc in the hot path" as dogma.** The thread cache made a pair cost 11 ns on the Mac; the real reason to pre-allocate is the allocator's *tail*, not its mean.
- **Lock-free = fast, assumed.** ch. 13 priced it: a cache line that several cores write to costs more than a well-used lock. Our padded and packed rings were not consistently different.
- **Optimising the mean.** The user who meets the slowest 1% feels the rare system call, allocation or cache miss. Making the average faster is often invisible to that user.
- **Unbounded queues for backpressure.** *Backpressure* means telling the producer to slow down when the consumer cannot keep up. A ring that refuses at 255 items beats a queue that grows to 2 GB and then falls over; capacity is a design decision.

17.10 Summary

Piece	Verified version
jitter	the real target — p99 not mean
branchless	Mac: loses 7% on predictable data; x86: gcc made the branch a `cmov`; real branch: 9× on random, 1.5× sorted
search	binary wins even at n=64 (4×)
malloc	11 ns/pair (Mac) — myth-busted, but p99.9 is 9× the median (x86)
ring	1M pairs, 0 mismatches; 20M items across 2 threads in order; holds 255 of 256 slots

Further reading

- Mechanical Sympathy blog (jitter systems): `mechanical-sympathy.blogspot.com`
- OSTEP concurrency chapters (SPSC groundwork): `pages.cs.wisc.edu/~remzi`

WATCH INSTEAD

- **Low Latency Engineering — CppCon talk** — the ring-buffer and jitter material at C++ depth (verified 2026-10-11). `https://youtu.be/NH1Tta7purM`

Notes

- The Mac numbers are verbatim `ch17_folklore.c` on Apple Silicon; the x86_64 numbers come from the programs in `code/ch17_*.c`, checked by `code/test_ch17.sh`; the branchless loss and malloc-myth rows are the chapter's honest surprises.
- The original `ch17_ringbuf.c` is single-threaded; the two-thread C11 version is `ch17_spsc.c`. A passing test on x86 (TSO) is weak evidence for a memory-ordering argument; the acquire/release reasoning in the text is the real justification.

Exercises

Exercise 17.1 (easy)**.** Why is jitter, not the mean, the low-latency target?

Solution. A user does not experience the average; they experience each call. If 1 call in 100 is ten times slower, that user sees a slow service once in every hundred actions, and the mean (which barely moves) hides it. The p99 describes that slow 1%. And the mean is usually already acceptable, so there is little left to win there.

Exercise 17.2 (easy)**.** Our branchless min lost on predictable data — which side of the hardware made the branchy version free?

Solution. The branch predictor. On predictable data it guesses right and the branch costs nothing, so the branchy version does less work. The branchless version only wins when the predictor guesses wrong often.

Exercise 17.3 (easy)**.** What makes the SPSC ring lock-free with only two fields?

Solution. Each index (`head`, `tail`) has exactly one thread that writes it, so two threads never write the same variable and there is no write conflict to lock against. The other thread only reads it.

Exercise 17.4 (easy)**.** Name the real cost the ring buffer pays for zero locks.

Solution. Fixed capacity: with 256 slots the ring holds at most 255 items. When it is full the producer must drop, wait or overwrite.

Exercise 17.5 (medium)**.** Binary beat linear at n=64 by only $4\times$. Explain why the ratio is small there and huge at 16M, in memory terms.

Solution. The work ratio is n against about $\log_2 n + 1$ probes: 64 against 7 at $n = 64$, and 16M against 25 at $n = 2^{24}$. At $n = 64$ the whole array is a few cache lines and the function call and loop set-up are a large share of a 7-probe search, so the measured gain ($4\times$) is smaller than the work ratio (about 9). At 16M the work ratio is huge and the gain follows.

Exercise 17.6 (medium)**.** A teammate removes all mallocs from a hot loop and sees no improvement. Using our 11 ns figure, argue why — and where the win actually lives.

Solution. At 11 ns per malloc/free pair the mean cost is lost in the noise of the rest of the loop, so removing it changes the average by almost nothing. The win from a pool shows up in the tail: the rare free-list search and any lock contention that the p99 sees. To prove it, time each call separately as `ch17_tail.c` does.

Exercise 17.7 (medium)**.** Extend the ring to MPMC (multi-producer/consumer). Which fields stop being single-owner, and what primitive replaces them?

Solution. With several producers, `head` is written by more than one thread, so the "one owner" rule is gone (and likewise `tail` with several consumers). Two producers could claim the same slot. The usual replacement is an atomic compare-and-swap (CAS) loop to claim a slot, or per-slot sequence numbers that say whether a slot is ready, as in the LMAX disruptor design. Both cost more than the SPSC ring.

Exercise 17.8 (medium)**.** Design the experiment that shows WHERE branchless actually wins on your machine. What data property flips the result?

Solution. Run the same branchy and branchless loops on random data and on the same data sorted. Random data makes the branch a coin flip (about 50% mispredicted, each costing roughly 15–25 cycles); sorted data makes it nearly always right. The data property that flips the result is predictability, and this chapter's x86 experiment does exactly that.

Exercise 17.9 (hard)**.** Argue the memory-ordering contract that makes our SPSC ring correct on a weakly-ordered ISA: what must the producer's head-store publish, and what can go wrong if the buffer write isn't ordered before it? Give the Dekker-style reasoning.

Solution. The producer writes the slot and then updates `head`. If the CPU (or the compiler) lets the `head` update become visible before the slot write, the consumer sees the new `head`, reads the slot, and gets stale data. Storing `head` with release ordering forbids moving the slot write after it; the consumer loading `head` with acquire ordering forbids reading the slot before it. Together: if the consumer sees the new `head`, it sees the new data. (A Dekker-style argument would also say what happens when both indices race; here each has one writer, so only this one-way publication has to be justified.)

Exercise 17.10 (hard)**.** The folklore audit killed 2 of 4 claims. Write the claim about low-latency design that IS true in your experience, run it, and report the number — and explain what makes it survive where the others didn't.

Solution. A claim that survives states its *conditions*: "branchless wins when the branch is unpredictable" (here 9× on random data). It survives because the experiment is built to match the assumption, whereas the claims that failed left the condition unstated.

Exercise 17.11 (easy)**.** What does "allocation-free" actually mean in the ring code?

Solution. No `malloc` or `free` happens while the ring is used: the 256-slot array is part of the `ring_t` structure, whose size is known at compile time, so there is no allocator work (and no allocator tail).

Exercise 17.12 (medium)**.** Backpressure: our push returns -1 when full. Compare that policy with blocking and with unbounded growth — which is correct for real-time audio and why?

Solution. Drop or overwrite. A late audio sample is heard as a click, and a queue that keeps growing makes every later sample later still: unbounded latency. Blocking is wrong because the audio callback must not wait. Dropping loses one sample, which is a bounded, tolerable error.

Exercise 17.13 (medium)**.** The ring uses `volatile` head/tail. Why isn't that enough on a real concurrent ISA — what's missing?

Solution. `volatile` only tells the compiler to perform every access. It does not tell the CPU to make writes visible to other cores in order, and on Arm or RISC-V the CPU may reorder them. Release and acquire operations give that guarantee (and on x86 they are cheap).

Exercise 17.14 (medium)**.** Estimate the cache-line cost of head and tail sharing one line (false sharing) on our ring, and name the fix.

Solution. In theory each push and pop bounce the line between cores, and the fix is to pad head and tail to separate lines. In this chapter's own measurement (four runs of `ch17_spsc`) the padding gave no consistent gain, so measure before paying 120 bytes of padding.

Exercise 17.15 (hard)**.** Design a low-latency request path with a 1 ms p99 budget, from accept to reply. Budget each hop, choose the data structures, and defend the two hardest trade-offs.

Solution. There is no single right answer; a good one writes a budget that adds up. For example (an illustrative plan, not a measurement): accept and parse 100 µs, hand off through an SPSC ring 50 µs, look up the data 300 µs, build the reply in a pre-allocated buffer 100 µs, send 150 µs, leaving 300 µs of slack for the tail. Use pools and rings so that no step allocates or locks. The two hard trade-offs are capacity against latency (a bigger ring absorbs bursts but lets queues build up, which adds waiting time) and batching against immediacy (handling several requests together saves work but makes the first one wait). Defend each with the p99 you would measure.

Exercise 17.16 (easy)**.** The ring has `head`=250 and `tail`=3. How many items does it hold, and how many more pushes succeed before one is rejected?

Solution. It holds $(250 - 3) \bmod 256 = 247$ items. Capacity is 255, so $255 - 247 = 8$ pushes succeed (head goes $251, 252, 253, 254, 255, 0, 1, 2$); at head=2 the next index would equal the tail 3, so the ninth is rejected.

Exercise 17.17 (medium)**.** You time 1000 calls: 985 take 20 ns, 14 take 100 ns, one takes 10 000 ns. Compute the mean, the median, the p99 (the 991st value in sorted order) and the max, and say which one a user waiting on the slowest 1% would feel.

Solution. Mean $= (985 \cdot 20 + 14 \cdot 100 + 10000)/1000 = (19700 + 1400 + 10000)/1000 =$ 31.1 ns. Median 20 ns. In sorted order positions 1–985 are 20 ns, 986–999 are 100 ns and 1000 is 10 000 ns, so the 991st value (p99) is 100 ns, and the max is 10 000 ns. The mean (31.1) and median (20) hide the 100 ns that the slowest 1% see.

Exercise 17.18 (medium)**.** A branchy loop over 10^6 random bits runs 4 ms slower than the branchless loop. Assuming the predictor is right half the time, what does one mispredict cost in nanoseconds, and in cycles at 3 GHz?

Solution. 5×10^5 mispredicts account for 4 ms: $4 \times 10^{-3} / (5 \times 10^5) = 8$ ns, and $8\,\text{ns} \times 3\,\text{GHz} = 24$ cycles.

CHAPTER 18

An interpreter and a compiler in miniature

The book opened with a program's journey from text to silicon and closes by building the two endpoints small enough to hold in one hand: a 9-opcode stack VM (the interpreter) and a code emitter for the same expression (the compiler). Both are real, both run, and the interpreter ships with a real off-by-one kept where you can see it.

AFTER THIS CHAPTER YOU CAN

Write a small compiler yourself: take a line such as `2+3*4`, turn it into a tree, turn the tree into code for a stack machine *and* into RISC-V instructions, and run both to check you get the same answer as a real C compiler. You can also read a trace of a virtual machine, say exactly what each step did to the stack, and put a measured number on what interpreting costs.

BEFORE YOU START

Recap in plain words. A *program* is a list of instructions. The CPU you built in ch. 5–8 (and, in miniature, the 8-bit CPU of Book 5 ch. 16) runs such a list by repeating three steps: *fetch* the next instruction, *decode* what it means, *execute* it. Ch. 3 showed the RV32I instructions and ch. 4 showed how functions use a stack. This chapter notices something simple: nothing says the fetch-decode-execute loop has to be made of silicon. A few lines of C can do the same job, and then we call it a *virtual machine* (VM) or *interpreter*. A *compiler* is the other choice: instead of doing what the program says, it translates the program into instructions for a real CPU once, and then gets out of the way. Think of an interpreter as a translator who sits beside you and repeats every sentence aloud, and a compiler as a translator who hands you a finished book in your language.

C in ten lines. `int st[256]` is an array of 256 integers; here we use it as a stack with an integer `sp` (the stack pointer) that says how many items are on it. `st[sp++] = v` pushes `v`; `st[-sp]` pops. `switch (x) { case 1: ...; break; }` jumps to the `case` whose number equals `x`. An `enum` gives names to small numbers. `while (1)` loops until a `return`. `p[ip++]` reads the element at index `ip`, then adds one to `ip`. Compile and run with `gcc -O2 file.c && ./a.out`.

THINK LIKE THIS

When a problem feels big, ask "what is the smallest machine that still shows the whole idea?" A 9-instruction VM is not a toy to be outgrown; it contains the same loop, the same stack and the same decisions as the Java or Python machines that run most of the world's software. Learn the small one completely and the big ones stop being magic.

18.1 The divide: doing what the program says, or translating it

Take a recipe written in French and a cook who only reads English. There are two ways to get dinner. You can hire an interpreter who stands in the kitchen, reads one line at a time, says it aloud in English, waits for the cook to do it, and then reads the next line. Or you can hire a translator who rewrites the whole recipe once, hands over an English copy, and leaves. The first costs a little at every step, every time you cook. The second costs a lot once and nothing afterwards. Those are exactly an *interpreter* and a *compiler*.

The same split exists for programs. An interpreter reads the program as *data*, one instruction at a time, and carries out each one with its own code. A compiler reads the program once and writes out a new program in the machine's own instructions, which the CPU then runs directly. Python's standard implementation interprets; C is normally compiled; Java and JavaScript engines do a mix (the JIT section below).

stage	interpreter	compiler
read source	every run	once
decode	per instruction	per AST node
execute	dispatch loop	native code
cost	10–100×	∼1×

Figure 18.1: The interpreter/compiler divide: the interpreter pays decode-per-step forever; the compiler pays it once at build time. Our VM took 28 dispatched steps for the chapter's program, and about 10× native time on a bare countdown loop (measured in this chapter, Linux x86_64).

CONCEPT: A PROGRAM IS JUST DATA

The idea that unlocks this whole chapter is that an instruction list is a *value* that another program can read, check and transform. An interpreter reads it and acts. A compiler reads it and writes out another one. Once you see programs as data, building either is an exercise in careful bookkeeping, not magic.

18.2 The interpreter: a stack machine

What is a stack machine? Start from $2 + 3$

A *stack* is a pile where you can only add or remove at the top, like plates. Suppose a calculator works on a stack. To compute $2 + 3$ you give it three commands: `PUSH 2` (put 2 on the pile), `PUSH 3`, then `ADD`. `ADD` takes the top two plates, adds them and puts the result back. No command ever names where its operands are: they are always "the top two plates". That is why a stack machine's instructions are tiny and why it is so easy to build one. Writing the operator after its operands (2 3 +) is called *postfix* notation, and it is exactly what stack code is.

For a longer formula such as $(6 \times 7) + 8$ the same idea works with no brackets at all: push 6, push 7, `MUL` (leaves 42), push 8, `ADD` (leaves 50). The order of the commands already carries the structure that brackets expressed.

op	stack after
PUSH 6	[6]
PUSH 7	[6,7]
MUL	[42]
PUSH 8	[42,8]
ADD	[50]
OUT	[] (prints 50)

Figure 18.2: The stack machine's trace on `(6*7)+8`: operands push, operators consume-and-push. No registers, no allocation — the simplest machine that computes.

`ch18_vm.c` implements 9 *opcodes* (operation codes, the numbers that name the instructions): PUSH, ADD, SUB, MUL, DIV, DUP (copy the top of the stack), OUT (print and remove the top), JNZ (jump if the top is not zero) and HALT, in one *dispatch loop*. Dispatch means: read the next opcode, pick the code that handles it, run it, repeat. The program computes $(6 \times 7) + 8$, prints 50, then counts down via a JNZ loop. The VM dispatches 28 instructions to do it (verified by running it, and again by counting by hand below). Of those 28, 13 do real work (5 arithmetic, 5 `OUT`, 3 `JNZ`) and 15 are stack bookkeeping (7 `PUSH`, 7 `DUP`, 1 `HALT`): about $28/13 \approx 2.2$ dispatches for each useful operation. That ratio is the interpreter's tax made visible, and it is a count of instructions, not of time (the time cost is measured later in the chapter).

```
50   3   2   1   0   (28 insn)
```

Figure 18.3: Real `ch18_vm` output verbatim: the arithmetic program prints 50, then the countdown loop prints 3,2,1 — *and a stray 0*, the last 0 coming from the `OUT` that follows the loop. A real off-by-one in the *program*, kept as the chapter's receipt.

Example 18.1 (Why does the countdown print a stray zero?)**.** The countdown prints `3 2 1 0`, but a person reading the description ("count down from 3") expects `3 2 1`. Where does the 0 come from? Read the program: inside the loop, each pass does `DUP OUT` (print

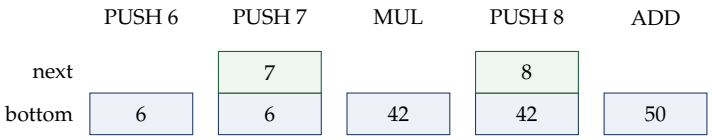
the counter), then `PUSH 1 SUB` (subtract 1), then `DUP JNZ` (jump back if the new value is not zero). When the counter reaches 0 the jump is not taken, the loop is left, and the next instruction is a final `OUT`, which prints the value still sitting on the stack: 0. The VM did exactly what it was told; the *program* printed one value too many. This is the loop-boundary mistake (an "off-by-one") that every hand-written loop in every language can have. Keeping it in the chapter is deliberate, because you will meet it again.

The fix is to delete that final `OUT`. The new bytecode `ch18_count_fixed.bc` is `1 3 6 7 1 1 3 6 8 2 0` and the loader `ch18_vmrun` (below) prints, verbatim,

```
3
2
1
instructions executed: 20
```

The count falls from 28 to 20 because the first part (the $6 \times 7 + 8$ arithmetic, 7 dispatches) and the final `OUT` (1) are gone.

18.3 Watching the stack: a trace



MUL pops 7 and 6, pushes 42; ADD pops 8 and 42, pushes 50

Figure 18.4: The operand stack of `ch18_vmrun` after each step of $6 \times 7 + 8$ (the real trace is in the worked example). Operands are never named: the top of the stack is always "the current value".

A stack machine is easiest to understand by watching it. The loader `ch18_vmrun.c` reads bytecode (a list of integers) from standard input and, with `-t`, prints one line per dispatched instruction showing the stack *before* that step. Unlike the first VM it also checks every push and pop, so a faulty program stops with a message instead of silently overwriting memory.

```
1    switch(op){
2      case HALT: printf("instructions executed: %ld\n",ins); return 0;
3      case PUSH: if(ip>=plen) die("missing operand"); push(prog[ip++]);
           break;
4      case ADD:{int b=pop(),a=pop();push(a+b);}break;
5      case SUB:{int b=pop(),a=pop();push(a-b);}break;
6      case MUL:{int b=pop(),a=pop();push(a*b);}break;
7      case DIV:{int b=pop(),a=pop(); if(b==0) die("divide by zero");
         push(a/b);}break;
8      case DUP:{int a=pop();push(a);push(a);}break;
9      case OUT: printf("%d\n",pop()); break;
10     case JNZ:{ if(ip>=plen) die("missing operand"); int t=prog[ip++];
           if(pop()) ip=t; }break;}}}
```

Listing 18.1: The dispatch loop of `ch18_vmrun.c`: one `case` per opcode, every pop and push checked.

Example 18.2 (Following the stack step by step: a real trace of $6 \times 7 + 8$)**.** Feeding it the bytecode `1 6 1 7 4 1 8 2 0` (PUSH 6, PUSH 7, MUL, PUSH 8, ADD, HALT) with `-t` gives, verbatim:

```
  1  ip=0   PUSH 6    stack before:
  2  ip=2   PUSH 7    stack before: 6
  3  ip=4   MUL       stack before: 6 7
  4  ip=5   PUSH 8    stack before: 42
  5  ip=7   ADD       stack before: 42 8
  6  ip=8   HALT      stack before: 50
instructions executed: 6
```

Read it line by line. Step 3: the stack is `6 7`; `MUL` pops the two top values, multiplies, pushes 42. Step 5: the stack is `42 8`; `ADD` leaves 50. Notice that no instruction says *which* registers to use. That is why stack code is so compact: the operands are always "the top two things".

Example 18.3 (What if the program is wrong? The checked VM says so)**.** A VM that trusts its input is a security hole (the first VM, with no checks, would let a long enough program write past its 256-entry stack). The checked loader turns three classic mistakes into messages. Verbatim:

```
PUSH 1, PUSH 0, DIV      ->  VM error at ip=5: divide by zero
(program: 6 6 6)          ->  VM error at ip=1: stack underflow
(300 times PUSH 1)       ->  VM error at ip=514: stack overflow
```

The second program is `DUP DUP DUP`: the first `DUP` tries to pop from an empty stack. The third pushes 300 values onto a 256-entry stack and is stopped at the 257th push. The habit of mind: whenever a machine has a fixed-size resource, ask "what happens at the edge?" and make the answer a deliberate error, not an accident.

18.4 The compiler: emit once

The compiler's trade is to do the thinking *before* the program runs. Our first compiler, `ch18_cc.c`, is as small as one can be: it knows one expression, $(a + b) \times c$, and *emits* (prints) the instructions that compute it, as text.

```
mov  w0, #6
mov  w1, #7
add  w0, w0, w1
mov  w1, #8
mul  w0, w0, w1
```

Figure 18.5: `ch18_cc` output: the same expression as 5 lines of arm64 — the compiler's advantage is that (a+b)×c becomes instructions *once*; there is no dispatch in the result.

The output is five arm64 instructions with no dispatch loop and no stack at run time. The constant operands are written straight into the instructions, and the compiler even computes

the answer itself (`# eval at compile time: 104`), a trick called *constant folding*. A plain interpreter, which looks at one instruction at a time as it runs, does not do this unless someone adds a separate pre-pass, and that pre-pass is already a small compiler.

THINK LIKE THIS

Ask of any work: "does this have to happen every time the program runs, or can it happen once, earlier?" Moving work from run time to build time is the single idea behind most compiler optimisations, and later in this chapter it is what the JIT does for the hot parts of a program.

18.5 A compiler you can read in one sitting

So far the "compiler" only printed lines it already knew. A real one has three jobs: *parse* the text (find its structure), maybe *optimise* it, and *generate* code. `ch18_expr.py` (about 120 lines of Python) does all three for integer expressions with `+ - * /` and parentheses, and can aim at two targets: our stack VM, or RISC-V assembly.

Why a tree? The text `2+3*4` is a flat line of characters, but its meaning has shape: the multiplication must happen first, so `3*4` is a unit and the addition combines `2` with that unit. A tree captures this: each node is an operator whose children are its operands. The compiler's first job is to build the tree.

Example 18.4 (First attempt: read left to right. It gets $2 + 3 \times 4$ wrong)**.** The simplest idea is to read left to right and apply each operator as you meet it. Our `-naive` flag does exactly that, treating every operator as equal. Its tree for `2+3*4` is $((2 + 3) \times 4)$ and its value is 20. The correct tree is $(2 + (3 \times 4))$ with value 14. Real output:

```
$ python3 ch18_expr.py --naive --target ast "2+3*4"
((2 + 3) * 4)
value: 20
$ python3 ch18_expr.py --target ast "2+3*4"
(2 + (3 * 4))
value: 14
```

The cure is to give the grammar one level per precedence: an *expression* is terms joined by $+/-$; a *term* is factors joined by $*//$; a *factor* is a number or a parenthesised expression. Because `expr` calls `term`, which calls `factor`, the multiplication gets grouped before the addition ever sees it.

```
1     def binary(self, sub, ops):
2         n = sub()
3         while self.peek() is not None and self.peek() in ops:
4             op = self.eat(); n = (op, n, sub())
5         return n
6     def expr(self):
7         if self.naive: # every operator at one level
```

```
8           return self.binary(self.factor, "+-*/")
9        return self.binary(self.term, "+-")
10    def term(self): return self.binary(self.factor, "*/")
11    def factor(self):
12       tok = self.peek()
13       if tok == "(":
14          self.eat("("); n = self.expr(); self.eat(")"); return n
15       if tok is not None and tok.isdigit(): self.eat(); return int(tok
             )
```

Listing 18.2: The parser of `ch18_expr.py`: a single helper `binary` builds left-leaning trees for any level of operators.

```
1  def gen_vm(n, out): # postorder walk = stack code
2     if isinstance(n, int): out += [PUSH, n]
3     else: gen_vm(n[1], out); gen_vm(n[2], out); out.append(OPC[n[0]])
4
5  TEMPS = ["t0", "t1", "t2", "t3", "t4", "t5", "t6"]
6  RVOP = {"+": "add", "-": "sub", "*": "mul", "/": "div"}
7  def gen_rv(n, free, out): # returns the register holding the value
8     if isinstance(n, int):
9        if not free: raise RuntimeError("out of temporary registers (no
             spilling)")
10       r = free.pop(0); out.append(f" li {r}, {n}"); return r
11    ra = gen_rv(n[1], free, out); rb = gen_rv(n[2], free, out)
12    out.append(f" {RVOP[n[0]]:<4} {ra}, {ra}, {rb}"); free.insert(0, rb
          ); return ra
```

Listing 18.3: Code generation. For the stack VM, walk the tree children-first ("postorder") and emit the operator last. For RISC-V, the same walk hands out temporary registers.

Example 18.5 (One expression, three kinds of output)**.** Compile $(6+7)\times 8$. Walking the tree children first gives the VM bytecode, the register walk gives RISC-V, and with `-fold` the compiler evaluates constants itself. Real outputs:

```
--target vmlist          --target rv32            --fold --target rv32
  0  PUSH 6                li   t0, 6               li   t0, 104
  2  PUSH 7                li   t1, 7               addi a0, t0, 0
  4  ADD                   add  t0, t0, t1
  5  PUSH 8                li   t1, 8
  7  MUL                   mul  t0, t0, t1
  8  OUT                   addi a0, t0, 0
  9  HALT
```

Running the bytecode in `ch18_vmrun` prints 104 after 7 dispatches. The RISC-V version needs no dispatch at all: 6 instructions, the last one just moving the answer into `a0`, the register that holds a function's result (ch. 4). Assembled with `clang -target=riscv32 -march=rv32im` and disassembled with `llvm-objdump`, the multiply is the machine word `b3 82 62 02`, which is `mul t0, t0, t1` (this needs the M extension; RV32I alone has no multiply, as ch. 3 noted). With folding the whole program collapses to one `li`, which is what `# eval at compile time:  104` meant earlier in the chapter.

Example 18.6 (How do we know the compiler is right? Compare it with gcc)**.** Correctness is checked, not assumed. `test_ch18.sh` compiles seven expressions, runs each on the VM, and compares with what gcc computes for the same C expression (including C's rule that integer division truncates). All seven match: `104, 50, 5, 14, 7, 1, -3` for `(6+7)*8, 6*7+8, 10-3-2, 2+3*4, 100/7/2, ((1+2)*(3+4))-5*(6-2), 7-10`. Check one by hand: $100/7 = 14$ (truncated), $14/2 = 7$; and $21 - 5 \cdot 4 = 1$. Programs using only + and − also run on this book's own RV32I simulator from ch. 3: the code for $(9 - 4) + (7 - 5)$ leaves 7 in `a0` after 8 retired instructions.

18.6 What does interpreting cost? Measure it

Folklore says "interpreters are 10 to 100 times slower". Let us see what our own VM does. `ch18_vm_bench.c` runs the same countdown loop from 100 million, natively and as bytecode (`PUSH N; L: PUSH 1, SUB, DUP, JNZ L; HALT`), four dispatches per iteration.

```
__attribute__((noinline)) static long vm(const int*p){
  int ip=0,sp=0; long ins=0;
  for(;;){ ins++;
    switch(p[ip++]){
      case HALT: return ins;
      case PUSH: st[sp++]=p[ip++]; break;
      case SUB:{int b=st[--sp],a=st[--sp]; st[sp++]=a-b;}break;
      case DUP:{int a=st[sp-1]; st[sp++]=a;}break;
      case JNZ:{int t=p[ip++]; if(st[--sp]) ip=t;}break;}}}
```

Listing 18.4: The VM side of `ch18_vm_bench.c`: the same switch dispatch as before, with no checks and no printing.

Example 18.7 (How much slower is interpreting? A fair baseline, then an unfair one)**.** Verbatim output (Linux x86_64 container, `gcc -O2`):

```
iterations: 100000000
native, register counter:    72.3 ms  (0.72 ns/iter)
native, memory counter  :   168.5 ms  (1.69 ns/iter)
stack VM                :   716.6 ms  (7.17 ns/iter, 400000002
    dispatches, 1.79 ns/dispatch)
VM / native(register) = 9.9x    VM / native(memory) = 4.3x
```

Two native baselines appear because the first one we tried, a counter declared `volatile` so that the compiler could not delete the loop, forces a store and reload on every pass and is itself slow (1.69 ns). Comparing against that flatters the VM (4.3×). The honest baseline keeps the counter in a register, which gives 9.9×. Over several runs the register-baseline ratio ranged from 8 to 16× on this shared container, so quote it as "about 10×", not as a precise figure. Each dispatch costs about 1.8 ns, roughly 5 cycles: that is the price of loading the opcode, jumping to its `case`, and touching the stack in memory. Folklore's 10–100× is therefore a fair description of a naive interpreter on a loop with almost no real work in it; real programs do more work per dispatch, which is why the figure shrinks for them.

18.7 The JIT compromise

Compiling costs time up front, and most code in a typical program runs only once or twice. A *JIT* (just-in-time) compiler splits the difference. It starts by interpreting everything, which costs nothing to start. It counts how often each piece of code runs. When a piece becomes *hot* (it runs thousands of times), the JIT compiles just that piece into machine code while the program is running. The cost of compiling is then paid once and recovered many times over, like the translator hired only for the chapters you re-read daily.

An optimising JIT goes one step further and *speculates*: it notices that a variable has always been an integer so far, compiles a version that assumes it always will be, and puts a cheap check (a *guard*) in front. If the guard ever fails, the engine abandons the fast code and falls back to the interpreter, which is called *deoptimising*. That is why the figure has three tiers.

tier	when it kicks in	cost model
interpreter	always	decode-per-step
baseline JIT	warm code	compile once, simple code
optimising JIT	hot code	speculates + deopts

Figure 18.6: The JIT answer to the divide: start interpreting, compile the hot paths at runtime. Python's CPython lives at row one; JVMs and JS engines climb all three.

18.8 The loop closes

You can now trace one line of source text all the way down. Text is parsed into a tree; the tree is translated to instructions (stack bytecode or RISC-V); those instructions are fetched, decoded and executed, by a C loop in the case of the VM or by the pipelined core of ch. 5 to 8 in the case of RISC-V; the core reads operands through the memory hierarchy of ch. 9 and 10; and the measurements of ch. 15 tell you how fast it all was. Each layer you met in this book is a place where one such translation happens.

this book built	the layer it closed
ch1 compile pipeline	source → binary
ch3–4 ISA/ABI	the contract
ch5–8 the core	the silicon
ch9–11 memory/OS	the environment
ch18 this machine	back to source

Figure 18.7: The loop closes: source text in, computation out, through every layer the book built. The 9-opcode VM is the same machine as ch8's SoC — one abstracts software, the other gates.

The VM is ch. 8's SoC in software: fetch-decode-execute with a `switch` for the decoder and a stack for the register file. The compiler is ch. 1's pipeline in miniature: syntax tree to

instructions. Everything between them — the ISA contract, the memory hierarchy, the OS services, the measurement discipline — is what this book spent 18 chapters building.

18.9 Common mistakes

- **"Interpreted = slow" without a number.** Ours measured about 10× native on a bare countdown loop (4× against a deliberately slow native baseline) and 2.2 dispatches per useful operation — real, and dependent on how much work each dispatch does.
- **The off-by-one at the loop boundary.** The printed trailing 0 is the `OUT` after the loop: test the exit condition and ask what is still on the stack when you leave.
- **A parser that ignores precedence.** `2+3*4` came out as 20; one grammar level per precedence level fixes it.
- **Thinking the compiler is magic.** Ours is about 120 lines of Python. The hard parts (choosing registers when there are too few, serious optimisation) are real, but the *idea* fits on a page.
- **JIT as free speed.** A JIT pays to compile, needs warm-up time before the fast code exists, and loses its work whenever a guard fails and it must deoptimise. The interpreter pays a little all the time; the JIT pays more at the start and sometimes twice.
- **Stack discipline bugs.** `OUT` removes what it prints, so `DUP` before `OUT` is what preserves the loop counter. One misplaced stack operation corrupts every later step.

18.10 Summary

Piece	Verified version
VM	9 opcodes, real run: 50 + countdown 3..0 (the 0 is the program's, fixed version 3..1)
overhead	28 dispatches, 13 of them useful work; ≈10× native on a bare loop (x86_64)
compiler	same expr → 5 arm64 lines, no dispatch; `ch18_expr.py`: parse, fold, emit VM or RV32 code
JIT	interpret cold, compile hot, speculate
closure	VM = ch8 SoC abstracted; book's loop closed

Further reading

- Crafting Interpreters (Nystrom): the book of this chapter. `craftinginterpreters.com`
- The Python bytecode interpreter (ceval.c): `github.com/python/cpython`

WATCH INSTEAD

- **Crafting Interpreters overview — Robert Nystrom talk** — the author's walkthrough of this exact build (verified 2026-10-11). https://youtu.be/8lC9FciP8Xg

Notes

- The original programs run verbatim on this machine; the trailing 0 in the countdown comes from the program's final `OUT`, kept as a receipt. The new programs (`ch18_vmrun.c`, `ch18_expr.py`, and `ch18_vm_bench.c`) are run and checked by `code/test_ch18.sh`; timings are from a shared Linux x86_64 container and vary by tens of percent between runs.
- The earlier text spoke of "14 abstract operations"; that figure cannot be derived from the program and was replaced by the counted 13 useful and 15 bookkeeping dispatches.
- The compiler's output is arm64 text emitted by the program — the book ends where ch1 began, one level up.

Exercises

Exercise 18.1 (easy)**.** Why can't the interpreter constant-fold like the compiler did?

Solution. The interpreter only ever looks at the instruction it is executing now; it does not look ahead to notice that two pushed constants will be multiplied. Folding needs a pass over the whole program before it runs, and a pass that rewrites a program into a better one is compilation.

Exercise 18.2 (easy)**.** What does the 2× dispatch overhead buy the VM?

Solution. Portability and safety. The VM is ordinary C, so it runs on any machine that has a C compiler, and it can check every operation (as `ch18_vmrun` does) instead of trusting the program. A compiler produces code for one specific CPU and that code runs unchecked.

Exercise 18.3 (easy)**.** Which register does the stack machine never need to name?

Solution. All of them. Operands are implicit ("the top of the stack"), so instructions carry no register fields. Register machines such as RV32I must name each source and destination register.

Exercise 18.4 (easy)**.** Fix the countdown's trailing 0. One-line change?

Solution. Delete the final `OUT`: bytecode `1 3 6 7 1 1 3 6 8 2 0` prints 3 2 1 in 20 dispatches (verified).

Exercise 18.5 (medium)**.** Add a CALL/RET pair to the VM. What stack discipline do the caller and callee need that PUSH/ADD never did?

Solution. `CALL` must remember where to come back to, and `RET` must find it. The caller leaves its arguments on the stack, `CALL` pushes the return address (or pushes it on a second "return stack"), and `RET` pops it into `ip`. The callee must leave the stack as the caller expects, for example only its result on top. That is the same calling convention idea as ch. 4's ABI, now in software.

Exercise 18.6 (medium)**.** The compiler baked (6+7)*8=104 at build time. What optimisation pass is that, and when does it break (overflow, UB)?

Solution. It is *constant folding*. It must give exactly the answer the program would have produced at run time, so it has to respect the language's rules. Our `ch18_expr.py` stops with an error if a folded result does not fit in 32 bits. In C, signed overflow is undefined behaviour, so a compiler is allowed to fold however it likes; in languages with defined wrap-around it must fold with wrap-around. Division by zero must also not be folded silently (our compiler reports it).

Exercise 18.7 (medium)**.** Design the VM's dispatch differently: threaded code (jump table per opcode). What does it remove and what does it cost?

Solution. A `switch` dispatch jumps from one shared place to the handler, so all opcodes share one indirect jump that the predictor finds hard to guess. Threaded code ends each handler with its own jump to the next handler, so the predictor sees one jump per opcode with its own history. That removes the shared bounds check and often mispredicts less. The costs are a less portable source style (computed gotos, a GCC extension) and code that is harder to read; the gain depends on the CPU and must be measured.

Exercise 18.8 (medium)**.** JIT speculation: your engine bets the loop counter stays int. What must it emit to stay correct when it doesn't?

Solution. Before using the integer-specialised code it must check that the value really is an integer (the guard). If not, it must jump to a *deoptimisation* path that rebuilds the interpreter's state from the compiled code's state and carries on in the interpreter. Correctness always comes before speed.

Exercise 18.9 (hard)**.** Prove or refute: the stack VM's dispatch count per useful operation grows with expression complexity. Count dispatches for an expression with n binary operators (compiled by `ch18_expr.py`) and explain the scaling.

Solution. An expression with n binary operators has $n+1$ numbers, so $n+1$ `PUSH`es, n operator instructions, one `OUT` and one `HALT`: $2n+3$ dispatches (check: $(6+7)\times 8$ has $n=2$ and 7 dispatches). That is linear in n; per operator it tends to 2, so the overhead does not grow.

Exercise 18.10 (hard)**.** Write the register-based VM version of this machine (reg[16] instead of stack). Compare program byte length and dispatch count on the same computation — which machine wins and when?

Solution. Register code names its operands, so each instruction is bigger, but the program needs fewer instructions because it has no `PUSH/DUP` bookkeeping. For $(6 + 7) \times 8$ the stack code has 7 dispatches and the register code (see the RV32 output) 5 plus a final move into `a0` (three constant loads and two arithmetic operations), so 6 instructions. Fewer dispatches generally mean a faster VM; the stack form is more compact. The rule: stack code wins on size, register code on speed, which is why modern VMs such as Lua's moved to registers.

Exercise 18.11 (easy)**.** What is the interpreter's `ip` in CPU terms?

Solution. The program counter. The CPU's PC holds the address of the next instruction; the VM's `ip` holds the index of the next opcode. Reading `p[ip++]` is fetch, the `switch` is decode, the case body is execute.

Exercise 18.12 (medium)**.** Why is the stack machine's code denser than a register machine's? Give the byte-count argument on $(6{\times}7) + 8$.

Solution. A stack `MUL` is just the opcode. A register `mul t0,t0,t1` must also encode three register numbers (RV32I uses 32 bits per instruction). In our VM encoding $(6 \times 7) + 8$ is `PUSH 6, PUSH 7, MUL, PUSH 8, ADD`: if opcodes and constants are one byte each, that is $2 + 2 + 1 + 2 + 1 = 8$ bytes, against $5 \cdot 4 = 20$ bytes for five 4-byte RISC-V instructions.

Exercise 18.13 (medium)**.** The JIT tier's "speculates and deopts" — what assumption does a type-specialising JIT actually bet on?

Solution. That the types it has *observed* so far will keep being the same, for example "this addition always receives integers". The guard checks the bet on every call, and a wrong bet costs a deoptimisation.

Exercise 18.14 (medium)**.** Our VM's stack lives in a C array. What breaks if a program pushes past 256, and what's the honest fix?

Solution. In the unchecked `ch18_vm.c` it silently writes past the array and corrupts whatever memory is next to it, which is how real exploits work. The honest fix is the one in `ch18_vmrun`: check the stack pointer on every push and pop and stop with an error, or grow the stack safely.

Exercise 18.15 (hard)**.** Design the minimal bytecode that a real JVM or WASM module would need beyond our 9 ops to run ch12's scheduler sim. List the semantic gaps (calls, locals, branches, memory).

Solution. Calls and returns with a frame for each call; local variables (load and store by index); conditional and unconditional branches (not only JNZ), ideally structured so they can be verified; and a block of addressable memory for arrays and structs. These four, together with more integer types and comparison operations, are the gaps between our toy and a real module such as WebAssembly.

Exercise 18.16 (easy)**.** Evaluate the bytecode `PUSH 2, PUSH 3, ADD, PUSH 4, MUL, OUT, HALT` by hand, writing the stack after each step. What does it print, and what infix expression is it?

Solution. Stacks: $[2]$, $[2,3]$, $[5]$, $[5,4]$, $[20]$; `OUT` prints 20. It is $(2+3)\times 4$, which is also what the naive parser computes for `2+3*4`.

Exercise 18.17 (medium)**.** Using the measured 7.17 ns per iteration and 4 dispatches per iteration, compute ns per dispatch, and convert to cycles at 2.8 GHz.

Solution. $7.17/4 = 1.79$ ns per dispatch; $1.79 \times 2.8 \approx 5.0$ cycles.

Exercise 18.18 (medium)**.** Compile `(1+2)*(3+4)` to RISC-V with the register scheme of `gen_rv`. How many temporary registers are live at the busiest moment, and which?

Solution. The code is `li t0,1, li t1,2, add t0,t0,t1, li t1,3, li t2,4, add t1,t1,t2, mul t0,t0,t1`. At the busiest point `t0, t1, t2` are live: three.

Bibliography

[1] J. Sannemo, *Principles of Algorithmic Problem Solving*, 2025. https://usaco.guide/PAPS1.pdf

Index

-O0, 32
-O2, 32
`.data`, 3
`.text`, 3

ABI names, 20
address, 11
address space, 5
aligned access, 13
alignment, 11
ALU, 40
Amdahl, 154
array, 11
assembler, 1
assembly language, 20
assembly line, 52

B-type, 20
base+offset addressing, 22
benchmark, 154
biased exponent, 14
big-endian, 12
blocking, 94
Book 5, 6
branch, 20
branch prediction, 64
bypass, 62

cache, 83, 94
callee-saved, 30
caller-saved, 30
calling convention, 29
calling overhead, 31
compilation pipeline, 1
compiler, 1, 192
compiler output, 32
condition variable, 132
conditional branch, 23
context switch, 119
control hazard, 64
CPI, 45
cpu8, 6
CPUlator, 23
critical path, 45

data hazard, 54
data memory, 40
datapath, 38
deadlock, 132
decoder, 39
directives, 23

ELF, 1
endianness, 11
`execve`, 1, 5
exponent, 14

file system, 143
floating point, 11
flush, 64
forwarding, 62
fsync, 143
funct3, 21
funct7, 21

hazard, 54

I-type, 20
IEEE 754, 11
IF/ID/EX/MEM/WB, 52
immediate, 21
immediate generator, 39
inode, 143
instruction encoding, 20
interpreter, 192

J-type, 20
jitter, 178
journal, 143

latency, 52
linker, 1

little-endian, 12
load, 20
load-use hazard, 63
load/store, 40
loader, 1
locality, 83
low latency, 178
LP64, 11

mantissa, 14
memory hierarchy, 83
memory mountain, 90
miss rate, 94
MLFQ, 119
mutex, 132

NEON, 163

object file, 1
`offsetof`, 13
opcode, 21
overflow, 12

padding, 13
page fault, 108
page table, 5, 108
PIC, 33
pipeline register, 52
pipelining, 52
pointer, 11
pointer arithmetic, 15
position-independent code, 33
preprocessor, 1
process, 119
program counter, 38
psABI, 29
pseudoinstruction, 20

R-type, 20
`ra`, 29
race condition, 132
register
    callee-saved, 29
    caller-saved, 29
register file, 20, 40
relocation, 1
relocation record, 4
ring buffer, 178

RV32I, 6, 20
RV32I core, 38

S-type, 20
scheduler, 119
section (ELF), 3
sign bit, 12
SIMD, 163
single-cycle processor, 38
`sp`, 29
stack frame, 29
stall, 63
state, 38
store, 20
struct padding, 11
sv32, 110
symbol (linker), 4
symbol table, 1

tail latency, 154
throughput, 52
tiling, 94
timing diagram, 53
TLB, 108
two's complement, 11
type punning, 11

U-type, 20
undefined symbol, 4

vectorisation, 163
virtual machine, 192
virtual memory, 108

word, 11

X-propagation, 66
x0, 20

zero register, 20